Date Published: April 01, 2026

Visit the Zebra Product Support page for the most up-to-date version of this documentation.

Mclass functions

Synopsis

The functions prefixed with Mclass make up the Classification module. The Classification module allows you to perform machine learning tasks, such as image classification, object detection, segmentation, and feature classification. The module also lets you import ONNX classifiers to perform these tasks.

Functions

MclassAlloc
MclassAllocResult
MclassControl
MclassControlEntry
MclassCopy
MclassCopyResult
MclassDraw
MclassDrawEntry
MclassEntryAddRegion
MclassExport
MclassFree
MclassGetHookInfo
MclassGetResult
MclassGetResultEntry
MclassGetResultStat
MclassHookFunction
MclassImport
MclassInquire
MclassInquireEntry
MclassPredict
MclassPrepareData
MclassPreprocess
MclassRestore
MclassSave
MclassSetHookInfo
MclassSplitDataset
MclassStatCalculate
MclassStream
MclassTrain

MclassAlloc

Synopsis

Allocate a classifier context, a dataset context, a data preparation context, a statistics classification context, or a training context.

Syntax

```
AIL_ID MclassAlloc(
    AIL_ID SysId,             //in
    AIL_INT64 ContextType,    //in
    AIL_INT64 ControlFlag,    //in
    AIL_ID *ContextClassIdPtr //out
)
```

Description

This function allocates a classifier context, a dataset context, a data preparation context, a statistics classification context, or a training context on the specified system. These contexts are for anomaly detection, feature classification, image classification, object detection, or segmentation.

In general, classifier contexts and dataset contexts are for training (MclassTrain()) and prediction (MclassPredict()), while training contexts, dataset contexts, and data preparation contexts are for training. You can import data to, and export data from, a dataset context by calling MclassImport() and MclassExport(). Datasets are typically exported after they are created and imported the next time they are needed.

When a context is no longer required, release it using MclassFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the context's lifetime and you must not manually free it.

Parameters

SysId INQ

Specifies the identifier of the system on which to allocate the context. Set this parameter to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the system																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

ContextType

Specifies the type of context to allocate. Set this parameter to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of context to allocate																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_CLASSIFIER_ANO_PREDEFINED	Specifies a predefined anomaly detection classifier context. This is for anomaly detection. For this context, you should set the ControlFlag parameter to M_ADNET). When you allocate a classifier context, you must usually train it (MclassTrain()) before predicting with it (MclassPredict()).	c		e	f	g	a		i		j	k	l	m	n	b
M_CLASSIFIER_CNN_PREDEFINED	Specifies a predefined CNN classifier context. This is for image classification. For this context, you should typically specify a specific CNN classifier that Aurora Imaging has built (that is, set the ControlFlag parameter to M_ICNET_...). CNN and image classification are used interchangeably unless otherwise specified. When you allocate a classifier context, you must usually train it (MclassTrain()) before predicting with it (MclassPredict()).	c		e	f	g	a		i		j	k	l	m	n	b
M_CLASSIFIER_DET_PREDEFINED	Specifies a predefined object detection classifier context. This is for object detection. For this context, you should set the ControlFlag parameter to M_ODNET). When you allocate a classifier context, you must usually train it (MclassTrain()) before predicting with it (MclassPredict()).	c		e	f	g	a		i		j	k	l	m	n	b
M_CLASSIFIER_ONNX	Specifies an ONNX classifier context. For this context, do not specify a specific classifier that Aurora Imaging has built (that is, set the ControlFlag parameter to M_DEFAULT). ONNX is an open-source machine learning format that you can use to create a classifier model. When you allocate an ONNX classifier context, you typically import a trained ONNX classifier model into it, and then predict with it MclassPredict(). The type of classification that is performed depends on the ONNX classifier that you created and trained prior to importing it.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLASSIFIER_SEG_PREDEFINED	Specifies a predefined segmentation classifier context. This is for segmentation. For this context, you should typically specify a specific segmentation classifier that Aurora Imaging has built (that is, set the ControlFlag parameter to M_CSNET_...). When you allocate a classifier context, you must usually train it (MclassTrain()) before predicting with it (MclassPredict()).	c		e	f	g	a		i		j	k	l	m	n	b
M_CLASSIFIER_TREE_ENSEMBLE	Specifies a tree ensemble classifier context. This is for feature classification. For this context, do not specify a specific classifier that Aurora Imaging has built (that is, set the ControlFlag parameter to M_DEFAULT). Tree ensemble and feature classification are used interchangeably unless otherwise specified. When you allocate a classifier context, you must usually train it (MclassTrain()) before predicting with it (MclassPredict()).	c		e	f	g	a		i		j	k	l	m	n	b

M_DATASET_FEATURES	Specifies a features dataset context. This context is for feature classification; it holds the feature data with which to train a tree ensemble classifier (M_CLASSIFIER_TREE_ENSEMBLE). For this context, do not specify a specific classifier that Aurora Imaging has built (that is, set the ControlFlag parameter to M_DEFAULT).	c		e	f	g	a		i		j	k	l	m	n	b
M_DATASET_IMAGES	Specifies an images dataset context. This context (dataset) is for image classification, segmentation, object detection, or anomaly detection; it holds the image data with which to train a CNN classifier (M_CLASSIFIER_CNN_PREDEFINED), a segmentation classifier (M_CLASSIFIER_SEG_PREDEFINED), an object detection classifier (M_CLASSIFIER_DET_PREDEFINED), or an anomaly detection classifier (M_CLASSIFIER_ANO_PREDEFINED). Image classification and segmentation support buffers with up to 1024 bands. You can also use an images dataset context with MclassPredict() to label images in an existing dataset (assisted labeling). For this context, do not specify a specific classifier that Aurora Imaging has built (that is, set the ControlFlag parameter to M_DEFAULT).	c		e	f	g	a		i		j	k	l	m	n	b
M_PREPARE_IMAGES_CNN	Specifies a CNN data preparation context. This context is for image classification; it holds the settings with which to prepare images for CNN training or prediction (for example, cropping, resizing, or augmenting), using MclassPrepareData(). For this context, do not specify a specific classifier that Aurora Imaging has built (that is, set the ControlFlag parameter to M_DEFAULT).	c		e	f	g	a		i		j	k	l	m	n	b
M_PREPARE_IMAGES_DET	Specifies an object detection data preparation context. This context is for object detection; it holds the settings with which to prepare images for object detection training or prediction (for example, cropping, resizing, or augmenting), using MclassPrepareData(). For this context, do not specify a specific classifier that Aurora Imaging has built (that is, set the ControlFlag parameter to M_DEFAULT).	c		e	f	g	a		i		j	k	l	m	n	b
M_PREPARE_IMAGES_SEG	Specifies a segmentation data preparation context. This context is for segmentation; it holds the settings with which to prepare images for segmentation training or prediction (for example, cropping, resizing, or augmenting), using MclassPrepareData(). For this context, do not specify a specific classifier that Aurora Imaging has built (that is, set the ControlFlag parameter to M_DEFAULT).	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_ANO	Specifies a statistics classification context for anomaly detection. This context holds the settings for which to calculate statistics on a dataset using MclassStatCalculate(). For this context, do not specify a specific classifier that Aurora Imaging has built (that is, set the ControlFlag parameter to M_DEFAULT).	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_CNN	Specifies a statistics classification context for image classification. This context holds the settings for which to calculate statistics on a dataset using MclassStatCalculate(). For this context, do not specify a specific classifier that Aurora Imaging has built (that is, set the ControlFlag parameter to M_DEFAULT).	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_DET	Specifies a statistics classification context for object detection. This context holds the settings for which to calculate statistics on a dataset using MclassStatCalculate(). For this context, do not specify a specific classifier that Aurora Imaging has built (that is, set the ControlFlag parameter to M_DEFAULT).	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_SEG	Specifies a statistics classification context for segmentation. This context holds the settings for which to calculate statistics on a dataset using MclassStatCalculate(). For this context, do not specify a specific classifier that Aurora Imaging has built (that is, set the ControlFlag parameter to M_DEFAULT).	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_TREE_ENSEMBLE	Specifies a statistics classification context for feature classification. This context holds the settings for which to calculate statistics on a dataset using MclassStatCalculate(). For this context, do not specify a specific classifier that Aurora Imaging has built (that is, set the ControlFlag parameter to M_DEFAULT).	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAIN_ANO	Specifies an anomaly detection training context. This context is for anomaly detection; it holds the training settings with which to train an anomaly detection classifier context, using MclassTrain(). For this context, do not specify a specific classifier that Aurora Imaging has built (that is, set the ControlFlag parameter to M_DEFAULT).	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAIN_CNN	Specifies a CNN training context. This context is for image classification; it holds the training settings with which to train a CNN classifier context, using MclassTrain(). For this context, do not specify a specific classifier that Aurora Imaging has built (that is, set the ControlFlag parameter to M_DEFAULT).	c		e	f	g	a		i		j	k	l	m	n	b

M_TRAIN_DET	Specifies an object detection training context. This context is for object detection; it holds the training settings with which to train an object detection classifier context, using MclassTrain(). For this context, do not specify a specific classifier that Aurora Imaging has built (that is, set the ControlFlag parameter to M_DEFAULT).	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAIN_SEG	Specifies a segmentation training context. This context is for segmentation; it holds the training settings with which to train a segmentation classifier context, using MclassTrain(). For this context, do not specify a specific classifier that Aurora Imaging has built (that is, set the ControlFlag parameter to M_DEFAULT).	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAIN_TREE_ENSEMBLE	Specifies a tree ensemble training context. This context is for feature classification; it holds the training settings with which to train a tree ensemble classifier context, using MclassTrain(). For this context, do not specify a specific classifier that Aurora Imaging has built (that is, set the ControlFlag parameter to M_DEFAULT).	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Specifies the predefined classifier to use. These classifiers are built by Aurora Imaging; you cannot modify their underlying architecture. If you are not allocating a predefined classifier context, set this parameter to M_DEFAULT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying no predefined classifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies no predefined classifier context. > M_DEFAULT is the only possible setting if you are not allocating a predefined classifier context. If you specify M_DEFAULT when allocating a predefined classifier context (M_CLASSIFIER_CNN_PREDEFINED, M_CLASSIFIER_DET_PREDEFINED, M_CLASSIFIER_ANO_PREDEFINED, or M_CLASSIFIER_SEG_PREDEFINED), you cannot directly train that classifier. In this case, you would typically copy a resulting trained classifier into that context (MclassCopyResult()).	c		e	f	g	a		i		j	k	l	m	n	b

To specify a predefined CNN classifier (for M_CLASSIFIER_CNN_PREDEFINED), set the ControlFlag parameter to one of the following values.

These classifiers are for image classification. They place no restriction on the minimum size of the training images (though they must all be the same size). When predicting, the size of the target images must be the same as the images used to train these classifiers.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a predefined CNN classifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_ICNET_COLOR_XL	Specifies an extra large ICNet classifier context that is for color images. This corresponds to the *ICNET_COLOR_XL.mclass* context file that Aurora Imaging Library installed. This classifier is intended for transfer learning. It has a recommended minimum receptive field of 195 pixels. You must use color (3-band) images with this classifier.	c		e	f	g	a		i		j	k	l	m	n	b
M_ICNET_M	Specifies a medium ICNet classifier context. This corresponds to the *ICNET_M.mclass* context file that Aurora Imaging Library installed. This classifier is for a complete training. It has a recommended minimum receptive field of 83 pixels. You can use images with up to 1024 bands (N-band) with this classifier.	c		e	f	g	a		i		j	k	l	m	n	b
M_ICNET_MONO_XL	Specifies an extra large ICNet classifier context that is for monochrome images. This corresponds to the *ICNET_MONO_XL.mclass* context file that Aurora Imaging Library installed. This classifier is intended for transfer learning. It has a recommended minimum receptive field of 195 pixels. You must use grayscale (1-band) images with this classifier.	c		e	f	g	a		i		j	k	l	m	n	b
M_ICNET_S	Specifies a small ICNet classifier context. This corresponds to the *ICNET_S.mclass* context file that Aurora Imaging Library installed. This classifier is for a complete training. It has a recommended minimum receptive field of 43 pixels. You can use images with up to 1024 bands (N-band) with this classifier.	c		e	f	g	a		i		j	k	l	m	n	b
M_ICNET_XL	Specifies an extra large ICNet classifier context. This corresponds to the *ICNET_XL.mclass* context file that Aurora Imaging Library installed. This classifier is for a complete training. It has a recommended minimum receptive field of 195 pixels. You can use images with up to 1024 bands (N-band) with this classifier.	c		e	f	g	a		i		j	k	l	m	n	b

To specify a predefined anomaly detection classifier (for M_CLASSIFIER_ANO_PREDEFINED), set the ControlFlag parameter to the following value.

This classifier is for anomaly detection. It places no restriction on the minimum size of the training images (though they must all be the same size). When predicting, the size of the target images must be the same as the images used to train this classifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a predefined anomaly detection classifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_ADNET	Specifies an ADNet classifier context. This corresponds to the *ADNET.mclass* context file that Aurora Imaging Library installed. This classifier is for a complete training.	c		e	f	g	a		i		j	k	l	m	n	b

To specify a predefined object detection classifier (for M_CLASSIFIER_DET_PREDEFINED), set the ControlFlag parameter to the following value.

This classifier is for object detection. It places no restriction on the minimum size of the training images (though they must all be the same size). When predicting, the size of the target images must be the same as the images used to train this classifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a predefined object detection classifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_ODNET	Specifies an ODNet classifier context. This corresponds to the *ODNET.mclass* context file that Aurora Imaging Library installed. This classifier is for a complete training. You cannot use float images with this classifier. Note, the receptive field concept does not apply to object detection.	c		e	f	g	a		i		j	k	l	m	n	b

To specify a predefined segmentation classifier (for M_CLASSIFIER_SEG_PREDEFINED), set the ControlFlag parameter to one of the following values.

These classifiers are for segmentation. They place no restriction on the minimum size of the training images. When predicting, the target image size can be smaller or bigger than the images used to train these classifiers (this requires calling MclassControl() with M_TARGET_IMAGE_SIZE_X and M_TARGET_IMAGE_SIZE_Y).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a predefined segmentation classifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_CSNET_COLOR_XL	Specifies an extra large CSNet classifier context that is for color images. This corresponds to the *CSNET_COLOR_XL.mclass* context file that Aurora Imaging Library installed. This classifier is intended for transfer learning. It has a minimum receptive field of 195 pixels that increases in step sizes of 32 pixels. You must use color (3-band) images with this classifier.	c		e	f	g	a		i		j	k	l	m	n	b
M_CSNET_M	Specifies a medium CSNet classifier context. This corresponds to the *CSNET_M.mclass* context file that Aurora Imaging Library installed. This classifier is for a complete training. It has a minimum receptive field of 83 pixels that increases in step sizes of 8 pixels. You can use images with up to 1024 bands (N-band) with this classifier.	c		e	f	g	a		i		j	k	l	m	n	b
M_CSNET_MONO_XL	Specifies an extra large CSNet classifier context that is for monochrome images. This corresponds to the *CSNET_MONO_XL.mclass* context file that Aurora Imaging Library installed. This classifier is intended for transfer learning. It has a minimum receptive field of 195 pixels that increases in step sizes of 32 pixels. You must use grayscale (1-band) images with this classifier.	c		e	f	g	a		i		j	k	l	m	n	b
M_CSNET_S	Specifies a small CSNet classifier context. This corresponds to the *CSNET_S.mclass* context file that Aurora Imaging Library installed. This classifier is for a complete training. It has a minimum receptive field of 43 pixels that increases in step sizes of 4 pixels. You can use images with up to 1024 bands (N-band) with this classifier.	c		e	f	g	a		i		j	k	l	m	n	b
M_CSNET_XL	Specifies an extra large CSNet classifier context. This corresponds to the *CSNET_XL.mclass* context file that Aurora Imaging Library installed. This classifier is for a complete training. It has a minimum receptive field of 195 pixels that increases in step sizes of 32 pixels. You can use images with up to 1024 bands (N-band) with this classifier.	c		e	f	g	a		i		j	k	l	m	n	b

ContextClassIdPtr

Specifies the address of the variable in which to write the classifier, dataset, data preparation, or training context identifier or specifies the data type that the function should use to return the context identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated classifier, dataset, data preparation, statistics classification, or training context ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated classifier, dataset, data preparation, statistics classification, or training context ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_CLASS_ID* is returned instead of a standard Aurora Imaging Library identifier. > This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the classifier, dataset, data preparation, statistics classification, or training context (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.	c		e	f	g	a		i		j	k	l	m	n	b

Address in which to write the classifier context identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated classifier context. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the data preparation context identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated data preparation context. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the dataset context identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated dataset context. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the statistics classification context identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated statistics classification context. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the training context identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated training context. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the classifier context identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_CLASS_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailclass11.lib.
DLL	Requires ail11.dll; ailclass11.dll.

MclassAllocResult

Synopsis

Allocate a classification result buffer to hold training, prediction, or statistic results.

Syntax

```
AIL_ID MclassAllocResult(
    AIL_ID SysId,            //in
    AIL_INT64 ResultType,    //in
    AIL_INT64 ControlFlag,   //in
    AIL_ID *ResultClassIdPtr //out
)
```

Description

This function allocates a classification result buffer, on the specified system, to store the results from an MclassTrain(), MclassPredict(), or MclassStatCalculate() operation.

When the classification result buffer is no longer required, release it using MclassFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the classification result buffer's lifetime and you must not manually free it.

Parameters

SysId

Specifies the identifier of the system on which to allocate the result buffer. This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the system identifier																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

ResultType

Specifies the type of classification result buffer to allocate. Set this parameter to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of classification result buffer to allocate																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_PREDICT_ANO_RESULT	Specifies an anomaly detection prediction result buffer. This buffer holds the results produced from calling MclassPredict() with a predefined anomaly detection classifier context (M_CLASSIFIER_ANO_PREDEFINED).	c		e	f	g	a		i		j	k	l	m	n	b
M_PREDICT_CNN_RESULT	Specifies a CNN prediction result buffer. This buffer holds the results produced from calling MclassPredict() with a predefined CNN classifier context (M_CLASSIFIER_CNN_PREDEFINED).	c		e	f	g	a		i		j	k	l	m	n	b
M_PREDICT_DET_RESULT	Specifies an object detection prediction result buffer. This buffer holds the results produced from calling MclassPredict() with a predefined object detection classifier context (M_CLASSIFIER_DET_PREDEFINED).	c		e	f	g	a		i		j	k	l	m	n	b
M_PREDICT_ONNX_RESULT	Specifies an ONNX prediction result buffer. This buffer holds the results produced from calling MclassPredict() with an ONNX classifier context (M_CLASSIFIER_ONNX).	c		e	f	g	a		i		j	k	l	m	n	b
M_PREDICT_SEG_RESULT	Specifies a segmentation prediction result buffer. This buffer holds the results produced from calling MclassPredict() with a predefined segmentation classifier context (M_CLASSIFIER_SEG_PREDEFINED).	c		e	f	g	a		i		j	k	l	m	n	b
M_PREDICT_TREE_ENSEMBLE_RESULT	Specifies a tree ensemble prediction result buffer. This buffer holds the results produced from calling MclassPredict() with a tree ensemble classifier context (M_CLASSIFIER_TREE_ENSEMBLE).	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_ANO_RESULT	Specifies a statistics classification result buffer for anomaly detection. This buffer holds the results produced from calling MclassStatCalculate() with a statistics classification context for anomaly detection (M_STAT_ANO).	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_CNN_RESULT	Specifies a statistics classification result buffer for image classification. This buffer holds the results produced from calling MclassStatCalculate() with a statistics classification context for image classification (M_STAT_CNN).	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_DET_RESULT	Specifies a statistics classification result buffer for object detection. This buffer holds the results produced from calling MclassStatCalculate() with a statistics classification context for object detection (M_STAT_DET).	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_SEG_RESULT	Specifies a statistics classification result buffer for segmentation. This buffer holds the results produced from calling MclassStatCalculate() with a statistics classification context for segmentation (M_STAT_SEG).	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_TREE_ENSEMBLE_RESULT	Specifies a statistics classification result buffer for feature classification. This buffer holds the results produced from calling MclassStatCalculate() with a statistics classification context for feature classification (M_STAT_TREE_ENSEMBLE).	c		e	f	g	a		i		j	k	l	m	n	b

M_TRAIN_ANO_RESULT	Specifies an anomaly training result buffer. This buffer holds the results produced from calling MclassTrain() with an anomaly detection training context (M_TRAIN_ANO).	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAIN_CNN_RESULT	Specifies a CNN training result buffer. This buffer holds the results produced from calling MclassTrain() with a CNN training context (M_TRAIN_CNN).	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAIN_DET_RESULT	Specifies an object detection training result buffer. This buffer holds the results produced from calling MclassTrain() with an object detection training context (M_TRAIN_DET).	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAIN_SEG_RESULT	Specifies a segmentation training result buffer. This buffer holds the results produced from calling MclassTrain() with a segmentation training context (M_TRAIN_SEG).	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAIN_TREE_ENSEMBLE_RESULT	Specifies a tree ensemble training result buffer. This buffer holds the results produced from calling MclassTrain() with a tree ensemble training context (M_TRAIN_TREE_ENSEMBLE).	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

ResultClassIdPtr

Specifies the address of the variable in which to write the classification result buffer identifier or specifies the data type that the function should use to return the result buffer identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated classification result buffer ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated classification result buffer ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_CLASS_ID* is returned instead of a standard Aurora Imaging Library identifier. > This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the classification result buffer (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the prediction result buffer identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated prediction result buffer. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the statistics classification result buffer identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated statistics classification result buffer. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the training result buffer identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated training result buffer. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the classifier result buffer identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_CLASS_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailclass11.lib.
DLL	Requires ail11.dll; ailclass11.dll.

MclassControl

Synopsis

Control a classifier context, dataset context, class definition, data preparation context, training context, training result buffer, or prediction result buffer.

Syntax

```
void MclassControl(
    AIL_ID ContextOrResultClassId, //out
    AIL_INT64 LabelOrIndex,        //in
    AIL_INT64 ControlType,         //in
    AIL_DOUBLE ControlValue        //in
)
```

Description

This function allows you to control a setting of a classifier context (or a class definition therein), a dataset context (or a class definition therein), a data preparation context (for image data), a training context, a training result buffer, or a prediction result buffer. You can typically inquire these settings, using MclassInquire(). These settings can affect the execution of MclassPrepareData(), MclassTrain(), or MclassPredict().

> Note that when using a C compiler (not a C++ or other compiler) in 64-bit mode, **MclassControl()** internally calls the *AIL_DOUBLE* version of this function (MclassControlDouble()). If you need to pass large integer values (including pointers), call the *AIL_INT64* version of this function (MclassControlInt64()).

Parameters

ContextOrResultClassId

Specifies the identifier of a context or result buffer to control. Set this parameter to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the context or result buffer																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ContextClassifierId	Specifies the identifier of a predefined CNN, predefined object detection, predefined segmentation, predefined anomaly detection, tree ensemble, or ONNX classifier context. These contexts must have been previously allocated on the required system using MclassAlloc() with M_CLASSIFIER_CNN_PREDEFINED, M_CLASSIFIER_DET_PREDEFINED, M_CLASSIFIER_SEG_PREDEFINED, M_CLASSIFIER_ANO_PREDEFINED, M_CLASSIFIER_TREE_ENSEMBLE, or M_CLASSIFIER_ONNX.	c		e	f	g	a		i		j	k	l	m	n	b
ContextDataPreparationId	Specifies the identifier of a data preparation context (for image data). This context must have been previously allocated on the required system using MclassAlloc() with M_PREPARE_IMAGES_CNN, M_PREPARE_IMAGES_DET or M_PREPARE_IMAGES_SEG.	c		e	f	g	a		i		j	k	l	m	n	b
ContextDatasetId	Specifies the identifier of an images or features dataset context. These contexts must have been previously allocated on the required system using MclassAlloc() with M_DATASET_IMAGES or M_DATASET_FEATURES.	c		e	f	g	a		i		j	k	l	m	n	b
ContextTrainId	Specifies the identifier of a CNN, object detection, segmentation, anomaly detection, or tree ensemble training context. These contexts must have been previously allocated on the required system using MclassAlloc() with M_TRAIN_CNN, M_TRAIN_DET, M_TRAIN_SEG, M_TRAIN_ANO, or M_TRAIN_TREE_ENSEMBLE.	c		e	f	g	a		i		j	k	l	m	n	b
ResultPredictId	Specifies the identifier of a CNN, object detection, segmentation, anomaly detection, tree ensemble, or ONNX prediction result buffer. These buffers must have been previously allocated on the required system using MclassAllocResult() with M_PREDICT_CNN_RESULT, M_PREDICT_DET_RESULT, M_PREDICT_SEG_RESULT, M_PREDICT_ANO_RESULT, M_PREDICT_TREE_ENSEMBLE_RESULT, or M_PREDICT_ONNX_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b
ResultTrainId	Specifies the identifier of a CNN, object detection, segmentation, anomaly detection, or tree ensemble training result buffer. These buffers must have been previously allocated on the required system using MclassAllocResult() with M_TRAIN_CNN_RESULT, M_TRAIN_DET_RESULT, M_TRAIN_SEG_RESULT, M_TRAIN_ANO_RESULT, or M_TRAIN_TREE_ENSEMBLE_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b

LabelOrIndex

Specifies what to control. Set this parameter to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying what to control																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTHOR_INDEX(AIL_INT AuthorIndex)	Specifies to control an author in a dataset context.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	AuthorIndex Set this parameter to the following:															
	0 <= *Value* < M_NUMBER_OF_AUTHORS — Specifies the index of the author to control. You can inquire the total number of authors, using M_NUMBER_OF_AUTHORS.	c		e	f	g	a		i		j	k	l	m	n	b

M_CLASS_INDEX(AIL_INT ClassIndex)	Specifies to control a class definition in a classifier context or a dataset context.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	ClassIndex Set this parameter to the following:															
	0 <= *Value* < M_NUMBER_OF_CLASSES Specifies the index of the class definition to control. You can inquire the total number of class definition indices, using M_NUMBER_OF_CLASSES.	c		e	f	g	a		i		j	k	l	m	n	b
M_CONTEXT	Specifies to control a global setting of a classifier context, dataset context, or data preparation context (for image data).	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT_SOURCE_LAYER	Specifies to control the input (initial) layer of the classifier in a classification context (for example, the source layer of the ONNX classifier). In this case, you must set the ContextOrResultClassId parameter to the identifier of a predefined CNN, object detection, segmentation, anomaly detection, or an ONNX classifier.	c		e	f	g	a		i		j	k	l	m	n	b
M_GENERAL	Specifies to control a general setting of a classification result buffer.	c		e	f	g	a		i		j	k	l	m	n	b

ControlType

Specifies the type of control to set.

See the Parameter associations section for possible values that can be specified.

ControlValue

Specifies the required value for the control.

See the Parameter associations section for possible values that can be specified.

Parameter associations

The tables below list possible values for the **ControlType** and **ControlValue** parameters.

- For a classifier context (segmentation or ONNX)
- For a classifier context (ONNX)
- For a classifier context (object detection)
- For a classifier context (object detection or anomaly detection)
- For a classifier context (CNN, object detection, segmentation, or anomaly detection)
- For a dataset context (images)
- For a dataset context (images or features)
- For a classifier or dataset context
- For a class definition in a classifier or dataset context (for CNN, object detection, segmentation, anomaly detection, tree ensemble, or ONNX)
- For a classifier context's source layer (ONNX)
- For a data preparation context (images)
- For a data preparation context (images) or a training context (anomaly detection or tree ensemble)
- For a statistics classification, classifier, or training context (CNN, object detection, segmentation, anomaly detection, or tree ensemble)
- For a training context (CNN, object detection, segmentation, anomaly detection)
- For a training context (CNN, object detection or segmentation)
- For a training context (CNN, segmentation, or feature classification)
- For a training context (segmentation)
- For a training context (anomaly detection)
- For a training context (segmentation or tree ensemble)
- For a training context (tree ensemble)
- For a training result buffer (CNN, object detection, segmentation, or tree ensemble)
- For a prediction result buffer (CNN, object detection, segmentation, tree ensemble, or ONNX) or a dataset (images or features)
- For a prediction result buffer (object detection)
- For a prediction or training result buffer (CNN, object detection, segmentation, tree ensemble, or ONNX)
- For statistics classification

To control a global setting of a predefined segmentation or ONNX classifier context, the ControlType and corresponding ControlValue parameter settings can be set to one of the following values, unless otherwise specified. In this case, set the ContextOrResultClassId parameter to the identifier of a predefined segmentation or ONNX classifier context, and set the LabelOrIndex parameter to M_CONTEXT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a classifier context (segmentation or ONNX)																	
ControlType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ControlValue																

M_TARGET_IMAGE_SIZE_X	Sets the X-size of the target image with which to call MclassPredict(), when using CSNet (M_CSNET_...) predefined segmentation classifiers, FCNet (M_FCNET_...) predefined legacy classifiers, or M_CLASSIFIER_ONNX ONNX classifiers. By default, Aurora Imaging Library assumes the target image size is the same as the default source layer size, which you can retrieve, using MclassInquire() with M_SIZE_X and M_SIZE_Y. M_TARGET_IMAGE_SIZE_X and M_TARGET_IMAGE_SIZE_Y must both be set to M_DEFAULT (or 0), or they must both be greater than 0. For CSNets, any target image size is allowed. Typically, cropping is the only valid method for predicting on different sized images. Resized images should not be used for prediction unless your classifier was trained using similarly resized images. For legacy FCNet classifiers, the increment by which you can increase the dimensions of the target image depends on the size of the classifier's source layer and step. The following calculations indicate how to establish these dimensions: • M_TARGET_IMAGE_SIZE_X = M_SIZE_X + *m* * M_STEP_X , where *m* is an integer that is greater than or equal to 0. • M_TARGET_IMAGE_SIZE_Y = M_SIZE_Y + *n* * M_STEP_Y , where *n* is an integer that is greater than or equal to 0. For ONNX classifiers, M_TARGET_IMAGE_SIZE_X must be set to a value greater than 0 if the X-size of the network is defined by the target image X-size; that is, if inquiring M_SIZE_X returns M_DEFINED_BY_TARGET_IMAGE_SIZE_X. Otherwise, M_TARGET_IMAGE_SIZE_X must be set to 0 (M_DEFAULT). INQ													
M_DEFAULT	Specifies the default value; the default value is 0. By default, Aurora Imaging Library assumes the target image size is the same as the default source layer size.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the X-size.	c		e	f	g	a		i		j	k	l	m	n	b
M_TARGET_IMAGE_SIZE_Y	Sets the Y-size of the target image with which to call MclassPredict(), when using CSNet (M_CSNET_...) predefined segmentation classifiers, FCNet (M_FCNET_...) predefined legacy classifiers, or M_CLASSIFIER_ONNX ONNX classifiers. By default, Aurora Imaging Library assumes the target image size is the same as the default source layer size, which you can retrieve, using MclassInquire() with M_SIZE_X and M_SIZE_Y. M_TARGET_IMAGE_SIZE_X and M_TARGET_IMAGE_SIZE_Y must both be set to M_DEFAULT (or 0), or they must both be greater than 0. For CSNets, any target image size is allowed. Typically, cropping is the only valid method for predicting on different sized images. Resized images should not be used for prediction unless your classifier was trained using similarly resized images. For legacy FCNet classifiers, the increment by which you can increase the dimensions of the target image depends on the size of the classifier's source layer and step. The following calculations indicate how to establish these dimensions: • M_TARGET_IMAGE_SIZE_X = M_SIZE_X + *m* * M_STEP_X , where *m* is an integer that is greater than or equal to 0. • M_TARGET_IMAGE_SIZE_Y = M_SIZE_Y + *n* * M_STEP_Y , where *n* is an integer that is greater than or equal to 0. For ONNX classifiers, M_TARGET_IMAGE_SIZE_Y must be set to a value greater than 0 if the Y-size of the network is defined by the target image Y-size; that is, if inquiring M_SIZE_Y returns M_DEFINED_BY_TARGET_IMAGE_SIZE_Y. Otherwise, M_TARGET_IMAGE_SIZE_Y must be set to 0 (M_DEFAULT). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0. By default, Aurora Imaging Library assumes the target image size is the same as the default source layer size.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the Y-size.	c		e	f	g	a		i		j	k	l	m	n	b

To control a global setting of an ONNX classifier context, the ControlType and corresponding ControlValue parameter settings can be set to one of the following values, unless otherwise specified. In this case, set the ContextOrResultClassId parameter to the identifier of an ONNX classifier context, and set the LabelOrIndex parameter to M_CONTEXT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a classifier context (ONNX)																
ControlType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ControlValue																
M_TARGET_IMAGE_SIZE_BAND	Sets the number of bands of the target image with which to call MclassPredict(). M_TARGET_IMAGE_SIZE_BAND must be set to a value greater than 0 if the number of input bands of the network is defined by the target image; that is, if inquiring M_SIZE_BAND returns M_DEFINED_BY_TARGET_IMAGE_SIZE_BAND. Otherwise, M_TARGET_IMAGE_SIZE_BAND must be set to 0 (M_DEFAULT). INQ															
M_DEFAULT	Specifies the default value; the default value is 0. By default, Aurora Imaging Library assumes the number of bands of the target image is the same as the number of bands of the default source layer.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the number of bands.	c		e	f	g	a		i		j	k	l	m	n	b

To control a global setting of a predefined object detection classifier context, the ControlType and corresponding ControlValue parameter settings can be set to one of the following values, unless otherwise specified. In this case, set the ContextOrResultClassId parameter to the identifier of a predefined object detection classifier context, and set the LabelOrIndex parameter to M_CONTEXT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a classifier context (object detection)																
ControlType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ControlValue																

M_NMS_IOU_THRESHOLD	Sets the intersection over union (IOU) threshold value used by the non-maximum suppression (NMS) algorithm to determine if a given box proposal is already covered by an existing box proposal. Aurora Imaging Library proposes many overlapping boxes that must be filtered. Of the two overlapping proposals, the one with the lowest detection score will be discarded. The higher the IOU threshold value, the more similarity is required between the boxes before one is discarded. INQ															
M_DEFAULT	Specifies the default value; the default value is 25.0.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the IOU threshold.	c		e	f	g	a		i		j	k	l	m	n	b
M_NMS_POST_K	Sets the maximum number of detections to keep after applying the non-maximum suppression (NMS) algorithm to the detection results specified by M_NMS_TOP_K. The rest of the detections will be discarded. Note, this must be less than or equal to the top K number of detection results (M_NMS_TOP_K). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 100.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the number of detections to keep.	c		e	f	g	a		i		j	k	l	m	n	b
M_NMS_TOP_K	Sets the top K number of detection results (in terms of score) that will have the non-maximum suppression (NMS) algorithm applied. A lower value reduces the computation complexity of the algorithm. Note, this must be greater than or equal to the maximum number of detections to keep after applying the NMS algorithm (M_NMS_POST_K). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1000.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the number of top detection results.	c		e	f	g	a		i		j	k	l	m	n	b

To control a global setting of a predefined object detection classifier context or a predefined anomaly detection classifier context, the ControlType and corresponding ControlValue parameter settings can be set to one of the following values, unless otherwise specified. In this case, set the ContextOrResultClassId parameter to the identifier of a predefined object detection classifier context or a predefined anomaly detection, and set the LabelOrIndex parameter to M_CONTEXT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a classifier context (object detection or anomaly detection)																
ControlType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ControlValue																
M_SCORE_THRESHOLD	Sets the minimum score threshold for a successful detection result. For object detection, only scores above this threshold are kept. For anomaly detection, images or pixels above this score are considered anomalous. INQ															
M_DEFAULT	Specifies the default value; the default value is 50.0.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the score threshold.	c		e	f	g	a		i		j	k	l	m	n	b

To control a global setting of a classifier context, the ControlType and corresponding ControlValue parameter settings can be set to one of the following values, unless otherwise specified. In this case, set the ContextOrResultClassId parameter to the identifier of a predefined CNN, predefined object detection, predefined segmentation, or predefined anomaly detection classifier context, and set the LabelOrIndex parameter to M_CONTEXT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a classifier context (CNN, object detection, segmentation, or anomaly detection)																
ControlType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ControlValue																
M_PREDICT_ENGINE	Sets the predict engine to use during MclassPredict(). You can get predict engine information by calling MclassInquire() with M_PREDICT_ENGINE_.... This setting does not apply to statistics classification contexts. You must have installed the appropriate Aurora Imaging Library add-ons and updates to perform GPU accelerated prediction using CUDA or OpenVINO.															
M_DEFAULT	Specifies the default predict engine that was set using the **Aurora Imaging Configurator** utility.	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* < M_NUMBER_OF_PREDICT_ENGINES	Specifies the index of the predict engine to use. To inquire a description of the predict engine associated with an index, call MclassInquire() with M_PREDICT_ENGINE_DESCRIPTION.	c		e	f	g	a		i		j	k	l	m	n	b

To control a global setting of a dataset context, the ControlType and corresponding ControlValue parameter settings can be set to the following values. In this case, set the ContextOrResultClassId parameter to the identifier of a dataset context. Unless otherwise specified, the following values apply to an images dataset context and you should set the LabelOrIndex parameter to M_CONTEXT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a dataset context (images)																
ControlType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ControlValue																

M_STOP_PREPARE	Stops the current execution of MclassPrepareData() on a dataset. This can only be done from another thread of higher priority, or from a user-defined classification hook function. If you stop the data preparation, MclassPrepareData() does not return any images in its destination folder. Aurora Imaging Library applies M_STOP_PREPARE on the destination dataset specified with MclassPrepareData(). M_STOP_PREPARE does not apply when you call MclassPrepareData() with images.															
M_DEFAULT	Specifies the default behavior.	c		e	f	g	a		i		j	k	l	m	n	b

To control a global setting of a dataset context, the ControlType and corresponding ControlValue parameter settings can be set to the following values. In this case, set the ContextOrResultClassId parameter to the identifier of a dataset context. Unless otherwise specified, the following values apply to an images or features dataset context and you should set the LabelOrIndex parameter to M_CONTEXT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a dataset context (images or features)

ControlType / ControlValue	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_ACTIVE_AUTHOR_INDEX	Sets the author that is automatically added to labeling operations performed on a dataset context. Aurora Imaging Library uses the current user unless you specify a different one. INQ															
M_DEFAULT	Same as M_CURRENT_USER.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTHOR_INDEX(AIL_INT AuthorIndex)	Specifies the index identifying the author that is automatically defined for each dataset entry. You can only specify the index of an existing author. Use M_AUTHOR_ADD to add a new author.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	AuthorIndex Set this parameter to the following.															
	0 <= *Value* <M_NUMBER_OF_AUTHORS Specifies the index. You can inquire the total number of author indices, using M_NUMBER_OF_AUTHORS.	c		e	f	g	a		i		j	k	l	m	n	b
M_CURRENT_USER	Specifies the system's current username as the author that is automatically defined for each dataset entry. If the system's current user name does not exist in the dataset, it is added at the end. Note that, in this case, authors are considered the same if their respective names are the same (regardless of their index).	c		e	f	g	a		i		j	k	l	m	n	b
M_ACTIVE_AUTHOR_UPDATE	Sets the behavior of the entry's author after a modification was performed on the entry. When disabled, changing an entry will never change its author's name. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ENABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to keep the existing author's information.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies that when an entry's information (an entry's region information) is modified, its author is automatically modified according to M_ACTIVE_AUTHOR_INDEX.	c		e	f	g	a		i		j	k	l	m	n	b
M_ANOMALY_DETECTION_FOLDER	Sets the path at which to save the anomaly detection scores. When working with relative paths, you must set the anomaly detection folder to be under the root path. This value only applies to an images dataset context. INQ	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*///DATASET///*") 1	Specifies the folder path from where the dataset context is restored, using MclassRestore(). If this path does not exist, Aurora Imaging Library uses the path in which your application's executable file is located. Note, if using a C compiler, you must pass the value as M_PTR_TO_AIL_INT(AIL_TEXT("*///DATASET///*")).	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*SegmentationFilePath*") 1	Specifies the anomaly detection file path. For example, you can specify the root path as, *AIL_TEXT("E:\\Manual\AnomalyDetection\Good")*. Note, if using a C compiler, you must pass the value as M_PTR_TO_AIL_INT(AIL_TEXT("*SegmentationFilePath*")).	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTHOR_ADD	Adds a new author to a dataset context, according to the specified name. Added authors are given an index and a key (UUID). Once you add an author, you can refer to it in a dataset's entries.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*NewAuthorName*") 1	Specifies the name of the author to add. Note, if using a C compiler, you must pass the value as M_PTR_TO_AIL_INT(AIL_TEXT("*NewAuthorName*")).	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTHOR_DELETE	Deletes authors from a dataset context, according to the specified index.	c		e	f	g	a		i		j	k	l	m	n	b

M_DEFAULT	Same as M_ALL.	c	e	f	g	a	i	j	k	l	m	n	b
M_AUTHOR_INDEX(AIL_INT Index)	Specifies the authors to delete from the dataset context. Note that you cannot delete the active author specified with M_ACTIVE_AUTHOR_INDEX. **Parameters** Index Set this parameter to the following:	c	e	f	g	a	i	j	k	l	m	n	b
	IndexValue — Specifies the index of the author to delete.	c	e	f	g	a	i	j	k	l	m	n	b
M_ALL	Specifies to delete all explicitly added authors. The active author (M_ACTIVE_AUTHOR_INDEX) is not deleted.	c	e	f	g	a	i	j	k	l	m	n	b
M_AUTHOR_DELETE_BY_KEY	Deletes an author from a dataset context, according to the specified key (UUID).	c	e	f	g	a	i	j	k	l	m	n	b
AIL_UUID Value	Specifies the key of the author to delete. The key is defined as an Aurora Imaging Library universal unique identifier (UUID). To specify an *AIL_UUID* value when using a C compiler, you must call the *AIL_UUID* version of this function (MclassControlAilUuid). When using a C++ or other compiler, **MclassControl**() internally calls the AIL_UUID version of this function. AIL_UUID version of MclassControl void MclassControlAilUuid (AIL_ID ContextOrResultClassId, AIL_INT64 LabelOrIndex, AIL_INT64 ControlType, AIL_UUID ControlValue)	c	e	f	g	a	i	j	k	l	m	n	b
M_AUTHOR_NAME	Sets the name of an author in the dataset context. You must set the LabelOrIndex parameter to M_AUTHOR_INDEX(). INQ	c	e	f	g	a	i	j	k	l	m	n	b
AIL_TEXT("*AuthorName*") 1	Specifies the name of an author. This name replaces the current author's name. Note, if using a C compiler, you must pass the value as M_PTR_TO_AIL_INT(AIL_TEXT("*AuthorName*")).	c	e	f	g	a	i	j	k	l	m	n	b
M_CONSOLIDATE_ENTRIES_INTO_FOLDER	Copies and reorganizes a dataset according to a specified destination directory. The structure of the directory follows the same one as MclassExport() without the icons and CSV files. In general, this means that within the destination folder, images of the dataset will be copied to the *Images* folder by class. If the dataset holds segmentation data, mask regions will be copied in the *Masks* folder and segmentation scores will be copied in the *Segmentation* folder. The paths of all entries, masks, and segmentation files are updated, as well as the *Mask* (M_REGION_MASKS_FOLDER), *Segmentation* (M_SEGMENTATION_FOLDER), and M_ANOMALY_DETECTION_FOLDER folders. Source files are not affected by this control type. It is recommended that you specify an empty destination folder each time you use M_CONSOLIDATE_ENTRIES_INTO_FOLDER. Class folders that already exist in the destination are not recreated. Existing files within that folder that have conflicting names are overwritten. Existing files without conflicting names remain unaffected. Entries without a ground truth are placed in the *Images* folder. Characters that are illegal in Windows file paths are replaced by the underscore character regardless of the operating system used. The maximum size of the file path is 259 characters. If this maximum is exceeded, Aurora Imaging Library attempts to shorten the folder class names. This value only applies to an images dataset context.	c	e	f	g	a	i	j	k	l	m	n	b
AIL_TEXT("*///DATASET///*") 1	Specifies the folder path from where the dataset context is restored, using MclassRestore(). If this path does not exist, Aurora Imaging Library uses the path in which your application's executable file is located. Note, if using a C compiler, you must pass the value as M_PTR_TO_AIL_INT(AIL_TEXT("*///DATASET///*")).	c	e	f	g	a	i	j	k	l	m	n	b
AIL_TEXT("*Path\\ToDstFolder*") 1	Specifies the file path. For example, you can specify "*C:\\ILoveZebra\\Dest*". Note, relative paths are supported; paths must not end with '\\'. Note, if using a C compiler, you must pass the value as M_PTR_TO_AIL_INT(AIL_TEXT("*Path\\ToDstFolder*")).	c	e	f	g	a	i	j	k	l	m	n	b
M_DONT_CARE_CLASS_DRAW_COLOR	Sets the color given for all regions where the value for M_CLASS_INDEX_GROUND_TRUTH has been set to M_DONT_CARE_CLASS. The effect of this value can be seen in MclassDrawEntry() with M_GROUND_TRUTH_IMAGE + M_PSEUDO_COLOR where any pixels assigned to M_DONT_CARE_CLASS will be given this color. The effect of this value can also be seen in MclassEntryAddRegion() with M_GROUND_TRUTH_IMAGE_COLOR where any pixels of this color in the input image buffer will be added as a region with M_CLASS_INDEX_GROUND_TRUTH set to M_DONT_CARE_CLASS. This value only applies to an images dataset context. INQ	c	e	f	g	a	i	j	k	l	m	n	b
M_DEFAULT	Same as M_COLOR_DONT_CARE_CLASS.	c	e	f	g	a	i	j	k	l	m	n	b

M_RGB888(AIL_INT Red, AIL_INT Green, AIL_INT Blue)	Specifies the RGB value to use as the region's color.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	Red Specifies the red band, as a value between 0 and 255.															
	Green Specifies the green band, as a value between 0 and 255.															
	Blue Specifies the blue band, as a value between 0 and 255.															
M_COLOR_BLACK	Specifies the color black.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_BLUE	Specifies the color blue.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_BRIGHT_GRAY	Specifies the color bright gray.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_CYAN	Specifies the color cyan.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_BLUE	Specifies the color dark blue.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_CYAN	Specifies the color dark cyan.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_GREEN	Specifies the color dark green.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_MAGENTA	Specifies the color dark magenta.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_RED	Specifies the color dark red.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_YELLOW	Specifies the color dark yellow.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DONT_CARE_CLASS	Specifies the color (255, 255, 255).	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_GRAY	Specifies the color gray.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_GREEN	Specifies the color green.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_LIGHT_BLUE	Specifies the color light blue.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_LIGHT_GRAY	Specifies the color light gray.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_LIGHT_GREEN	Specifies the color light green.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_LIGHT_WHITE	Specifies the color light white.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_MAGENTA	Specifies the color magenta.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_RED	Specifies the color red.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_WHITE	Specifies the color white.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_YELLOW	Specifies the color yellow.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENTRY_ADD	Adds a new entry to a dataset context. A new entry is added after the last one. For every entry added, a key (*AIL_UUID*) is generated to uniquely identify that entry. For an images dataset, this automatically adds a full frame region that you can refer to using index 0. You can import data, such as entries, from a CSV file to a dataset context, using MclassImport(). You can also use Aurora Imaging CoPilot to interactively create, modify, and export datasets and their entries.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Implements the default behavior.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENTRY_DELETE	Deletes an entry from a dataset context, according to the specified index.	c		e	f	g	a		i		j	k	l	m	n	b

0 <= *Value* <= M_NUMBER_OF_ENTRIES	Specifies the index of the entry to delete. The default value is 0.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENTRY_DELETE_ALL	Deletes all entries in a dataset context.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Implements the default behavior.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENTRY_DELETE_BY_KEY	Deletes an entry from a dataset context, according to the specified key (UUID).	c		e	f	g	a		i		j	k	l	m	n	b
AIL_UUID Value	Specifies the key of the entry to delete. The key is defined as an Aurora Imaging Library universal unique identifier (UUID). To specify an *AIL_UUID* value when using a C compiler, you must call the *AIL_UUID* version of this function (MclassControlAilUuid). When using a C++ or other compiler, **MclassControl**() internally calls the AIL_UUID version of this function. AIL_UUID version of MclassControl void MclassControlAilUuid (AIL_ID ContextOrResultClassId, AIL_INT64 LabelOrIndex, AIL_INT64 ControlType, AIL_UUID ControlValue)	c		e	f	g	a		i		j	k	l	m	n	b
M_MAKE_FILE_PATHS_ABSOLUTE	Makes the file path of every entry in a dataset context into an absolute path. This is done by concatenating the current root path (M_ROOT_PATH value) to the relative path (M_ENTRY_IMAGE_PATH, M_REGION_MASK_PATH, M_SEGMENTATION_PATH, M_ANOMALY_DETECTION_PATH M_REGION_MASKS_FOLDER, M_SEGMENTATION_FOLDER, and M_ANOMALY_DETECTION_FOLDER) of all entries. This results in every entry having a complete path, and the root path (M_ROOT_PATH) being replaced with an empty string (*AIL_TEXT("")*). This value only applies to an images dataset context.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Implements the default behavior.	c		e	f	g	a		i		j	k	l	m	n	b
M_MAKE_FILE_PATHS_RELATIVE	Makes the file path of every entry in a dataset context into a relative path. This is done by parsing the absolute file path (M_ENTRY_IMAGE_PATH_ABS, M_SEGMENTATION_PATH_ABS, M_ANOMALY_DETECTION_PATH_ABS, and M_REGION_MASK_PATH_ABS) of all entries to find a common root path. If a common root path is found, the file paths of all entries are modified to use relative paths, and the root path (M_ROOT_PATH) is set to the newly found common root path. In addition, if M_REGION_MASKS_FOLDER, M_SEGMENTATION_FOLDER, and M_ANOMALY_DETECTION_FOLDER are set, those folders' paths are also involved in the search for the common root path. Note, if MobjInquire() with M_OBJECT_FILE_FOLDER returns a non-empty string value, and the common path has a part that is the same as the object's file folder, Aurora Imaging Library will substitute this value with "///DATASET/". If M_OBJECT_FILE_FOLDER returns an empty string, Aurora Imaging Library uses the common root path that was found. For example, if the file paths of two dataset entries are *A:\work\examples\ProcessingRocks\Classification\Image1.mim* and *A:\work\examples\ProcessingRocks\Classification\Image2.mim*, Aurora Imaging Library would establish the common path as *A:\work\examples\ProcessingRocks\Classification*. In this case, if M_OBJECT_FILE_FOLDER returns *A:\work\examples\ProcessingRocks\Classification*, Aurora Imaging Library would set the root path (M_ROOT_PATH) to *///DATASET///*. However, if M_OBJECT_FILE_FOLDER returns an empty string, Aurora Imaging Library would set the root path to *A:\work\examples\ProcessingRocks\Classification*. This value only applies to an images dataset context.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Implements the default behavior.	c		e	f	g	a		i		j	k	l	m	n	b
M_NO_CLASS_DRAW_COLOR	Sets the color used for pixels that are either not assigned to any region or that you would like to not be assigned to any region. The effect of this value can be seen in MclassDrawEntry() with M_GROUND_TRUTH_IMAGE + M_PSEUDO_COLOR where any unlabeled pixels will be given this color if M_NO_REGION_PIXEL_CLASS is set to M_NO_CLASS. The effect of this value can also be seen in MclassEntryAddRegion() with M_GROUND_TRUTH_IMAGE_COLOR where any pixels of this color in the input image buffer will not be added as a region. This value only applies to an images dataset context. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_COLOR_NO_CLASS.	c		e	f	g	a		i		j	k	l	m	n	b
M_RGB888(AIL_INT Red, AIL_INT Green, AIL_INT Blue)	Specifies the RGB value to use as the region's color.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	Red Specifies the red band, as a value between 0 and 255.															
	Green Specifies the green band, as a value between 0 and 255.															

		Blue Specifies the blue band, as a value between 0 and 255.													
	M_COLOR_BLACK	Specifies the color black.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_BLUE	Specifies the color blue.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_BRIGHT_GRAY	Specifies the color bright gray.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_CYAN	Specifies the color cyan.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_DARK_BLUE	Specifies the color dark blue.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_DARK_CYAN	Specifies the color dark cyan.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_DARK_GREEN	Specifies the color dark green.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_DARK_MAGENTA	Specifies the color dark magenta.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_DARK_RED	Specifies the color dark red.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_DARK_YELLOW	Specifies the color dark yellow.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_GRAY	Specifies the color gray.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_GREEN	Specifies the color green.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_LIGHT_BLUE	Specifies the color light blue.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_LIGHT_GRAY	Specifies the color light gray.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_LIGHT_GREEN	Specifies the color light green.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_LIGHT_WHITE	Specifies the color light white.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_MAGENTA	Specifies the color magenta.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_NO_CLASS	Specifies the color (16, 0, 0).	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_RED	Specifies the color red.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_WHITE	Specifies the color white.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_YELLOW	Specifies the color yellow.	c		e	f	g	a		i		j	k	l	m	n	b
M_NO_REGION_PIXEL_CLASS		Sets the class of any pixels that are not associated with any regions and therefore do not have an assigned value for M_CLASS_INDEX_GROUND_TRUTH. This control lets you determine whether you want to interpret these region-less values as having no class or being a part of a specific class. This control can be useful for interpreting unlabeled pixels as part of a specific class. For example, if you have two classes, one being the object you are interested in and the other being the background class, you have the option of only labeling the objects that you are interested in and then setting the M_NO_REGION_PIXEL_CLASS to be the index of your background class. In this case, during training, all pixels without a region will be interpreted as having the same ground truth index as the background class. M_NO_REGION_PIXEL_CLASS is associated with the UUID of the class at the specified index at the time that it is set. If the corresponding class is later deleted, M_NO_REGION_PIXEL_CLASS will default to M_NO_CLASS. The effect of this value can be seen in MclassDrawEntry() with M_GROUND_TRUTH_IMAGE + M_PSEUDO_COLOR where any pixels that do not have a label will be given the M_CLASS_DRAW_COLOR of the ground truth index specified by value of M_NO_REGION_PIXEL_CLASS. To change the color of the pixels assigned to M_NO_CLASS, use the M_NO_CLASS_DRAW_COLOR control. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_NO_CLASS.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DONT_CARE_CLASS	Specifies that all pixels not assigned to a region will not contribute to training loss.	c		e	f	g	a		i		j	k	l	m	n	b
	M_NO_CLASS	Specifies that no class index is used for pixels not assigned to a region.	c		e	f	g	a		i		j	k	l	m	n	b

0 <= *Value* < M_NUMBER_OF_CLASSES	Specifies the index of the class that is used for pixels not assigned to a region.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_MASKS_FOLDER	Sets the path at which to save the region masks as they are added by MclassEntryAddRegion(). When working with relative paths, you must set the region masks folder to be under the root path. This value only applies to an images dataset context. INQ	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*///DATASET///*") 1	Specifies the folder path from where the dataset context is restored, using MclassRestore(). If this path does not exist, Aurora Imaging Library uses the path in which your application's executable file is located. Note, if using a C compiler, you must pass the value as M_PTR_TO_AIL_INT(AIL_TEXT("*///DATASET///*")).	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*RegionMaskFilePath*") 1	Specifies the region mask file path. For example, you can specify the root path as, *AIL_TEXT("E:\\Manual\RegionMasks\Good")*. Note, if using a C compiler, you must pass the value as M_PTR_TO_AIL_INT(AIL_TEXT("*RegionMaskFilePath*")).	c		e	f	g	a		i		j	k	l	m	n	b
M_ROOT_PATH	Sets the root path that is used for the file path of a dataset's entry images, region masks, and segmentation scores. The file path is set, using MclassControlEntry() with M_ENTRY_IMAGE_PATH, M_REGION_MASK_PATH, M_SEGMENTATION_PATH, and M_ANOMALY_DETECTION_PATH. Note, this is also related to (affects) M_REGION_MASKS_FOLDER, M_SEGMENTATION_FOLDER, and M_ANOMALY_DETECTION_FOLDER. This value only applies to an images dataset context. INQ	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("") 1	Specifies an empty path. This indicates that all entries' file paths are absolute paths. Note, if using a C compiler, you must pass the value as M_PTR_TO_AIL_INT(AIL_TEXT("")).	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*///DATASET///*") 1	Specifies the folder path from where the dataset context is restored, using MclassRestore(). If this path does not exist, Aurora Imaging Library uses the path in which your application's executable file is located. Note, if using a C compiler, you must pass the value as M_PTR_TO_AIL_INT(AIL_TEXT("*///DATASET///*")).	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*RootFilePath*") 1	Specifies the root file path. For example, you can specify the root path as, *AIL_TEXT("E:\\Manual\LabeledIt\Good")*. Note, if using a C compiler, you must pass the value as M_PTR_TO_AIL_INT(AIL_TEXT("*RootFilePath*")).	c		e	f	g	a		i		j	k	l	m	n	b
M_SEGMENTATION_FOLDER	Sets the path at which to save the segmentation scores. When working with relative paths, you must set the segmentation folder to be under the root path. This value only applies to an images dataset context. INQ	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*///DATASET///*") 1	Specifies the folder path from where the dataset context is restored, using MclassRestore(). If this path does not exist, Aurora Imaging Library uses the path in which your application's executable file is located. Note, if using a C compiler, you must pass the value as M_PTR_TO_AIL_INT(AIL_TEXT("*///DATASET///*")).	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*SegmentationFilePath*") 1	Specifies the segmentation file path. For example, you can specify the root path as, *AIL_TEXT("E:\\Manual\Segmentation\Good")*. Note, if using a C compiler, you must pass the value as M_PTR_TO_AIL_INT(AIL_TEXT("*SegmentationFilePath*")).	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

To control a classifier or dataset context, set the ControlType parameter to one of the values below. In this case, set the ContextOrResultClassId parameter to the identifier of a classifier or dataset context, and set the LabelOrIndex parameter to M_CONTEXT, unless otherwise specified.

Note, these values apply to a classifier context or an images or a features dataset context, unless otherwise specified.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a classifier or dataset context																
ControlType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ControlValue																
M_CLASS_ADD	Adds a new class definition to a classifier or dataset context, with the specified name. Aurora Imaging Library assigns an index to the new class definition such that the index is 1 greater than the highest existing class definition index.															
M_NULL	Specifies to add a new class definition with no explicitly defined name. Aurora Imaging Library defines a default name, in a "Class ##" format, where "##" is the index value.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*NewClassName*") 1	Specifies the name of the class definition to add. Note, if using a C compiler, you must pass the value as M_PTR_TO_AIL_INT(AIL_TEXT("*NewClassName*")).	c		e	f	g	a		i		j	k	l	m	n	b

M_CLASS_DELETE	Deletes a class definition from a classifier or dataset context, according to the specified index. If an entry in a dataset context references this deleted class definition, that reference is also deleted. Dataset entries without class definitions will cause an error when you call MclassTrain().	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ALL.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLASS_INDEX(AIL_INT ClassIndex)	Specifies a class definition to delete. If a region (other than region 0) refers to the specified class to delete, that region is removed.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	ClassIndex Set this parameter to the following:															
	0 <= *Value* <M_NUMBER_OF_CLASSES Specifies the index. You can inquire the total number of class definition indices, using M_NUMBER_OF_CLASSES.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL	Specifies to delete all class definitions. All regions other than region 0 will be removed.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLASS_DELETE_BY_KEY	Deletes a class definition from a classifier or dataset context, according to the specified key (UUID). If an entry in a dataset context references this deleted class definition, that reference is also deleted. Dataset entries without class definitions will cause an error when you call MclassTrain().	c		e	f	g	a		i		j	k	l	m	n	b
AIL_UUID Value	Specifies the key of the class definition to delete. The key is defined as an Aurora Imaging Library universal unique identifier (UUID). To specify an *AIL_UUID* value when using a C compiler, you must call the *AIL_UUID* version of this function (MclassControlAilUuid). When using a C++ or other compiler, **MclassControl**() internally calls the AIL_UUID version of this function. If a region (other than region 0) is referred to the specified class, the region is removed. AIL_UUID version of MclassControl void MclassControlAilUuid (AIL_ID ContextOrResultClassId, AIL_INT64 LabelOrIndex, AIL_INT64 ControlType, AIL_UUID ControlValue)	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

To control a class definition, the ControlType and corresponding ControlValue parameter settings can be set to the following values. Unless otherwise specified, set the ContextOrResultClassId parameter to the identifier of a classifier context (predefined CNN, object detection, predefined segmentation, predefined anomaly detection, tree ensemble, or ONNX) or a dataset context (images or features), and set the LabelOrIndex parameter to M_CLASS_INDEX().

Note, class definitions are added to a dataset context, using M_CLASS_ADD.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a class definition in a classifier or dataset context (for CNN, object detection, segmentation, anomaly detection, tree ensemble, or ONNX)

ControlType / ControlValue	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_ANOMALOUS	Sets whether the specified class is considered anomalous. Note, the ContextOrResultClassId parameter must be the identifier of an anomaly detection classifier context or images dataset used for anomaly detection. INQ															
M_DEFAULT	Same as M_FALSE.	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that images containing this class, either at the image or pixel level, will not be considered anomalous unless it also contains a class with M_ANOMALOUS set to M_TRUE.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that images containing this class, either at the image or pixel level, will be considered anomalous.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLASS_DRAW_COLOR	Sets the color assigned to a class definition. This color is used as a visual label and helps to identify class definitions. Draw operations use the class definition color. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_RGB888(AIL_INT Red, AIL_INT Green, AIL_INT Blue)	Specifies the RGB value to use as the region's color.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	Red Specifies the red band, as a value between 0 and 255.															

		Green Specifies the green band, as a value between 0 and 255.														
		Blue Specifies the blue band, as a value between 0 and 255.														
	M_COLOR_BLACK	Specifies the color black.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_BLUE	Specifies the color blue.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_BRIGHT_GRAY	Specifies the color bright gray.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_CYAN	Specifies the color cyan.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_DARK_BLUE	Specifies the color dark blue.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_DARK_CYAN	Specifies the color dark cyan.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_DARK_GREEN	Specifies the color dark green.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_DARK_MAGENTA	Specifies the color dark magenta.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_DARK_RED	Specifies the color dark red.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_DARK_YELLOW	Specifies the color dark yellow.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_GRAY	Specifies the color gray.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_GREEN	Specifies the color green.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_LIGHT_BLUE	Specifies the color light blue.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_LIGHT_GRAY	Specifies the color light gray.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_LIGHT_GREEN	Specifies the color light green.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_LIGHT_WHITE	Specifies the color light white.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_MAGENTA	Specifies the color magenta.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_RED	Specifies the color red.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_WHITE	Specifies the color white.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_YELLOW	Specifies the color yellow.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLASS_ICON_ID		Sets the class definition's sample image. This image helps visually identify the class definition. This does not affect calculations. When you call **MclassControl**() with M_CLASS_ICON_ID, Aurora Imaging Library copies the image to the context and assigns it a new identifier. Aurora Imaging Library returns this new identifier, if you inquire M_CLASS_ICON_ID. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_NULL	Specifies that the image associated with the class definition is freed. Note that if the specified class definition does not have an associated image, this setting has no effect.	c		e	f	g	a		i		j	k	l	m	n	b
	Image identifier	Specifies the identifier of the image buffer to use for the class definition's sample image.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLASS_NAME		Sets the name of a class definition that is currently in the dataset context. This name replaces the current class definition's name. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	AIL_TEXT("*ClassDefinitionName*") 1	Specifies the name of a class definition. > Note, if using a C compiler, you must pass the value as M_PTR_TO_AIL_INT(AIL_TEXT("*ClassDefinitionName*")).	c		e	f	g	a		i		j	k	l	m	n	b

M_CLASS_WEIGHT	Sets the class definition's weight. In this case, you must set the ContextOrResultClassId parameter to the identifier of a features dataset context, an images dataset context being used for segmentation, or a tree ensemble classifier context. The specified weight can only have an effect if M_CLASS_WEIGHT_MODE is set to M_USER_DEFINED. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the weight.	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

To control the source layer of the classifier in an ONNX classifier context, the ControlType parameter can be set to one of the following values. In this case, set the LabelOrIndex parameter to M_DEFAULT_SOURCE_LAYER.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a classifier context's source layer (ONNX)																
ControlType ControlValue	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_BAND_ORDER	Sets the order of the input bands of the network. This corresponds to the order in which the network interprets the bands of the images that were used in the training phase. If the bands in the target image (MclassPredict()) do not follow this order, Aurora Imaging Library internally compensates by temporarily copying the bands to be ordered as the network expects.															
M_DEFAULT	Same as M_RGB.	c		e	f	g	a		i		j	k	l	m	n	b
M_BGR	Specifies that the bands are ordered according to the BGR (Blue, Green, Red) convention.	c		e	f	g	a		i		j	k	l	m	n	b
M_RGB	Specifies that the bands are ordered according to the RGB (Red, Green, Blue) convention.	c		e	f	g	a		i		j	k	l	m	n	b

To control a global setting of a data preparation context (for image data), the ControlType and corresponding ControlValue parameter settings can be set to one of the following values. In this case, set the ContextOrResultClassId parameter to the identifier of a data preparation context, and set the LabelOrIndex parameter to M_CONTEXT. These values affect the execution of MclassPrepareData().

Data preparation values related to augmentation (M_AUGMENT_...) only have an effect if augmentations are enabled. This can be done by calling **MclassControl**() and enabling preset augmentations (M_PRESET...). You can also enable other types of augmentation operations (M_AUG_..._OP) by calling MimControl() with the identifier of the data preparation context's internal augmentation context, which you can inquire using M_AUGMENT_CONTEXT_ID.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a data preparation context (images)																
ControlType ControlValue	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_AUGMENT_BALANCING	Sets a value that represents the proportion of augmented entries that will go towards class balancing. INQ															
M_DEFAULT	Specifies the default value; the default value is 80.0.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the proportion of augmented entries that will go towards class balancing. For example, if this is set to 80.0, then 80 percent of the augmented entries specified by M_AUGMENT_NUMBER_ABSOLUTE will go towards balancing the dataset whereas the last 20 percent (mainly for image classification) will go evenly to all classes such that they can all have some augmentation. M_AUGMENT_BALANCING only applies when calling MclassPrepareData() with datasets. If you call MclassPrepareData() with image buffers, M_AUGMENT_BALANCING is ignored.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUGMENT_NUMBER_ABSOLUTE	Sets the number of augmented entries that Aurora Imaging Library should add to the destination dataset. M_AUGMENT_NUMBER_ABSOLUTE only applies when calling MclassPrepareData() with datasets. If you call MclassPrepareData() with image buffers, M_AUGMENT_NUMBER_ABSOLUTE is ignored. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the number of augmented entries that should be added to the destination dataset. If you do not specify any augmentations, either with M_PRESET_... or with the internal augmentation context (M_AUGMENT_CONTEXT_ID), no images are added to the destination dataset. The actual number of augmented entries added to the destination dataset will be less than the augment number if there are any problems with the images in the source dataset. To check that all images were augmented properly you can check the M_STATUS hook info in the M_PREPARE_ENTRY_POST hook.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUGMENT_NUMBER_FACTOR	Sets the ratio with which to establish the number of augmentations to add to the destination dataset. M_AUGMENT_NUMBER_FACTOR only applies when calling MclassPrepareData() with datasets. If you call MclassPrepareData() with image buffers, M_AUGMENT_NUMBER_FACTOR is ignored. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 3.0. If you do not specify any augmentations, either with M_PRESET_... or with the internal augmentation context (M_AUGMENT_CONTEXT_ID), no images are added to the destination dataset.	c		e	f	g	a		i		j	k	l	m	n	b

Value >= 0.0	Specifies the ratio. Aurora Imaging Library applies the ratio value to the source dataset to establish the number of augmentations in the destination dataset. For example, setting M_AUGMENT_NUMBER_FACTOR to 3.0 indicates a 3:1 ratio between augmented images (destination dataset) and original images (source dataset). Specifying 0.0 results in no augmented entries being added.	c	e	f	g	a	i	j	k	l	m	n	b
M_AUGMENT_NUMBER_MODE	Sets the mode with which to establish the number of augmentations that should be added to the destination dataset when calling MclassPrepareData(). INQ	c	e	f	g	a	i	j	k	l	m	n	b
M_DEFAULT	Same as M_AUTO.	c	e	f	g	a	i	j	k	l	m	n	b
M_ABSOLUTE	Specifies that the number of augmentations is established with M_AUGMENT_NUMBER_ABSOLUTE.	c	e	f	g	a	i	j	k	l	m	n	b
M_AUTO	Specifies that Aurora Imaging Library automatically establishes the number of augmentations. When augmenting a dataset, M_AUTO is the same as M_FACTOR. When augmenting an image buffer, M_AUTO is the same as M_DISABLE.	c	e	f	g	a	i	j	k	l	m	n	b
M_DISABLE	Specifies that no augmentations are performed. This does not affect other data preparations (these are still performed).	c	e	f	g	a	i	j	k	l	m	n	b
M_FACTOR	Specifies that the number of augmentations is established with M_AUGMENT_NUMBER_FACTOR.	c	e	f	g	a	i	j	k	l	m	n	b
M_CENTER_MODE	Sets the mode with which to establish the center of the destination image. INQ	c	e	f	g	a	i	j	k	l	m	n	b
M_DEFAULT	Same as M_AUTO.	c	e	f	g	a	i	j	k	l	m	n	b
M_AUTO	Specifies that the center of the destination image is taken to be the center of the source image. If a hook-handler function was called due to an M_PREPARE_ENTRY_PRE event, the user-defined centers will be over written by the centers specified by MclassSetHookInfo() with M_CENTER_X and M_CENTER_Y.	c	e	f	g	a	i	j	k	l	m	n	b
M_USER_DEFINED	Specifies the center for cropping is taken to be the user-defined M_CENTER_X and M_CENTER_Y. If a hook-handler function was called due to an M_PREPARE_ENTRY_PRE event, the user-defined centers will be over written by the centers specified by MclassSetHookInfo() with M_CENTER_X and M_CENTER_Y. This mode requires that the M_CENTER_X and M_CENTER_Y be selected such that in combination with the size the cropped image does not exceed the boundaries of the source image.	c	e	f	g	a	i	j	k	l	m	n	b
M_CENTER_X	Sets the X-coordinate of the center point used by M_CENTER_MODE. M_CENTER_X only has an effect if M_CENTER_MODE is set to M_USER_DEFINED. INQ	c	e	f	g	a	i	j	k	l	m	n	b
Value >= 0.0	Specifies the X-coordinate.	c	e	f	g	a	i	j	k	l	m	n	b
M_CENTER_Y	Sets the Y-coordinate of the center point used by M_CENTER_MODE. M_CENTER_Y only has an effect if M_CENTER_MODE is set to M_USER_DEFINED. INQ	c	e	f	g	a	i	j	k	l	m	n	b
Value >= 0.0	Specifies the Y-coordinate.	c	e	f	g	a	i	j	k	l	m	n	b
M_DESTINATION_FOLDER_MODE	Sets the mode Aurora Imaging Library uses to manage the destination folder in which it saves the prepared images. INQ	c	e	f	g	a	i	j	k	l	m	n	b
M_DEFAULT	Same as M_CREATE.	c	e	f	g	a	i	j	k	l	m	n	b
M_CREATE	Specifies that the data preparation process creates a unique *prepare* destination folder. Each time you call MclassPrepareData(), prepared images are written to this unique folder. Previously created folders are not overwritten. Aurora Imaging Library creates the unique *prepare* folders under the folder specified with M_PREPARED_DATA_FOLDER; for example, *M_PREPARED_DATA_FOLDER/prepare1*, *M_PREPARED_DATA_FOLDER/prepare2*, *M_PREPARED_DATA_FOLDER/prepare...*, *M_PREPARED_DATA_FOLDER/preparex*. Note, Aurora Imaging Library uses the first available folder by following this hierarchy.	c	e	f	g	a	i	j	k	l	m	n	b
M_OVERWRITE	Specifies that the data preparation process uses the same *prepare* destination folder. Each time you call MclassPrepareData(), prepared images are written to this folder. Aurora Imaging Library creates the *prepare* folder under the folder specified with M_PREPARED_DATA_FOLDER; that is, *M_PREPARED_DATA_FOLDER/prepare*. When using overwrite mode, you cannot make concurrent calls to MclassPrepareData().	c	e	f	g	a	i	j	k	l	m	n	b
M_PREPARE_ORIGINAL_IMAGES	Sets the mode with which to add source entries to the destination dataset, when cropping/resizing images with MclassPrepareData(). By default, no cropping or resizing is done; in this case, MclassPrepareData() adds (copies) the original (source) images as-is. INQ	c	e	f	g	a	i	j	k	l	m	n	b
M_DEFAULT	Same as M_AUTO.	c	e	f	g	a	i	j	k	l	m	n	b

M_AUTO	Specifies that the source dataset entries, for cropped/resized images, are added to the destination dataset, if they were not previously added. In this case, multiple calls with the same destination dataset will result in the cropped/resized images only being present in the destination dataset once.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies that the source dataset entries, for cropped/resized images, are not added to the destination dataset.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies that the source entries, for cropped/resized images, are added to the destination dataset, regardless of whether they were previously added. This can result in repeated entries/images.	c		e	f	g	a		i		j	k	l	m	n	b
M_PREPARED_DATA_FOLDER	Sets the path to the destination folder in which to save the prepared dataset. INQ	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("") 1	Specifies an empty path. This implies the destination folder is the default destination folder (*C:\Users\Public\Documents\Classification* on Windows or */var/lib/aurora_imaging/Classification* in the user's home directory on Linux). Note, if using a C compiler, you must pass the value as M_PTR_TO_AIL_INT(AIL_TEXT("")).	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*DestinationFolder*") 1	Specifies the destination folder where the prepared dataset will be saved. Note, if using a C compiler, you must pass the value as M_PTR_TO_AIL_INT(AIL_TEXT("*DestinationFolder*")).	c		e	f	g	a		i		j	k	l	m	n	b
M_PRESET_ASPECT_RATIO	Sets whether to enable the aspect ratio augmentation operation. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable the aspect ratio augmentation operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable the aspect ratio augmentation operation with preset values.	c		e	f	g	a		i		j	k	l	m	n	b
M_PRESET_CROP	Sets whether to enable the crop augmentation operation. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable the crop augmentation operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable the crop augmentation operation with preset values.	c		e	f	g	a		i		j	k	l	m	n	b
M_PRESET_FLIP	Sets whether to enable the flip augmentation operation. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable the flip augmentation operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable the flip augmentation operation with preset values.	c		e	f	g	a		i		j	k	l	m	n	b
M_PRESET_GAMMA	Sets whether to enable the gamma augmentation operation. This operation is not supported for binary (1-bit) and grayscale (1-band) images. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable the gamma augmentation operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable the gamma augmentation operation with preset values.	c		e	f	g	a		i		j	k	l	m	n	b
M_PRESET_HUE_OFFSET	Sets whether to enable the hue offset augmentation operation. This operation is not supported for binary (1-bit) and grayscale (1-band) images. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable the hue offset augmentation operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable the hue offset augmentation operation with preset values.	c		e	f	g	a		i		j	k	l	m	n	b
M_PRESET_INTENSITY_ADD	Sets whether to enable the intensity add augmentation operation. This operation is not supported for binary (1-bit) images. INQ	c		e	f	g	a		i		j	k	l	m	n	b

M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable the intensity add augmentation operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable the intensity add augmentation operation with preset values.	c		e	f	g	a		i		j	k	l	m	n	b
M_PRESET_INTENSITY_MULTIPLY	Sets whether to enable the intensity multiply augmentation operation. This operation is not supported for binary (1-bit) images. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable the intensity multiply augmentation operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable the intensity multiply augmentation operation with preset values.	c		e	f	g	a		i		j	k	l	m	n	b
M_PRESET_LIGHTING_DIRECTIONAL	Sets whether to enable the directional lighting augmentation operation. Modifying an image's directional lighting can be seen as a type of add-ramp operation. This operation is not supported for binary (1-bit) images. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable the directional lighting augmentation operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable the directional lighting augmentation operation with preset values.	c		e	f	g	a		i		j	k	l	m	n	b
M_PRESET_NOISE_GAUSSIAN_ADDITIVE	Sets whether to enable the Gaussian additive noise augmentation operation. This operation is not supported for binary (1-bit) images. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable the Gaussian additive noise augmentation operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable the Gaussian additive noise augmentation operation with preset values.	c		e	f	g	a		i		j	k	l	m	n	b
M_PRESET_NOISE_SALT_PEPPER	Sets whether to enable the salt and pepper augmentation operation. This operation is not supported for binary (1-bit) images. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable the salt and pepper noise augmentation operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable the salt and pepper noise augmentation operation with preset values.	c		e	f	g	a		i		j	k	l	m	n	b
M_PRESET_REDUCE	Sets whether to enable the reduce augmentation operation. The reduce operation creates an image with the mean values and puts the reduced image at a random location with the regions translated accordingly. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable the reduce augmentation operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable the reduce augmentation operation with preset values.	c		e	f	g	a		i		j	k	l	m	n	b
M_PRESET_ROTATION	Sets whether to enable the rotation augmentation operation. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable the rotation augmentation operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable the rotation augmentation operation with preset values.	c		e	f	g	a		i		j	k	l	m	n	b
M_PRESET_SATURATION_GAIN	Sets whether to enable the saturation gain augmentation operation. This operation is not supported for binary (1-bit) and grayscale (1-band) images. INQ	c		e	f	g	a		i		j	k	l	m	n	b

M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable the saturation gain augmentation operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable the saturation gain augmentation operation with preset values.	c		e	f	g	a		i		j	k	l	m	n	b
M_PRESET_SCALE	Sets whether to enable the scale augmentation operation. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable the scale augmentation operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable the scale augmentation operation with preset values.	c		e	f	g	a		i		j	k	l	m	n	b
M_PRESET_SMOOTH_GAUSSIAN	Sets whether to enable the smooth Gaussian augmentation operation. This operation is not supported for binary (1-bit) images. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable the smooth Gaussian augmentation operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable the smooth Gaussian augmentation operation with preset values.	c		e	f	g	a		i		j	k	l	m	n	b
M_PRESET_TRANSLATION	Sets whether to enable the translation augmentation operation. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable the translation augmentation operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable the translation augmentation operation with preset values.	c		e	f	g	a		i		j	k	l	m	n	b
M_RESIZE_SCALE_FACTOR	Sets the resize scale factor to apply between the contents of the source and destination images. While the control affects the contents of an image by cropping or adding black borders, it does not affect the size x/y of the destination image. Black borders will be ignored during training. For example, a resize scale factor of 0.5 will result in the contents of the destination images being half the size of the source images, with the remainder of the destination image filled with a black border. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_FILL_DESTINATION_BILINEAR	Specifies to make all entries scale to fill the destination image size x/y determined by M_SIZE_MODE, M_SIZE_X, M_SIZE_Y and classifier passed in the MclassPrepareData() call using bilinear interpolation. This mode is meant for the case where you would like to resize all images to a common size x/y without cropping or adding black borders. When in this mode, M_CENTER_MODE, M_CENTER_X, and M_CENTER_Y are all ignored and not used even if they are set.	c		e	f	g	a		i		j	k	l	m	n	b
M_FILL_DESTINATION_NEAREST_NEIGHBOR	Specifies to make all entries scale to fill the destination image size x/y determined by M_SIZE_MODE, M_SIZE_X, M_SIZE_Y and classifier passed in the MclassPrepareData() call using nearest neighbor interpolation. This mode is meant for the case where you would like to resize all images to a common size x/y without cropping or adding black borders. When in this mode, M_CENTER_MODE, M_CENTER_X, and M_CENTER_Y are all ignored and not used even if they are set.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 < *Value* <= 16.0	Specifies the factor by which to resize. If the factor is set to 1.0 (default), there is no resizing.	c		e	f	g	a		i		j	k	l	m	n	b
M_SIZE_MODE	Sets the mode with which to establish the size of the destination image. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_AUTO.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO	Specifies that the size of the destination image will be the same as the source image if no classifier was given. If a trained classifier is given, then the size of the destination image will be the same as the source layer. If an untrained classifier is given, then the size of the destination image will be the same as the source image size. Note, if you are using a legacy FCNet classifier, and it is untrained, it will try to adapt the source image size to the nearest classifier formula size. When selecting a size based on the classifier formula, the one selected will be the nearest one when rounding down. In this case, the size of the destination will always be less than or equal to the size of the source image and never larger.	c		e	f	g	a		i		j	k	l	m	n	b
M_USER_DEFINED	Specifies the size of the destination image will be the user-defined M_SIZE_X and M_SIZE_Y. Both M_SIZE_X and M_SIZE_Y must be specified in this mode. When M_SIZE_MODE is set to M_USER_DEFINED, the classifier (ClassifierContextClassId) in MclassPrepareData() must not be specified.	c		e	f	g	a		i		j	k	l	m	n	b

M_SIZE_X	Sets the width for the destination image. To use this control, M_SIZE_MODE must be set to M_USER_DEFINED and the classifier (ClassifierContextClassId) in MclassPrepareData() must not be specified. INQ	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the width.	c		e	f	g	a		i		j	k	l	m	n	b
M_SIZE_Y	Sets the height for the destination image. To use this control, M_SIZE_MODE must be set to M_USER_DEFINED and the classifier (ClassifierContextClassId) in MclassPrepareData() must not be specified. INQ	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the height.	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

To control a global setting of a data preparation context (for image data) or a training context (anomaly detection or tree ensemble), the ControlType and corresponding ControlValue parameter settings can be set to one of the following values. Set the ContextOrResultClassId parameter to the identifier of a data preparation context (for image data) or the identifier of a training context (anomaly detection or tree ensemble), and set the LabelOrIndex parameter to M_CONTEXT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a data preparation context (images) or a training context (anomaly detection or tree ensemble)																	
ControlType / ControlValue	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_SEED_MODE	Sets the mode with which to initialize the seed used for anomaly detection and tree ensemble training or image data augmentation. Aurora Imaging Library uses the seed value to generate the random numbers needed to train an anomaly detection context or a tree ensemble. Specifically, Aurora Imaging Library uses randomness to: INQ • Select the entries for each training set (if you do not disable M_BOOTSTRAP). • Select the features to consider when splitting a node. To control the maximum number of features considered, use M_NODE_MAX_NUMBER_OF_FEATURES_MODE and M_NODE_MAX_NUMBER_OF_FEATURES. • To shuffle the values of a feature for all entries of the development set (or out-of-bag set), when calculating the feature importance (M_FEATURE_IMPORTANCE_MODE) using permutation (M_PERMUTATION).															
M_DEFAULT	Specifies the default mode with which to initialize the seed. Same as M_USER_DEFINED for training contexts. Same as M_AUTO for data preparation contexts.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO	Specifies that the seed is chosen at random, and that Aurora Imaging Library generates and uses a new seed each time you call MclassTrain(). To retrieve the used seed, call MclassGetResult() with M_GENERAL and M_SEED_VALUE. If you retrain with the same seed, you can reproduce the same training (Aurora Imaging Library uses the same random values).	c		e	f	g	a		i		j	k	l	m	n	b
M_USER_DEFINED	Specifies the user-defined seed set with M_SEED_VALUE.	c		e	f	g	a		i		j	k	l	m	n	b
M_SEED_VALUE	Sets the seed value that Aurora Imaging Library uses to generate random numbers. These random numbers are used to train a tree ensemble or to generate the random augmentations required by image data preparation. Aurora Imaging Library ignores this explicitly specified seed value if M_SEED_MODE is set to M_AUTO. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 1	Specifies the seed value.	c		e	f	g	a		i		j	k	l	m	n	b

To control a global setting of a statistics classification context, a classifier context, or a training context, the ControlType and corresponding ControlValue parameter settings can be set to one of the following values, unless otherwise specified. In this case, set the ContextOrResultClassId parameter to the identifier of a statistics classification context, a predefined CNN, predefined object detection, predefined segmentation, predefined anomaly detection, or tree ensemble training or classifier context, and set the LabelOrIndex parameter to M_CONTEXT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a statistics classification, classifier, or training context (CNN, object detection, segmentation, anomaly detection, or tree ensemble)																	
ControlType / ControlValue	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_TIMEOUT	Sets the maximum calculation time, in msec. If you specify a training context, the timeout applies to MclassTrain(). If you specify a classifier context, the timeout applies to MclassPredict(). If you specify a statistics classification context, the timeout applies to MclassStatCalculate(). INQ																
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies an infinite amount of time.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the maximum time, in msec.	c		e	f	g	a		i		j	k	l	m	n	b

To control a global setting of a CNN, object detection, segmentation, or anomaly detection training context, the ControlType and corresponding ControlValue parameter settings can be set to one of the following values. In this case, set the ContextOrResultClassId parameter to the identifier of a CNN, object detection, segmentation, or anomaly detection

training context, and set the LabelOrIndex parameter to M_CONTEXT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a training context (CNN, object detection, segmentation, anomaly detection)																	
ControlType / ControlValue	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_MINI_BATCH_SIZE	Sets the size of each mini-batch. Aurora Imaging Library uses mini-batches to split the training dataset to help make the training process more efficient. You can automatically modify this setting, if you modify M_RESET_TRAINING_VALUES. INQ																
M_DEFAULT	Specifies the default value. For image classification (M_TRAIN_CNN), the default is 32. For segmentation (M_TRAIN_SEG) and object detection (M_TRAIN_DET), the default is 4. For anomaly detection (M_TRAIN_ANO), the default is 1.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the size.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAIN_ENGINE	Sets the engine (processing unit) that Aurora Imaging Library uses to perform the training process. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_AUTO.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO	Specifies the engine automatically. Typically, Aurora Imaging Library uses the best available GPU.	c		e	f	g	a		i		j	k	l	m	n	b
M_CPU	Specifies the CPU.	c		e	f	g	a		i		j	k	l	m	n	b
M_GPU	Specifies the GPU. You must have installed the appropriate Aurora Imaging Library add-ons and updates to train on your GPU.	c		e	f	g	a		i		j	k	l	m	n	b

To control a global setting of a CNN, object detection or segmentation training context, the ControlType and corresponding ControlValue parameter settings can be set to one of the following values. In this case, set the ContextOrResultClassId parameter to the identifier of a CNN, object detection or segmentation training context, and set the LabelOrIndex parameter to M_CONTEXT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a training context (CNN, object detection or segmentation)																	
ControlType / ControlValue	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FOCAL_LOSS_GAMMA	Sets a focusing value for the focal loss when training. This only has an effect when M_LOSS_FUNCTION_TYPE is set to M_FOCAL_LOSS. This is only available for classification and segmentation training contexts (M_TRAIN_CNN and M_TRAIN_SEG). INQ																
M_DEFAULT	Specifies the default value; the default value is 2.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the focusing value. Greater values reduce the relative loss for well-classified images while putting more focus on hard to classify images.	c		e	f	g	a		i		j	k	l	m	n	b
M_INITIAL_LEARNING_RATE	Sets the initial learning rate for training. The learning rate determines the degree to which the classifier's weights are modified at each mini-batch during training. You can automatically modify this setting, if you modify M_RESET_TRAINING_VALUES. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value. For image classification (M_TRAIN_CNN), the default is 0.005. For segmentation (M_TRAIN_SEG) and object detection (M_TRAIN_DET), the default is 0.001.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the learning rate.	c		e	f	g	a		i		j	k	l	m	n	b
M_LEARNING_RATE_DECAY	Sets the learning rate decay for training. The learning rate decay determines how fast the learning rate decreases during training. This can help prevent excessive weights. You can automatically modify this setting, if you modify M_RESET_TRAINING_VALUES. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value. For image classification (M_TRAIN_CNN), the default is 0.1. For segmentation (M_TRAIN_SEG) and object detection (M_TRAIN_DET), the default is 0.05.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the learning rate decay.	c		e	f	g	a		i		j	k	l	m	n	b
M_LOSS_FUNCTION_TYPE	Sets the type of loss function for training. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_AUTO.	c		e	f	g	a		i		j	k	l	m	n	b

M_AUTO	Specifies that Aurora Imaging Library automatically establishes the type of loss function to use. For image classification, Aurora Imaging Library uses M_CROSS_ENTROPY. For object detection, Aurora Imaging Library uses M_DET_LOSS. For segmentation, Aurora Imaging Library uses M_FOCAL_LOSS.	c		e	f	g	a		i		j	k	l	m	n	b
M_CROSS_ENTROPY	Specifies a cross entropy type of loss. This is only available for classification and segmentation training contexts (M_TRAIN_CNN and M_TRAIN_SEG).	c		e	f	g	a		i		j	k	l	m	n	b
M_DET_LOSS	Specifies a detection type of loss. This is only available for object detection training contexts (M_TRAIN_DET).	c		e	f	g	a		i		j	k	l	m	n	b
M_DICE_LOSS	Specifies a dice type of loss. This is only available for segmentation training contexts (M_TRAIN_SEG).	c		e	f	g	a		i		j	k	l	m	n	b
M_FOCAL_LOSS	Specifies a focal type of loss. To set a focusing value for the focal loss, use M_FOCAL_LOSS_GAMMA. This is only available for classification and segmentation training contexts (M_TRAIN_CNN and M_TRAIN_SEG).	c		e	f	g	a		i		j	k	l	m	n	b
M_MAX_EPOCH	Sets the maximum number of epochs. An epoch refers to one complete cycle through the data that the classifier must learn during training. You can automatically modify this setting, if you modify M_RESET_TRAINING_VALUES. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 60.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the maximum number.	c		e	f	g	a		i		j	k	l	m	n	b
M_RESET_TRAINING_VALUES	Sets the value of all training mode controls (hyperparameters) automatically, according to the specified training mode. Note, only a complete training is performed for object detection and anomaly detection. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_COMPLETE.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPLETE	Specifies a complete training mode. Typically, this is for completely restarting the training of a CNN, object detection or segmentation classifier context, or for training a CNN, object detection or segmentation classifier context that is not trained. A compete training is also performed for anomaly detection, though this value cannot be explicitly specified. The training mode controls, and what M_COMPLETE automatically sets them to, are listed below. Unless otherwise specified, values apply for image classification (M_TRAIN_CNN), object detection (M_TRAIN_DET), anomaly detection (M_TRAIN_ANO), and segmentation (M_TRAIN_SEG).	c		e	f	g	a		i		j	k	l	m	n	b

M_INITIAL_LEARNING_RATE	0.005 (image classification) or 0.001 (object detection and segmentation)
M_LEARNING_RATE_DECAY	0.1 (image classification) or 0.05 (object detection and segmentation)
M_MAX_EPOCH	60
M_MINI_BATCH_SIZE	32 (image classification), 4 (object detection and segmentation), or 1 (anomaly detection)
M_SCHEDULER_TYPE	M_CYCLICAL_DECAY (image classification and segmentation) or M_FACTOR_DECAY (object detection)
For anomaly detection, a complete training is always performed (though M_COMPLETE cannot be explicitly set). This complete training also affects M_TRAIN_MODE, M_MINI_BATCHES_PER_SAMPLING, and M_SAMPLES_FIXED_SIZE. Refer to these values for details.	

> When using M_COMPLETE, the classifier architecture adapts to certain qualities of the training dataset (image X- and Y-size, image bands, and the number of classes). All of the classifier's weights (learnable parameters) are randomly initialized prior to a complete training.

M_FINE_TUNING	Specifies a fine tuning training mode. Typically, this is for a CNN or segmentation classifier context that was already trained on a specific classification problem, and that you must fine tune with new data. If you specify M_FINE_TUNING with a classifier context that was not sufficiently trained, you will get an error. The training mode controls, and what M_FINE_TUNING automatically sets them to, are listed below. Unless otherwise specified, values apply for image classification (M_TRAIN_CNN) and segmentation (M_TRAIN_SEG). M_INITIAL_LEARNING_RATE: 0.001 M_LEARNING_RATE_DECAY: 0.1 (image classification) or 0.05 (segmentation) M_MAX_EPOCH: 60 M_MINI_BATCH_SIZE: 32 (image classification) or 4 (segmentation) M_SCHEDULER_TYPE: M_DECAY When using M_FINE_TUNING, the classifier architecture is unchangeable. Therefore, the classifier's weights must be complete, and all the related qualities of the training dataset (image X- and Y-size, image bands, and the number of classes) must comply to the classifier's architecture. A fine-tuning training starts by using the classifier's current weights (learnable parameters). M_FINE_TUNING is not supported for object detection.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRANSFER_LEARNING	Specifies a transfer learning mode. Typically, this is for a CNN or segmentation classifier context that was already trained on a specific classification problem, and that you must train on a similar (but new) problem. If you specify M_TRANSFER_LEARNING with a context that was not sufficiently trained, you will get an error. The training mode controls, and what M_TRANSFER_LEARNING automatically sets them to, are listed below. Unless otherwise specified, values apply for image classification (M_TRAIN_CNN) and segmentation (M_TRAIN_SEG). M_INITIAL_LEARNING_RATE: 0.005 (image classification) or 0.001 (segmentation) M_LEARNING_RATE_DECAY: 0.1 (image classification) or 0.05 (segmentation) M_MAX_EPOCH: 60 M_MINI_BATCH_SIZE: 32 (image classification) or 4 (segmentation) M_SCHEDULER_TYPE: M_CYCLICAL_DECAY When using M_TRANSFER_LEARNING, the classifier architecture adapts to certain qualities of the training dataset (image X- and Y-size, and the number of classes). During training, some of the classifier's weights (learnable parameters) are randomly initialized and affected by the training (those related to classifying features), while others remain unaffected by the training (those related to extracting features). M_TRANSFER_LEARNING is not supported for object detection.	c		e	f	g	a		i		j	k	l	m	n	b
M_SCHEDULER_TYPE	Sets the schedule with which to adjust the learning rate. You can automatically modify this setting, if you modify M_RESET_TRAINING_VALUES. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value. For image classification and segmentation, the default is M_CYCLICAL_DECAY. For object detection, the default is M_FACTOR_DECAY.	c		e	f	g	a		i		j	k	l	m	n	b
M_CYCLICAL_DECAY	Specifies to decay the learning rate at an internally established cyclical schedule.	c		e	f	g	a		i		j	k	l	m	n	b
M_DECAY	Specifies to decay the learning rate, as weights are updated.	c		e	f	g	a		i		j	k	l	m	n	b
M_FACTOR_DECAY	Specifies to decay the learning rate by a factor at every specified epoch.	c		e	f	g	a		i		j	k	l	m	n	b
M_SPLIT_PERCENTAGE	Sets the percentage value, for the training dataset, used by MclassTrain() when splitting a single dataset into a training and development dataset. For example, if you call MclassTrain() with a single dataset, and you set M_SPLIT_PERCENTAGE to 80 percent (M_DEFAULT), the training dataset will contain 80 percent of the entries in the single dataset, and the development dataset will contain 20 percent of the entries. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 80.0.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the percentage.	c		e	f	g	a		i		j	k	l	m	n	b
M_SPLIT_SEED_MODE	Sets the split seed mode to use when splitting a single dataset into a training and development dataset. For example, this can be set to randomly assign each entry to the training set or development set using M_RANDOM or, a fixed selection process using M_FIXED. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_RANDOM.	c		e	f	g	a		i		j	k	l	m	n	b

	M_FIXED	Specifies to use the M_SPLIT_CONTEXT_FIXED_SEED predefined split classification context, for image classification (CNN) or feature classification (tree ensemble), or the M_SPLIT_SEG_CONTEXT_FIXED_SEED predefined split classification context, for segmentation.	c		e	f	g	a		i		j	k	l	m	n	b
	M_RANDOM	Specifies to use the M_SPLIT_CONTEXT_DEFAULT predefined split classification context, for image classification (CNN) or feature classification (tree ensemble), or the M_SPLIT_SEG_CONTEXT_DEFAULT predefined split classification context, for segmentation.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAIN_DESTINATION_FOLDER		Sets the path to the destination folder in which to save training results. For image classification training (M_TRAIN_CNN) and object detection training (M_TRAIN_DET), this specifies the location of the prepared images that will be stored on disk. For segmentation training (M_TRAIN_SEG), this determines the destination for both the segmentation scores and the prepared images that will be stored on disk. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	AIL_TEXT("") 1	Specifies an empty path. This indicates that you are using the default path, which you can inquire using M_TRAIN_DESTINATION_FOLDER + M_DEFAULT_PATH. > Note, if using a C compiler, you must pass the value as M_PTR_TO_AIL_INT(AIL_TEXT("")).	c		e	f	g	a		i		j	k	l	m	n	b
	AIL_TEXT("*TrainingResultsPath*") 1	Specifies the training results file path. For example, you can specify the root path as, *AIL_TEXT("E:\\Manual\TrainingResults\Good")*. > Note, if using a C compiler, you must pass the value as M_PTR_TO_AIL_INT(AIL_TEXT("*TrainingResultsPath*")).	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

To set a global setting of a CNN, segmentation, or tree ensemble training context, set the ControlType parameter to the value below. In this case, set the ContextOrResultClassId parameter to the identifier of a CNN, segmentation, or tree ensemble training context, and set the LabelOrIndex parameter to M_CONTEXT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a training context (CNN, segmentation, or feature classification)																	
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																
M_SUPPORT_MISSING_GROUND_TRUTH		Sets whether the training process allows the training and development datasets to contain entries that do not have a ground truth. An entry's ground truth is specified using MclassControlEntry() with M_CLASS_INDEX_GROUND_TRUTH. For segmentation, when M_NO_REGION_PIXEL_CLASS is set to M_NO_CLASS, entries without a descriptor based region (M_DESCRIPTOR_BASED) are considered not to have a ground truth. INQ															
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that entries without a ground truth are not allowed. In this case, if you call MclassTrain(), and one or more entries in the training or development datasets do not have a ground truth, you will get an error.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that entries without a ground truth are allowed. In this case, if you call MclassTrain(), and one or more entries in the training or development datasets do not have a ground truth, Aurora Imaging Library ignores them.	c		e	f	g	a		i		j	k	l	m	n	b

To control a global setting of a segmentation training context, the ControlType and corresponding ControlValue parameter settings can be set to one of the following values. In this case, set the ContextOrResultClassId parameter to the identifier of a segmentation training context, and set the LabelOrIndex parameter to M_CONTEXT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a training context (segmentation)																	
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																
M_CLASS_WEIGHT_STRENGTH		Sets the class weight strength when training with an inverse class frequency weight mode (M_INVERSE_CLASS_FREQUENCY). This setting has no effect when training with other weight modes. INQ															
	M_DEFAULT	Specifies the default value; the default value is 50.0.	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 100.0	Specifies the proportion of weighting to assign to M_INVERSE_CLASS_FREQUENCY. Reducing this value reduces the emphasis on the less represented classes.	c		e	f	g	a		i		j	k	l	m	n	b
M_FEATURE_SIZE_X		Sets the width (X-size) of the smallest feature to segment. This acts as a lower bound to the X-size of the class regions that the classifier expects during training. This control might be useful if the minimum feature size is larger than the minimum receptive field of the classifier. The default feature size (M_MIN) is typically the best possible setting. This has no effect when fine tuning (M_FINE_TUNING). If images have been resized during data preparation (either by MclassPrepareData() or an internal data preparation context), this value must be set relative to the final prepared images. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_MIN.	c		e	f	g	a		i		j	k	l	m	n	b
	M_MIN	Specifies the smallest size possible.	c		e	f	g	a		i		j	k	l	m	n	b

Value > 0	Specifies that the classifier will adjust to encompass features of the size you set, provided they are larger than the minimum possible feature size.	c		e	f	g	a		i		j	k	l	m	n	b
M_FEATURE_SIZE_Y	Sets the height (Y-size) of the smallest feature to segment. This acts as a lower bound to the Y-size of the class regions that the classifier expects during training. This control might be useful if the minimum feature size is larger than the minimum receptive field of the classifier. The default feature size (M_MIN) is typically the best possible setting. This has no effect when fine tuning (M_FINE_TUNING). If images have been resized during data preparation (either by MclassPrepareData() or an internal data preparation context), this value must be set relative to the final prepared images. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_SAME.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN	Specifies the smallest size possible.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAME	Specifies to use the same value as M_FEATURE_SIZE_X.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies that the classifier will adjust to encompass features of the size you set, provided they are larger than the minimum possible feature size.	c		e	f	g	a		i		j	k	l	m	n	b
M_USE_MASK_DESCRIPTORS	Sets whether to use mask descriptors for training. You must enable M_USE_MASK_DESCRIPTORS if you disable M_USE_POLYGON_DESCRIPTORS (both can be enabled, but both cannot be disabled). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ENABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to use mask descriptors for training.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to use mask descriptors for training.	c		e	f	g	a		i		j	k	l	m	n	b
M_USE_POLYGON_DESCRIPTORS	Sets whether to use polygon descriptors for training. You must enable M_USE_POLYGON_DESCRIPTORS if you disable M_USE_MASK_DESCRIPTORS (both can be enabled, but both cannot be disabled). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ENABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to use polygon descriptors for training.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to use polygon descriptors for training.	c		e	f	g	a		i		j	k	l	m	n	b

To control a global setting of an anomaly detection training context, the ControlType and corresponding ControlValue parameter settings can be set to one of the following values. In this case, set the ContextOrResultClassId parameter to the identifier of an anomaly detection training context, and set the LabelOrIndex parameter to M_CONTEXT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a training context (anomaly detection)																
ControlType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ControlValue																
M_MINI_BATCHES_PER_SAMPLING	Sets how many mini-batches are loaded into memory before sampling when M_TRAIN_MODE is set to M_ITERATIVE_SAMPLING.															
M_DEFAULT	Specifies the default value; the default value is 1.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 1	Specifies the number of mini-batches.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAMPLES_FIXED_SIZE	Sets the number of samples learned by the classifier. The higher the number, the bigger the size of the network (bigger networks can have a better/higher capacity to handle more complex cases).	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 4000.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the number of samples.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAIN_MODE	Sets the mode in which the samples are selected. By default, Aurora Imaging Library tries to perform a global type of training (M_GLOBAL_SAMPLING), but this can lead to running out of GPU memory. In this case, it is recommended to set M_TRAIN_MODE to M_ITERATIVE_SAMPLING, and then increase M_MINI_BATCHES_PER_SAMPLING until you maximize GPU usage. Note that both modes result in similar accuracy.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_AUTO.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO	Specifies to automatically select the mode in which the samples are selected. This is the same as M_GLOBAL_SAMPLING.	c		e	f	g	a		i		j	k	l	m	n	b

	M_GLOBAL_SAMPLING	Specifies a global selection mode. This mode loads the whole training set into memory and then samples it. M_GLOBAL_SAMPLING is memory intensive but faster than M_ITERATIVE_SAMPLING since the sampling occurs only once.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ITERATIVE_SAMPLING	Specifies an iterative selection mode. This mode partitions the training set and samples each partition iteratively. M_ITERATIVE_SAMPLING uses less memory than M_GLOBAL_SAMPLING but takes longer since the sampling process is repeated at each iteration.	c		e	f	g	a		i		j	k	l	m	n	b

To control a global setting of a segmentation or tree ensemble training context, the ControlType and corresponding ControlValue parameter settings can be set to one of the following values. In this case, set the ContextOrResultClassId parameter to the identifier of a segmentation or tree ensemble training context, unless otherwise specified, and set the LabelOrIndex parameter to M_CONTEXT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a training context (segmentation or tree ensemble)																	
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																
M_CLASS_WEIGHT_MODE		Sets how to establish the weight of a class definition. INQ															
	M_DEFAULT	Same as M_AUTO.	c		e	f	g	a		i		j	k	l	m	n	b
	M_AUTO	Specifies that Aurora Imaging Library automatically sets how the weight is established. For segmentation training contexts, if the loss function type is M_FOCAL_LOSS or M_CROSS_ENTROPY, M_AUTO is the same as M_INVERSE_CLASS_FREQUENCY. If the loss function type is M_DICE_LOSS, M_AUTO is the same as M_SQUARED_INVERSE_CLASS_FREQUENCY. For image classification and tree ensemble training contexts, M_AUTO is the same as M_NONE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_BALANCE	Specifies a balanced weight. This setting is only for tree ensemble.	c		e	f	g	a		i		j	k	l	m	n	b
	M_INVERSE_CLASS_FREQUENCY	Specifies to use weight factors equal to the inverse of each class frequency. You can further adjust the weight factors with M_CLASS_WEIGHT_STRENGTH. This setting is only for segmentation.	c		e	f	g	a		i		j	k	l	m	n	b
	M_NONE	Specifies no weight.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SQUARED_INVERSE_CLASS_FREQUENCY	Specifies to use weight factors equal to the squared inverse of each class frequency. You can further adjust the weight factors with M_CLASS_WEIGHT_STRENGTH. This setting is only for segmentation.	c		e	f	g	a		i		j	k	l	m	n	b
	M_USER_DEFINED	Specifies to use the weight defined with M_CLASS_WEIGHT.	c		e	f	g	a		i		j	k	l	m	n	b

To control a global setting of a tree ensemble training context, the ControlType and corresponding ControlValue parameter settings can be set to one of the following values. In this case, set the ContextOrResultClassId parameter to the identifier of a tree ensemble training context, and set the LabelOrIndex parameter to M_CONTEXT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a training context (tree ensemble)																	
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																
M_BOOTSTRAP		Sets whether to use bootstrap aggregation for the training dataset entries to produce multiple and separate training sets when building trees. INQ															
	M_DEFAULT	Same as M_WITH_REPLACEMENT.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies not to use bootstrap aggregation.	c		e	f	g	a		i		j	k	l	m	n	b
	M_WITH_REPLACEMENT	Specifies to use bootstrap aggregation, and allow replacement (chosen entries in a training set can be chosen again).	c		e	f	g	a		i		j	k	l	m	n	b
	M_WITHOUT_REPLACEMENT	Specifies to use bootstrap aggregation, and not allow replacement (chosen entries in a training set cannot be chosen again).	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPUTE_OUT_OF_BAG_RESULTS		Sets whether to use out-of-bag dataset entries to estimate the generalization accuracy during training. Note, bagging information is typically unreliable if your training dataset has augmented entries. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_ENABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies not to use out-of-bag entries.	c		e	f	g	a		i		j	k	l	m	n	b

M_ENABLE	Specifies to use out-of-bag entries.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPUTE_PROXIMITY_MATRIX	Sets whether to calculate the proximity measure matrix when building trees. This is a normalized *N* x *N* matrix, where *N* is the number of entries. The intersection value between pairs of entries in the matrix defines their proximity, which refers to the average number of trees that occupy the same terminal node for those entries. For example, if entries i and *j* are in the same terminal node, their proximity increases. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to calculate the proximity measure matrix when building trees.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to calculate the proximity measure matrix when building trees.	c		e	f	g	a		i		j	k	l	m	n	b
M_FEATURE_IMPORTANCE_MODE	Sets how to establish the importance of the features. Every entry in a features dataset refers to a set of features. For example, every entry can refer to a set of blob features, such as area, perimeter, and Feret diameter. Specific values for these features are specified, for each entry, using MclassControlEntry() with M_RAW_DATA. By enabling a feature importance mode, Aurora Imaging Library establishes, during training, whether certain features are more important than others in determining the class to which the input data belongs. For example, the perimeter feature can end up being very important to establishing a successful classification, while the area feature can end up being almost irrelevant. To retrieve the resulting importance of features after training, call MclassGetResult() with M_FEATURE_IMPORTANCE. You can then modify your set of features and retrain, which can help improve the tree ensemble classifier's performance and accuracy. Note, M_FEATURE_IMPORTANCE_MODE does not directly affect training. It allows you to retrieve which features are more important; you can then use this information to modify your dataset (for example, you can specify only the important features) and retrain. In general, the fastest modes with which to establish the feature importance are M_MEAN_DECREASE_IMPURITY, M_PERMUTATION, and M_DROP_COLUMN, while the modes with which to most accurately establish the feature importance are M_DROP_COLUMN, M_PERMUTATION, and M_MEAN_DECREASE_IMPURITY. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_MEAN_DECREASE_IMPURITY.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable the feature importance mode. In this case, you cannot retrieve any information about the feature importance.	c		e	f	g	a		i		j	k	l	m	n	b
M_DROP_COLUMN	Specifies that the feature importance is based on the drop column of features. In this case, the more the elimination of a feature affects accuracy, the more importance that feature is given. To specify the set with which to calculate this type of feature importance, use M_FEATURE_IMPORTANCE_SET. To use M_DROP_COLUMN, you must train with a development dataset or compute out-of-bag results (that is, enable M_COMPUTE_OUT_OF_BAG_RESULTS).	c		e	f	g	a		i		j	k	l	m	n	b
M_MEAN_DECREASE_IMPURITY	Specifies that the feature importance is based on a decreasing impurity process. In this case, the more a feature affects a proper node splitting, the more important it is. Proper splitting means that the two sets resulting from splitting the node are significantly purer (better) than the set that was input to the node.	c		e	f	g	a		i		j	k	l	m	n	b
M_PERMUTATION	Specifies that the feature importance is based on feature permutation. In this case, the more the shuffling of a feature affects accuracy, the more importance that feature is given. To specify the set with which to calculate this type of feature importance, use M_FEATURE_IMPORTANCE_SET. To use M_PERMUTATION, you must train with a development dataset or compute out-of-bag results (that is, enable M_COMPUTE_OUT_OF_BAG_RESULTS).	c		e	f	g	a		i		j	k	l	m	n	b
M_FEATURE_IMPORTANCE_SET	Sets how to establish the group of features with which to determine feature importance when using drop column or permutation. M_FEATURE_IMPORTANCE_SET only has an effect if M_FEATURE_IMPORTANCE_MODE is set to M_DROP_COLUMN or M_PERMUTATION. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_AUTO.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO	Specifies to automatically establish the feature set to use. If available, Aurora Imaging Library uses M_DEV_DATASET; otherwise, Aurora Imaging Library uses M_OUT_OF_BAG.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEV_DATASET	Specifies to use the development dataset.	c		e	f	g	a		i		j	k	l	m	n	b
M_OUT_OF_BAG	Specifies to use the out-of-bag set. In this case, you must not disable M_BOOTSTRAP. Note, bagging information is typically unreliable if your data is augmented.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_IMPURITY_DECREASE	Sets a decrease of impurity requirement for splitting a node. That is, Aurora Imaging Library will split a node if it induces a decrease of impurity greater than or equal to the specified value. The weighted impurity decrease equation is as follows: *N_t* / *N* * (*impurity* - *N_t_R* / *N_t* * *right_impurity* - *N_t_L* / *N_t* * *left_impurity*), where *N* is the total number of dataset entries, *N_t* is the number of dataset entries related to the current node, *N_t_L* is the number of dataset entries related to the left child, and *N_t_R* is the number of dataset entries related to the right child. *N*, *N_t*, *N_t_R*, and *N_t_L* all refer to the weighted sum, if weights (entry weights and class weights) are used. INQ	c		e	f	g	a		i		j	k	l	m	n	b

	M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0.0	Specifies the impurity value.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_NUMBER_OF_ENTRIES_LEAF		Sets the minimum number of training dataset entries, or a percentage of the minimum number of related dataset entries, that Aurora Imaging Library uses to identify a leaf node. Aurora Imaging Library uses this value to establish M_MIN_NUMBER_OF_ENTRIES_LEAF_MODE. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 1.	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 < *MinNumPercent* <= 100.0	Specifies the minimum number of entries, as a percentage. You should only use a percentage when M_MIN_NUMBER_OF_ENTRIES_LEAF_MODE is set to M_USER_DEFINED_PERCENTAGE. If you specify a percentage value that results in the minimum number of entries equaling 0, Aurora Imaging Library uses 1 as the minimum number; that is, if Round((Value/100) * *NumberEntriesInTrainDataset*) == 0, then *MinNumberEntriesLeaf* = 1.	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 1	Specifies the minimum number of entries. You should only use this value when M_MIN_NUMBER_OF_ENTRIES_LEAF_MODE is set to M_USER_DEFINED_VALUE.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_NUMBER_OF_ENTRIES_LEAF_MODE		Sets how to establish the minimum number of related dataset entries required to be at a leaf node. A split point at any depth will only be considered if it leaves at least the minimum number of entries (leaf training entries) in each of the left and right branches. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_USER_DEFINED_VALUE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_USER_DEFINED_PERCENTAGE	Specifies to use a percentage, determined from the minimum number of entries defined with M_MIN_NUMBER_OF_ENTRIES_LEAF.	c		e	f	g	a		i		j	k	l	m	n	b
	M_USER_DEFINED_VALUE	Specifies to use the minimum number of entries defined with M_MIN_NUMBER_OF_ENTRIES_LEAF.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_NUMBER_OF_ENTRIES_SPLIT		Sets the minimum number of training dataset entries, or a percentage of the minimum number of related dataset entries, that Aurora Imaging Library uses to determine how to best split the internal nodes of the trees. Aurora Imaging Library uses this value to establish M_MIN_NUMBER_OF_ENTRIES_SPLIT_MODE. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 2.	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 < *MinNumPercent* <= 100.0	Specifies the minimum number of entries, as a percentage. This only has an effect if M_MIN_NUMBER_OF_ENTRIES_SPLIT_MODE is set to M_USER_DEFINED_PERCENTAGE. If you specify a percentage value that results in the minimum number of entries equaling 0, Aurora Imaging Library uses 2 as the minimum number; that is, if Round((Value/100) * *NumberEntriesInTrainDataset*) == 0, then *MinNumberEntriesSplit* = 2.	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 1	Specifies the minimum number of entries. This only has an effect if M_MIN_NUMBER_OF_ENTRIES_SPLIT_MODE is set to M_USER_DEFINED_VALUE.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_NUMBER_OF_ENTRIES_SPLIT_MODE		Sets how to establish the minimum number of related dataset entries required to split an internal node. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_USER_DEFINED_VALUE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_USER_DEFINED_PERCENTAGE	Specifies to use a percentage, determined from the minimum number of entries defined with M_MIN_NUMBER_OF_ENTRIES_SPLIT.	c		e	f	g	a		i		j	k	l	m	n	b
	M_USER_DEFINED_VALUE	Specifies to use the minimum number of entries defined with M_MIN_NUMBER_OF_ENTRIES_SPLIT.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_WEIGHT_FRACTION_LEAF		Sets the minimum weighted fraction of the sum total of weights (of all the dataset entries) required to be at a leaf node. Entries have equal weight when M_CLASS_WEIGHT_MODE is set to M_NONE. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 0.5	Specifies the minimum weighted fraction.	c		e	f	g	a		i		j	k	l	m	n	b
M_NODE_MAX_NUMBER_OF_FEATURES		Sets the maximum number of features in the training dataset, or a percentage of the maximum number of features, that Aurora Imaging Library can use to determine how to best split the nodes of the trees. M_NODE_MAX_NUMBER_OF_FEATURES only has an effect if M_NODE_MAX_NUMBER_OF_FEATURES_MODE is set to M_USER_DEFINED_VALUE or M_USER_DEFINED_PERCENTAGE. If M_NODE_MAX_NUMBER_OF_FEATURES_MODE is set to M_LOG2, and the number of features in the dataset is 1, Aurora Imaging Library trains with 1 feature, even though the calculation results in 0; that is, log(1) = 0. If M_NODE_MAX_NUMBER_OF_FEATURES_MODE is equal to M_USER_DEFINED_VALUE, and M_NODE_MAX_NUMBER_OF_FEATURES is higher than the actual number of features in the training dataset, Aurora Imaging Library uses all the features in the dataset. INQ	c		e	f	g	a		i		j	k	l	m	n	b

	M_DEFAULT	Same as M_ALL.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ALL	Specifies no maximum.	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *MaxNumPercent* <= 100.0	Specifies the percentage with which to determine the maximum number of features. This only has an effect if M_NODE_MAX_NUMBER_OF_FEATURES_MODE is set to M_USER_DEFINED_PERCENTAGE. If you specify a percentage value that results in the maximum number of features equaling 0, Aurora Imaging Library uses 1 feature as the maximum number; that is, if Round((Value/100)**NumberOfFeaturesInTrainDataset*) == 0, then *MaxNumberOfFeatures* = 1 (see M_NODE_MAX_NUMBER_OF_FEATURES_MODE).	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 1	Specifies the value with which to determine the maximum number of features. This only has an effect if M_NODE_MAX_NUMBER_OF_FEATURES_MODE is set to M_USER_DEFINED_VALUE. If you specify a maximum value that is greater than the total number of features available, Aurora Imaging Library uses the total number of features available.	c		e	f	g	a		i		j	k	l	m	n	b
M_NODE_MAX_NUMBER_OF_FEATURES_MODE		Sets how to establish the maximum number of features that Aurora Imaging Library uses to determine how to best split the nodes of the trees. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_SQUARE_ROOT.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ALL	Specifies that all available features are used.	c		e	f	g	a		i		j	k	l	m	n	b
	M_LOG2	Specifies to use a base 2 logarithm, determined from the total number of features. This can be described as: M_LOG2 = Round(Log2(*NumberOfFeatures*)).	c		e	f	g	a		i		j	k	l	m	n	b
	M_SQUARE_ROOT	Specifies to use a square root, determined from the total number of features. This can be described as: M_SQUARE_ROOT = Round(Log2(*NumberOfFeatures*)).	c		e	f	g	a		i		j	k	l	m	n	b
	M_USER_DEFINED_PERCENTAGE	Specifies to use a percentage, determined from the total number of features, where the percentage is set by M_NODE_MAX_NUMBER_OF_FEATURES. This can be described as: M_USER_DEFINED_PERCENTAGE = Round(*NumberOfFeatures** *MaxNumberOfFeatures*/100).	c		e	f	g	a		i		j	k	l	m	n	b
	M_USER_DEFINED_VALUE	Specifies to use the maximum number of features defined with M_NODE_MAX_NUMBER_OF_FEATURES.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_TREES		Sets the number of trees in the tree ensemble train context.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 10.	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 1	Specifies the amount of trees in the tree ensemble.	c		e	f	g	a		i		j	k	l	m	n	b
M_SPLIT_QUALITY_TYPE		Sets how to measure the quality of a node split. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_GINI.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENTROPY	Specifies that Aurora Imaging Library uses an Entropy based function to measure the quality of a node split.	c		e	f	g	a		i		j	k	l	m	n	b
	M_GINI	Specifies that Aurora Imaging Library uses a Gini based function to measure the quality of a node split.	c		e	f	g	a		i		j	k	l	m	n	b
M_TREE_MAX_DEPTH		Sets the maximum depth of the trees in the tree ensemble. The depth of a tree (M_TREE_MAX_DEPTH) is the depth of its deepest node. The depth of a node is the number of edges (connectors) from that node to the tree's root node. A root node has a depth of 0. To retrieve the actual depth achieved after training, call MclassGetResult() with M_TREE_DEPTHS_ACHIEVED. The depth achieved is always less than or equal to the maximum tree depth (M_TREE_MAX_DEPTH). INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_NONE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_NONE	Specifies no maximum depth.	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0	Specifies the maximum depth.	c		e	f	g	a		i		j	k	l	m	n	b
M_TREE_MAX_NUMBER_OF_LEAF_NODES		Sets the maximum number of terminal nodes in the tree. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_NONE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_NONE	Specifies no maximum.	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 1	Specifies the maximum number of terminal nodes in the tree.	c		e	f	g	a		i		j	k	l	m	n	b

To control a general setting of a training result buffer, the ControlType and corresponding ControlValue parameter settings can be set to one of the following values. In this case, set the ContextOrResultClassId parameter to the identifier of a CNN, object detection, segmentation, or tree ensemble training result buffer, unless otherwise specified, and set the LabelOrIndex parameter to M_GENERAL.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a training result buffer (CNN, object detection, segmentation, or tree ensemble)																	
ControlType / ControlValue	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_STOP_TRAIN	Stops the current execution of MclassTrain(). This can only be done from another thread of higher priority, or from a user-defined classification hook function. If you stop training a CNN, object detection or segmentation classifier context, and 1 or more epochs are completed, results are typically available, except those related to development accuracy, training accuracy, development error rate, and training error rate. If you stop training a tree ensemble classifier context, and 1 or more trees are completed, all results are available. If 1 or more epochs are completed, Aurora Imaging Library saves the trained classifier results that MclassTrain() produces up to the point that you stop training. You can copy these results into a classifier context, using MclassCopyResult().															
M_DEFAULT	Specifies the default behavior.	c		e	f	g	a		i		j	k	l	m	n	b
M_UPDATE_TRAINED_PARAMS	Updates the internal and hidden parameters (weights) of the classifier. This only applies when training a CNN or segmentation classifier, which you must specify with the ContextOrResultClassId parameter.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_NOW.	c		e	f	g	a		i		j	k	l	m	n	b
M_NOW	Specifies to immediately update the training parameters network.	c		e	f	g	a		i		j	k	l	m	n	b

To control a general setting of a prediction result buffer, the ControlType and corresponding ControlValue parameter settings can be set to one of the following values. Unless otherwise specified, set the ContextOrResultClassId parameter to the identifier of a CNN, object detection, segmentation, tree ensemble, or ONNX prediction result buffer, and set the LabelOrIndex parameter to M_GENERAL. Note, it is possible to specify M_STOP_PREDICT when calling MclassPredict() with a target dataset; this requires setting the ContextOrResultClassId parameter to the identifier of the output dataset (images or features), and setting the LabelOrIndex parameter to M_CONTEXT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a prediction result buffer (CNN, object detection, segmentation, tree ensemble, or ONNX) or a dataset (images or features)																	
ControlType / ControlValue	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_STOP_PREDICT	Stops the current execution of MclassPredict(). This can only be done from another thread of higher priority, or from a user-defined classification hook function. If you stop the prediction, Aurora Imaging Library does not return any results. Note, to stop the prediction of a target dataset, you must set the ContextOrResultClassId parameter to the identifier of the output dataset (produced by MclassPredict()), and set the LabelOrIndex parameter to M_CONTEXT.															
M_DEFAULT	Specifies the default behavior.	c		e	f	g	a		i		j	k	l	m	n	b

To control a general setting of a prediction result buffer, the ControlType and corresponding ControlValue parameter settings can be set to one of the following values. Unless otherwise specified, set the ContextOrResultClassId parameter to the identifier of an object detection prediction result buffer, and set the LabelOrIndex parameter to M_GENERAL.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a prediction result buffer (object detection)																	
ControlType / ControlValue	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_RESULT_OUTPUT_UNITS	Sets which units and coordinate system to use for prediction results. This setting only affects positional results within the Aurora Imaging Library Classification module. Note, this setting can be changed at any time to get results in the required units. INQ															
M_DEFAULT	Same as M_ACCORDING_TO_CALIBRATION.	c		e	f	g	a		i		j	k	l	m	n	b
M_ACCORDING_TO_CALIBRATION	Specifies to use world units if the target image is associated with a camera calibration context; otherwise, specifies to use pixel units.	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL	Specifies to use pixel units with respect to the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_WORLD	Specifies to use world units with respect to the relative coordinate system. Using MclassGetResult() will generate an error if the target image is not associated with a calibration context.	c		e	f	g	a		i		j	k	l	m	n	b

To control a general setting of a prediction or training result buffer, the ControlType and corresponding ControlValue parameter settings can be set to one of the following values. Unless otherwise specified, set the ContextOrResultClassId parameter to the identifier of a CNN, object detection, segmentation, or tree ensemble prediction or training result buffer, or an ONNX prediction result buffer, and set the LabelOrIndex parameter to M_GENERAL.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a prediction or training result buffer (CNN, object detection, segmentation, tree ensemble, or ONNX)

ControlType / ControlValue	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_RESET	Removes all results from the classification result buffer. Note that this does not delete the result buffer identifier, as is the case with MclassFree().															
M_DEFAULT	Specifies the default behavior.	c		e	f	g	a		i		j	k	l	m	n	b

To control statistics classification, the ControlType and corresponding ControlValue parameter settings can be set to one of the following values. In addition, you can also specify M_TIMEOUT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For statistics classification

ControlType / ControlValue	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_MATCH_IOU_THRESHOLD	Sets the IOU threshold for an object detection stat context, based on which predicted boxes are compared to their matched ground truth boxes. By comparing the IOU of each matched pair, the boxes are classified on the confusion matrix. To use this control, you must set the ContextOrResultClassId parameter to the identifier of a statistics classification context for object detection (M_STAT_DET) and set the LabelOrIndex parameter to M_CONTEXT. INQ															
M_DEFAULT	Specifies the default value; the default value is 50.0.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the IOU threshold.	c		e	f	g	a		i		j	k	l	m	n	b
M_MODE	Sets whether to calculate statistics for anomaly detection on an image level (default) or a pixel level. On an image level, the problem is an image classification one, whereas on the pixel level it is a segmentation problem. To use this control, you must set the ContextOrResultClassId parameter to the identifier of a statistics classification context for anomaly detection (M_STAT_ANO) and set the LabelOrIndex parameter to M_CONTEXT. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_IMAGE_LEVEL.	c		e	f	g	a		i		j	k	l	m	n	b
M_IMAGE_LEVEL	Specifies to calculate statistics for anomaly detection on an image level. This takes into account M_WHOLE_IMAGE regions to identify anomalous entries. For a given entry, if its M_WHOLE_IMAGE region has no M_CLASS_INDEX_GROUND_TRUTH, it is considered non anomalous unless there exist at least one anomalous M_DESCRIPTOR_BASED region.	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL_LEVEL	Specifies to calculate statistics for anomaly detection on a pixel level. This only takes into account M_DESCRIPTOR_TYPE_POLYGON and M_DESCRIPTOR_TYPE_MASK regions of the entry to identify anomalous pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_SCORE_THRESHOLD_STEP	Sets the step increment of threshold values from 0.0 to 100.0. Last threshold might be less than 100.0 if the step setting leads past it. To use this control, you must set the ContextOrResultClassId parameter to the identifier of any statistics classification context (M_STAT_ANO, M_STAT_CNN, M_STAT_DET, M_STAT_SEG, or M_STAT_TREE_ENSEMBLE) and set the LabelOrIndex parameter to M_CONTEXT. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.001.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the step increment for threshold.	c		e	f	g	a		i		j	k	l	m	n	b
M_STOP_CALCULATE	Stops the current execution of MclassStatCalculate(). This can only be done from another thread of higher priority, or from a user-defined classification hook function. You cannot call MclassInquire() with M_STOP_CALCULATE. To use this control, you must set the ContextOrResultClassId parameter to the identifier of any statistics classification result buffer (M_STAT_ANO_RESULT, M_STAT_CNN_RESULT, M_STAT_DET_RESULT, M_STAT_SEG_RESULT, or M_STAT_TREE_ENSEMBLE_RESULT) and set the LabelOrIndex parameter to M_GENERAL.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default behavior.	c		e	f	g	a		i		j	k	l	m	n	b

Type-specific versions of the function when using a C compiler under 64-bit

```
void MclassControlInt64 (AIL_ID ContextOrResultClassId, AIL_INT64 LabelOrIndex, AIL_INT64 ControlType, AIL_INT64 ControlValue)
```

Details

Parameters

ContextOrResultClassId

See ContextOrResultClassId of the main function for a description.

LabelOrIndex

See LabelOrIndex of the main function for a description.

ControlType

See ControlType of the main function for a description.

ControlValue

See ControlValue of the main function for a description.

```
void MclassControlDouble (AIL_ID ContextOrResultClassId, AIL_INT64 LabelOrIndex, AIL_INT64 ControlType, AIL_DOUBLE ControlValue)
```

Details

Parameters

ContextOrResultClassId

See ContextOrResultClassId of the main function for a description.

LabelOrIndex

See LabelOrIndex of the main function for a description.

ControlType

See ControlType of the main function for a description.

ControlValue

See ControlValue of the main function for a description.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailclass11.lib.
DLL	Requires ail11.dll; ailclass11.dll.

MclassControlEntry

Synopsis

Control an entry in a dataset context.

Syntax

```
void MclassControlEntry(
    AIL_ID DatasetContextClassId, //in
    AIL_INT64 EntryIndex,         //in
    AIL_UUID EntryKey,            //in
    AIL_INT64 RegionIndex,        //in
    AIL_INT64 ControlType,        //in
    AIL_DOUBLE ControlValue,      //in
    const void *ControlValuePtr,  //in
    AIL_INT ControlValuePtrSize   //in
)
```

Description

This function allows you to control a setting of an entry in a dataset context using its index or key. You can typically inquire these settings, using MclassInquireEntry().

To add entries to, or delete entries from, a dataset context, use MclassControl(). You can also import data from a CSV file to a dataset context, using MclassImport().

Parameters

DatasetContextClassId

Specifies the dataset on which to control settings. These contexts are allocated using MclassAlloc() with M_DATASET_IMAGES or M_DATASET_FEATURES.

EntryIndex

Specifies the entry to control, using the entry's index. Set this parameter to one of the following values.

> Note that EntryIndex and EntryKey cannot simultaneously be set to M_DEFAULT and M_DEFAULT_KEY respectively.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the index of the entry to control																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies that the entry index is not required. In this case, you must set the EntryKey parameter to the key (UUID) of the entry to control.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the index of the entry to control. In this case, you must set the EntryKey parameter to M_DEFAULT_KEY.	c		e	f	g	a		i		j	k	l	m	n	b

EntryKey

Specifies the entry to control, using the entry's unique key (UUID). Set this parameter to one of the following values.

> Note that EntryIndex and EntryKey cannot simultaneously be set to M_DEFAULT and M_DEFAULT_KEY respectively.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the key (UUID) of the entry to control																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_KEY	Specifies that the entry's key (UUID) is not required. In this case, you must set the EntryIndex parameter to the index of the entry to control.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_UUID Value	Specifies the key (UUID) of the entry to control. In this case, you must set the EntryIndex parameter to M_DEFAULT. The key is defined as an Aurora Imaging Library universal unique identifier (UUID). To get an entry's key, call MclassInquireEntry() with M_ENTRY_KEY.	c		e	f	g	a		i		j	k	l	m	n	b

RegionIndex

Specifies to control the entry, or to control the entry's region. Set this parameter to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying whether to control the entry or the entry's region																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies to control the entry. You must specify M_DEFAULT if you are using a features dataset context.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_INDEX(AIL_INT RegionIdx)	Specifies to control the entry's region. This value only applies to an images dataset context.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	RegionIdx Set this parameter to the following:															

			0 <= *Value* < M_NUMBER_OF_REGIONS	Specifies a specific region within the entry. Note that there is always at least one region in an entry.	c		e	f	g	a		i		j	k	l	m	n	b

ControlType

Specifies the type of control to set.

See the Parameter associations section for possible values that can be specified.

ControlValue

Specifies the value for the control. If this information is not required, set this parameter to **M_DEFAULT**.

See the Parameter associations section for possible values that can be specified.

ControlValuePtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE>]
- AIL_CONST_TEXT_PTR [optionally, in C++: a reference to an AIL_STRING]
- M_NULL

Specifies the address of the variable which contains information for the type of control to set. If this information is not required, set this parameter to **M_NULL**.

See the Parameter associations section for possible values that can be specified.

ControlValuePtrSize

Specifies the number of elements in ControlValuePtr. If this information is not needed, set this parameter to **M_DEFAULT**.

Parameter associations

The tables below list possible values for the **ControlType**, **ControlValue**, and **ControlValuePtr** parameters.

- For an entry in a dataset context (images or features)
- For an entry or a region of an entry in a dataset context (images or features)

To control an entry in a dataset, the ControlType and corresponding ControlValue and ControlValuePtr parameter settings can be set to the following values. In this case, set the DatasetContextClassId parameter to the identifier of either an images dataset context or a features dataset context, unless otherwise specified, and set the RegionIndex parameter to M_DEFAULT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For an entry in a dataset context (images or features)																	
ControlType ControlValue ControlValuePtr	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)	
M_AUGMENTATION_SOURCE	Sets whether an entry is an augmented entry, according to its index. For entries added to a dataset by MclassPrepareData(), this key refers to the prepared original image should it be present in the dataset. INQ																
ControlValue	Specifies one of the following.	c		e	f	g	a		i		j	k	l	m	n	b	
M_NOT_AUGMENTED	Specifies that the entry is not an augmented entry.	c		e	f	g	a		i		j	k	l	m	n	b	
M_UNKNOWN	Specifies that the entry is augmented, however, the original data is not known.	c		e	f	g	a		i		j	k	l	m	n	b	
Value >= 0	Specifies that the entry is augmented, and that it originates from (was performed with) the specified entry index. The entry from which the augmentation originates is considered a source or parent entry.	c		e	f	g	a		i		j	k	l	m	n	b	
ControlValuePtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b	
M_AUGMENTATION_SOURCE_KEY	Sets whether an entry is an augmented entry, according to its key (UUID). For entries added to a dataset by MclassPrepareData(), this key refers to the prepared original image should it be present in the dataset.	c		e	f	g	a		i		j	k	l	m	n	b	
ControlValue	Specifies one of the following.INQ	c		e	f	g	a		i		j	k	l	m	n	b	
M_NULL_KEY	Specifies that the entry is not an augmented entry.	c		e	f	g	a		i		j	k	l	m	n	b	
M_UNKNOWN_KEY	Specifies that the entry is augmented, and that the entry key (UUID) from which it was augmented is not known.	c		e	f	g	a		i		j	k	l	m	n	b	

AIL_UUID Value	Specifies that the entry is augmented, and that it originates from (was performed with) the specified key. The entry from which the augmentation originates is considered a source or parent entry. The key is defined as an Aurora Imaging Library universal unique identifier (UUID). To specify an *AIL_UUID* value when using a C compiler, you must call the *AIL_UUID* version of this function (MclassControlEntryAilUuid). When using a C++ or other compiler, **MclassControlEntry**() internally calls the AIL_UUID version of this function. AIL_UUID version of MclassControlEntry void MclassControlEntryAilUuid (AIL_ID DatasetContextClassId, AIL_INT64 EntryIndex, AIL_UUID EntryKey, AIL_INT64 RegionIndex, AIL_INT64 ControlType, AIL_UUID ControlValue, const void *ControlValuePtr, AIL_INT ControlValuePtrSize)	c		e	f	g	a		i		j	k	l	m	n	b
ControlValuePtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_ENTRY_IMAGE_PATH	Sets the location from which to get the entry's image, such that M_ROOT_PATH + M_ENTRY_IMAGE_PATH gives a valid path. If M_ROOT_PATH is empty, M_ENTRY_IMAGE_PATH should be an absolute path, otherwise it should be a relative path to M_ROOT_PATH. The maximum size of the entire file path (M_ROOT_PATH + M_ENTRY_IMAGE_PATH) is 259 characters. This value only applies to an images dataset context. INQ	c		e	f	g	a		i		j	k	l	m	n	b
ControlValue	This parameter must be set to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b
ControlValuePtr	Data type info: Data type: *address of an AIL_CONST_TEXT_PTR [optionally, in C++: AIL_STRING]* Specifies one of the following.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*///DATASET///FilePath*") 1	Specifies the file path under the folder path from where the dataset context is restored, using MclassRestore(). If the ///DATASET/// path does not exist, Aurora Imaging Library uses the path in which your application's executable file is located.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*FilePath*") 1	Specifies the file path.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENTRY_USER_STRING	Sets a user string to store meta information. INQ	c		e	f	g	a		i		j	k	l	m	n	b
ControlValue	This parameter must be set to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b
ControlValuePtr	Data type info: Data type: *address of an AIL_CONST_TEXT_PTR [optionally, in C++: AIL_STRING]* Specifies the following.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*String*") 1	Specifies a string containing meta information. This string must not contain commas.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENTRY_WEIGHT	Sets a weight that indicates the importance of the feature entry. This value only applies to a features dataset context. INQ	c		e	f	g	a		i		j	k	l	m	n	b
ControlValue	Specifies one of the following.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the weight, as a *double*.	c		e	f	g	a		i		j	k	l	m	n	b
ControlValuePtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_RAW_DATA	Sets an array of values (a set of features). In this case, set the DatasetContextClassId parameter to the identifier of a features dataset context. This value only applies to a features dataset context. INQ	c		e	f	g	a		i		j	k	l	m	n	b
ControlValue	Specifies the following.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies that this parameter is not required.	c		e	f	g	a		i		j	k	l	m	n	b
ControlValuePtr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE>]* Specifies the feature values.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_DELETE	Sets the index of a region to remove from a specified entry. This value only applies to an images dataset context.	c		e	f	g	a		i		j	k	l	m	n	b
ControlValue	Specifies the following.	c		e	f	g	a		i		j	k	l	m	n	b

0 < *Value* < M_NUMBER_OF_REGIONS	Specifies a specific region to delete within the entry. Note that there is always at least one region in an entry. Region index 0 cannot be deleted.	c	e	f	g	a	i	j	k	l	m	n	b
ControlValuePtr	Data type info: Data type: *Set this parameter to M_NULL*	c	e	f	g	a	i	j	k	l	m	n	b
M_USER_CONFIDENCE	Sets how confident you are in the entry's ground truth (M_CLASS_INDEX_GROUND_TRUTH). The confidence is not used in any calculations; it is for information purposes only. For example, when sharing datasets among multiple users, it can be useful to know how confident you should be of an entry's ground truth. You must decide the confidence scale to use; all confidence values should follow the chosen convention. This value only applies to an images dataset context. INQ	c	e	f	g	a	i	j	k	l	m	n	b
ControlValue	Specifies one of the following.	c	e	f	g	a	i	j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0.	c	e	f	g	a	i	j	k	l	m	n	b
Value	Specifies the user confidence.	c	e	f	g	a	i	j	k	l	m	n	b
ControlValuePtr	Data type info: Data type: *Set this parameter to M_NULL*	c	e	f	g	a	i	j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

To control an entry or to control a region of an entry, the ControlType and corresponding ControlValue and ControlValuePtr parameter settings can be set to the following values. In this case, set the DatasetContextClassId parameter to the identifier of a dataset context and, for an images dataset context, set the RegionIndex parameter to M_REGION_INDEX(), and for a features dataset context, set the RegionIndex parameter to M_DEFAULT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For an entry or a region of an entry in a dataset context (images or features)															
ControlType ControlValue ControlValuePtr	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_AUTHOR_NAME	Sets the author's name. INQ														
ControlValue	This parameter must be set to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b
ControlValuePtr	Data type info: Data type: *address of an AIL_CONST_TEXT_PTR [optionally, in C++: AIL_STRING]* Specifies the following.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*Name*") 1	Specifies the author's name.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLASS_INDEX_GROUND_TRUTH	Sets the index of the class definition that represents the entry's ground truth. INQ	c		e	f	g	a		i		j	k	l	m	n	b
ControlValue	Specifies one of the following.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies to delete the entry's ground truth, when using whole image regions. To inquire if you are using whole image regions (MclassInquireEntry()), M_REGION_TYPE must return M_WHOLE_IMAGE. To delete the ground truth, you must also set the ControlValuePtr parameter to **M_NULL** and the ControlValuePtrSize parameter to 0.	c		e	f	g	a		i		j	k	l	m	n	b
M_DONT_CARE_CLASS	Specifies that pixels assigned to the region do not contribute to the training loss, when using descriptor based regions (that is, any region index other than 0). To inquire if you are using descriptor based regions (MclassInquireEntry()), M_REGION_TYPE must return M_DESCRIPTOR_BASED. This don't care class (M_DONT_CARE_CLASS) is always available as a class definition that you can assign as the ground truth to a descriptor based region (that is, you do not need to add a don't care class definition). Typically, regions assigned to the don't care class will be areas of the image where the class of the object is unknown. As well, these regions can be assigned to the border pixels between two classes where labeling the edge between them can be difficult. You can set the color of the don't care regions with the M_DONT_CARE_CLASS_DRAW_COLOR control. This value only applies to an images dataset context.	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* < M_NUMBER_OF_CLASSES	Specifies the class definition index (ground truth), as an *integer*.	c		e	f	g	a		i		j	k	l	m	n	b
ControlValuePtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_DESCRIPTOR	Replaces the descriptors for the region. The same restrictions to the descriptors apply here as they do in MclassEntryAddRegion(). You must pass either an image buffer identifier or a graphics list identifier. Region index 0 cannot be replaced. This will only replace descriptors of the same type. If you replace a mask descriptor, the region mask path (M_REGION_MASK_PATH) will remain the same and the mask file on the disk will be overwritten. This control is mainly meant for modifying existing region descriptors. When adding them for the first time it is recommended to use MclassEntryAddRegion(). This value only applies to an images dataset context.	c		e	f	g	a		i		j	k	l	m	n	b

ControlValue	Specifies one of the following.	c	e	f	g	a	i	j	k	l	m	n	b
2D graphics list identifier	Sets the identifier of the 2D graphics list from which to establish the descriptor for the region. Graphics must not be defined in world units.	c	e	f	g	a	i	j	k	l	m	n	b
Image buffer identifier	Sets the identifier of the image buffer from which to establish the added region. This image buffer must not have an ROI associated with it (MbufSetRegion()).	c	e	f	g	a	i	j	k	l	m	n	b
ControlValuePtr	Data type info: Data type: *Set this parameter to M_NULL*	c	e	f	g	a	i	j	k	l	m	n	b
M_DESCRIPTOR_ADD	Adds new descriptors for the region. The resulting group of descriptors must comply with the same restrictions as in MclassEntryAddRegion(). You must pass either an image buffer identifier or a graphics list identifier. Region index 0 cannot be modified. This control is mainly meant for adding to existing region descriptors. When adding them for the first time it is recommended to use MclassEntryAddRegion(). This value only applies to an images dataset context.	c	e	f	g	a	i	j	k	l	m	n	b
ControlValue	Specifies one of the following.	c	e	f	g	a	i	j	k	l	m	n	b
2D graphics list identifier	Sets the identifier of the 2D graphics list from which to establish the descriptor for the region. Graphics must not be defined in world units.	c	e	f	g	a	i	j	k	l	m	n	b
Image buffer identifier	Sets the identifier of the image buffer from which to establish the added region. This image buffer must not have an ROI associated with it (MbufSetRegion()).	c	e	f	g	a	i	j	k	l	m	n	b
ControlValuePtr	Data type info: Data type: *Set this parameter to M_NULL*	c	e	f	g	a	i	j	k	l	m	n	b
M_DESCRIPTOR_DELETE	Deletes all descriptors of a given type. No descriptors can be deleted from region index 0. This value only applies to an images dataset context.	c	e	f	g	a	i	j	k	l	m	n	b
ControlValue	Specifies one of the following.	c	e	f	g	a	i	j	k	l	m	n	b
M_ALL_DESCRIPTORS	Specifies to delete all descriptors.	c	e	f	g	a	i	j	k	l	m	n	b
M_DESCRIPTOR_TYPE_BOX	Specifies to delete all box-type descriptors.	c	e	f	g	a	i	j	k	l	m	n	b
M_DESCRIPTOR_TYPE_MASK	Specifies to delete all mask-type descriptors.	c	e	f	g	a	i	j	k	l	m	n	b
M_DESCRIPTOR_TYPE_POLYGON	Specifies to delete all polygon-type descriptors.	c	e	f	g	a	i	j	k	l	m	n	b
ControlValuePtr	Data type info: Data type: *Set this parameter to M_NULL*	c	e	f	g	a	i	j	k	l	m	n	b
M_REGION_MASK_PATH	Sets the mask path of a region descriptor. The buffer file that this path points to should be a 1-band buffer where all non-zero pixels are assumed to be part of the region. As well, this buffer should have the same size as the entry image pointed to by the control M_ENTRY_IMAGE_PATH. You cannot set the mask path for region index 0. This value only applies to an images dataset context. INQ	c	e	f	g	a	i	j	k	l	m	n	b
ControlValue	This parameter must be set to **M_DEFAULT**.	c	e	f	g	a	i	j	k	l	m	n	b
ControlValuePtr	Data type info: Data type: *address of an AIL_CONST_TEXT_PTR [optionally, in C++: AIL_STRING]* Specifies one of the following.	c	e	f	g	a	i	j	k	l	m	n	b
AIL_TEXT("*///DATASET///FilePath*") 1	Specifies the file path under the folder path from where the dataset context is restored, using MclassRestore(). If the ///DATASET/// path does not exist, Aurora Imaging Library uses the path in which your application's executable file is located.	c	e	f	g	a	i	j	k	l	m	n	b
AIL_TEXT("*FilePath*") 1	Specifies the file path.	c	e	f	g	a	i	j	k	l	m	n	b
M_REGION_USE	Sets whether the specified region will be used. This value only applies to an images dataset context.	c	e	f	g	a	i	j	k	l	m	n	b
ControlValue	Specifies one of the following.	c	e	f	g	a	i	j	k	l	m	n	b
M_DEFAULT	Same as M_USE.	c	e	f	g	a	i	j	k	l	m	n	b
M_IGNORE	Specifies that the region is to be ignored.	c	e	f	g	a	i	j	k	l	m	n	b
M_USE	Specifies that the region can be used.	c	e	f	g	a	i	j	k	l	m	n	b

	ControlValuePtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailclass11.lib.
DLL	Requires ail11.dll; ailclass11.dll.

MclassCopy

Synopsis

Copy data from one classification context to another.

Syntax

```
void MclassCopy(
    AIL_ID SrcContextClassId, //in
    AIL_INT64 SrcIndex,       //in
    AIL_ID DstContextClassId, //out
    AIL_INT64 DstIndex,       //in
    AIL_INT64 CopyType,       //in
    AIL_INT64 ControlFlag     //in
)
```

Description

This function copies data from the specified source classification context to the specified destination classification context. The source and the destination context must be for the same type of classifier (predefined CNN, predefined object detection, predefined segmentation, predefined anomaly detection, tree ensemble, or an ONNX classifier). For example, you can copy data from one images dataset to another, or from an images dataset to a predefined CNN classifier context.

Parameters

SrcContextClassId

Specifies the identifier of the source classification context from which to copy. Set this parameter to one of the following values.

For specifying the source classification context	
Value	Description
ContextClassifierId	Specifies the identifier of a predefined CNN, predefined object detection, predefined segmentation, predefined anomaly detection, tree ensemble, or an ONNX classifier context. This context is allocated using MclassAlloc() with M_CLASSIFIER_CNN_PREDEFINED, M_CLASSIFIER_DET_PREDEFINED, M_CLASSIFIER_SEG_PREDEFINED, M_CLASSIFIER_ANO_PREDEFINED, M_CLASSIFIER_TREE_ENSEMBLE, or M_CLASSIFIER_ONNX. > This is not available when the CopyType parameter is set to M_AUTHORS or M_DATASET.
ContextDatasetFeaturesId	Specifies the identifier of a features dataset context. This context is allocated using MclassAlloc() with M_DATASET_FEATURES. If the source is a features dataset context, the destination must be a features dataset context or a tree ensemble classifier context.
ContextDatasetImagesId	Specifies the identifier of an images dataset context. This context is allocated using MclassAlloc() with M_DATASET_IMAGES. If the source is an images dataset context, the destination must be an images dataset context, a predefined CNN classifier context, a predefined object detection classifier context, a predefined segmentation classifier context, or a predefined anomaly detection classifier context.

SrcIndex

Reserved for future expansion and must be set to **M_DEFAULT**.

DstContextClassId

Specifies the identifier of the destination classification context in which to copy. Set this parameter to one of the following values.

For specifying the destination classification context	
Value	Description
ContextClassifierId	Specifies the identifier of a predefined CNN, predefined object detection, predefined segmentation, predefined anomaly detection, tree ensemble, or an ONNX classifier context. This context is allocated using MclassAlloc() with M_CLASSIFIER_CNN_PREDEFINED, M_CLASSIFIER_DET_PREDEFINED, M_CLASSIFIER_SEG_PREDEFINED, M_CLASSIFIER_ANO_PREDEFINED, M_CLASSIFIER_TREE_ENSEMBLE, or M_CLASSIFIER_ONNX. > This is not available when the CopyType parameter is set to M_AUTHORS or M_DATASET.
ContextDatasetFeaturesId	Specifies the identifier of the features dataset context. This context is allocated using MclassAlloc() with M_DATASET_FEATURES. > This is not available when the CopyType parameter is set to M_DATASET and SrcContextClassId is set to an identifier of an images dataset context.
ContextDatasetImagesId	Specifies the identifier of the images dataset context. This context is allocated using MclassAlloc() with M_DATASET_IMAGES. > This is not available when the CopyType parameter is set to M_DATASET and SrcContextClassId is set to an identifier of a features dataset context.

DstIndex

Reserved for future expansion and must be set to **M_DEFAULT**.

CopyType

Specifies the type of copy operation to perform.

Specifies the type of copy operation	
Value	Description

M_AUTHORS	Copies authors from the source dataset to the destination dataset or classifier object. This overwrites any existing authors in the destination object. If the destination is a dataset, it should not contain any entries. > When using M_AUTHORS, you must set the SrcContextClassId parameter to the identifier of a features dataset or images dataset context. The DstContextClassId parameter must be set to the identifier of a features dataset or images dataset context.
M_CLASS_DEFINITIONS	Copies the class definitions from the source classification context to the destination classification context. Class definitions already in the destination are overwritten. > When using M_CLASS_DEFINITIONS, you must set the SrcContextClassId parameter to the identifier of a predefined CNN classifier, a predefined object detection classifier, a predefined segmentation classifier, a predefined anomaly detection classifier, tree ensemble classifier, ONNX classifier, features dataset, or images dataset context. The DstContextClassId parameter must be set to the identifier of a predefined CNN classifier, a predefined object detection classifier, a predefined segmentation classifier, a predefined anomaly detection classifier, tree ensemble classifier, ONNX classifier, features dataset, or images dataset context.
M_DATASET	Copies the entire dataset. This overwrites the entries, class definitions, authors, and controls related to the dataset (for example, M_ROOT_PATH). Note, included in the entries are the mask and segmentation paths (the files themselves are not copied). > When using M_DATASET, you must set the SrcContextClassId parameter to the identifier of a features or images dataset context. The DstContextClassId parameter must be set to the identifier of a features or images dataset context. The source and the destination context must be for the same type of dataset.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailclass11.lib.
DLL	Requires ail11.dll; ailclass11.dll.

MclassCopyResult

Synopsis

Copy data from a training result buffer to a classifier or dataset context.

Syntax

```
void MclassCopyResult(
    AIL_ID SrcResultClassId,  //in
    AIL_INT64 SrcIndex,       //in
    AIL_ID DstContextClassId, //out
    AIL_INT64 DstIndex,       //in
    AIL_INT64 CopyType,       //in
    AIL_INT64 ControlFlag     //in
)
```

Description

This function copies data from a training result buffer to a classifier or dataset context. For example, you can copy the trained classifier result to a classifier context. The result buffer and classifier or dataset context must be compatible (that is, they must be for the same type of classifier).

You can only copy trained results after calling MclassTrain(). Once you copy the trained results into a classifier context, you can continue to train that context (MclassTrain()) or you can predict with it (MclassPredict()).

Parameters

SrcResultClassId

Specifies the identifier of the source training result buffer from which to copy. The result buffer must have been allocated using MclassAllocResult(). The type of result buffer that you can specify depends on the copy operation (CopyType). Set this parameter to the following value.

For specifying the source result	
Value	Description
ResultBufTrainId	Specifies the identifier of a training result buffer (M_TRAIN_CNN_RESULT, M_TRAIN_DET_RESULT, M_TRAIN_SEG_RESULT, M_TRAIN_ANO_RESULT, or M_TRAIN_TREE_ENSEMBLE_RESULT) that holds the results produced from calling MclassTrain() with a CNN training context, an object detection training context, a segmentation training context, an anomaly detection training context, or tree ensemble training context.

SrcIndex

Reserved for future expansion and must be set to **M_DEFAULT**.

DstContextClassId

Specifies the identifier of the destination classifier or dataset context in which to copy. The classifier or dataset context must have been allocated using MclassAlloc(). The type of result buffer that you can specify depends on the copy operation (CopyType). Set this parameter to one of the following values.

For specifying the destination classification context	
Value	Description
ContextClassifierPredefinedId	Specifies the identifier of a predefined CNN, object detection, segmentation, or anomaly detection classifier context (M_CLASSIFIER_CNN_PREDEFINED, M_CLASSIFIER_DET_PREDEFINED, M_CLASSIFIER_SEG_PREDEFINED, or M_CLASSIFIER_ANO_PREDEFINED).
ContextClassifierTreeEnsembleId	Specifies the identifier of a tree ensemble classifier context (M_CLASSIFIER_TREE_ENSEMBLE).
ContextDatasetFeaturesId	Specifies the identifier of a features dataset context (M_DATASET_FEATURES).
ContextDatasetImagesId	Specifies the identifier of an images dataset context (M_DATASET_IMAGES).

DstIndex

Reserved for future expansion and must be set to **M_DEFAULT**.

CopyType

Specifies the type of copy operation to perform.

Specifies the type of copy operation	
Value	Description
M_DEV_DATASET_PREDICT_SCORES	Copies the predicted scores of the development dataset established by MclassTrain(). To perform this copy operation, you must set the SrcResultClassId parameter to the identifier of a CNN or tree ensemble training result buffer, and the DstContextClassId parameter to the identifier of a features or images dataset context. If you used a tree ensemble classifier and did not specify a development dataset with MclassTrain(), nothing is copied. Copies are only performed if the corresponding entry already exists in the destination dataset (otherwise, nothing is copied).
M_OUT_OF_BAG_PREDICT_SCORES	Copies the predicted scores of the out-of-bag entries established by MclassTrain(). To perform this copy operation, you must set the SrcResultClassId parameter to the identifier of a tree ensemble training result buffer and the DstContextClassId parameter to the identifier of a features dataset context. Note, bagging information is typically unreliable if your training dataset has augmented entries.

M_PREPARED_DEV_DATASET	Copies the development dataset from the result to the destination dataset. To perform this copy operation, you must set the SrcResultClassId parameter to the identifier of a CNN, object detection, segmentation, or anomaly detection training result buffer, and the DstContextClassId parameter to the identifier of an images dataset context.
M_PREPARED_TRAIN_DATASET	Copies the train dataset from the result to the destination dataset. To perform this copy operation, you must set the SrcResultClassId parameter to the identifier of a CNN, object detection, segmentation, or anomaly detection training result buffer, and the DstContextClassId parameter to the identifier of an images dataset context.
M_TRAIN_DATASET_PREDICT_SCORES	Copies the predicted scores of the training dataset established by MclassTrain(). To perform this copy operation, you must set the SrcResultClassId parameter to the identifier of a CNN or tree ensemble training result buffer, and the DstContextClassId parameter to the identifier of a features or images dataset context. Copies are only performed if the corresponding entry already exists in the destination dataset (otherwise, nothing is copied).
M_TRAINED_CLASSIFIER	Copies the trained classifier context established by MclassTrain(). To perform this copy operation, you must set the SrcResultClassId parameter to the identifier of a training result buffer, and the DstContextClassId parameter to the identifier of a classifier context. This is available for CNN, object detection, segmentation, anomaly detection, and tree ensemble classifiers; both the SrcResultClassId and the DstContextClassId parameter must contain the same type of classifier.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailclass11.lib.
DLL	Requires ail11.dll; ailclass11.dll.

MclassDraw

Synopsis

Draw specific features of a classifier context, a dataset context, or a prediction result buffer.

Syntax

```
void MclassDraw(
    AIL_ID ContextGraId,           //in
    AIL_ID ContextOrResultClassId, //in
    AIL_ID DstImageBufOrListGraId, //out
    AIL_INT64 Operation,           //in
    AIL_INT64 Index,               //in
    AIL_INT64 ControlFlag          //in
)
```

Description

This function draws specific features of a classifier context, a dataset context, or a prediction result buffer. Drawing operations are typically performed in a destination image buffer. You cannot draw from a tree ensemble result buffer, and you cannot perform any drawing operations with ONNX.

By default, this function performs the drawing operation for every class in the specified context or result. To draw for a specific class or specific instance of a class, specify its index (Index).

Parameters

ContextGraId

Specifies the identifier of the 2D graphics context to use when drawing. This parameter must be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the 2D graphics context																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies that the default 2D graphics context of the current Aurora Imaging Library application is used. Note that there is a different default 2D graphics context for each thread.	c		e	f	g	a		i		j	k	l	m	n	b
2D graphics context identifier	Specifies a valid 2D graphics context identifier, which you have allocated using MgraAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

ContextOrResultClassId

Specifies the identifier of the classifier or dataset context, or prediction result buffer from which to draw.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the classifier or dataset context, or prediction result buffer																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ContextClassifierPredefinedId	Specifies the identifier of the predefined CNN, object detection, segmentation, or anomaly detection classifier context. This context is allocated using MclassAlloc() with M_CLASSIFIER_CNN_PREDEFINED, M_CLASSIFIER_DET_PREDEFINED, M_CLASSIFIER_SEG_PREDEFINED, or M_CLASSIFIER_ANO_PREDEFINED.	c		e	f	g	a		i		j	k	l	m	n	b
ContextClassifierTreeEnsembleId	Specifies the identifier of a tree ensemble classifier context. This context is allocated using MclassAlloc() with M_CLASSIFIER_TREE_ENSEMBLE.	c		e	f	g	a		i		j	k	l	m	n	b
ContextDatasetId	Specifies the identifier of an images or features dataset context. These contexts must have been previously allocated on the required system using MclassAlloc() with M_DATASET_IMAGES or M_DATASET_FEATURES.	c		e	f	g	a		i		j	k	l	m	n	b
ResultClassPredictId	Specifies the identifier of the CNN, object detection, segmentation, or anomaly detections, prediction result buffer. This buffer is allocated using MclassAllocResult() with M_PREDICT_CNN_RESULT, M_PREDICT_DET_RESULT M_PREDICT_SEG_RESULT, or M_PREDICT_ANO_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b

DstImageBufOrListGraId

Specifies the identifier of the destination image buffer or LUT buffer in which to draw. You should typically specify an image, which can be any valid Aurora Imaging Library image buffer allocated using MbufAlloc...(). By drawing into a display's overlay buffer, you can also annotate an image non-destructively. To specify a LUT, you must use the M_DRAW_CLASS_COLOR_LUT drawing operation.

When drawing from a CNN or segmentation prediction result buffer, the expected size of the destination image buffer is, by default, the same as the target image. This assumes prediction results are from a typical image classification process.

When drawing from an object detection prediction result buffer, the destination image buffer in which to draw should either be the same image used in the prediction, or a graphics list.

Note, the classification map (the output of the network) for a segmentation prediction result is, by default, scaled by M_RECEPTIVE_FIELD_STRIDE_X and M_RECEPTIVE_FIELD_STRIDE_Y, and placed at M_RECEPTIVE_FIELD_OFFSET_X and M_RECEPTIVE_FIELD_OFFSET_Y (these scale and map values are retrievable using MclassGetResult()). If you add M_REPLICATE_BORDER to the drawing operation, Aurora Imaging Library expands (copies) the edges of the classification map to reach the edges of the destination buffer.

You can modify the default scale and offset map behavior by adding M_NETWORK_OUTPUT_SIZE to the drawing operation. This performs the drawing operation according to the exact classification map (output of the network) without scaling or expanding. In this case, the destination buffer size should be equal to the resulting X- and Y-size of the classification map; to retrieve these dimensions, call MclassGetResult() with M_PREDICTION_SIZE_X and M_PREDICTION_SIZE_Y.

Operation

Specifies the drawing operation to perform.

The following Operation parameter values can be set for a predefined classifier context (CNN, segmentation, or object detection), a tree ensemble classifier context, or a dataset context (images or features).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a predefined (CNN, segmentation, or object detection) or tree ensemble classifier context, or a dataset context (images or features)																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DRAW_CLASS_COLOR_LUT	Draws the color (LUT) associated to the class. This color is defined using MclassControl() with M_CLASS_DRAW_COLOR. When performing this operation, you must set the destination buffer (DstImageBufOrListGrald) to the identifier of an Aurora Imaging Library LUT buffer. To inquire the width of this LUT, use M_NUMBER_OF_CLASSES.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_CLASS_ICON	Draws the icon image associated to the class. This image is defined using MclassControl() with M_CLASS_ICON_ID. When performing this operation, you must set the destination buffer (DstImageBufOrListGrald) to the identifier of an Aurora Imaging Library image buffer. To inquire the format of this buffer (DstImageBufOrListGrald), such as the X- and Y-size, use M_CLASS_ICON_ID (the dimension of the destination image in which you draw should be the same as the class description's icon image).	c		e	f	g	a		i		j	k	l	m	n	b

The following Operation parameter values can be set for a CNN or segmentation prediction result buffer. These operations require an image buffer as the destination in which to draw (DstImageBufOrListGrald).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a CNN or segmentation prediction result buffer																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DRAW_BEST_INDEX_CONTOUR_IMAGE +	Draws the contours of the best classes. If you do not specify the M_PSEUDO_COLOR combination value, Aurora Imaging Library uses the result type M_BEST_INDEX_IMAGE_TYPE to determine the type of the image to use. If you specify M_PSEUDO_COLOR, the image type must be 3 bands, 8-bit unsigned. Note, this drawing operation supports combination values from the For modifying the drawing operation table. The combination values that you can specify depend on the type of prediction result buffer that you are using: • For CNN prediction results, you can specify M_NETWORK_OUTPUT_SIZE, M_PSEUDO_COLOR, and M_REPLICATE_BORDER. • For segmentation prediction results, you can specify M_PSEUDO_COLOR.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_BEST_INDEX_IMAGE +	Draws an image using the index value of the best class at each point. If you do not specify the M_PSEUDO_COLOR combination value, Aurora Imaging Library uses the result type M_BEST_INDEX_IMAGE_TYPE to determine the type of the image to use. If you specify M_PSEUDO_COLOR, the image type must be 3 bands, 8-bit unsigned. Note, this drawing operation supports combination values from the For modifying the drawing operation table. The combination values that you can specify depend on the type of prediction result buffer that you are using: • For CNN prediction results, you can specify M_NETWORK_OUTPUT_SIZE, M_PSEUDO_COLOR, and M_REPLICATE_BORDER. • For segmentation prediction results, you can specify M_PSEUDO_COLOR.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_BEST_SCORE_IMAGE +	Draws an image using the best score at each point. If you do not specify the M_PSEUDO_COLOR combination value, the image type must be float or unsigned. If you specify M_PSEUDO_COLOR, the image type must be 3 bands, 8-bit unsigned. Note, this drawing operation supports combination values from the For modifying the drawing operation table. The combination values that you can specify depend on the type of prediction result buffer that you are using: • For CNN prediction results, you can specify M_NETWORK_OUTPUT_SIZE, M_PSEUDO_COLOR, and M_REPLICATE_BORDER. • For segmentation prediction results, you can specify M_PSEUDO_COLOR.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_CLASS_SCORES +	Draws (fills) an image using the scores of a specific class. This drawing operation is not possible when drawing general results; you must set the Index parameter to a specific class. Note, this drawing operation supports combination values from the For modifying the drawing operation table. The combination values that you can specify depend on the type of prediction result buffer that you are using: • For CNN prediction results, you can specify M_NETWORK_OUTPUT_SIZE and M_REPLICATE_BORDER. If you specify M_NETWORK_OUTPUT_SIZE, the drawing is done with the classification map size (the ControlFlag parameter must be set to its default). If you do not specify the M_NETWORK_OUTPUT_SIZE combination value, you can set the ControlFlag parameter to M_BILINEAR_INTERPOLATION or M_RECEPTIVE_FIELD_PATCH. In all cases, when using M_NETWORK_OUTPUT_SIZE, the image type must be float or 8-bit unsigned. • For segmentation prediction results, you cannot specify any combination value. • You cannot specify M_PSEUDO_COLOR for any predefined classifier.	c		e	f	g	a		i		j	k	l	m	n	b

The following Operation parameter values can be set for an object detection prediction result buffer. For these operations, the destination in which to draw (DstImageBufOrListGrald) should be either the same image used in the prediction, or a graphics list.

Multiple drawing operations for object detection can be used together by adding them. For example, M_DRAW_BOX + M_DRAW_BOX_NAME. This is only available for values in this table.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For an object detection prediction result buffer																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DRAW_BOX	Draws the bounding boxes for all instances, a specific class, or a specific instance (only one box will be drawn in this case). The drawn color is specified in the class definitions. To change the line thickness of the box, you can use MgraControl() with M_LINE_THICKNESS on the ContextGraId.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_BOX_CENTER	Draws the center positions of the bounding boxes for all instances, a specific class, or a specific instance (only one center position will be drawn in this case). The drawn color is specified in the class definitions. To change the line thickness of the center position, you can use MgraControl() with M_LINE_THICKNESS with the graphics context specified by the ContextGraId parameter.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_BOX_NAME	Draws the class names for bounding boxes for all instances, a specific class, or a specific instance (only one name will be drawn in this case). The class names are drawn using the graphics context specified by the ContextGraId parameter. You can modify the text settings of the drawn names using MgraControl(). If used with M_DRAW_BOX, the class name is drawn on top of the outline of the bounding box. If used with M_DRAW_BOX_CENTER, the class name is drawn near the center of the bounding box. Text alignment controls set with MgraControl() are relative to these locations.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_BOX_SCORE	Draws the class scores for bounding boxes for all instances, a specific class, or a specific instance (only one score will be drawn in this case). The class scores are drawn using the graphics context specified by the ContextGraId parameter. You can modify the text settings of the drawn scores using MgraControl(). If used with M_DRAW_BOX, the class score is drawn on top of the outline of the bounding box. If used with M_DRAW_BOX_CENTER, the class score is drawn near the center of the bounding box. Text alignment controls set with MgraControl() are relative to these locations.	c		e	f	g	a		i		j	k	l	m	n	b

The following Operation parameter values can be set for an anomaly detection prediction result buffer. For these operations, the destination in which to draw (DstImageBufOrListGraId) should be either the same image used in the prediction, or a graphics list.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For an anomaly detection prediction result buffer																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DRAW_ANOMALY_CONTOUR_MASK	Draws the contour of the anomalies using the color of the graphics context specified by the ContextGraId parameter. To change the color, you can use MgraControl() with M_COLOR on the graphics context specified by the ContextGraId parameter. Note, DstImageBufOrListGraId must be an image buffer of type M_UNSIGNED. Additionally, if the graphics context specified by the ContextGraId parameter is not grayscale, DstImageBufOrListGraId must be 3 bands and 8-bit.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_ANOMALY_HEATMAP	Draws the anomaly scores projected as a heat map for each pixel location in the image. Note, DstImageBufOrListGraId must be an 8-bit and 3-band image buffer of type M_UNSIGNED.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_ANOMALY_MASK	Draws the anomalous pixels using the color of the graphics context specified by the ContextGraId parameter. To change the color, you can use MgraControl() with M_COLOR on the graphics context specified by the ContextGraId parameter. Note, DstImageBufOrListGraId must be an image buffer of type M_UNSIGNED. Additionally, if the graphics context specified by the ContextGraId parameter is not grayscale, DstImageBufOrListGraId must be 3 bands and 8-bit.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_ANOMALY_SCORES	Draws the anomaly score for each pixel location in the image. Note, DstImageBufOrListGraId must be a 1-band image buffer of type M_FLOAT.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for M_DRAW_BEST_INDEX_CONTOUR_IMAGE; M_DRAW_BEST_INDEX_IMAGE; M_DRAW_BEST_SCORE_IMAGE; M_DRAW_CLASS_SCORES.

You can add one or more of the following values to the above-mentioned values to specify a modification to the drawing operation.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For modifying the drawing operation																	
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NETWORK_OUTPUT_SIZE	Draws according to the exact classification map size (output of the network) without scaling or expanding. In this case, the expected size of the destination buffer (DstImageBufOrListGraId) is the same as the X- and Y-size of M_PREDICTION_SIZE_X and M_PREDICTION_SIZE_Y. You cannot use M_NETWORK_OUTPUT_SIZE with M_REPLICATE_BORDER.	c		e	f	g	a		i		j	k	l	m	n	b
M_PSEUDO_COLOR	Draws using the pseudo colors defined by calling MclassControl() with M_CLASS_DRAW_COLOR. For all drawing operations, if you specify M_PSEUDO_COLOR, the destination buffer (DstImageBufOrListGraId) image type must be 3 bands, 8-bit unsigned. Depending on the draw operation, you can use M_PSEUDO_COLOR with other combination values.	c		e	f	g	a		i		j	k	l	m	n	b
M_REPLICATE_BORDER	Draws with borders expanded to the size of the destination buffer (DstImageBufOrListGraId). You cannot use M_REPLICATE_BORDER with M_NETWORK_OUTPUT_SIZE.	c		e	f	g	a		i		j	k	l	m	n	b

Index

Specifies what to use to perform the drawing operation.

Set this parameter to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying what to use to perform the drawing operation																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_CONTEXT, if the ContextOrResultClassId parameter is a predefined CNN, segmentation, object detection, or anomaly detection classifier context, a tree ensemble classifier context, an images dataset context, or a features dataset context; same as M_GENERAL, if the ContextOrResultClassId is a CNN, segmentation, object detection, or anomaly detection prediction result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLASS_INDEX(AIL_INT Index)	Specifies that the drawing operation is for a specific class in the specified context or result buffer (ContextOrResultClassId). You can draw for a specific class when specifying a CNN prediction result buffer (M_PREDICT_CNN_RESULT), an object detection prediction result buffer (M_PREDICT_DET_RESULT), a segmentation prediction result buffer (M_PREDICT_SEG_RESULT), a predefined CNN classifier context (M_CLASSIFIER_CNN_PREDEFINED), a predefined object detection classifier context (M_CLASSIFIER_DET_PREDEFINED), a predefined segmentation classifier context (M_CLASSIFIER_SEG_PREDEFINED), a tree ensemble classifier context (M_CLASSIFIER_TREE_ENSEMBLE), an images dataset context(M_DATASET_IMAGES), or a features dataset context (M_DATASET_FEATURES).	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	Index Set this parameter to the following:															
	Value >= 0 — Specifies the index of the class for which to draw.	c		e	f	g	a		i		j	k	l	m	n	b
M_INSTANCE_INDEX(AIL_INT Index)	Specifies that the drawing operation is for a specific instance in the specified object detection prediction result buffer (ContextOrResultClassId).	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	Index Set this parameter to the following:															
	Value >= 0 — Specifies the index of the instance for which to draw.	c		e	f	g	a		i		j	k	l	m	n	b
M_CONTEXT	Specifies that the drawing operation is for every class in the classifier context or the dataset context (ContextOrResultClassId).	c		e	f	g	a		i		j	k	l	m	n	b
M_GENERAL	Specifies that the drawing operation is for every class in the CNN, segmentation, object detection, or anomaly detection prediction result buffer (ContextOrResultClassId).	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Specifies how to control the drawing operation.

For all drawing operations except M_DRAW_CLASS_SCORES, set this parameter to M_DEFAULT. Typically, these drawing operations apply to segmentation prediction results.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying how to control the class score drawing																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Implements the default behavior. This is the only possible setting for all drawing operations except M_DRAW_CLASS_SCORES.	c		e	f	g	a		i		j	k	l	m	n	b
M_BILINEAR_INTERPOLATION	Scales the classification map with a bilinear calculation, when using M_DRAW_CLASS_SCORES. The scale is established in receptive field calculations. The scaled image is placed at the offset of the receptive field.	c		e	f	g	a		i		j	k	l	m	n	b
M_RECEPTIVE_FIELD_PATCH	Fills the corresponding receptive fields with calculated scores, when using M_DRAW_CLASS_SCORES. Scores are drawn in order, from low to high. The best score is therefore drawn over lower scores.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailclass11.lib.
DLL	Requires ail11.dll; ailclass11.dll.

MclassDrawEntry

Synopsis

Draw region descriptors, classification results, segmentation results, object detection results, or anomaly detection results from an entry in an images dataset context.

Syntax

```
void MclassDrawEntry(
    AIL_ID ContextGraId,           //in
    AIL_ID DatasetContextClassId,  //in
    AIL_ID DstImageBufOrListGraId, //out
    AIL_INT64 Operation,           //in
    AIL_INT64 EntryIndex,          //in
    AIL_UUID EntryKey,             //in
    AIL_INT64 TaskType,            //in
    AIL_INT64 Index,               //in
    AIL_INT *DstLabelArrayPtr,     //out
    AIL_INT64 ControlFlag          //in
)
```

Description

This function draws specific region descriptors, classification results, segmentation results, object detection results, or anomaly detection results from an entry in an images dataset context. Drawing operations are typically performed in a destination image buffer.

For datasets with more than 256 class definitions, using default colors will result in repeated colors.

Parameters

ContextGraId

Specifies the identifier of the graphics context to use for the draw operation.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the 2D graphics context

Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies that the default 2D graphics context of the current Aurora Imaging Library application is used. Note that there is a different default 2D graphics context for each thread.	c		e	f	g	a		i		j	k	l	m	n	b
2D graphics context identifier	Specifies a valid 2D graphics context identifier, which you have allocated using MgraAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

DatasetContextClassId

Specifies the images dataset context from which to draw the entry.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the image dataset context

Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
DatasetContextClassId	Specifies the identifier of an images dataset context. This context must have been previously allocated on the required system using MclassAlloc() with M_DATASET_IMAGES.	c		e	f	g	a		i		j	k	l	m	n	b

DstImageBufOrListGraId

Specifies the identifier of the destination image buffer or 2D graphics list in which to draw. This can be either an image buffer or a graphics list identifier depending on the draw operation.

When drawing from an object detection prediction result stored in an entry of a dataset, the destination in which to draw should either be the same image used in the prediction, or a graphics list.

Operation

Specifies the drawing operation. Set this parameter to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the drawing operation

Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DESCRIPTOR_TYPE_BOX +	Draws the bounding box descriptor from the specified region using the color specified in the dataset (ContextGraId). To change the color, call MgraControl() with M_COLOR on the specified graphics context (ContextGraId). To check for the presence of a bounding box descriptor, call MclassInquireEntry() with M_NUMBER_OF_DESCRIPTOR_TYPE_BOX. Note that when no descriptor is present in the region, nothing is drawn. M_DESCRIPTOR_TYPE_BOX is only available for object detection.	c		e	f	g	a		i		j	k	l	m	n	b

M_DESCRIPTOR_TYPE_MASK +	Draws the mask descriptor from the specified region using the color specified in the dataset (ContextGrald). To change the color, call MgraControl() with M_COLOR on the specified graphics context (ContextGrald). To check for the presence of a mask descriptor, call MclassInquireEntry() with M_NUMBER_OF_DESCRIPTOR_TYPE_MASK. Note that the destination must be an image buffer and not a graphics list. When no descriptor is present in the region, nothing is drawn. M_DESCRIPTOR_TYPE_MASK is only available for segmentation and anomaly detection.	c		e	f	g	a		i		j	k	l	m	n	b
M_DESCRIPTOR_TYPE_POLYGON +	Draws the polygon descriptor from the specified region using the color specified in the dataset (ContextGrald). To change the color, call MgraControl() with M_COLOR on the specifies graphics context (ContextGrald). To check for the presence of a polygon descriptor, call MclassInquireEntry() with M_NUMBER_OF_DESCRIPTOR_TYPE_POLYGON. Note that the required size of the DstLabelArrayPtr is M_NUMBER_OF_DESCRIPTOR_TYPE_POLYGON for a single region. When no descriptor is present in the region, nothing is drawn. M_DESCRIPTOR_TYPE_POLYGON is only available for segmentation and anomaly detection.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_ANOMALY_CONTOUR_MASK	Draws the contour of the anomalies using the color of the graphics context specified by the ContextGrald parameter. To change the color, you can use MgraControl() with M_COLOR on the graphics context specified by the ContextGrald parameter. Note, DstImageBufOrListGrald must be an image buffer of type M_UNSIGNED. Additionally, if the graphics context specified by the ContextGrald parameter is not grayscale, DstImageBufOrListGrald must be 3 bands and 8-bit.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_ANOMALY_HEATMAP	Draws the anomaly scores for each pixel location in the image. Note, DstImageBufOrListGrald must be an 8-bit and 3-band image buffer of type M_UNSIGNED.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_ANOMALY_MASK	Draws the anomalous pixels using the color of the graphics context specified by the ContextGrald parameter. To change the color, you can use MgraControl() with M_COLOR on the graphics context specified by the ContextGrald parameter. Note, DstImageBufOrListGrald must be an image buffer of type M_UNSIGNED. Additionally, if the graphics context specified by the ContextGrald parameter is not grayscale, DstImageBufOrListGrald must be 3 bands and 8-bit.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_ANOMALY_SCORES	Draws the anomaly scores for each pixel location in the image. Note, DstImageBufOrListGrald must be a 1-band image buffer of type M_FLOAT.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_BEST_INDEX_CONTOUR_IMAGE +	Draws the contours of the best classes. If you do not specify the M_PSEUDO_COLOR combination value, Aurora Imaging Library uses the result type M_BEST_INDEX_IMAGE_TYPE to determine the type of the image to use. If you specify M_PSEUDO_COLOR, the image type must be 3 bands, 8-bit unsigned. Note, this drawing operation supports combination values from the For modifying the draw entry operation table. The combination values that you can specify depend on the type of predefined classifier that you are using (MclassGetResultEntry() with M_CLASSIFIER_PREDEFINED_TYPE): • For M_ICNET_... or legacy M_FCNET_..., you can specify M_NETWORK_OUTPUT_SIZE, M_PSEUDO_COLOR, and M_REPLICATE_BORDER. • For M_CSNET_..., you can specify M_PSEUDO_COLOR.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_BEST_INDEX_IMAGE +	Draws an image using the index value of the best class at each point. If you do not specify the M_PSEUDO_COLOR combination value, Aurora Imaging Library uses the result type M_BEST_INDEX_IMAGE_TYPE to determine the type of the image to use. If you specify M_PSEUDO_COLOR, the image type must be 3 bands, 8-bit unsigned. Note, this drawing operation supports combination values from the For modifying the draw entry operation table. The combination values that you can specify depend on the type of predefined classifier that you are using (MclassGetResultEntry() with M_CLASSIFIER_PREDEFINED_TYPE): • For M_ICNET_... or legacy M_FCNET_..., you can specify M_NETWORK_OUTPUT_SIZE, M_PSEUDO_COLOR, and M_REPLICATE_BORDER. • For M_CSNET_..., you can specify M_PSEUDO_COLOR.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_BEST_SCORE_IMAGE +	Draws an image using the best score at each point. If you do not specify the M_PSEUDO_COLOR combination value, the image type must be float or unsigned. If you specify M_PSEUDO_COLOR, the image type must be 3 bands, 8-bit unsigned. Note, this drawing operation supports combination values from the For modifying the draw entry operation table. The combination values that you can specify depend on the type of predefined classifier that you are using (MclassGetResultEntry() with M_CLASSIFIER_PREDEFINED_TYPE): • For M_ICNET_... or legacy M_FCNET_..., you can specify M_NETWORK_OUTPUT_SIZE, M_PSEUDO_COLOR, and M_REPLICATE_BORDER. • For M_CSNET_..., you can specify M_PSEUDO_COLOR.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_BOX	Draws the bounding boxes for all instances, a specific class, or a specific instance (only one box will be drawn in this case). The drawn color is specified in the class definitions. To change the line thickness of the box, call MgraControl() with M_LINE_THICKNESS on the specified graphics context (ContextGrald). Multiple drawing operations for object detection can be used together by adding them. For example, M_DRAW_BOX + M_DRAW_BOX_NAME.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_BOX_CENTER	Draws the center positions of the bounding boxes for all instances, a specific class, or a specific instance (only one center position will be drawn in this case). The drawn color is specified in the class definitions. To change the line thickness of center position, call MgraControl() with M_LINE_THICKNESS on the specified graphics context (ContextGrald). Multiple drawing operations for object detection can be used together by adding them. For example, M_DRAW_BOX + M_DRAW_BOX_NAME.	c		e	f	g	a		i		j	k	l	m	n	b

M_DRAW_BOX_NAME	Draws the class names for bounding boxes for all instances, a specific class, or a specific instance (only one name will be drawn in this case). The class names are drawn using the graphics context specified by the ContextGraId parameter. You can modify the text settings of the drawn names using MgraControl(). Multiple drawing operations for object detection can be used together by adding them. For example, M_DRAW_BOX + M_DRAW_BOX_NAME. If used with M_DRAW_BOX, the class name is drawn on top of the outline of the bounding box. If used with M_DRAW_BOX_CENTER, the class name is drawn near the center of the bounding box. Text alignment controls set with MgraControl() are relative to these locations.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_BOX_SCORE	Draws the class scores for bounding boxes for all instances, a specific class, or a specific instance (only one score will be drawn in this case). The class scores are drawn using the graphics context specified by the ContextGraId parameter. You can modify the text settings of the drawn scores using MgraControl(). Multiple drawing operations for object detection can be used together by adding them. For example, M_DRAW_BOX + M_DRAW_BOX_NAME. If used with M_DRAW_BOX, the class score is drawn on top of the outline of the bounding box. If used with M_DRAW_BOX_CENTER, the class score is drawn near the center of the bounding box. Text alignment controls set with MgraControl() are relative to these locations.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_CLASS_SCORES +	Draws (fills) an image using the scores of a specific class. This drawing operation is not possible when drawing general results; you must set the Index parameter to a specific class. Note, this drawing operation supports combination values from the For modifying the draw entry operation table. The combination values that you can specify depend on the type of predefined classifier that you are using (MclassGetResultEntry() with M_CLASSIFIER_PREDEFINED_TYPE): • For M_ICNET_... or legacy M_FCNET_..., you can specify M_NETWORK_OUTPUT_SIZE and M_REPLICATE_BORDER. If you specify M_NETWORK_OUTPUT_SIZE, the drawing is done with the classification map size (the ControlFlag parameter must be set to its default). If you do not specify the M_NETWORK_OUTPUT_SIZE combination value, you can set the ControlFlag parameter to M_BILINEAR_INTERPOLATION or M_RECEPTIVE_FIELD_PATCH. In all cases, when using M_NETWORK_OUTPUT_SIZE, the image type must be float or 8-bit unsigned. • For M_CSNET_..., you cannot specify any combination value. • You cannot specify M_PSEUDO_COLOR for any predefined classifier.	c		e	f	g	a		i		j	k	l	m	n	b
M_GROUND_TRUTH_IMAGE +	Draws all the mask regions using their class index for pixel values. Unlabeled pixels are not drawn. Pixels must not belong to multiple regions that have different values for M_CLASS_INDEX_GROUND_TRUTH. Note that the destination must be an image buffer and not a graphics list. For anomaly detection, non-anomalous values are drawn with index 0, anomalous pixels are drawn with index 1 and all don't care pixels are drawn with index -1.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for M_DESCRIPTOR_TYPE_BOX; M_DESCRIPTOR_TYPE_MASK; M_DESCRIPTOR_TYPE_POLYGON; M_DRAW_BEST_INDEX_CONTOUR_IMAGE; M_DRAW_BEST_INDEX_IMAGE; M_DRAW_BEST_SCORE_IMAGE; M_DRAW_CLASS_SCORES; M_GROUND_TRUTH_IMAGE.

You can add one of the following values to the above-mentioned values to specify a modification to the drawing operation.

Valid combinations can depend on many factors, such as the specified drawing operation and classifier type. See the combination values for details.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For modifying the draw entry operation																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_GROUND_TRUTH_INDEX	Draws the descriptor from the specified region using M_CLASS_INDEX_GROUND_TRUTH for pixel values. This combination value cannot be applied to M_GROUND_TRUTH_IMAGE.	c		e	f	g	a		i		j	k	l	m	n	b
M_NETWORK_OUTPUT_SIZE	Draws according to the exact classification map size (output of the network) without scaling or expanding. In this case, the expected size of the destination buffer (DstImageBufOrListGraId) is the same as the X- and Y-size of M_NETWORK_OUTPUT_SIZE. You cannot use M_NETWORK_OUTPUT_SIZE with: • M_DESCRIPTOR_TYPE_... drawing operations. • M_CSNET_... predefined classifiers. • M_REPLICATE_BORDER.	c		e	f	g	a		i		j	k	l	m	n	b
M_PSEUDO_COLOR +	Draws the descriptor from the specified region using color specified by M_CLASS_DRAW_COLOR for the class index of the region. If the operation is set to M_GROUND_TRUTH_IMAGE, M_PSEUDO_COLOR draws all the mask regions using colors specified in class definitions. This draws an image analogous to the input image of MclassEntryAddRegion() with M_GROUND_TRUTH_IMAGE_COLOR. In this case, you can use M_PSEUDO_COLOR with other combination values.	c		e	f	g	a		i		j	k	l	m	n	b
M_REPLICATE_BORDER	Draws with borders expanded to the size of the destination buffer (DstImageBufOrListGraId). You cannot use M_REPLICATE_BORDER with: • M_DESCRIPTOR_TYPE_... drawing operations. • M_CSNET_... predefined classifiers. • M_NETWORK_OUTPUT_SIZE.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_GROUND_TRUTH_IMAGE; M_PSEUDO_COLOR.

You can add the following value to the above-mentioned values to specify a modification to the drawing operation.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For modifying the ground truth																	
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NO_REGION_PIXELS	Specifies that unlabeled pixels are assigned values according to M_NO_REGION_PIXEL_CLASS. If M_NO_REGION_PIXEL_CLASS is set to M_NO_CLASS, pixels are set to -2. For an 8 + M_UNSIGNED buffer, that resolves to 0xFE (that is, 254). Otherwise if M_NO_REGION_PIXEL_CLASS is set to an index, the pixel will be given the same value of this index. Combined with M_PSEUDO_COLOR, this draws all the mask regions using the color specified in class definitions. If M_NO_REGION_PIXEL_CLASS is set to M_NO_CLASS, pixels are set to the color specified by M_NO_CLASS_DRAW_COLOR. This draws an image analogous to MclassEntryAddRegion() with M_GROUND_TRUTH_IMAGE_COLOR, but draws unlabeled pixels.	c		e	f	g	a		i		j	k	l	m	n	b

EntryIndex

Specifies the index of the entry to draw.

When you specify an index with EntryIndex, you must set the EntryKey parameter to M_DEFAULT_KEY.

> Note that both EntryIndex and EntryKey cannot be set to M_DEFAULT and M_DEFAULT_KEY simultaneously.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the index of an entry																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies that the index of an entry is not required.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the index of an entry.	c		e	f	g	a		i		j	k	l	m	n	b

EntryKey

Specifies the key (*AIL_UUID*) of the entry to which regions are added. The key is defined as an Aurora Imaging Library universal unique identifier (UUID).

When you specify an entry key with EntryKey, you must set the EntryIndex parameter to M_DEFAULT.

> Note that both EntryIndex and EntryKey cannot be set to M_DEFAULT and M_DEFAULT_KEY simultaneously.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the unique key of an entry																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_KEY	Specifies that the unique key of an entry is not required.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the unique key of an entry.	c		e	f	g	a		i		j	k	l	m	n	b

TaskType

Specifies the task (image classification, segmentation, object detection, or anomaly detection) for which the drawing operation is performed. A dataset entry might contain results from more than one task.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of task for which to draw results																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_AUTO.	c		e	f	g	a		i		j	k	l	m	n	b
M_ANOMALY_DETECTION	Specifies that you are drawing for anomaly detection.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO	Specifies the task for which to get results automatically, given the content in the dataset used (DatasetContextClassId). If the dataset selected is an images dataset, and it contains only image classification results, M_AUTO is the same as M_CLASSIFICATION. If the dataset contains only segmentation results, M_AUTO is the same as M_SEGMENTATION. If the dataset only contains object detection results, M_AUTO is the same as M_OBJECT_DETECTION. If the dataset only contains anomaly detection results, M_AUTO is the same as M_ANOMALY_DETECTION. If the dataset contains two types of results, then M_AUTO is the same as M_SEGMENTATION (if segmentation results are present), or M_CLASSIFICATION (if no segmentation results are present). If the dataset contains image classification, segmentation, and object detection results, M_AUTO is the same as M_SEGMENTATION. If the dataset contains segmentation and anomaly detection results, M_AUTO is the same as M_SEGMENTATION. If the dataset selected is a features dataset, M_AUTO is the same as M_CLASSIFICATION, since segmentation and object detection are not supported for a features dataset. To determine whether M_CLASSIFICATION, M_SEGMENTATION, M_OBJECT_DETECTION, or M_ANOMALY_DETECTION results are available in the dataset, use the M_PREDICT_INFO result. Typically, a dataset should be set up for a specific task (either image classification, segmentation, object detection, or anomaly detection).	c		e	f	g	a		i		j	k	l	m	n	b
M_CLASSIFICATION	Specifies that you are drawing for image classification.	c		e	f	g	a		i		j	k	l	m	n	b

M_OBJECT_DETECTION	Specifies that you are drawing for object detection.	c		e	f	g	a		i		j	k	l	m	n	b
M_SEGMENTATION	Specifies that you are drawing for segmentation.	c		e	f	g	a		i		j	k	l	m	n	b

Index

Specifies what to use to perform the drawing operation. Set this parameter to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying what to use to perform the drawing operation																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the function call refers to the entry and not to a specific region or instance. The draw operation will be applied in a sequential manner to all regions or instances associated with the image. Note, this is not available when Operation is set to M_DRAW_CLASS_SCORES.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLASS_INDEX(AIL_INT Index)	Specifies that the drawing operation is for a specific class in the specified dataset context (DatasetContextClassId). Note, this is not available when Operation is set to M_DESCRIPTOR_TYPE_BOX, M_DESCRIPTOR_TYPE_MASK, M_DESCRIPTOR_TYPE_POLYGON, or M_GROUND_TRUTH_IMAGE.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	Index Set this parameter to the following:															
	Value — Specifies the index of the class for which to draw.	c		e	f	g	a		i		j	k	l	m	n	b
M_INSTANCE_INDEX(AIL_INT Index)	Specifies that the drawing operation is for a specific instance in the specified dataset context (DatasetContextClassId). This value only applies to an images dataset context. Note, this is only available when Operation is set to M_DRAW_BOX, M_DRAW_BOX_CENTER, M_DRAW_BOX_NAME, or M_DRAW_BOX_SCORE.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	Index Set this parameter to the following:															
	Value — Specifies the index of the instance for which to draw.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_INDEX(AIL_INT RegionIdx)	Specifies the function call refers to a specific region within the entry. This value only applies to an images dataset context. Note, this is only available when Operation is set to M_DESCRIPTOR_TYPE_BOX, M_DESCRIPTOR_TYPE_MASK, or M_DESCRIPTOR_TYPE_POLYGON.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	RegionIdx Set this parameter to the following:															
	0 < *Value* < M_NUMBER_OF_REGIONS — Specifies a specific region within the entry. Note that there is always at least one region in an entry.	c		e	f	g	a		i		j	k	l	m	n	b

DstLabelArrayPtr

Specifies the address of the array in which to write the labels that have been automatically given to the copied or moved graphics in the destination graphics list. Set this parameter to **M_NULL** if the destination (DstImageBufOrListGraId) is an image buffer.

To retrieve the size that this array must have, use MclassInquireEntry() with M_NUMBER_OF_DESCRIPTOR_TYPE_BOX, M_NUMBER_OF_DESCRIPTOR_TYPE_MASK, or M_NUMBER_OF_DESCRIPTOR_TYPE_POLYGON, when the drawing operation (Operation) is set to M_DESCRIPTOR_TYPE_BOX, M_DESCRIPTOR_TYPE_MASK, or M_DESCRIPTOR_TYPE_POLYGON, respectively.

ControlFlag

Specifies how to control the drawing operation.

For all drawing operations except M_DRAW_CLASS_SCORES, set this parameter to M_DEFAULT. If entries were predicted by M_ICNET_... or legacy M_FCNET_... predefined classifiers, other values can be used. Typically, these drawing operations apply to prediction results from segmentation.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying how to control the class score drawing																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_DEFAULT	Implements the default behavior. This is the only possible setting for all drawing operations except M_DRAW_CLASS_SCORES.	c		e	f	g	a		i		j	k	l	m	n	b
M_BILINEAR_INTERPOLATION	Scales the classification map with a bilinear calculation. The scale is a network property and established in receptive field calculations. The scaled image is placed at the offset of the receptive field. This value is only supported for the M_DRAW_CLASS_SCORES drawing operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_RECEPTIVE_FIELD_PATCH	Fills the corresponding receptive fields with calculated scores. Scores are drawn in order, from low to high. The best score is therefore drawn over lower scores. This value is only supported for the M_DRAW_CLASS_SCORES drawing operation.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailclass11.lib.
DLL	Requires ail11.dll; ailclass11.dll.

MclassEntryAddRegion

Synopsis

Add regions to an entry in an images dataset context used for object detection or segmentation.

Syntax

```
void MclassEntryAddRegion(
    AIL_ID DatasetContextClassId,          //in
    AIL_INT64 EntryIndex,                  //in
    AIL_UUID EntryKey,                     //in
    AIL_INT64 DescriptorType,              //in
    AIL_ID DescriptorId,                   //in
    AIL_CONST_TEXT_PTR DescriptorFileName, //in
    AIL_INT64 ClassIndexGroundTruth,       //in
    AIL_INT64 ControlFlag                  //in
)
```

Description

This function adds regions to an entry in an images dataset context used for object detection or segmentation. This function should not be used for image or feature classification.

Each time you call **MclassEntryAddRegion**(), one or more regions are added to the specified entry. The number and type of regions that are added is determined by the region's descriptor (DescriptorType). For example, you can add one or more regions from a mask (image) or a graphics list.

Each added region has a ground truth assigned to it. Depending on the descriptor, the added region's ground truth is established with the ClassIndexGroundTruth or DescriptorId parameter. The pixels in the entry image that correspond to the added region are identified with this ground truth.

The first region added has an index value of 1. Region 0 is created by default for all entries and is reserved for image classification. The ground truth of the entry itself applies to region 0, and has no effect on the added regions.

To control and inquire about regions, call MclassControlEntry() and MclassInquireEntry(). To draw region information, call MclassDrawEntry(). Some internal descriptor information about regions, such as their polygon, can be drawn but not inquired.

Parameters

DatasetContextClassId

Specifies the identifier of the images dataset context containing the entries for which to add the regions. This dataset context should be used for object detection or segmentation and must have been previously allocated on the required system using MclassAlloc() with M_DATASET_IMAGES.

EntryIndex

Specifies the index of the entry to which regions are added.

When specifying an index with the EntryIndex parameter, you must set the EntryKey parameter to M_DEFAULT_KEY. You cannot set the EntryIndex parameter to M_DEFAULT and the EntryKey parameter to M_DEFAULT_KEY simultaneously.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the entry's index																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies that the EntryIndex parameter is not required.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the index of an entry.	c		e	f	g	a		i		j	k	l	m	n	b

EntryKey

Specifies the key (*AIL_UUID*) of the entry to which regions are added. The key is defined as an Aurora Imaging Library universal unique identifier (UUID).

When specifying a key with the EntryKey parameter, you must set the EntryIndex parameter to M_DEFAULT. You cannot set the EntryKey parameter to M_DEFAULT_KEY and the EntryIndex parameter to M_DEFAULT simultaneously.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the entry's key																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_KEY	Specifies that the EntryKey parameter is not required.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_UUID Value	Specifies the key (*AIL_UUID*) of an entry.	c		e	f	g	a		i		j	k	l	m	n	b

DescriptorType

Specifies the descriptor type of the region to add to the entry.

See the Parameter associations section for possible values that can be specified.

DescriptorId

Specifies the identifier of the Aurora Imaging Library object that describes the region to add. If this parameter is not needed, set it to **M_NULL**.

See the Parameter associations section for possible values that can be specified.

DescriptorFileName

Specifies the file that describes the region to add. If this parameter is not needed, set it to **M_NULL**.

See the Parameter associations section for possible values that can be specified.

ClassIndexGroundTruth

Specifies the index of the class definition that represents the added regions' ground truth. If this parameter is not needed, set it to **M_DEFAULT**.

See the Parameter associations section for possible values that can be specified.

ControlFlag

Reserved for future expansion. This parameter must be set to **M_DEFAULT**.

Parameter associations

The table below lists possible values for the **DescriptorType**, **DescriptorId**, **DescriptorFileName**, and **ClassIndexGroundTruth** parameters.

- For establishing the region to add to the entry

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For establishing the region to add to the entry																	
DescriptorType DescriptorId DescriptorFileName ClassIndexGroundTruth	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DESCRIPTOR_TYPE_BOX	Adds a new bounding box region to the specified entry. The pixels in the entry image that correspond to the areas within the bounding box are considered part of the ground truth established for this region (ClassIndexGroundTruth). The bounding box must not be rotated by any angle. M_DESCRIPTOR_TYPE_BOX is only available for object detection.															
2D graphics list identifier	Sets the identifier of the 2D graphics list from which to establish the added region. The 2D graphics list must contain a single rectangle that is not defined in world units.	c		e	f	g	a		i		j	k	l	m	n	b
DescriptorFileName	This parameter is not needed and must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
ClassIndexGroundTruth	Sets the index of the class definition that represents the added region's ground truth.	c		e	f	g	a		i		j	k	l	m	n	b
M_DESCRIPTOR_TYPE_MASK	Adds an image (mask) region to the specified entry. The pixels in the entry image that correspond to the non-zero pixels (masked) in the specified image are considered part of the ground truth specified for this region (ClassIndexGroundTruth). You must use a single band image, and its dimensions must be the same as the entry image, as specified by M_ENTRY_IMAGE_PATH (MclassControlEntry()). This image must not have an ROI associated with it (MbufSetRegion()). M_DESCRIPTOR_TYPE_MASK is only available for segmentation.	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies that this parameter is not needed and the added region will be established from an image file (DescriptorFileName).	c		e	f	g	a		i		j	k	l	m	n	b
DescriptorFileName	Sets the image file from which to establish the added region. You must specify a valid file and path. The location of this file is specified by M_REGION_MASK_PATH (MclassControl()). You can specify relative paths; therefore, if the root path is set with M_ROOT_PATH (MclassControl()), then the following must represent a valid path: *RootPath\DescriptorFileName*.	c		e	f	g	a		i		j	k	l	m	n	b
ClassIndexGroundTruth	Sets the index of the class definition that represents the added region's ground truth.	c		e	f	g	a		i		j	k	l	m	n	b
Image buffer identifier	Sets the identifier of the image buffer from which to establish the added region. This image will be saved on disk at the location specified by M_REGION_MASKS_FOLDER (MclassControl()).	c		e	f	g	a		i		j	k	l	m	n	b
DescriptorFileName	This parameter is not needed and must be set to **M_NULL**. The added region is established with an image buffer (DescriptorId).	c		e	f	g	a		i		j	k	l	m	n	b
ClassIndexGroundTruth	Sets the index of the class definition that represents the added region's ground truth.	c		e	f	g	a		i		j	k	l	m	n	b
M_DESCRIPTOR_TYPE_POLYGON	Adds a region with one or more polygons to the specified entry. The pixels in the entry image that correspond to the areas within the polygons are considered part of the ground truth established for this region (ClassIndexGroundTruth). M_DESCRIPTOR_TYPE_POLYGON is only available for segmentation.	c		e	f	g	a		i		j	k	l	m	n	b
2D graphics list identifier	Sets the identifier of the 2D graphics list from which to establish the added region. The 2D graphics list must contain polygons that are not defined in world units.	c		e	f	g	a		i		j	k	l	m	n	b
DescriptorFileName	This parameter is not needed and must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
ClassIndexGroundTruth	Sets the index of the class definition that represents the added region's ground truth.	c		e	f	g	a		i		j	k	l	m	n	b

M_GROUND_TRUTH_IMAGE	Adds regions to the entry according to the ground truth indices in the specified image. The pixel values in this image must represent class definition index values. The corresponding pixels in the entry image have that class definition as their ground truth. The number of regions added corresponds to the number of class definitions in the ground truth image. M_GROUND_TRUTH_IMAGE is similar to M_GROUND_TRUTH_IMAGE_COLOR, except you are providing the index of the class definitions, instead of their color. If you are adding many regions, using M_GROUND_TRUTH_IMAGE can be more efficient than adding the regions one by one. For each class definition (region) detected, an image (mask) is saved to disk at the location specified by M_REGION_MASKS_FOLDER (MclassControl()). If the specified ground truth image has an index value that does not correspond to an existing class definition index, Aurora Imaging Library creates a class definition with that index, and all the missing class definitions (indices) up to it. This is the same behavior as MclassImport(). An index (pixel value) of -1 in the specified ground truth image indicates a don't care value. Pixels in the entry image that correspond to the -1 pixels in the ground truth image do not contribute to the training loss, but are available to be assigned to a region. This has the same effect as setting M_CLASS_INDEX_GROUND_TRUTH to M_DONT_CARE_CLASS (MclassControlEntry()). An index (pixel value) of -2 in the specified ground truth image indicates an unlabeled value. Pixels in the entry image that correspond to the -2 pixels in the ground truth image are not part of any region. Be cautious of how don't care pixels (-1) and unlabeled pixels (-2) get resolved, given the data type and depth of your images. For example, when using 8-bit unsigned images, unlabeled pixels (-2) resolve to 0xFE (that is, 254); if your dataset already holds 254 or more class definitions, an Aurora Imaging Library error is generated, requesting an image buffer with a greater depth (for example, 16- bit unsigned).	c		e	f	g	a		i		j	k	l	m	n	b
Image buffer identifier	Sets the identifier of the image buffer from which to establish the added regions. This image buffer must not have an ROI associated with it (MbufSetRegion()).	c		e	f	g	a		i		j	k	l	m	n	b
DescriptorFileName	This parameter is not needed and must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
ClassIndexGroundTruth	This parameter is not needed and must be set to **M_DEFAULT**. The ground truth is established from the image buffer (DescriptorId).	c		e	f	g	a		i		j	k	l	m	n	b
M_GROUND_TRUTH_IMAGE_COLOR	Adds regions to the entry according to the ground truth colors in the specified image. The pixel values in this image must represent class definition colors. The corresponding pixels in the entry image have that class definition as their ground truth. The number of regions added corresponds to the number of class definition colors in the ground truth image. M_GROUND_TRUTH_IMAGE_COLOR is similar to M_GROUND_TRUTH_IMAGE, except you are providing the color of the class definitions, instead of their index. For each class definition (region) detected, an image (mask) is saved to disk at the location specified by M_REGION_MASKS_FOLDER (MclassControl()). If the specified ground truth image has a color that does not correspond to an existing class definition color, Aurora Imaging Library creates a class definition with that color. Class definition colors are set with M_CLASS_DRAW_COLOR (MclassControl()). A color (pixel value) of M_DONT_CARE_CLASS_DRAW_COLOR (MclassControl()) in the specified ground truth image indicates a don't care value. Pixels in the entry image that correspond to M_DONT_CARE_CLASS_DRAW_COLOR in the ground truth image do not contribute to the training loss, but are available to be assigned to a region. A color (pixel value) of M_NO_CLASS_DRAW_COLOR (MclassControl()) in the specified ground truth image indicates an unlabeled value. Pixels in the entry image that correspond to M_NO_CLASS_DRAW_COLOR in the ground truth image are not part of any region. When using M_GROUND_TRUTH_IMAGE_COLOR, the M_CLASS_DRAW_COLOR, M_DONT_CARE_CLASS_DRAW_COLOR, and M_NO_CLASS_DRAW_COLOR controls must have unique colors.	c		e	f	g	a		i		j	k	l	m	n	b
Image buffer identifier	Sets the identifier of the image buffer from which to establish the added regions. This image buffer must not have an ROI associated with it (MbufSetRegion()).	c		e	f	g	a		i		j	k	l	m	n	b
DescriptorFileName	This parameter is not needed and must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
ClassIndexGroundTruth	This parameter is not needed and must be set to **M_DEFAULT**. The ground truth is established from the specified image buffer (DescriptorId).	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailclass11.lib.
DLL	Requires ail11.dll; ailclass11.dll.

MclassExport

Synopsis

Export data from a dataset context, training result buffer, or a tree ensemble classifier context to a single file or to multiple files in an Aurora Imaging Library folder structure.

Syntax

```
void MclassExport(
    AIL_CONST_TEXT_PTR FileNameOrFolderPath, //in
    AIL_INT64 FileOrFolderFormat,            //in
    AIL_ID ClassId,                          //in
    AIL_INT64 Index,                         //in
    AIL_INT64 ExportType,                    //in
    AIL_INT64 ControlFlag                    //in
)
```

Description

This function exports data from a dataset context, training result buffer, or a tree ensemble classifier context to a single file or to multiple files in an Aurora Imaging Library folder structure. A dataset can be exported to a CSV file. Image datasets can also be exported to multiple files in an Aurora Imaging Library folder structure. Training results are exported to TXT files. Tree ensemble classifiers (trees) are exported to DOT files.

You would typically use this function to export an entire images dataset, using M_IMAGE_DATASET_FOLDER and M_COMPLETE. The following is an example of the resulting Aurora Imaging Library folder structure after such an export. The folder names represent what Aurora Imaging Library uses; the file names are fabricated. Datasets for image classification do not have descriptor, segmentation, or detection data; exporting such datasets would not result in those folders and files.

DstFolder[1] ├──Icons │ ├──Class1.mim │ ├──Class2.mim ├──Images │ ├──Class1 │ │ ├──Image1.mim │ │ └──Image2.mim │ ├──Class2 │ │ ├──Image1.mim │ │ └──Image2.mim │ ├──Image1.mim │ └──Image2.mim ├──Masks │ ├──GUID1.mim │ └──GUID2.mim ├──Segmentations │ ├──GUID1_0.mbufi │ ├──GUID2_0.mbufi │ ├──GUID3_0.mbufi │ ├──GUID4_0.mbufi │ ├──GUID5_0.mbufi │ ├──GUID6_0.mbufi ├──Anomalies │ ├──GUID1.mbufi │ ├──GUID2.mbufi │ ├──GUID3.mbufi │ ├──GUID4.mbufi │ ├──GUID5.mbufi │ ├──GUID6.mbufi ├──authors.csv ├──class_definitions.csv ├──controls.csv ├──descriptor_boxes.csv ├──descriptor_masks.csv ├──descriptor_polygons.csv ├──entries.csv └──results_segmentations.csv └──results_anomaly_detection.csv └──results_detection.csv	[1]For more information about this folder structure, see the Importing data from a folder or CSV file section of Chapter 48: Datasets.

To import data, use MclassImport().

Parameters

FileNameOrFolderPath

Specifies the name and path of the file in which to export, or specifies the path to the folder in which to export multiple files in an Aurora Imaging Library folder structure. The function handles (internally) the opening and closing of the file. If you specify a file that already exists, it will be overwritten.

The specified file name must use the CSV, DOT, or TXT file extension. CSV and DOT files represent common formats that hold comma separated values and tree graph descriptions, respectively. You should use a CSV file when exporting from a dataset context. When exporting a tree ensemble classifier from a tree ensemble training result buffer or a tree ensemble classifier context, use a DOT file. A TXT file extension must be used when exporting training results from a CNN, object detection, segmentation, anomaly detection, or tree ensemble training result buffer.

Set this parameter to one of the following values.

For specifying the file name and path																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which to interactively specify the file's drive, directory, and name.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*FileName*") 1	Specifies the file's drive, directory, and name (for example, *"E:\Manual\MySweetSweetDataset.csv"*). To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the file name string with *"remote:///"* (for example, *"remote:///E:\Manual\MyEvenSweeterDataset.dot"*).	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

FileOrFolderFormat

Specifies the format of the file or folder in which to export. Set this parameter to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the file format																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FORMAT_CSV	Specifies a CSV file format. This value is supported when exporting from a dataset context.	c		e	f	g	a		i		j	k	l	m	n	b
M_FORMAT_DOT	Specifies a DOT file format. This value is supported when exporting a tree from a tree ensemble training result buffer or a tree ensemble classifier context.	c		e	f	g	a		i		j	k	l	m	n	b
M_FORMAT_TXT	Specifies a TXT file format. This value is supported when exporting from a CNN, segmentation, or tree ensemble training result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_IMAGE_DATASET_FOLDER	Specifies a path to a folder in which to create the Aurora Imaging Library folder structure for an images dataset. Note, the export does not complete if two classes (or more) have the same name, or if the creation of the folders would exceed MAX_PATH.	c		e	f	g	a		i		j	k	l	m	n	b
M_IMAGE_DATASET_FOLDER_ALLOW_RENAME	Specifies a path to a folder in which to create the Aurora Imaging Library folder structure for an images dataset (allows the renaming of classes to make sure the export completes). Note, M_IMAGE_DATASET_FOLDER_ALLOW_RENAME, ignores the restrictions on M_IMAGE_DATASET_FOLDER to make sure that the exports completes. It does so by renaming duplicate and reserved class names, and shortening the names of the folders that are too long. Using this operation might cause MclassImport() to incorrectly import the dataset.	c		e	f	g	a		i		j	k	l	m	n	b

ClassId

Specifies the identifier of the Aurora Imaging Library Classification module object from which to export. Set this parameter to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the Classification module object from which to export																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ContextDatasetId	Specifies the identifier of a features or images dataset context. This context is allocated using MclassAlloc() with M_DATASET_FEATURES or M_DATASET_IMAGES.	c		e	f	g	a		i		j	k	l	m	n	b
ContextTreeEnsembleClassiferId	Specifies the identifier of a tree ensemble classifier context. This context is allocated using MclassAlloc() with M_CLASSIFIER_TREE_ENSEMBLE.	c		e	f	g	a		i		j	k	l	m	n	b
ResultBufTrainId	Specifies the identifier of a CNN, object detection, segmentation, or tree ensemble training result buffer. This buffer is allocated using MclassAllocResult() with M_TRAIN_CNN_RESULT, M_TRAIN_DET_RESULT, M_TRAIN_SEG_RESULT, or M_TRAIN_TREE_ENSEMBLE_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b

Index

Specifies the tree to export from a tree ensemble training result buffer or tree ensemble classifier context. To export a tree, you must set the ExportType parameter to M_TRAIN_TREE. If you are not exporting a tree, you must set the Index parameter to M_DEFAULT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the index of the tree to export																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies that you are not exporting a tree.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the index of the tree to export to a DOT file.	c		e	f	g	a		i		j	k	l	m	n	b

ExportType

Specifies the type of export operation to perform. Available operations depend on the context or result buffer from which you are exporting.

If you are exporting from a dataset context, you can set the ExportType parameter to one of the following values. In this case, the FileNameOrFolderPath parameter must be set to either a CSV file or a folder path, the FileOrFolderFormat parameter must be set accordingly based on FileNameOrFolderPath parameter, the ClassId parameter must be set to the

identifier of a dataset context, and the Index parameter must be set to M_DEFAULT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying what to export from a dataset context																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_AUTHORS	Specifies to export the authors to a CSV file.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLASS_DEFINITIONS	Specifies to export the class definitions to a CSV file.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPLETE	Specifies to export all the information of the dataset; this includes: entries (images and CSV), segmentation files (Aurora Imaging Library buffers), prediction results (CSV), descriptors (images and CSV), icons (images), class definitions (CSV), controls (CSV), and authors (CSV). This is typically the most useful setting for image datasets. To use it, the FileOrFolderFormat parameter must be set to M_IMAGE_DATASET_FOLDER or M_IMAGE_DATASET_FOLDER_ALLOW_RENAME. > You cannot specify M_COMPLETE with Distributed Aurora Imaging Library.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENTRIES	Specifies to export the entries to a CSV file.	c		e	f	g	a		i		j	k	l	m	n	b

If you are exporting from a CNN, object detection, segmentation, or tree ensemble training result buffer, you can set the ExportType parameter to the following values.

When exporting from a CNN, object detection or segmentation training result buffer, the FileNameOrFolderPath parameter must be set to a CSV file, the FileOrFolderFormat parameter must be set to M_FORMAT_CSV, the ClassId parameter must be set to the identifier of a CNN, object detection, or segmentation training result buffer, and the Index parameter must be set to M_DEFAULT.

When exporting from a tree ensemble training result buffer, the FileNameOrFolderPath parameter must be set to either a DOT or TXT file. The FileOrFolderFormat parameter must be set accordingly based on FileNameOrFolderPath parameter. To export tree(s), use M_FORMAT_DOT with M_TRAIN_TREE. To export a training report, use M_FORMAT_TXT with M_TRAIN_REPORT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying what to export from a training result buffer																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_TRAIN_REPORT	Specifies to export a train report to a TXT file. If you are exporting from a CNN, object detection or segmentation training result buffer, the TXT file contains the CNN, object detection or segmentation training results of each epoch. If you are exporting from a tree ensemble training result buffer, the TXT file contains results from the training. For example, the TXT file contains results such as feature importance, train accuracy, and number of trained trees.	c		e	f	g	a		i		j	k	l	m	n	b

To export a tree from a tree ensemble training result buffer or a tree ensemble classifier context, use the following value.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying what to export from a tree ensemble training result buffer or classifier context																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_TRAIN_TREE	Specifies to export the resulting tree to a DOT file.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailclass11.lib.
DLL	Requires ail11.dll; ailclass11.dll.

MclassFree

Synopsis

Free a classifier context, a dataset context, a data preparation context, a training context, a statistics classification context, a training result buffer, a prediction result buffer, or a statistics classification result buffer.

Syntax

```
void MclassFree(
    AIL_ID ClassObjectId //in
)
```

Description

This function deletes the specified classifier context, dataset context, data preparation context, training context, statistics classification context, training result buffer, prediction result buffer, or statistics classification result buffer identifier and releases any memory associated with it. All contexts and result buffers allocated on a particular system must be freed before the system can be freed, unless M_UNIQUE_ID was specified during allocation.

Parameter

ClassObjectId

Specifies the identifier of the classifier context, dataset context, data preparation context, training context, statistics classification context, training result buffer, prediction result buffer, or statistics classification result buffer to free. These must have been successfully allocated with MclassAlloc(), MclassAllocResult(), MclassRestore(), or MclassImport() before calling **MclassFree**().

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailclass11.lib.
DLL	Requires ail11.dll; ailclass11.dll.

MclassGetHookInfo

Synopsis

Get information about a hooked classification object event.

Syntax

```
AIL_INT MclassGetHookInfo(
    AIL_ID EventId,     //in
    AIL_INT64 InfoType, //in
    void *UserVarPtr    //out
)
```

Description

This function retrieves information about the event that caused the hook-handler function to be called. This function should only be called within the scope of a classification object hook-handler function call (see MclassHookFunction()).

Parameters

EventId

Specifies the classification object event identifier received by the hook-handler function. See MclassHookFunction() for more information. The event will be of the type passed to the HookType parameter of the hook-handler function.

See the Parameter associations section for possible values that can be specified.

InfoType

Specifies the type of information to get.

See the Parameter associations section for possible values that can be specified.

UserVarPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]
- array of type AIL_ID [optionally, in C++: a reference to a std::vector<AIL_ID>]
- array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]
- array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]
- array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]
- array of type AIL_TEXT_CHAR [optionally, in C++: a reference to an AIL_STRING]
- array of type AIL_UUID [optionally, in C++: a reference to a std::vector<AIL_UUID>]
- AIL_DOUBLE
- AIL_ID
- AIL_INT
- AIL_INT32
- AIL_INT64
- AIL_UUID

Specifies the address in which to write the requested information. Note that when getting parameter values, the UserVarPtr parameter should be of the same data type as value of the selected parameter.

See the Parameter associations section for possible values that can be returned.

Parameter associations

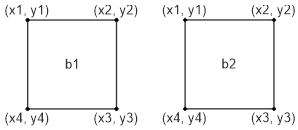
The table below lists possible values for the **EventId** and **InfoType** parameters and possible values returned to the **UserVarPtr** parameter.

- For getting information about a hooked classification object event

For getting information about a hooked classification object event	
EventId InfoType UserVarPtr - Possible values returned	Description
An M_DATASETS_PREPARED event ID	Specifies that the event that called the hook-handler function was due to an M_DATASETS_PREPARED event type on a CNN, object detection, segmentation, or anomaly detection training context object.
M_RESULT_ID +	Retrieves the identifier of the result buffer. If the hook-handler function was called due to an M_DATASETS_PREPARED event type when training, you can pass this result buffer identifier to MclassCopyResult() to get the prepared training (M_PREPARED_TRAIN_DATASET) and development (M_PREPARED_DEV_DATASET) datasets.
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*
An M_EPOCH_TRAINED event ID	Specifies that the event that called the hook-handler function was due to an M_EPOCH_TRAINED event type. Unless otherwise specified, this applies to a CNN, object detection or segmentation training context object.
M_DEV_DATASET_ACCURACY +	Retrieves the accuracy on the development dataset using the trained context obtained at the end of the epoch. Note, this value is not available for object detection.
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*

0.0 <= *Value* <= 100.0	Specifies the accuracy on the development dataset, as a percentage.
M_DEV_DATASET_CONFUSION_MATRIX +	Retrieves an array that represents the confusion matrix calculated using the development dataset. The confusion matrix contains information about how many entries were correctly and incorrectly classified during training in the development dataset. Note, this value is not available for object detection.
UserVarPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*
M_DEV_DATASET_CONFUSION_MATRIX_SIZE_X +	Retrieves the X-dimension (number of rows) of the confusion matrix calculated using the development dataset. Note, this value is not available for object detection.
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*
Value >= 1	Specifies the number of rows in the confusion matrix.
M_DEV_DATASET_CONFUSION_MATRIX_SIZE_Y +	Retrieves the Y-dimension (number of columns) of the confusion matrix calculated using the development dataset. Note, this value is not available for object detection.
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*
Value >= 1	Specifies the number of columns in the confusion matrix.
M_DEV_DATASET_EPOCH_ERROR_ENTRIES +	Retrieves the array containing the AIL_UUIDs of each image in the development dataset that produced an error. For segmentation, UUIDs are also listed for entries that have invalid labels. A label can be invalid if Aurora Imaging Library failed to restore a mask, if the mask size does not match the size of the entry image, if not all the pixels in the image are labeled, or if there is an overlap of pixels between different classes.
UserVarPtr extra info	Data type info: Data type: *array of type AIL_UUID [optionally, in C++: a reference to a std::vector<AIL_UUID>]*
M_DEV_DATASET_ERROR_RATE +	Retrieves the error rate on the development dataset using the trained context obtained at the end of the epoch. The error rate is a measurement of how frequently the classifier incorrectly classifies images in the dataset. Note, this value is not available for object detection.
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*
0.0 <= *Value* <= 100.0	Specifies the error on the development dataset, as a percentage.
M_DEV_DATASET_ID +	Retrieves the identifier of the development dataset context.
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*
M_DEV_DATASET_IOU_MEAN +	Retrieves the mean IOU (Intersection over Union) metric of the development dataset after the last epoch. The IOU compares the area and location of the predicted region to the ground truth region. Note, this value is only available for segmentation.
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*
M_DEV_DATASET_LOSS +	Retrieves the loss metric of the development dataset after the last epoch. You can use the loss to evaluate the lack of confidence (doubt) associated with the training. Note, this value is not available for CNN.
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*
M_DEV_DATASET_USED_ENTRIES +	Retrieves the array that contains the AIL_UUID of each image in the development dataset used during training.
UserVarPtr extra info	Data type info: Data type: *array of type AIL_UUID [optionally, in C++: a reference to a std::vector<AIL_UUID>]*
M_EPOCH_INDEX +	Retrieves the index of the last epoch. An epoch is one complete pass of the training dataset.
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*
M_MINI_BATCH_PER_EPOCH +	Retrieves the number of mini-batches per epoch. A mini-batch is a subset of the dataset. The weights of the classifier are updated after each mini-batch iteration.
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*
M_RESULT_ID +	Retrieves the identifier of the result buffer.
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*
M_TRAIN_CONTEXT_ID +	Retrieves the identifier of the training context.
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*
M_TRAIN_DATASET_ACCURACY +	Retrieves the accuracy of the training dataset using the trained context obtained at the end of the epoch. Note, this value is not available for object detection.

UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*
0.0 <= *Value* <= 100.0	Specifies the accuracy of the training dataset, as a percentage.
M_TRAIN_DATASET_CONFUSION_MATRIX +	Retrieves an array that represents the confusion matrix calculated using the training dataset. The confusion matrix contains information about how many entries were correctly and incorrectly classified during training in the development dataset. Note, this value is not available for object detection.
UserVarPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*
M_TRAIN_DATASET_CONFUSION_MATRIX_SIZE_X +	Retrieves the X-dimension (number of rows) of the confusion matrix calculated using the training dataset. Note, this value is not available for object detection.
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*
Value >= 1	Specifies the number of rows in the confusion matrix.
M_TRAIN_DATASET_CONFUSION_MATRIX_SIZE_Y +	Retrieves the Y-dimension (number of columns) of the confusion matrix calculated using the training dataset. Note, this value is not available for object detection.
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*
Value >= 1	Specifies the number of columns in the confusion matrix.
M_TRAIN_DATASET_EPOCH_ERROR_ENTRIES +	Retrieves the array containing the AIL_UUIDs of each image in the training dataset that produced an error. For segmentation, UUIDs are also listed for entries that have invalid labels. A label can be invalid if Aurora Imaging Library failed to restore a mask, if the mask size does not match the size of the entry image, if not all the pixels in the image are labeled, or if there is an overlap of pixels between different classes.
UserVarPtr extra info	Data type info: Data type: *array of type AIL_UUID [optionally, in C++: a reference to a std::vector<AIL_UUID>]*
M_TRAIN_DATASET_ERROR_RATE +	Retrieves the error rate of the training dataset using the trained context obtained at the end of the epoch. Note, this value is not available for object detection.
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*
0.0 <= *Value* <= 100.0	Specifies the error of the training dataset, as a percentage.
M_TRAIN_DATASET_ID +	Retrieves the identifier of the training dataset. This is unavailable for M_DATASETS_PREPARED.
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*
M_TRAIN_DATASET_IOU_MEAN +	Retrieves the mean IOU (Intersection over Union) metric of the training dataset after the last epoch. Note, this value is only available for segmentation.
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*
M_TRAIN_DATASET_USED_ENTRIES +	Retrieves the array that contains the AIL_UUID of each image in the training dataset used during the training phase.
UserVarPtr extra info	Data type info: Data type: *array of type AIL_UUID [optionally, in C++: a reference to a std::vector<AIL_UUID>]*
M_TRAINED_PARAMETERS_UPDATED +	Retrieves whether the current CNN, object detection or segmentation classifier parameters are up to date.
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*
M_FALSE	Specifies that the current parameters are not up to date.
M_TRUE	Specifies that the current parameters are up to date.
An M_MINI_BATCH_TRAINED event ID	Specifies that the event that called the hook-handler function was due to an M_MINI_BATCH_TRAINED event type on a CNN, object detection, segmentation, or anomaly detection training context object.
M_DEV_DATASET_ID +	Retrieves the identifier of the development dataset context. This is unavailable for M_DATASETS_PREPARED.
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*
M_EPOCH_INDEX +	Retrieves the index of the last epoch.
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*
M_MINI_BATCH_INDEX +	Retrieves the index of the last mini-batch.
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*
M_MINI_BATCH_LOSS +	Retrieves the loss at the end of training a mini-batch. You can use the loss to evaluate the lack of confidence (doubt) associated with the training.

UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*
M_MINI_BATCH_PER_EPOCH +	Retrieves the number of mini-batches per epoch.
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*
M_RESULT_ID +	Retrieves the identifier of the result buffer.
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*
M_TRAIN_CONTEXT_ID +	Retrieves the identifier of the training context. This is unavailable for M_DATASETS_PREPARED.
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*
M_TRAIN_DATASET_ID +	Retrieves the identifier of the training dataset. This is unavailable for M_DATASETS_PREPARED.
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*
An M_PREDICT_ENTRY event ID	Specifies that the event that called the hook-handler function was due to an M_PREDICT_ENTRY event type on a CNN, object detection classifier, segmentation, anomaly detection or tree ensemble context object.
M_BEST_CLASS_INDEX +	Retrieves the index of the class with the highest score, for the last entry being predicted. For image or feature classification, M_BEST_CLASS_INDEX retrieves one index. For segmentation, M_BEST_CLASS_INDEX retrieves an array of values (indices) that can be arranged in a 2D image, where the size is equal to M_PREDICTION_SIZE_X * M_PREDICTION_SIZE_Y. Each value is the index of the best class score for that X- Y-pixel location. The index of the class begins at 0. For object detection, M_BEST_CLASS_INDEX retrieves an array of values representing the index of the best class score for each detected instance. The size of the array is equal to M_NUMBER_OF_INSTANCES.
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*
M_BOX_4_CORNERS +	Retrieves an array containing the coordinates of the four corners of the specified bounding boxes. The 8 coordinates of the first box (b1) are grouped, followed by the 8 coordinates of the second box (b2), repeating this process for all specified boxes in the order b1x1, b1y1, b1x2, b1y2, b1x3, b1y3, b1x4, b1y4, b2x1, b2y1, b2x2, b2y2, b2x3, b2y3, b2x4, b2y4, etc.  The size of the array will be equal to the number of specified instances (M_NUMBER_OF_INSTANCES) multiplied by 8.
UserVarPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*
M_CENTER_X +	Retrieves an array of the X-coordinates of the centers of the specified bounding boxes. The size of the array is equal to M_NUMBER_OF_INSTANCES.
UserVarPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*
M_CENTER_Y +	Retrieves an array of the Y-coordinates of the centers of the specified bounding boxes. The size of the array is equal to M_NUMBER_OF_INSTANCES.
UserVarPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*
M_ENTRY_KEY +	Retrieves the key of the last entry being predicted.
UserVarPtr extra info	Data type info: Data type: *address of an AIL_UUID*
M_HEIGHT +	Retrieves an array containing the heights of the specified bounding boxes. The size of the array is equal to M_NUMBER_OF_INSTANCES.
UserVarPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*
M_INPUT_DATASET_ID +	Retrieves the identifier of the dataset whose entries get predicted (MclassPredict() with TargetAilObjectId).
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*
M_NUMBER_OF_ENTRIES_IN_ERROR +	Retrieves the current number of entries that could not be predicted.
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*
M_NUMBER_OF_INSTANCES +	Retrieves the current number of instances in the predicted entry for object detection.
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*
M_NUMBER_OF_PREDICTED_ENTRIES +	Retrieves the current number of entries that were predicted.
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*

M_OUTPUT_DATASET_ID +	Retrieves the identifier of the dataset in which the entries and their predictions are written (MclassPredict() with DatasetContextOrResultClassId).
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*
M_PREDICT_CONTEXT_ID +	Retrieves the identifier of the trained classifier context used to predict dataset entries (MclassPredict() with ContextClassId).
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*
M_PREDICT_SCORE +	Retrieves the highest class score of the last entry being predicted. For image or feature classification, M_PREDICT_SCORE retrieves one score. For segmentation, M_PREDICT_SCORE retrieves an array of values (scores) that can be arranged in a 2D image, where the size is equal to M_PREDICTION_SIZE_X * M_PREDICTION_SIZE_Y. Each value is the highest score for that X- Y- pixel location. For object detection, M_PREDICT_SCORE, retrieves an array of values representing the highest class score for each detected instance. The size of the array is equal to M_NUMBER_OF_INSTANCES.
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*
M_PREDICT_SCORE_AVERAGE +	Retrieves the average prediction score of the predicted entries. This is not available for segmentation.
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*
M_PREDICT_SCORE_MAX +	Retrieves the maximum prediction score of the predicted entries. This is not available for segmentation.
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*
M_PREDICT_SCORE_MIN +	Retrieves the minimum prediction score of predicted entries. This is not available for segmentation.
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*
M_PREDICTION_SIZE_X +	Retrieves the number of class scores, along the X-direction. This is only available for segmentation.
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*
M_PREDICTION_SIZE_Y +	Retrieves the number of class scores, along the Y-direction. This is only available for segmentation.
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*
M_STATUS +	Retrieves the status of the last predicted entry. M_PREDICT_ENTRY event types can occur with CNN, segmentation, object detection, anomaly detection, or tree ensemble classifiers.
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*
M_COMPLETE	Specifies that the prediction completed.
M_IMAGE_FILE_INVALID	Specifies that the image file associated with the source entry is not a valid image.
M_IMAGE_FILE_NOT_FOUND	Specifies that the image file associated with the source entry was not found.
M_INTERNAL_ERROR	Specifies that an internal error occurred during the current entry predict.
M_INVALID_BUFFER_SIZE	Specifies that the entry has invalid dimensions; the height/width of the entry does not match the size of the classifier source layer or the size of the specified target image. This applies to M_PREDICT_ENTRY with a segmentation classifier.
M_INVALID_BUFFER_SIZE_BAND	Specifies that the entry has an invalid number of bands; the number of bands of the entry does not match the number of bands of the classifier source layer or the of the specified target image. This applies to M_PREDICT_ENTRY with a segmentation classifier.
M_INVALID_BUFFER_TYPE	Specifies that the entry has an invalid type; the entry type does not match the classifier source layer type or the of the specified target image type.
M_INVALID_NUMBER_OF_FEATURES	Specifies that the entry has an invalid number of features. This applies to M_PREDICT_ENTRY with a tree ensemble classifier.
M_INVALID_REGION_BUFFER	Specifies that the entry has no valid region of interest (ROI).
M_TASK_TYPE +	Retrieves the type of task performed by the M_PREDICT_ENTRY event.
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*
M_ANOMALY_DETECTION	Specifies that the prediction was for an anomaly detection task type.
M_CLASSIFICATION	Specifies that the prediction was for a classification task type.
M_OBJECT_DETECTION	Specifies that the prediction was for an object detection task type.

M_SEGMENTATION	Specifies that the prediction was for a segmentation task type.
M_WIDTH +	Retrieves an array containing the widths of the specified bounding boxes for object detection. The size of the array is equal to M_NUMBER_OF_INSTANCES.
UserVarPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*
An M_PREPARE_ENTRY_POST event ID	Specifies that the event that called the hook-handler function was due to an M_PREPARE_ENTRY_POST event type on an object detection, CNN, or segmentation data preparation context object.
M_DST_DATASET_ID +	Retrieves the identifier of the destination dataset.
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*
M_DST_ENTRIES_START_INDEX +	Get the destination entry start index for the last mini-batch of prepared images. That batch extends from the start index up to the end of the destination dataset.
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*
M_NUMBER_OF_PREPARED_SRC_ENTRIES +	Retrieves the number of entries from the source dataset that have been prepared so far. You will always get a value greater than or equal to 0.
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*
M_SRC_DATASET_ID +	Retrieves the identifier of the source dataset that is specified by the source entry index (M_SRC_ENTRY_INDEX).
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*
M_SRC_ENTRY_INDEX +	Retrieves the source entry index of the entry that was prepared. You will always get a value greater than or equal to 0.
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*
M_STATUS +	Retrieves the status of the last prepared entry. M_PREPARE_ENTRY_POST event types can occur with object detection, CNN, or segmentation data preparation contexts.
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*
M_COMPLETE	Specifies that the entry was prepared successfully.
M_FAILED_TO_SAVE_IMAGE	Specifies that the prepared image could not be saved at the required destination.
M_FLOAT_IMAGE_NOT_NORMALIZED	Specifies that float images must be normalized between 0 and 1.
M_IMAGE_FILE_INVALID	Specifies that the image file associated with the source entry exists but is not restorable.
M_IMAGE_FILE_NOT_FOUND	Specifies that the image file associated with the source entry was not found.
M_INVALID_AUG_OP_FOR_1_BAND_BUFFER	Specifies that not all augmentation operations were valid for 1 band buffers and a 1 band buffer was given.
M_INVALID_AUG_OP_FOR_1_BIT_BUFFER	Specifies that not all augmentation operations were valid for 1 bit buffers and a 1 bit buffer was given.
M_INVALID_AUG_OP_FOR_BUFFER_SIZE_BAND	Specifies that not all augmentation operations were valid for multi-band buffers with 2 bands or more than 3 bands and that such a buffer was given. This applies to M_PREPARE_ENTRY_POST with a CNN or segmentation data preparation context.
M_INVALID_BUFFER_SIGN_FOR_AUG	Specifies that the augmentation required an unsigned image buffer and a signed buffer was given.
M_INVALID_BUFFER_SIZE_BAND	Specifies that a buffer with an invalid size band was given; specifically, multi-band buffers with 2 bands or more than 3 bands are not supported. This applies to M_PREPARE_ENTRY_POST with an object detection data preparation context.
M_INVALID_CENTER	Specifies that the given center, for the M_CENTER_X and M_CENTER_Y controls, is outside the source image.
M_MASK_FILE_NOT_FOUND	Specifies that one of the mask files associated with the source entry was not found. This applies to M_PREPARE_ENTRY_POST with a segmentation data preparation context.
M_RESIZED_IMAGE_TOO_SMALL	Specifies that the size mode and resize scale factor resulted in a destination image too small to create; this can only occur when no classifier was passed.
M_SOURCE_TOO_SMALL_FOR_DERICHE_OP	Specifies that the source image is too small for Deriche operations; it must be at least 3 pixels.
An M_PREPARE_ENTRY_PRE event ID	Specifies that the event that called the hook-handler function was due to an M_PREPARE_ENTRY_PRE event type on a CNN or segmentation data preparation context object.

M_IMAGE_ID +	Retrieves the buffer identifier of the restored source entry about to be augmented. Use your algorithm (for example the Blob Analysis module) on this buffer identifier to find the center that you want to set (M_CENTER_X and M_CENTER_Y).
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*
M_SRC_DATASET_ID +	Retrieves the identifier of the source dataset that is specified by the source entry index (M_SRC_ENTRY_INDEX).
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*
M_SRC_ENTRY_INDEX +	Retrieve the source entry index of the entry that was prepared. You will always get a value greater than or equal to 0.
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*
An M_SAMPLE_ADDED event ID	Specifies that the event that called the hook-handler function was due to an M_SAMPLE_ADDED event type on an anomaly detection training context object.
M_CURRENT_SAMPLE_INDEX +	Retrieves the index of the newly added sample.
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*
M_CURRENT_SAMPLE_LOSS +	Retrieves the loss of the newly added sample.
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*
M_ITERATION_INDEX +	Retrieves the index of the current iteration.
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*
M_NUMBER_OF_ITERATIONS +	Retrieves the total number of iterations.
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*
M_RESULT_ID +	Retrieves the identifier of the training result buffer.
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*
An M_TREE_TRAINED event ID	Specifies that the event that called the hook-handler function was due to an M_TREE_TRAINED event type on a tree ensemble training context object.
M_DEV_DATASET_ID +	Retrieves the identifier of the development dataset context. This is unavailable for M_DATASETS_PREPARED.
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*
M_RESULT_ID +	Retrieves the identifier of the result buffer.
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*
M_TRAIN_CONTEXT_ID +	Retrieves the identifier of the training context. This is unavailable for M_DATASETS_PREPARED.
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*
M_TRAIN_DATASET_ID +	Retrieves the identifier of the training dataset. This is unavailable for M_DATASETS_PREPARED.
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*
M_TREE_DEPTH_ACHIEVED +	Retrieves the tree depth of the current tree. The current tree is equivalent to the last tree added to the tree ensemble.
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*
M_TREE_INDEX +	Retrieves the index of the tree being trained.
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*
M_TREE_NUMBER_OF_LEAF_NODES_ACHIEVED +	Retrieves the number of leaf nodes of the current tree. The current tree is equivalent to the last tree added to the tree ensemble.
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*
An M_WRITE_NEW_FILE event ID	Specifies that the event that called the hook-handler function was due to an M_WRITE_NEW_FILE event type on a CNN, segmentation, or anomaly detection dataset context object.
M_NEW_FILE_PATH +	Retrieves the path of the most recently created file on disk.
UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*
AIL_TEXT("*NewFilePath*") 1	Specifies the path.

[1] If the value was passed in a variable, it won't be returned enclosed in AIL_TEXT().

Combination value for [when EventId=An M_EPOCH_TRAINED event ID: M_DEV_DATASET_CONFUSION_MATRIX; M_DEV_DATASET_EPOCH_ERROR_ENTRIES; M_DEV_DATASET_USED_ENTRIES; M_TRAIN_DATASET_CONFUSION_MATRIX; M_TRAIN_DATASET_EPOCH_ERROR_ENTRIES; M_TRAIN_DATASET_USED_ENTRIES;], [when

EventId=An M_PREDICT_ENTRY event ID: M_BEST_CLASS_INDEX; M_BOX_4_CORNERS; M_CENTER_X; M_CENTER_Y; M_HEIGHT; M_PREDICT_SCORE; M_WIDTH].

You can add the following value to the above-mentioned values to determine the required number of elements in the array (array size).

For determining the required number of elements in the array (array size)	
InfoType combination value	Description
UserVarPtr - Possible values returned	
M_NB_ELEMENTS	Retrieves the required array size (number of elements) to store the returned values.
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*

Combination value for [when EventId=An M_WRITE_NEW_FILE event ID: M_NEW_FILE_PATH].

You can add the following value to the above-mentioned value to get the size of a string.

For getting the size of a string	
InfoType combination value	Description
UserVarPtr - Possible values returned	
M_STRING_SIZE +	Retrieves the number of characters in the string. This number accounts for every character, including the terminating null character.
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*

Combination value for the values of the InfoType parameter listed in For getting information about a hooked classification object event. (exceptions might exist)

You can add the following value to the above-mentioned values to determine whether an event result is available to be returned.

For determining whether event results are available to be returned	
InfoType	Description
UserVarPtr - Possible values returned	
M_AVAILABLE	Retrieves whether the requested information is available for retrieval.
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*
M_FALSE	Specifies that the requested information is not available.
M_TRUE	Specifies that the requested information is available.

Combination values for the values of the InfoType parameter listed in For getting information about a hooked classification object event; (exceptions might exist) and for the following value: M_STRING_SIZE.

You can add one of the following values to the above-mentioned values to cast the requested information to a required data type.

> Note: In C++, when using a standard vector (std::vector) instead of an array, the values listed in this combination table must not be used. If you need to use a specific data type, instantiate the vector with the necessary data type.

For specifying the data type	
InfoType	Description
UserVarPtr - Possible values returned	
M_TYPE_AIL_DOUBLE	Casts the requested information to an *AIL_DOUBLE*.
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*
M_TYPE_AIL_ID	Casts the requested information to an *AIL_ID*.
UserVarPtr extra info	Data type info: Data type: *address of an AIL_ID* **|** *array of type AIL_ID [optionally, in C++: a reference to a std::vector<AIL_ID>]*
M_TYPE_AIL_INT	Casts the requested information to an *AIL_INT*.
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT* **|** *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]*
M_TYPE_AIL_INT32	Casts the requested information to an *AIL_INT32*.
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT32* **|** *array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]*
M_TYPE_AIL_INT64	Casts the requested information to an *AIL_INT64*.
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64* **|** *array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]*

Return value

The returned value is **M_NULL** if successful. If the operation fails, a non-null (!**M_NULL**) value is returned.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailclass11.lib.
DLL	Requires ail11.dll; ailclass11.dll.

MclassGetResult

Synopsis

Get results from a training or prediction result buffer.

Syntax

```
void MclassGetResult(
    AIL_ID ResultClassId,   //in
    AIL_INT64 LabelOrIndex, //in
    AIL_INT64 ResultType,   //in
    void *ResultArrayPtr    //out
)
```

Description

This function retrieves results of the specified type from a training or prediction result buffer. Result buffers are allocated using MclassAllocResult(). You can either retrieve training results, which are available after calling MclassTrain(), or you can retrieve prediction results, which are available after calling MclassPredict().

Parameters

ResultClassId

Specifies the identifier of the classification result buffer from which to retrieve results. You can get results from a CNN (M_TRAIN_CNN_RESULT), object detection (M_TRAIN_DET_RESULT), segmentation (M_TRAIN_SEG_RESULT), anomaly detection (M_TRAIN_ANO_RESULT) or tree ensemble (M_TRAIN_TREE_ENSEMBLE_RESULT) training result buffer, or a CNN (M_PREDICT_CNN_RESULT), object detection (M_PREDICT_DET_RESULT), segmentation (M_PREDICT_SEG_RESULT), anomaly detection (M_PREDICT_ANO_RESULT) tree ensemble (M_PREDICT_TREE_ENSEMBLE_RESULT), or ONNX (M_PREDICT_ONNX_RESULT) prediction result buffer.

LabelOrIndex

Specifies what to retrieve. The values in this table are available for all tasks unless otherwise specified. Set this parameter to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying what to retrieve																		
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_GENERAL.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLASS_INDEX(AIL_INT ClassIndex)	Specifies the index of the class for which to get results. You can get the number of included classes in the result buffer using M_NUMBER_OF_CLASSES.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	ClassIndex Specifies the index of a class.															
M_INSTANCE_INDEX(AIL_INT InstanceIndex)	Specifies the index of the instance for which to get results. You can get the number of included instances in the result buffer using M_NUMBER_OF_INSTANCES. This is only available for object detection result buffers.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	InstanceIndex Specifies the index of an instance.															
M_OUTPUT_INDEX(AIL_INT OutputIndex)	Specifies the index of the output of an ONNX model for which to get results. This is only available for ONNX result buffers.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	OutputIndex Specifies the index of an output.															
M_ALL_INSTANCES	Specifies to retrieve results for all instances. This is only available for object detection result buffers.	c		e	f	g	a		i		j	k	l	m	n	b
M_GENERAL	Specifies to retrieve global results from the result buffer.	c		e	f	g	a		i		j	k	l	m	n	b

ResultType

Specifies the type of result to retrieve.

See the Parameter associations section for possible values that can be specified.

ResultArrayPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]
- array of type AIL_FLOAT [optionally, in C++: a reference to a std::vector<AIL_FLOAT>]
- array of type AIL_ID [optionally, in C++: a reference to a std::vector<AIL_ID>]

- array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]
- array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]
- array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]
- array of type AIL_UINT8 [optionally, in C++: a reference to a std::vector<AIL_UINT8>]
- array of type AIL_UUID [optionally, in C++: a reference to a std::vector<AIL_UUID>]
- AIL_DOUBLE
- AIL_FLOAT
- AIL_ID
- AIL_INT
- AIL_INT32
- AIL_INT64
- AIL_UINT8

Specifies the address of the array in which to write results.

See the Parameter associations section for possible values that can be returned.

Parameter associations

The tables below list possible values for the **ResultType** parameter and possible values returned to the **ResultArrayPtr** parameter.

- For training (CNN or segmentation) - retrieving general results
- For training (CNN, object detection, segmentation, or anomaly detection) - retrieving general results
- For training (CNN, object detection, or segmentation) - retrieving general results
- For training (segmentation) - retrieving general results
- For training (segmentation or object detection) - retrieving general results
- For training (tree ensemble) - retrieving general results
- For training (CNN, segmentation, or tree ensemble) - retrieving general results
- For training (CNN, object detection, segmentation, anomaly detection or tree ensemble) - retrieving general results
- For prediction (CNN, object detection, segmentation, or ONNX) - retrieving general results
- For prediction (CNN, segmentation, or anomaly detection) - retrieving general results
- For prediction (CNN or segmentation) - retrieving general results
- For prediction (CNN, segmentation, or tree ensemble) - retrieving general results
- For prediction (CNN, object detection, segmentation, or tree ensemble) - retrieving general or specific results
- For prediction (CNN, segmentation, or tree ensemble) - retrieving general or class-specific results
- For prediction (object detection) - retrieving general results
- For prediction (object detection) - retrieving general or class-specific results
- For prediction (object detection) - retrieving results for a specific class, a specific instance, or all instances
- For prediction (anomaly detection) - retrieving general results
- For prediction (ONNX) - retrieving general results
- For prediction (ONNX) - retrieving results for a specific output
- For training (CNN, object detection, segmentation, anomaly detection, or tree ensemble) or prediction (CNN, object detection, segmentation, anomaly detection, tree ensemble, or ONNX) - retrieving the operation's resulting status

To retrieve general results from a CNN or segmentation training result buffer (ResultClassId), the ResultType can be set to one of the following values. In this case, set the LabelOrIndex parameter to M_GENERAL.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For training (CNN or segmentation) - retrieving general results																	
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned																	
M_DEV_DATASET_EPOCH_ACCURACY +	Retrieves an array containing the development dataset accuracy for each epoch. Note that the required array size, which you can retrieve with M_NB_ELEMENTS, is equivalent to the M_MAX_EPOCH (MclassControl()) of the training context used.																
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the development dataset accuracy for each epoch, as a percentage.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEV_DATASET_EPOCH_ERROR_RATE +	Retrieves an array containing the development dataset error rate for each epoch. Note that the required array size, which you can retrieve with M_NB_ELEMENTS, is equivalent to the M_MAX_EPOCH (MclassControl()) of the training context used.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the development dataset error rate for each epoch, as a percentage.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAIN_DATASET_EPOCH_ACCURACY +	Retrieves an array containing the training dataset accuracy for each epoch. Note that the required array size, which you can retrieve with M_NB_ELEMENTS, is equivalent to the M_MAX_EPOCH (MclassControl()) of the training context used.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the training dataset accuracy for each epoch, as a percentage.	c		e	f	g	a		i		j	k	l	m	n	b

M_TRAIN_DATASET_EPOCH_ERROR_RATE +	Retrieves an array containing the training dataset error rate for each epoch. Note that the required array size, which you can retrieve with M_NB_ELEMENTS, is equivalent to the M_MAX_EPOCH (MclassControl()) of the training context used.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the training dataset error rate for each epoch, as a percentage.	c		e	f	g	a		i		j	k	l	m	n	b

To retrieve general results from a CNN, object detection, segmentation, or anomaly detection training result buffer (ResultClassId), the ResultType can be set to one of the following values. In this case, set the LabelOrIndex parameter to M_GENERAL.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For training (CNN, object detection, segmentation, or anomaly detection) - retrieving general results																
ResultType ResultArrayPtr - Possible values returned	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEV_DATASET_ERROR_ENTRIES +	Retrieves an array containing the AIL_UUID of each image in the development dataset that produced an error, at least once, during training. For segmentation, UUIDs are also listed for entries that have invalid labels. A label can be invalid if Aurora Imaging Library failed to restore a mask, if the mask size does not match the size of the entry image, if not all the pixels in the image are labeled, or if there is an overlap of pixels between different classes.															
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_UUID [optionally, in C++: a reference to a std::vector<AIL_UUID>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEV_DATASET_ERROR_ENTRIES_STATUS +	Retrieves the error status of each entry in the development dataset that produced an error. Use M_DEV_DATASET_ERROR_ENTRIES to get the UUID of the entries that produced an error.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_FAILED_TO_RESTORE_GROUND_TRUTH_MASK	Specifies that the ground truth mask entry failed to be restored from the file. This only applies to segmentation tasks.	c		e	f	g	a		i		j	k	l	m	n	b
M_GROUND_TRUTH_MASK_DOES_NOT_EXIST	Specifies that the ground truth mask entry does not exist in the dataset. This only applies to segmentation tasks.	c		e	f	g	a		i		j	k	l	m	n	b
M_GROUND_TRUTH_MASK_INVALID_DIMENSIONS	Specifies that the ground truth entry has invalid dimensions; the height/width of the entry does not match the size of the classifier source layer or the size of the specified target image. This only applies to segmentation tasks.	c		e	f	g	a		i		j	k	l	m	n	b
M_IMAGE_FILE_INVALID	Specifies that the image file associated with the source entry is not a valid image.	c		e	f	g	a		i		j	k	l	m	n	b
M_IMAGE_FILE_NOT_FOUND	Specifies that the image file associated with the source entry was not found.	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID_BUFFER_SIZE	Specifies that the entry has invalid dimensions; the height/width of the entry does not match the size of the classifier source layer or the size of the specified target image. This applies to M_PREDICT_ENTRY with a segmentation classifier.	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID_BUFFER_SIZE_BAND	Specifies that the entry has an invalid number of bands; the number of bands of the entry does not match the number of bands of the classifier source layer or the of the specified target image. This applies to M_PREDICT_ENTRY with a segmentation classifier.	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID_BUFFER_TYPE	Specifies that the entry has an invalid type; the entry type does not match the classifier source layer type or the of the specified target image type.	c		e	f	g	a		i		j	k	l	m	n	b
M_OVERLAPPING_GROUND_TRUTH_PIXELS	Specifies that the ground truth entry contains pixels with overlapping classes. This only applies to segmentation tasks.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNLABELED_GROUND_TRUTH_PIXELS	Specifies that the ground truth entry contains unlabeled pixels. This only applies to segmentation tasks.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAINABLE_DATASETS +	Retrieves the status of the datasets in train results to determine whether you can perform a complete training using this CNN, object detection, segmentation, or anomaly detection training result buffer with an **M_NULL** identifier for both datasets of MclassTrain().	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DATASETS_STATUS_INVALID	Specifies that training cannot be done with these datasets.	c		e	f	g	a		i		j	k	l	m	n	b
M_DATASETS_STATUS_MISSING_GT	Specifies that training can be done if missing GT (ground truth) is supported. To support missing GT, enable M_SUPPORT_MISSING_GROUND_TRUTH (MclassControl()).	c		e	f	g	a		i		j	k	l	m	n	b
M_DATASETS_STATUS_VALID	Specifies that training can be done with these datasets.	c		e	f	g	a		i		j	k	l	m	n	b

M_TRAIN_DATASET_ERROR_ENTRIES +	Retrieves an array containing the AIL_UUID of each image in the training dataset that produced an error, at least once, during training. For segmentation, UUIDs are also listed for entries that have invalid labels. A label can be invalid if Aurora Imaging Library failed to restore a mask, if the mask size does not match the size of the entry image, if not all the pixels in the image are labeled, or if there is an overlap of pixels between different classes.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_UUID [optionally, in C++: a reference to a std::vector<AIL_UUID>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAIN_DATASET_ERROR_ENTRIES_STATUS +	Retrieves the error status of each entry in the training dataset that produced an error. Use M_TRAIN_DATASET_ERROR_ENTRIES to get the UUID of the entries that produced an error.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_FAILED_TO_RESTORE_GROUND_TRUTH_MASK	Specifies that the ground truth mask entry failed to be restored from the file. This only applies to segmentation tasks.	c		e	f	g	a		i		j	k	l	m	n	b
M_GROUND_TRUTH_MASK_DOES_NOT_EXIST	Specifies that the ground truth mask entry does not exist in the dataset. This only applies to segmentation tasks.	c		e	f	g	a		i		j	k	l	m	n	b
M_GROUND_TRUTH_MASK_INVALID_DIMENSIONS	Specifies that the ground truth entry has invalid dimensions; the height/width of the entry does not match the size of the classifier source layer or the size of the specified target image. This only applies to segmentation tasks.	c		e	f	g	a		i		j	k	l	m	n	b
M_IMAGE_FILE_INVALID	Specifies that the image file associated with the source entry is not a valid image.	c		e	f	g	a		i		j	k	l	m	n	b
M_IMAGE_FILE_NOT_FOUND	Specifies that the image file associated with the source entry was not found.	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID_BUFFER_SIZE	Specifies that the entry has invalid dimensions; the height/width of the entry does not match the size of the classifier source layer or the size of the specified target image. This applies to M_PREDICT_ENTRY with a segmentation classifier.	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID_BUFFER_SIZE_BAND	Specifies that the entry has an invalid number of bands; the number of bands of the entry does not match the number of bands of the classifier source layer or the of the specified target image. This applies to M_PREDICT_ENTRY with a segmentation classifier.	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID_BUFFER_TYPE	Specifies that the entry has an invalid type; the entry type does not match the classifier source layer type or the of the specified target image type.	c		e	f	g	a		i		j	k	l	m	n	b
M_OVERLAPPING_GROUND_TRUTH_PIXELS	Specifies that the ground truth entry contains pixels with overlapping classes. This only applies to segmentation tasks.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNLABELED_GROUND_TRUTH_PIXELS	Specifies that the ground truth entry contains unlabeled pixels. This only applies to segmentation tasks.	c		e	f	g	a		i		j	k	l	m	n	b

To retrieve general results from a CNN, object detection, or segmentation training result buffer (ResultClassId), the ResultType can be set to one of the following values. In this case, set the LabelOrIndex parameter to M_GENERAL.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For training (CNN, object detection, or segmentation) - retrieving general results															
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned															
M_LAST_EPOCH_UPDATED_PARAMETERS +	Retrieves the index of the epoch where classifier weights were updated the last time.														
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID	Specifies that the parameters were not updated.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the index of the epoch.	c		e	f	g	a		i		j	k	l	m	n	b
M_MINI_BATCH_PER_EPOCH +	Retrieves the number of mini-batches per epoch.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the number of mini-batches per epoch.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAIN_DATASET_MINI_BATCH_LOSS +	Retrieves an array containing the loss error values for each mini-batch during training. You can use the loss to evaluate the lack of confidence (doubt) associated with the classification during training. Note that the required array size, which you can retrieve with M_NB_ELEMENTS, is equivalent to M_MAX_EPOCH (MclassControl()) * M_MINI_BATCH_PER_EPOCH (**MclassGetResult**()) of the training context used.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

	0.0 <= *Value* <= 1.0	Specifies the training dataset loss for each mini-batch.	c		e	f	g	a		i		j	k	l	m	n	b

To retrieve general results from a segmentation training result buffer (ResultClassId), the ResultType can be set to one of the following values. In this case, set the LabelOrIndex parameter to M_GENERAL.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For training (segmentation) - retrieving general results																	
ResultType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ResultArrayPtr - Possible values returned																
M_DEV_DATASET_EPOCH_IOU_MEAN +		Retrieves an array containing the mean IOU (Intersection over Union) metric, as a percentage, of the development dataset using the trained context after each epoch.															
	ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 100.0	Specifies the mean IOU (Intersection over Union) metric of the development dataset, as a percentage.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAIN_DATASET_EPOCH_IOU_MEAN +		Retrieves an array containing the mean IOU (Intersection over Union) metric, as a percentage, of the training dataset using the trained context after each epoch.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 100.0	Specifies the mean IOU (Intersection over Union) metric of the training dataset, as a percentage.	c		e	f	g	a		i		j	k	l	m	n	b

To retrieve general results from a segmentation or object detection training result buffer (ResultClassId), the ResultType can be set to one of the following values. In this case, set the LabelOrIndex parameter to M_GENERAL.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For training (segmentation or object detection) - retrieving general results																	
ResultType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ResultArrayPtr - Possible values returned																
M_DEV_DATASET_EPOCH_LOSS +		Retrieves an array containing the loss metric of the development dataset after the last epoch. Note that the required array size, which you can retrieve with M_NB_ELEMENTS, is equivalent to the M_MAX_EPOCH (MclassControl()) of the training context used.															
	ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 1.0	Specifies the loss metric of the development dataset after each epoch.	c		e	f	g	a		i		j	k	l	m	n	b

To retrieve general results from a tree ensemble training result buffer (ResultClassId), the ResultType can be set to one of the following values. In this case, set the LabelOrIndex parameter to M_GENERAL.

Note, bagging information is typically unreliable if your training dataset has augmented entries.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For training (tree ensemble) - retrieving general results																	
ResultType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ResultArrayPtr - Possible values returned																
M_DEV_DATASET_ACCURACY_AFTER_EACH_TREE +		Retrieves an array containing the accuracies on the development dataset after adding each tree to the ensemble.															
	ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEV_DATASET_ERROR_RATE_AFTER_EACH_TREE +		Retrieves an array containing the error rates after on the development dataset after adding each tree to the ensemble.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_FEATURE_IMPORTANCE +		Retrieves an array containing the importance of each feature when training the classifier. This result can be disabled if M_FEATURE_IMPORTANCE_MODE is set to M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_ENTRIES_OUT_OF_BAG +		Retrieves the number of out-of-bag dataset entries. This is only available if M_COMPUTE_OUT_OF_BAG_RESULTS is set to M_ENABLE and M_BOOTSTRAP is not set to M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

	Value >= 0	Specifies the number of out-of-bag dataset entries.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_FEATURES +		Retrieves the number of features.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 1	Specifies the number of features.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_TREES_TRAINED +		Retrieves the number of trained trees. This result is equivalent to the number of trees in the classifier.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 1	Specifies the number of trained trees.	c		e	f	g	a		i		j	k	l	m	n	b
M_OUT_OF_BAG_ACCURACY +		Retrieves the out-of-bag error accuracy. This is only available if M_COMPUTE_OUT_OF_BAG_RESULTS is set to M_ENABLE and M_BOOTSTRAP is not set to M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 100.0	Specifies the out-of-bag accuracy, as a percentage.	c		e	f	g	a		i		j	k	l	m	n	b
M_OUT_OF_BAG_ACCURACY_AFTER_EACH_TREE +		Retrieves the array containing the accuracies on the out-of-bag set after adding each tree to the ensemble. This is only available if M_COMPUTE_OUT_OF_BAG_RESULTS is set to M_ENABLE and M_BOOTSTRAP is not set to M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_OUT_OF_BAG_CONFUSION_MATRIX +		Retrieves the confusion matrix obtained using the out-of-bag set. This is a square matrix. Aurora Imaging Library returns the matrix values, row-by-row, as an array. The array size is equivalent to the square of the total amount of classes, which is the same M_OUT_OF_BAG_CONFUSION_MATRIX_SIZE_X x M_OUT_OF_BAG_CONFUSION_MATRIX_SIZE_Y, or M_NB_ELEMENTS. This is only available if M_COMPUTE_OUT_OF_BAG_RESULTS is set to M_ENABLE and M_BOOTSTRAP is not set to M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_OUT_OF_BAG_CONFUSION_MATRIX_SIZE_X +		Retrieves the X-dimension of the confusion matrix. This value is calculated using the out-of-bag set. This is only available if M_COMPUTE_OUT_OF_BAG_RESULTS is set to M_ENABLE and M_BOOTSTRAP is not set to M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 1	Specifies the X-dimension of the confusion matrix.	c		e	f	g	a		i		j	k	l	m	n	b
M_OUT_OF_BAG_CONFUSION_MATRIX_SIZE_Y +		Retrieves the Y-dimension of the confusion matrix. This value is calculated using the out-of-bag set. This is only available if M_COMPUTE_OUT_OF_BAG_RESULTS is set to M_ENABLE and M_BOOTSTRAP is not set to M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 1	Specifies the Y-dimension of the confusion matrix.	c		e	f	g	a		i		j	k	l	m	n	b
M_OUT_OF_BAG_ERROR_RATE +		Retrieves the out-of-bag error rate. This is only available if M_COMPUTE_OUT_OF_BAG_RESULTS is set to M_ENABLE and M_BOOTSTRAP is not set to M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 100.0	Specifies the out-of-bag error rate.	c		e	f	g	a		i		j	k	l	m	n	b
M_OUT_OF_BAG_ERROR_RATE_AFTER_EACH_TREE +		Retrieves the array containing the accuracies on the out-of-bag set after adding each tree to the ensemble. This is only available if M_COMPUTE_OUT_OF_BAG_RESULTS is set to M_ENABLE and M_BOOTSTRAP is not set to M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_PROXIMITY_MATRIX +	Retrieves the proximity matrix. This is a square matrix. Aurora Imaging Library returns the matrix values, row-by-row, as an array. The array size is equivalent M_PROXIMITY_MATRIX_SIZE_X x M_PROXIMITY_MATRIX_SIZE_Y or M_NB_ELEMENTS. In the proximity matrix, the entry in cell (*j*, *k*) is a measure of similarity (or distance) between the entries to which row *j* and column *k* correspond. The proximity matrix result can be enabled or disabled using M_COMPUTE_PROXIMITY_MATRIX in the train context.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_PROXIMITY_MATRIX_SIZE_X +	Retrieves the X-dimension of the proximity matrix.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 1	Specifies the X-dimension of the proximity matrix.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROXIMITY_MATRIX_SIZE_Y +	Retrieves the Y-dimension of the proximity matrix.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 1	Specifies the Y-dimension of the proximity matrix.	c		e	f	g	a		i		j	k	l	m	n	b
M_SEED_VALUE +	Retrieves the seed used for the training dataset.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 1	Specifies the seed used for the training dataset.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAIN_DATASET_ACCURACY_AFTER_EACH_TREE +	Retrieves the array containing the accuracies on the training dataset after adding each tree to the classifier.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAIN_DATASET_ERROR_RATE_AFTER_EACH_TREE +	Retrieves the array containing the error rates on the training dataset after adding each tree to the classifier.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TREE_DEPTHS_ACHIEVED +	Retrieves the array containing the depth achieved by each tree in the classifier.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TREE_NUMBER_OF_LEAF_NODES_ACHIEVED +	Retrieves the array containing the number of leaf nodes for each tree in the classifier.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

To retrieve general results from a CNN, segmentation or tree ensemble training result buffer (ResultClassId), the ResultType parameter can be set to one of the following values. In this case, set the LabelOrIndex parameter to M_GENERAL.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For training (CNN, segmentation, or tree ensemble) - retrieving general results																
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned																
M_DEV_DATASET_ACCURACY +	Retrieves the accuracy on the development dataset using the trained context.															
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the accuracy on the development dataset, as a percentage.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEV_DATASET_CONFUSION_MATRIX +	Retrieves the confusion matrix obtained using the development dataset. This is a square matrix. Aurora Imaging Library returns the matrix values, row-by-row, as an array. The array size is equivalent to the square of the total amount of classes, which is the same M_DEV_DATASET_CONFUSION_MATRIX_SIZE_X x M_DEV_DATASET_CONFUSION_MATRIX_SIZE_Y, or M_NB_ELEMENTS.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEV_DATASET_CONFUSION_MATRIX_SIZE_X +	Retrieves the X-dimension of the confusion matrix. This value is calculated using the development dataset.	c		e	f	g	a		i		j	k	l	m	n	b

	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 1	Specifies the X-dimension of the confusion matrix.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEV_DATASET_CONFUSION_MATRIX_SIZE_Y +		Retrieves the Y-dimension of the confusion matrix. This value is calculated using the development dataset.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 1	Specifies the Y-dimension of the confusion matrix.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEV_DATASET_ERROR_RATE +		Retrieves the error rate on the development dataset. This can be expressed as 100.0 - M_DEV_DATASET_ACCURACY.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 100.0	Specifies the accuracy error rate on the development dataset, as a percentage.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAIN_DATASET_ACCURACY +		Retrieves the accuracy on the training dataset using the trained context.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 100.0	Specifies the accuracy on the training dataset, as a percentage.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAIN_DATASET_CONFUSION_MATRIX +		Retrieves the confusion matrix obtained using the training dataset. This is a square matrix. Aurora Imaging Library returns the matrix values, row-by-row, as an array. The array size is equivalent to the square of the total amount of classes, which is the same M_TRAIN_DATASET_CONFUSION_MATRIX_SIZE_X x M_TRAIN_DATASET_CONFUSION_MATRIX_SIZE_Y, or M_NB_ELEMENTS.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAIN_DATASET_CONFUSION_MATRIX_SIZE_X +		Retrieves the X-dimension of the confusion matrix. This value is calculated using the training dataset.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 1	Specifies the X-dimension of the confusion matrix.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAIN_DATASET_CONFUSION_MATRIX_SIZE_Y +		Retrieves the Y-dimension of the confusion matrix. This value is calculated using the training dataset.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 1	Specifies the Y-dimension of the confusion matrix.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAIN_DATASET_ERROR_RATE +		Retrieves the error rate on the training dataset. This can be expressed as 100.0 - M_TRAIN_DATASET_ACCURACY.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 100.0	Specifies the accuracy error rate on the training dataset, as a percentage.	c		e	f	g	a		i		j	k	l	m	n	b

To retrieve general results from a CNN, object detection, segmentation, anomaly detection, or tree ensemble training result buffer (ResultClassId), the ResultType parameter can be set to one of the following values. In this case, set the LabelOrIndex parameter to M_GENERAL.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For training (CNN, object detection, segmentation, anomaly detection or tree ensemble) - retrieving general results																
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned																
M_DEV_DATASET_USED_ENTRIES +	Retrieves an array containing the AIL_UUID of each image in the development dataset used during the training phase.															
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_UUID [optionally, in C++: a reference to a std::vector<AIL_UUID>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAIN_DATASET_USED_ENTRIES +	Retrieves an array containing the AIL_UUID of each image in the training dataset used during the training phase.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_UUID [optionally, in C++: a reference to a std::vector<AIL_UUID>]*	c		e	f	g	a		i		j	k	l	m	n	b

To retrieve general results from any prediction result buffer (ResultClassId), the ResultType can be set to one of the following values. In this case, set the LabelOrIndex parameter to M_GENERAL.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For prediction (CNN, object detection, segmentation, or ONNX) - retrieving general results																
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned																

M_CLASSIFIER_PREDEFINED_TYPE +		Retrieves the type of the predefined classifier.													
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c	e	f	g	a	i	j	k	l	m	n	b
	M_ADNET	Specifies an ADNet classifier context for anomaly detection.	c	e	f	g	a	i	j	k	l	m	n	b
	M_CSNET_COLOR_XL	Specifies an extra large CSNet classifier context that is for color images. This is a predefined classifier for segmentation.	c	e	f	g	a	i	j	k	l	m	n	b
	M_CSNET_M	Specifies a medium CSNet classifier context. This is a predefined classifier for segmentation.	c	e	f	g	a	i	j	k	l	m	n	b
	M_CSNET_MONO_XL	Specifies an extra large CSNet classifier context that is for monochrome images. This is a predefined classifier for segmentation.	c	e	f	g	a	i	j	k	l	m	n	b
	M_CSNET_S	Specifies a small CSNet classifier context. This is a predefined classifier for segmentation.	c	e	f	g	a	i	j	k	l	m	n	b
	M_CSNET_XL	Specifies an extra large CSNet classifier context. This is a predefined classifier for segmentation.	c	e	f	g	a	i	j	k	l	m	n	b
	M_CUSTOM	Specifies a custom classifier context. This is a predefined classifier for image classification, segmentation, or object detection.	c	e	f	g	a	i	j	k	l	m	n	b
	M_FCNET_COLOR_XL	Specifies an extra large legacy FCNet classifier context that is for color images. This is a predefined classifier for image classification or segmentation.	c	e	f	g	a	i	j	k	l	m	n	b
	M_FCNET_M	Specifies a medium legacy FCNet classifier context. This is a predefined classifier for image classification or segmentation.	c	e	f	g	a	i	j	k	l	m	n	b
	M_FCNET_MONO_XL	Specifies an extra large legacy FCNet classifier context that is for monochrome images. This is a predefined classifier for image classification or segmentation.	c	e	f	g	a	i	j	k	l	m	n	b
	M_FCNET_S	Specifies a small legacy FCNet classifier context. This is a predefined classifier for image classification or segmentation.	c	e	f	g	a	i	j	k	l	m	n	b
	M_FCNET_XL	Specifies an extra large legacy FCNet classifier context. This is a predefined classifier for image classification or segmentation.	c	e	f	g	a	i	j	k	l	m	n	b
	M_ICNET_COLOR_XL	Specifies an extra large ICNet classifier context that is for color images. This is a predefined classifier for image classification.	c	e	f	g	a	i	j	k	l	m	n	b
	M_ICNET_M	Specifies a medium ICNet classifier context. This is a predefined classifier for image classification.	c	e	f	g	a	i	j	k	l	m	n	b
	M_ICNET_MONO_XL	Specifies an extra large ICNet classifier context that is for monochrome images. This is a predefined classifier for image classification.	c	e	f	g	a	i	j	k	l	m	n	b
	M_ICNET_S	Specifies a small ICNet classifier context. This is a predefined classifier for image classification.	c	e	f	g	a	i	j	k	l	m	n	b
	M_ICNET_XL	Specifies an extra large ICNet classifier context. This is a predefined classifier for image classification.	c	e	f	g	a	i	j	k	l	m	n	b
	M_ODNET	Specifies an ODNet classifier context. This is a predefined classifier for object detection.	c	e	f	g	a	i	j	k	l	m	n	b
	M_USER_ONNX	Specifies an ONNX classifier context.	c	e	f	g	a	i	j	k	l	m	n	b

To retrieve general results from a CNN, segmentation, or anomaly detection prediction result buffer (ResultClassId), the ResultType can be set to one of the following values. In this case, set the LabelOrIndex parameter to M_GENERAL.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For prediction (CNN, segmentation, or anomaly detection) - retrieving general results

ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned																
M_PREDICTION_SIZE_X +	Retrieves the number of predictions the deep network has made according to the target image size. For image classification, the map size will typically be 1x1 for a target image the same size as the network's source layer, meaning a single prediction for the whole target image. For a larger target image, the map size can be larger, meaning the classifier has convolved the larger target image and made many predictions doing so. For segmentation, the network outputs a prediction for each pixel in the target image, so the map size will be the same size as the target image. For anomaly detection, this shows the output size of the pixel level anomaly detection results.															
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Retrieves the number of entries in the dataset.	c		e	f	g	a		i		j	k	l	m	n	b
M_PREDICTION_SIZE_Y +	Retrieves the number of predictions the deep network has made according to the target image size. For image classification, the map size will typically be 1x1 for a target image the same size as the network's source layer, meaning a single prediction for the whole target image. For a larger target image, the map size can be larger, meaning the classifier has convolved the larger target image and made many predictions doing so. For segmentation, the network outputs a prediction for each pixel in the target image, so the map size will be the same size as the target image. For anomaly detection, this shows the output size of the pixel level anomaly detection results.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Retrieves the number of entries in the dataset.	c		e	f	g	a		i		j	k	l	m	n	b

To retrieve general results from a CNN or segmentation prediction result buffer (ResultClassId), the ResultType can be set to one of the following values. In this case, set the LabelOrIndex parameter to M_GENERAL.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For prediction (CNN or segmentation) - retrieving general results

ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned																
M_BEST_INDEX_IMAGE_TYPE +	Retrieves the image type to provide when calling MclassDraw() with M_DRAW_BEST_INDEX_IMAGE or M_DRAW_BEST_INDEX_CONTOUR_IMAGE.															
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_UNSIGNED + 8	Specifies that the image type is 8-bit unsigned.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNSIGNED + 16	Specifies that the image type is 16-bit unsigned.	c		e	f	g	a		i		j	k	l	m	n	b
M_RECEPTIVE_FIELD_OFFSET_X +	Retrieves the offset along the X-axis, from the top-left corner of the target image, needed to place the first class score at the center of its receptive field. To calculate the center of the receptive field of each result returned for M_PREDICTION_SCORES (*Y*, *X*) in the target image, use the following formula: (*X* * M_RECEPTIVE_FIELD_STRIDE_X + M_RECEPTIVE_FIELD_OFFSET_X; *Y* * M_RECEPTIVE_FIELD_STRIDE_Y + M_RECEPTIVE_FIELD_OFFSET_Y). This value is unavailable if your dataset results were produced using MclassPredict() with an ONNX classifier (that is, calling MclassInquire() with M_CLASSIFIER_PREDEFINED_TYPE must not return M_USER_ONNX).	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the classification map offset along the X-axis.	c		e	f	g	a		i		j	k	l	m	n	b
M_RECEPTIVE_FIELD_OFFSET_Y +	Retrieves the offset along the Y-axis, from the top-left corner of the target image, needed to place the first class score at the center of its receptive field. To calculate the center of the receptive field of each result returned for M_PREDICTION_SCORES (*Y*, *X*) in the target image, use the following formula: (*X* * M_RECEPTIVE_FIELD_STRIDE_X + M_RECEPTIVE_FIELD_OFFSET_X; *Y* * M_RECEPTIVE_FIELD_STRIDE_Y + M_RECEPTIVE_FIELD_OFFSET_Y). This value is unavailable if your dataset results were produced using MclassPredict() with an ONNX classifier (that is, calling MclassInquire() with M_CLASSIFIER_PREDEFINED_TYPE must not return M_USER_ONNX).	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the classification map offset along the Y-axis.	c		e	f	g	a		i		j	k	l	m	n	b
M_RECEPTIVE_FIELD_SIZE_X +	Retrieves the size of the receptive field along the X-axis.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

Value >= 0	Specifies the size of the receptive field along the X-axis.	c		e	f	g	a		i		j	k	l	m	n	b
M_RECEPTIVE_FIELD_SIZE_Y +	Retrieves the size of the receptive field along the Y-axis.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the size of the receptive field along the Y-axis.	c		e	f	g	a		i		j	k	l	m	n	b
M_RECEPTIVE_FIELD_STRIDE_X +	Retrieves the stride spacing the receptive field centers in the target image along the X-axis. To calculate the center of the receptive field of each result returned for M_PREDICTION_SCORES (*Y, X*) in the target image, use the following formula: (*X* * M_RECEPTIVE_FIELD_STRIDE_X + M_RECEPTIVE_FIELD_OFFSET_X; *Y* * M_RECEPTIVE_FIELD_STRIDE_Y + M_RECEPTIVE_FIELD_OFFSET_Y). This value is unavailable if your dataset results were produced using MclassPredict() with an ONNX classifier (that is, calling MclassInquire() with M_CLASSIFIER_PREDEFINED_TYPE must not return M_USER_ONNX).	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the classification map scale along the X-axis.	c		e	f	g	a		i		j	k	l	m	n	b
M_RECEPTIVE_FIELD_STRIDE_Y +	Retrieves the stride spacing the receptive field centers in the target image along the Y-axis. To calculate the center of the receptive field of each result returned for M_PREDICTION_SCORES (*Y, X*) in the target image, use the following formula: (*X* * M_RECEPTIVE_FIELD_STRIDE_X + M_RECEPTIVE_FIELD_OFFSET_X; *Y* * M_RECEPTIVE_FIELD_STRIDE_Y + M_RECEPTIVE_FIELD_OFFSET_Y). This value is unavailable if your dataset results were produced using MclassPredict() with an ONNX classifier (that is, calling MclassInquire() with M_CLASSIFIER_PREDEFINED_TYPE must not return M_USER_ONNX).	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the classification map scale along the Y-axis.	c		e	f	g	a		i		j	k	l	m	n	b

To retrieve general results from a CNN, segmentation, or tree ensemble prediction result buffer (ResultClassId), the ResultType can be set to one of the following values. In this case, set the LabelOrIndex parameter to M_GENERAL.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For prediction (CNN, segmentation, or tree ensemble) - retrieving general results																
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned																
M_NUMBER_OF_CLASSES +	Retrieves the number of classes. For CNN or segmentation, this refers to the number of outputs in the network's last layer. For tree ensemble, this refers to the number of tree ensemble in the classifier. This can also be used in a tree ensemble prediction with LabelOrIndex set to M_GENERAL.															
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Retrieves the number of entries in the dataset.	c		e	f	g	a		i		j	k	l	m	n	b

To retrieve general results from a CNN, object detection, segmentation, or tree ensemble prediction result buffer (ResultClassId), the ResultType can be set to one of the following values. In this case, set the LabelOrIndex parameter to M_GENERAL.

To retrieve a class or instance specific result from an object detection result buffer, the LabelOrIndex parameter can be set to M_ALL_INSTANCES, M_CLASS_INDEX(), or M_INSTANCE_INDEX().

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For prediction (CNN, object detection, segmentation, or tree ensemble) - retrieving general or specific results																
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned																

M_BEST_CLASS_INDEX +	Retrieves the index of the class with the highest score. For image classification, M_BEST_CLASS_INDEX retrieves one index. For segmentation, the array (map) of values retrieved can be arranged in a 2D image, where the size is equal to M_PREDICTION_SIZE_X multiplied by M_PREDICTION_SIZE_Y. Each value is the index of the class having the best score for that X- Y-pixel location. The index of the class begins at 0. For object detection, to retrieve the class index with the highest score for all instances, set the LabelOrIndex parameter to M_ALL_INSTANCES. The array retrieved will have size equal to the number of instances (**MclassGetResult**() with M_NUMBER_OF_INSTANCES and LabelOrIndex set to M_GENERAL) and will contain the class index with the highest score for each instance. For object detection, to retrieve the class index with the highest score for all instances of a given class, set the LabelOrIndex parameter to M_CLASS_INDEX(). The array retrieved will have size equal to the number of instances (**MclassGetResult**() with M_NUMBER_OF_INSTANCES and LabelOrIndex set to M_CLASS_INDEX()) and will contain the class index with the highest score for each instance of a given class. For object detection, to retrieve the class index with the highest score for a specific instance, set the LabelOrIndex parameter to M_INSTANCE_INDEX().														
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the index of the class with the highest score.	c		e	f	g	a		i		j	k	l	m	n	b
M_BEST_CLASS_SCORE +	Retrieves the highest class score. For image classification, M_BEST_CLASS_SCORE retrieves one score. For segmentation, the array (map) of values retrieved can be arranged in a 2D image, where the size is equal to M_PREDICTION_SIZE_X multiplied by M_PREDICTION_SIZE_Y. Each value is the best class score for that X- Y-pixel location. For object detection, to retrieve the highest score for all instances, set the LabelOrIndex parameter to M_ALL_INSTANCES. The array retrieved will have size equal to the number of instances (**MclassGetResult**() with M_NUMBER_OF_INSTANCES and LabelOrIndex set to M_GENERAL) and will contain the highest score for each instance. For object detection, to retrieve the highest score for all instances of a given class, set the LabelOrIndex parameter to M_CLASS_INDEX(). The array retrieved will have size equal to the number of instances (**MclassGetResult**() with M_NUMBER_OF_INSTANCES and LabelOrIndex set to M_CLASS_INDEX()) and will contain the highest score for each instance of a given class. For object detection, to retrieve the highest score for a specific instance, set the LabelOrIndex parameter to M_INSTANCE_INDEX().	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the best class score.	c		e	f	g	a		i		j	k	l	m	n	b

To retrieve general results from a CNN, segmentation, or tree ensemble prediction result buffer, set the LabelOrIndex parameter to M_GENERAL; to retrieve class-specific results from a CNN or segmentation prediction result buffer, set the LabelOrIndex parameter the index of a specific class (using M_CLASS_INDEX()). In these cases, the ResultType can be set to one of the following values. Note, you cannot receive class-specific results from a tree ensemble result buffer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For prediction (CNN, segmentation, or tree ensemble) - retrieving general or class-specific results																
ResultType ResultArrayPtr - Possible values returned	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NUMBER_OF_PREDICTIONS +	Retrieves the total number of class scores. For general results obtained from a CNN or segmentation predict result buffer, the total corresponds to M_PREDICTION_SIZE_X * M_PREDICTION_SIZE_Y * M_NUMBER_OF_CLASSES scores. The first batch of M_PREDICTION_SIZE_X * M_PREDICTION_SIZE_Y scores are for the first class and the subsequent batches are for the remaining classes. In other words, the data is organized planar-wise. For general results obtained from a tree ensemble predict result buffer, the total corresponds to M_NUMBER_OF_CLASSES. For class-specific results (LabelOrIndex set to M_CLASS_INDEX()) obtained from a CNN or segmentation predict result buffer, the total corresponds to M_PREDICTION_SIZE_X * M_PREDICTION_SIZE_Y.															
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Retrieves the total number of class scores.	c		e	f	g	a		i		j	k	l	m	n	b
M_PREDICTION_SCORES +	Retrieves an array of all the class scores. The number of returned values is M_NUMBER_OF_PREDICTIONS. For general results obtained from a CNN or segmentation prediction result buffer, the returned values are organized planar-wise in a 3d volume of size M_PREDICTION_SIZE_X * M_PREDICTION_SIZE_Y * M_NUMBER_OF_CLASSES. Note, each band contains the score of one given class. The index in this returned array of the score of class C at pixel X, Y is: index = (C * MapSizeX * MapSizeY) + (Y * MapSizeX) + X. For general results obtained from a tree ensemble prediction result buffer, the number of class scores is equivalent to M_NUMBER_OF_PREDICTIONS. For class-specific results (LabelOrIndex set to M_CLASS_INDEX()) obtained from a CNN or segmentation prediction result buffer, the returned values are organized in a vector that corresponds to a 2D volume of size M_PREDICTION_SIZE_X * M_PREDICTION_SIZE_Y. The index of the score at location X, Y is: index = (Y * M_PREDICTION_SIZE_X) + X.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies all of the class scores.	c		e	f	g	a		i		j	k	l	m	n	b

To retrieve general results from an object detection prediction result buffer, set the LabelOrIndex parameter to M_GENERAL. In this case, the ResultType parameter can be set to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For prediction (object detection) - retrieving general results																	
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned																	
M_SCORE_THRESHOLD +	Retrieves the score threshold from the classifier context.																
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Retrieves the score threshold.	c		e	f	g	a		i		j	k	l	m	n	b

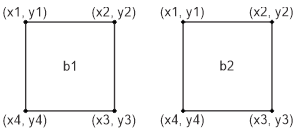
To retrieve general or class-specific results from an object detection prediction result buffer, set the LabelOrIndex parameter to M_GENERAL or M_CLASS_INDEX(), respectively. In these cases, the ResultType parameter can be set to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For prediction (object detection) - retrieving general or class-specific results																	
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned																	
M_NUMBER_OF_INSTANCES +	Retrieves the number of instances.																
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Retrieves the number of instances.	c		e	f	g	a		i		j	k	l	m	n	b

To retrieve results for a specific class, a specific instance, or all instances from an object detection prediction result buffer, set the LabelOrIndex parameter to M_CLASS_INDEX(), M_INSTANCE_INDEX(), or M_ALL_INSTANCES. In these cases, you can set the ResultType parameter to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For prediction (object detection) - retrieving results for a specific class, a specific instance, or all instances																	
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned																	
M_BOX_4_CORNERS +	Retrieves an array containing the coordinates of the four corners of the specified bounding boxes. The 8 coordinates of the first box (b1) are grouped, followed by the 8 coordinates of the second box (b2), repeating this process for all specified boxes in the order b1x1, b1y1, b1x2, b1y2, b1x3, b1y3, b1x4, b1y4, b2x1, b2y1, b2x2, b2y2, b2x3, b2y3, b2x4, b2y4, etc.  The size of the array will be equal to the number of specified instances (M_NUMBER_OF_INSTANCES) multiplied by 8.															
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CENTER_X +	Retrieves the X-coordinates of the centers of the specified bounding boxes. When retrieving results for all instances, or a specific class, the X-coordinates are returned in an array. The size of the array can be retrieved with M_NUMBER_OF_INSTANCES. Only one coordinate will be returned if you are retrieving results for a specific instance.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* **|** *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_CENTER_Y +	Retrieves the Y-coordinates for the centers of the specified bounding boxes. When retrieving results for all instances, or a specific class, the Y-coordinates are returned in an array. The size of the array can be retrieved with M_NUMBER_OF_INSTANCES. Only one coordinate will be returned if you are retrieving results for a specific instance.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* **|** *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_HEIGHT +	Retrieves the heights of the specified bounding boxes. When retrieving results for all instances, or a specific class, the heights are returned in an array. The size of the array can be retrieved with M_NUMBER_OF_INSTANCES. Only one height will be returned if you are retrieving results for a specific instance.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* **|** *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

M_WIDTH +	Retrieves the widths of the specified bounding boxes. When retrieving results for all instances, or a specific class, the widths are returned in an array. The size of the array can be retrieved with M_NUMBER_OF_INSTANCES. Only one coordinate will be returned if you are retrieving results for a specific instance.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* | *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

To retrieve general results from an anomaly detection prediction result buffer, set the LabelOrIndex parameter to M_GENERAL. In this case, the ResultType parameter can be set to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For prediction (anomaly detection) - retrieving general results																
ResultType ResultArrayPtr - Possible values returned	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_IMAGE_PREDICTED_ANOMALOUS +	Retrieves whether or not the image is considered anomalous.															
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that the image is normal.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that the image is anomalous.	c		e	f	g	a		i		j	k	l	m	n	b
M_IMAGE_SCORE +	Retrieves the anomaly score of the whole image.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the anomaly score of the image.	c		e	f	g	a		i		j	k	l	m	n	b
M_MASK_IMAGE +	Retrieves whether or not each pixel in the image is considered anomalous. The array must have a size of M_PREDICTION_SIZE_X x M_PREDICTION_SIZE_Y.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL_SCORES +	Retrieves the anomaly score of each pixel in the image. The array must have a size of M_PREDICTION_SIZE_X x M_PREDICTION_SIZE_Y.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

To retrieve general results from an ONNX prediction result buffer, set the LabelOrIndex parameter to M_GENERAL. In this case, the ResultType parameter can be set to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For prediction (ONNX) - retrieving general results																
ResultType ResultArrayPtr - Possible values returned	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NUMBER_OF_OUTPUTS +	Retrieves the number of outputs of the ONNX model.															
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 1	Retrieves the number of outputs.	c		e	f	g	a		i		j	k	l	m	n	b

To retrieve general results from an ONNX prediction result buffer, set the LabelOrIndex parameter to M_OUTPUT_INDEX(). In this case, the ResultType parameter can be set to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For prediction (ONNX) - retrieving results for a specific output																
ResultType ResultArrayPtr - Possible values returned	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DATA_TYPE +	Retrieves the tensor element type.															
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_FLOAT+16	Specifies that the ONNX type is float16. This is not a basic C++ data type thus its representation is platform dependent. The recommended Aurora Imaging Library data type is AIL_UINT16.	c		e	f	g	a		i		j	k	l	m	n	b
M_FLOAT+32	Specifies that the ONNX type is float. The recommended Aurora Imaging Library data type is AIL_FLOAT.	c		e	f	g	a		i		j	k	l	m	n	b

	M_FLOAT+64	Specifies that the ONNX type is double. The recommended Aurora Imaging Library data type is AIL_DOUBLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SIGNED+8	Specifies that the ONNX type is int8. The recommended Aurora Imaging Library data type is AIL_INT8.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SIGNED+16	Specifies that the ONNX type is int16. The recommended Aurora Imaging Library data type is AIL_INT16.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SIGNED+32	Specifies that the ONNX type is int32. The recommended Aurora Imaging Library data type is AIL_INT32.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SIGNED+64	Specifies that the ONNX type is int64. The recommended Aurora Imaging Library data type is AIL_INT64.	c		e	f	g	a		i		j	k	l	m	n	b
	M_UNSIGNED+8	Specifies that the ONNX type is uint8. The recommended Aurora Imaging Library data type is AIL_UINT8.	c		e	f	g	a		i		j	k	l	m	n	b
	M_UNSIGNED+16	Specifies that the ONNX type is uint8. The recommended Aurora Imaging Library data type is AIL_UINT8.	c		e	f	g	a		i		j	k	l	m	n	b
M_OUTPUT_DATA +		Retrieves the output tensor data. You can retrieve the number of elements in the tensor with M_NB_ELEMENTS. Apply the corresponding Aurora Imaging Library data type modifier to retrieve the output data in the appropriate format otherwise, elements are converted to AIL_DOUBLE which can be detrimental, especially for float16 and int64 ONNX data types.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_OUTPUT_RAW +		Retrieves the output tensor raw data. Retrieves the output tensor raw data. You can retrieve the size (in bytes) of the tensor with M_NB_ELEMENTS. This result must be cast using the M_TYPE_AIL_UINT8 modifier.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_UINT8 [optionally, in C++: a reference to a std::vector<AIL_UINT8>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_OUTPUT_SHAPE +		Retrieves the output tensor shape array. You can retrieve the number of elements in the array with M_NB_ELEMENTS. The number of elements composing the shape equals the dimensionality (rank) of the tensor. The values of the shape correspond to the lengths of the dimensions of the tensor. The product of all lengths gives the total number of elements in the tensor.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 1	Retrieves the output tensor shape array.	c		e	f	g	a		i		j	k	l	m	n	b

To retrieve the status of the training or prediction operation from a result buffer (ResultClassId), the ResultType can be set to the following value. In this case, set the LabelOrIndex parameter to M_GENERAL. Unless otherwise specified, the following values apply to a CNN, object detection, segmentation, anomaly detection, or tree ensemble training result buffer, or a CNN, object detection, segmentation, anomaly detection, tree ensemble, or ONNX prediction result buffer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For training (CNN, object detection, segmentation, anomaly detection, or tree ensemble) or prediction (CNN, object detection, segmentation, anomaly detection, tree ensemble, or ONNX) - retrieving the operation's resulting status

ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned																
M_STATUS +	Retrieves information regarding the state of the operation.															
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPLETE	Specifies that the operation completed successfully.	c		e	f	g	a		i		j	k	l	m	n	b
M_CURRENTLY_PREDICTING	Specifies that the prediction operation is currently ongoing. You can only get this status if you are retrieving it from another thread. This only applies if you are retrieving results from a CNN, object detection, segmentation, anomaly detection, tree ensemble, or ONNX prediction result buffer (ResultClassId).	c		e	f	g	a		i		j	k	l	m	n	b
M_CURRENTLY_TRAINING	Specifies that the training operation is currently ongoing. You can only get this status if you are retrieving it from another thread. This only applies if you are retrieving results from a CNN, object detection, segmentation, anomaly detection, or tree ensemble training result buffer (ResultClassId).	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERNAL_ERROR	Specifies that an unexpected error occurred during the operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID_REGION_BUFFER	Specifies that the target image had an invalid region buffer. This only applies if you are retrieving results from a CNN, object detection, segmentation, anomaly detection, tree ensemble, or ONNX prediction result buffer (ResultClassId).	c		e	f	g	a		i		j	k	l	m	n	b
M_NO_VALID_DEVSET_ENTRY	Specifies that the target image had an invalid region buffer.	c		e	f	g	a		i		j	k	l	m	n	b

M_NON_FINITE_VALUE_DETECTED	Specifies that the training terminated because a non-finite value was detected when the network's parameters were last updated. These values include NaN (Not a Number) and INF (infinity) cases. Such invalid values are often caused by numerically unstable computations that occur during training due to a learning rate that is too high. This only applies if you are retrieving results from a CNN object detection, or segmentation training result buffer (ResultClassId).	c		e	f	g	a		i		j	k	l	m	n	b
M_NOT_ENOUGH_GPU_MEMORY	Specifies that a memory allocation error occurred while training on the GPU. This only applies if you are retrieving results from a CNN, object detection, segmentation, or anomaly detection training result buffer (ResultClassId).	c		e	f	g	a		i		j	k	l	m	n	b
M_NOT_ENOUGH_MEMORY	Specifies that a memory allocation error occurred during the operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_PREDICT_NOT_PERFORMED	Specifies that the prediction operation was not performed. This is the initial status. This will be the status shown if MclassPredict() is terminated early by an Aurora Imaging Library error. This only applies if you are retrieving results from a CNN, object detection, segmentation, anomaly detection, tree ensemble, or ONNX prediction result buffer (ResultClassId).	c		e	f	g	a		i		j	k	l	m	n	b
M_STOPPED_BY_REQUEST	Specifies that the current execution of the operation was explicitly stopped (MclassControl() with M_STOP_PREDICT or M_STOP_TRAIN).	c		e	f	g	a		i		j	k	l	m	n	b
M_TIMEOUT_REACHED	Specifies that the operation ended because the timeout limit was reached (MclassControl() with M_TIMEOUT).	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAINING_NOT_PERFORMED	Specifies that the training operation was not performed. This is the initial status. This only applies if you are retrieving results from a CNN, object detection, segmentation, anomaly detection, or tree ensemble training result buffer (ResultClassId).	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for the values listed in For training (CNN, object detection, segmentation, anomaly detection or tree ensemble) - retrieving general results, For prediction (CNN, object detection, segmentation, or tree ensemble) - retrieving general or specific results, For prediction (object detection) - retrieving results for a specific class, a specific instance, or all instances; and for the following values: M_DEV_DATASET_EPOCH_ERROR_RATE; M_DEV_DATASET_ACCURACY_AFTER_EACH_TREE; M_DEV_DATASET_ERROR_RATE_AFTER_EACH_TREE; M_FEATURE_IMPORTANCE; M_OUT_OF_BAG_ACCURACY_AFTER_EACH_TREE; M_OUT_OF_BAG_CONFUSION_MATRIX; M_OUT_OF_BAG_ERROR_RATE_AFTER_EACH_TREE; M_TRAIN_DATASET_ACCURACY_AFTER_EACH_TREE; M_TRAIN_DATASET_ERROR_RATE_AFTER_EACH_TREE; M_TREE_DEPTHS_ACHIEVED; M_TREE_NUMBER_OF_LEAF_NODES_ACHIEVED; M_DEV_DATASET_CONFUSION_MATRIX; M_TRAIN_DATASET_CONFUSION_MATRIX; M_PREDICTION_SCORES; M_OUTPUT_DATA; M_OUTPUT_RAW; M_OUTPUT_SHAPE.

You can add the following value to the above-mentioned values to determine the required number of elements in the array (array size).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For determining the required number of elements in the array (array size)																
ResultType combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned																
M_NB_ELEMENTS	Retrieves the required array size (number of elements) to store the returned values.															
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for
Tables:

- For training (CNN or segmentation) - retrieving general results
- For training (CNN, object detection, segmentation, or anomaly detection) - retrieving general results
- For training (CNN, object detection, or segmentation) - retrieving general results
- For training (segmentation) - retrieving general results
- For training (segmentation or object detection) - retrieving general results
- For training (tree ensemble) - retrieving general results
- For training (CNN, segmentation, or tree ensemble) - retrieving general results
- For training (CNN, object detection, segmentation, anomaly detection or tree ensemble) - retrieving general results
- For prediction (CNN, object detection, segmentation, or ONNX) - retrieving general results
- For prediction (CNN, segmentation, or anomaly detection) - retrieving general results
- For prediction (CNN or segmentation) - retrieving general results
- For prediction (CNN, segmentation, or tree ensemble) - retrieving general results
- For prediction (CNN, object detection, segmentation, or tree ensemble) - retrieving general or specific results
- For prediction (CNN, segmentation, or tree ensemble) - retrieving general or class-specific results
- For training (CNN, object detection, segmentation, anomaly detection, or tree ensemble) or prediction (CNN, object detection, segmentation, anomaly detection, tree ensemble, or ONNX) - retrieving the operation's resulting status

the values listed in all the parameter association tables except **For prediction (object detection) - retrieving general results**, **For prediction (object detection) - retrieving general or class-specific results**, **For prediction (object detection) - retrieving results for a specific class, a specific instance, or all instances**, **For prediction (anomaly detection) - retrieving general results**, **For prediction (ONNX) - retrieving general results**, **For prediction (ONNX) - retrieving results for a specific output**.

You can add the following value to the above-mentioned values to determine whether a result is available to be returned.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For determining whether results are available to be returned																
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned																

M_AVAILABLE		Retrieves whether the requested result type is available for retrieval.														
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_FALSE	Specifies that the requested result type is not available.	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRUE	Specifies that the requested result type is available.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for
Tables:

- For training (CNN or segmentation) - retrieving general results
- For training (CNN, object detection, segmentation, or anomaly detection) - retrieving general results
- For training (CNN, object detection, or segmentation) - retrieving general results
- For training (segmentation) - retrieving general results
- For training (segmentation or object detection) - retrieving general results
- For training (tree ensemble) - retrieving general results
- For training (CNN, segmentation, or tree ensemble) - retrieving general results
- For training (CNN, object detection, segmentation, anomaly detection or tree ensemble) - retrieving general results
- For prediction (CNN, object detection, segmentation, or ONNX) - retrieving general results
- For prediction (CNN, segmentation, or anomaly detection) - retrieving general results
- For prediction (CNN or segmentation) - retrieving general results
- For prediction (CNN, segmentation, or tree ensemble) - retrieving general results
- For prediction (CNN, object detection, segmentation, or tree ensemble) - retrieving general or specific results
- For prediction (CNN, segmentation, or tree ensemble) - retrieving general or class-specific results
- For prediction (object detection) - retrieving general results
- For prediction (object detection) - retrieving general or class-specific results
- For prediction (anomaly detection) - retrieving general results
- For prediction (ONNX) - retrieving general results
- For prediction (ONNX) - retrieving results for a specific output
- For training (CNN, object detection, segmentation, anomaly detection, or tree ensemble) or prediction (CNN, object detection, segmentation, anomaly detection, tree ensemble, or ONNX) - retrieving the operation's resulting status

the values listed in all the parameter association tables except **For prediction (object detection) - retrieving results for a specific class, a specific instance, or all instances**.

You can add one of the following values to the above-mentioned values to cast the requested results to the required data type.

> Note: In C++, when using a standard vector (std::vector) instead of an array, the values listed in this combination table must not be used. If you need to use a specific data type, instantiate the vector with the necessary data type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the data type																	
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned																
M_TYPE_AIL_DOUBLE	Casts the requested results to an *AIL_DOUBLE*.															
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* **|** *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_FLOAT	Casts the requested results to an *AIL_FLOAT*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_FLOAT [optionally, in C++: a reference to a std::vector<AIL_FLOAT>]* **|** *address of an AIL_FLOAT*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_ID	Casts the requested results to an *AIL_ID*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_ID [optionally, in C++: a reference to a std::vector<AIL_ID>]* **|** *address of an AIL_ID*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT	Casts the requested results to an *AIL_INT*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]* **|** *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT32	Casts the requested results to an *AIL_INT32*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]* **|** *address of an AIL_INT32*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT64	Casts the requested results to an *AIL_INT64*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]* **|** *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_UINT8	Casts the requested results to an *AIL_UINT8*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_UINT8 [optionally, in C++: a reference to a std::vector<AIL_UINT8>]* **|** *address of an AIL_UINT8*	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailclass11.lib.
DLL	Requires ail11.dll; ailclass11.dll.

MclassGetResultEntry

Synopsis

Get results from an entry in a dataset.

Syntax

```
void MclassGetResultEntry(
    AIL_ID DatasetContextClassId, //in
    AIL_INT64 EntryIndex,         //in
    AIL_UUID EntryKey,            //in
    AIL_INT64 TaskType,           //in
    AIL_INT64 LabelOrIndex,       //in
    AIL_INT64 ResultType,         //in
    void *ResultArrayPtr          //out
)
```

Description

This function retrieves results of the specified type from an entry in an images or a features dataset context.

Before retrieving results from a dataset, you must call MclassTrain() or MclassPredict() with it. If you want to get results from a dataset after calling MclassTrain(), you must copy the dataset-specific results from the result buffer that MclassTrain() produces to a dataset (MclassCopyResult()) before calling **MclassGetResultEntry**(). This copy is not required for MclassPredict(), since it can write results directly in the dataset.

Parameters

DatasetContextClassId

Specifies the identifier of the images or features dataset context from which to get results. This context is allocated using MclassAlloc() with M_DATASET_IMAGES or M_DATASET_FEATURES and you must have called MclassTrain() and copied the results using MclassCopyResult() or have called MclassPredict() with it.

EntryIndex

Specifies the index of the entry for which to get results.

When specifying an index with the EntryIndex parameter, you must set the EntryKey parameter to M_DEFAULT_KEY. You cannot set the EntryIndex parameter to M_DEFAULT and the EntryKey parameter to M_DEFAULT_KEY simultaneously.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the entry's index																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies that the EntryIndex parameter is not required.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the index of an entry.	c		e	f	g	a		i		j	k	l	m	n	b

EntryKey

Specifies the key (*AIL_UUID*) of the entry for which to get results. The key is defined as an Aurora Imaging Library universal unique identifier (UUID).

When specifying a key with the EntryKey parameter, you must set the EntryIndex parameter to M_DEFAULT. You cannot set the EntryKey parameter to M_DEFAULT_KEY and the EntryIndex parameter to M_DEFAULT simultaneously.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the entry's key																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_KEY	Specifies that the EntryKey parameter is not required.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_UUID Value	Specifies the key (*AIL_UUID*) of an entry.	c		e	f	g	a		i		j	k	l	m	n	b

TaskType

Specifies the task (classification, segmentation, object detection, or anomaly detection) for which to get results. An entry in an images dataset can contain classification, segmentation, object detection, or anomaly detection results. An entry in a features dataset can only contain classification results.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of task for which to get results																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_AUTO.	c		e	f	g	a		i		j	k	l	m	n	b
M_ANOMALY_DETECTION	Specifies that you are getting anomaly detection results.	c		e	f	g	a		i		j	k	l	m	n	b

M_AUTO	Specifies the task for which to get results automatically, given the content in the dataset used (DatasetContextClassId). If the dataset selected is an images dataset, and it contains only image classification results, M_AUTO is the same as M_CLASSIFICATION. If the dataset contains only segmentation results, M_AUTO is the same as M_SEGMENTATION. If the dataset only contains object detection results, M_AUTO is the same as M_OBJECT_DETECTION. If the dataset only contains anomaly detection results, M_AUTO is the same as M_ANOMALY_DETECTION. If the dataset contains two types of results, then M_AUTO is the same as M_SEGMENTATION (if segmentation results are present), or M_CLASSIFICATION (if no segmentation results are present). If a result type (ResultType) is specified that only applies to object detection, M_AUTO is the same as M_OBJECT_DETECTION. If the dataset contains image classification, segmentation, object detection, and anomaly detection results, M_AUTO is the same as M_SEGMENTATION. If the dataset selected is a features dataset, M_AUTO is the same as M_CLASSIFICATION, since segmentation and object detection are not supported for a features dataset. To determine whether M_CLASSIFICATION, M_SEGMENTATION, M_OBJECT_DETECTION, or M_ANOMALY_DETECTION results are available in the dataset, use the M_PREDICT_INFO result. Typically, a dataset should be set up for a specific task (either image classification, segmentation, object detection, or anomaly detection).	c		e	f	g	a		i		j	k	l	m	n	b
M_CLASSIFICATION	Specifies that you are getting classification results (image or feature).	c		e	f	g	a		i		j	k	l	m	n	b
M_OBJECT_DETECTION	Specifies that you are getting object detection results.	c		e	f	g	a		i		j	k	l	m	n	b
M_SEGMENTATION	Specifies that you are getting segmentation results.	c		e	f	g	a		i		j	k	l	m	n	b

LabelOrIndex

Specifies the class for which to get the entry result, or specifies that you are getting a general entry result. The values in this table are available for all tasks unless otherwise specified. Set this parameter to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the class for which to get the entry result, or specifies to get a general entry result																
Value	Description		(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_GENERAL.		c		e	f	g	a		i		j	k	l	m	n	b
M_CLASS_INDEX(AIL_INT ClassIndex)	Specifies the index of the class for which to get the entry result. To get the number of classes, use M_NUMBER_OF_CLASSES.		c		e	f	g	a		i		j	k	l	m	n	b
		Parameters															
		ClassIndex Specifies the index of a class.															
M_INSTANCE_INDEX(AIL_INT InstanceIndex)	Specifies the index of the instance for which to get results. You can get the number of included instances in the result buffer using M_NUMBER_OF_INSTANCES. This is only available for images datasets with object detection results.		c		e	f	g	a		i		j	k	l	m	n	b
		Parameters															
		InstanceIndex Specifies the index of an instance.															
M_ALL_INSTANCES	Specifies to retrieve results for all instances. This is only available for images datasets with object detection results.		c		e	f	g	a		i		j	k	l	m	n	b
M_GENERAL	Specifies to get a general entry result.		c		e	f	g	a		i		j	k	l	m	n	b

ResultType

Specifies the type of result to retrieve from the dataset entry.

See the Parameter associations section for possible values that can be specified.

ResultArrayPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]
- array of type AIL_ID [optionally, in C++: a reference to a std::vector<AIL_ID>]
- array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]
- array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]
- array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]
- AIL_DOUBLE
- AIL_ID
- AIL_INT
- AIL_INT32
- AIL_INT64

Specifies the address of the array in which to write results.

See the Parameter associations section for possible values that can be returned.

Parameter associations

The tables below list possible values for the **ResultType** parameter and possible values returned to the **ResultArrayPtr** parameter.

- For retrieving general entry results (images or features dataset)
- For retrieving general entry results (images dataset)
- For retrieving general or class-specific entry results
- For retrieving results for a specific class, a specific object detection instance, or all instances (images dataset)

To retrieve general results from an entry in the images or features dataset (DatasetContextClassId), the ResultType parameter can be set to one of the following values. In this case, you must set the LabelOrIndex parameter to M_GENERAL unless otherwise specified. You can retrieve results for any task (TaskType).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving general entry results (images or features dataset)																		
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned																
M_BEST_CLASS_INDEX +	Retrieves the index of the class with the highest score for the entry. For classification, M_BEST_CLASS_INDEX retrieves one index. For segmentation, M_BEST_CLASS_INDEX retrieves an array of values (indices) that can be arranged in a 2D image, where the size is equal to M_PREDICTION_SIZE_X * M_PREDICTION_SIZE_Y. Each value is the index of the best class score for that X- Y-pixel location. The index of the class begins at 0. For object detection, to retrieve the class index with the highest score for all instances, set the LabelOrIndex parameter to M_ALL_INSTANCES. The size of the retrieved array is equal to the number of instances (**MclassGetResultEntry**() with M_NUMBER_OF_INSTANCES and LabelOrIndex set to M_GENERAL) and will contain the class index with the highest score for each instance. For object detection, to retrieve the class index with the highest score for all instances of a given class, set the LabelOrIndex parameter to M_CLASS_INDEX(). The size of the retrieved array is equal to the number of instances (**MclassGetResultEntry**() with M_NUMBER_OF_INSTANCES and LabelOrIndex set to M_CLASS_INDEX()) and will contain the class index with the highest score for each instance of a given class. For object detection, to retrieve the class index with the highest score for a specific instance, set the LabelOrIndex parameter to M_INSTANCE_INDEX().															
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the index of the class with the highest score for the entry.	c		e	f	g	a		i		j	k	l	m	n	b
M_BEST_CLASS_SCORE +	Retrieves the highest class score for the entry. For classification, M_BEST_CLASS_SCORE retrieves one score. For segmentation, M_BEST_CLASS_SCORE retrieves an array of values (scores) that can be arranged in a 2D image, where the size is equal to M_PREDICTION_SIZE_X * M_PREDICTION_SIZE_Y. Each value is the best class score for that X-Y-pixel location. The index of the class begins at 0. For object detection, to retrieve the highest score for all instances, set the LabelOrIndex parameter to M_ALL_INSTANCES. The size of the retrieved array is equal to the number of instances (**MclassGetResultEntry**() with M_NUMBER_OF_INSTANCES and LabelOrIndex set to M_GENERAL) and will contain the highest score for each instance. For object detection, to retrieve the highest score for all instances of a given class, set the LabelOrIndex parameter to M_CLASS_INDEX(). The size of the retrieved array is equal to the number of instances (**MclassGetResultEntry**() with M_NUMBER_OF_INSTANCES and LabelOrIndex set to M_CLASS_INDEX()) and will contain the highest score for each instance of a given class. For object detection, to retrieve the highest score for a specific instance, set the LabelOrIndex parameter to M_INSTANCE_INDEX().	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the best class score.	c		e	f	g	a		i		j	k	l	m	n	b
M_IMAGE_PREDICTED_ANOMALOUS +	Retrieves the anomaly class index at the image-level.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that the image is normal.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that the image is anomalous.	c		e	f	g	a		i		j	k	l	m	n	b
M_IMAGE_SCORE +	Retrieves the anomaly score of the image.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the best class score.	c		e	f	g	a		i		j	k	l	m	n	b
M_MASK_IMAGE +	Retrieves the anomaly class index for each pixel in the image.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that the pixel is normal.	c		e	f	g	a		i		j	k	l	m	n	b

M_TRUE	Specifies that the pixel is anomalous.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_CLASSES +	Retrieves the total number of classes available. This refers to the number of outputs in the classifier's last layer.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the number of classes.	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL_SCORES +	Retrieves the anomaly score of each pixel in the image.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the pixel's anomaly score.	c		e	f	g	a		i		j	k	l	m	n	b

To retrieve general results from an entry in the images dataset (DatasetContextClassId), the ResultType parameter can be set to one of the following values. In this case, you must set DatasetContextClassId parameter to the identifier of an images dataset, and the LabelOrIndex parameter to M_GENERAL. You can retrieve results for any task (TaskType).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving general entry results (images dataset)																
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned																
M_BEST_INDEX_IMAGE_TYPE +	Retrieves the image type to provide when calling MclassDraw() or MclassDrawEntry() with M_DRAW_BEST_INDEX_IMAGE or M_DRAW_BEST_INDEX_CONTOUR_IMAGE.															
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_UNSIGNED + 8	Specifies that the image type is 8-bit unsigned.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNSIGNED + 16	Specifies that the image type is 16-bit unsigned.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLASSIFIER_PREDEFINED_TYPE +	Retrieves the type of the predefined classifier.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_ADNET	Specifies an ADNet classifier context for anomaly detection.	c		e	f	g	a		i		j	k	l	m	n	b
M_CSNET_COLOR_XL	Specifies an extra large CSNet classifier context that is for color images. This is a predefined classifier for segmentation (M_SEGMENTATION).	c		e	f	g	a		i		j	k	l	m	n	b
M_CSNET_M	Specifies a medium CSNet classifier context. This is a predefined classifier for segmentation (M_SEGMENTATION).	c		e	f	g	a		i		j	k	l	m	n	b
M_CSNET_MONO_XL	Specifies an extra large CSNet classifier context that is for monochrome images. This is a predefined classifier for segmentation (M_SEGMENTATION).	c		e	f	g	a		i		j	k	l	m	n	b
M_CSNET_S	Specifies a small CSNet classifier context. This is a predefined classifier for segmentation (M_SEGMENTATION).	c		e	f	g	a		i		j	k	l	m	n	b
M_CSNET_XL	Specifies an extra large CSNet classifier context. This is a predefined classifier for segmentation (M_SEGMENTATION).	c		e	f	g	a		i		j	k	l	m	n	b
M_CUSTOM	Specifies a custom classifier context. This is a predefined classifier for image classification (M_CLASSIFICATION), segmentation (M_SEGMENTATION, or object detection (M_OBJECT_DETECTION).	c		e	f	g	a		i		j	k	l	m	n	b
M_FCNET_COLOR_XL	Specifies an extra large legacy FCNet classifier context that is for color images. This is a predefined classifier for image classification (M_CLASSIFICATION) or segmentation (M_SEGMENTATION).	c		e	f	g	a		i		j	k	l	m	n	b
M_FCNET_M	Specifies a medium legacy FCNet classifier context. This is a predefined classifier for image classification (M_CLASSIFICATION) or segmentation (M_SEGMENTATION).	c		e	f	g	a		i		j	k	l	m	n	b
M_FCNET_MONO_XL	Specifies an extra large legacy FCNet classifier context that is for monochrome images. This is a predefined classifier for image classification (M_CLASSIFICATION) or segmentation (M_SEGMENTATION).	c		e	f	g	a		i		j	k	l	m	n	b
M_FCNET_S	Specifies a small legacy FCNet classifier context. This is a predefined classifier for image classification (M_CLASSIFICATION) or segmentation (M_SEGMENTATION).	c		e	f	g	a		i		j	k	l	m	n	b

	M_FCNET_XL	Specifies an extra large legacy FCNet classifier context. This is a predefined classifier for image classification (M_CLASSIFICATION) or segmentation (M_SEGMENTATION).	c		e	f	g	a		i		j	k	l	m	n	b
	M_ICNET_COLOR_XL	Specifies an extra large ICNet classifier context that is for color images. This is a predefined classifier for image classification (M_CLASSIFICATION).	c		e	f	g	a		i		j	k	l	m	n	b
	M_ICNET_M	Specifies a medium ICNet classifier context. This is a predefined classifier for image classification (M_CLASSIFICATION).	c		e	f	g	a		i		j	k	l	m	n	b
	M_ICNET_MONO_XL	Specifies an extra large ICNet classifier context that is for monochrome images. This is a predefined classifier for image classification (M_CLASSIFICATION).	c		e	f	g	a		i		j	k	l	m	n	b
	M_ICNET_S	Specifies a small ICNet classifier context. This is a predefined classifier for image classification (M_CLASSIFICATION).	c		e	f	g	a		i		j	k	l	m	n	b
	M_ICNET_XL	Specifies an extra large ICNet classifier context. This is a predefined classifier for image classification (M_CLASSIFICATION).	c		e	f	g	a		i		j	k	l	m	n	b
	M_ODNET	Specifies an ODNet classifier context.	c		e	f	g	a		i		j	k	l	m	n	b
	M_USER_ONNX	Specifies an ONNX classifier context.	c		e	f	g	a		i		j	k	l	m	n	b
M_PREDICT_INFO +		Retrieves whether the entry contains results from the specified task (TaskType). For example, if your task is segmentation, you will get M_TRUE if the entry contains segmentation results and M_FALSE if it does not. When using M_PREDICT_INFO, you must specify a specific task (you cannot set the TaskType parameter to M_AUTO or M_DEFAULT). If you never called MclassTrain() or MclassPredict() with the specified dataset (DatasetContextClassId), M_PREDICT_INFO always returns M_FALSE.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_FALSE	Specifies that the entry does not contain results from the specified task (M_CLASSIFICATION, M_SEGMENTATION, M_OBJECT_DETECTION, or M_ANOMALY_DETECTION).	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRUE	Specifies that the entry does contain results from the specified task (M_CLASSIFICATION, M_SEGMENTATION, M_OBJECT_DETECTION, or M_ANOMALY_DETECTION).	c		e	f	g	a		i		j	k	l	m	n	b
M_PREDICTION_SIZE_X +		Retrieves the number of class scores available for the entry, along the X-direction.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0	Specifies the number of class scores available for the entry, along the X-direction. For image classification (which uses the entire entry image), M_PREDICTION_SIZE_X always returns 1.	c		e	f	g	a		i		j	k	l	m	n	b
M_PREDICTION_SIZE_Y +		Retrieves the number of class scores available for the entry, along the Y-direction.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0	Specifies the number of class scores available for the entry, along the Y-direction. For image classification (which uses the entire entry image), M_PREDICTION_SIZE_Y always returns 1.	c		e	f	g	a		i		j	k	l	m	n	b
M_RECEPTIVE_FIELD_OFFSET_X +		Retrieves the offset along the X-axis, from the top-left corner of the target image, needed to place the first class score at the center of its receptive field. To calculate the center of the receptive field of each result returned for M_PREDICTION_SCORES (*Y, X*) in the target image, use the following formula: (*X* * M_RECEPTIVE_FIELD_STRIDE_X + M_RECEPTIVE_FIELD_OFFSET_X; *Y* * M_RECEPTIVE_FIELD_STRIDE_Y + M_RECEPTIVE_FIELD_OFFSET_Y). This ResultType value is unavailable if your dataset results (DatasetContextClassId) were produced using MclassPredict() with an ONNX classifier (that is, calling MclassInquire() with M_CLASSIFIER_PREDEFINED_TYPE must not return M_USER_ONNX).	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0	Specifies the receptive field offset along the X-axis.	c		e	f	g	a		i		j	k	l	m	n	b

M_RECEPTIVE_FIELD_OFFSET_Y +	Retrieves the offset along the Y-axis, from the top-left corner of the target image, needed to place the first class score at the center of its receptive field. To calculate the center of the receptive field of each result returned for M_PREDICTION_SCORES (*Y*, *X*) in the target image, use the following formula: (*X* * M_RECEPTIVE_FIELD_STRIDE_X + M_RECEPTIVE_FIELD_OFFSET_X; *Y* * M_RECEPTIVE_FIELD_STRIDE_Y + M_RECEPTIVE_FIELD_OFFSET_Y). This ResultType value is unavailable if your dataset results (DatasetContextClassId) were produced using MclassPredict() with an ONNX classifier (that is, calling MclassInquire() with M_CLASSIFIER_PREDEFINED_TYPE must not return M_USER_ONNX).	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the receptive field offset along the Y-axis.	c		e	f	g	a		i		j	k	l	m	n	b
M_RECEPTIVE_FIELD_SIZE_X +	Retrieves the size of the receptive field along the X-axis. This ResultType value is unavailable if your dataset results (DatasetContextClassId) were produced using MclassPredict() with an ONNX classifier (that is, calling MclassInquire() with M_CLASSIFIER_PREDEFINED_TYPE must not return M_USER_ONNX).	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the size of the receptive field along the X-axis.	c		e	f	g	a		i		j	k	l	m	n	b
M_RECEPTIVE_FIELD_SIZE_Y +	Retrieves the size of the receptive field along the Y-axis. This ResultType value is unavailable if your dataset results (DatasetContextClassId) were produced using MclassPredict() with an ONNX classifier (that is, calling MclassInquire() with M_CLASSIFIER_PREDEFINED_TYPE must not return M_USER_ONNX).	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the size of the receptive field along the Y-axis.	c		e	f	g	a		i		j	k	l	m	n	b
M_RECEPTIVE_FIELD_STRIDE_X +	Retrieves the stride spacing the receptive field centers in the target image along the X-axis. To calculate the center of the receptive field of each result returned for M_PREDICTION_SCORES (*Y*, *X*) in the target image, use the following formula: (*X* * M_RECEPTIVE_FIELD_STRIDE_X + M_RECEPTIVE_FIELD_OFFSET_X; *Y* * M_RECEPTIVE_FIELD_STRIDE_Y + M_RECEPTIVE_FIELD_OFFSET_Y). This ResultType value is unavailable if your dataset results (DatasetContextClassId) were produced using MclassPredict() with an ONNX classifier (that is, calling MclassInquire() with M_CLASSIFIER_PREDEFINED_TYPE must not return M_USER_ONNX).	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the stride spacing the receptive fields along the X-axis.	c		e	f	g	a		i		j	k	l	m	n	b
M_RECEPTIVE_FIELD_STRIDE_Y +	Retrieves the stride spacing the receptive field centers in the target image along the Y-axis. To calculate the center of the receptive field of each result returned for M_PREDICTION_SCORES (*Y*, *X*) in the target image, use the following formula: (*X* * M_RECEPTIVE_FIELD_STRIDE_X + M_RECEPTIVE_FIELD_OFFSET_X; *Y* * M_RECEPTIVE_FIELD_STRIDE_Y + M_RECEPTIVE_FIELD_OFFSET_Y). This ResultType value is unavailable if your dataset results (DatasetContextClassId) were produced using MclassPredict() with an ONNX classifier (that is, calling MclassInquire() with M_CLASSIFIER_PREDEFINED_TYPE must not return M_USER_ONNX).	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the stride spacing the receptive fields along the Y-axis.	c		e	f	g	a		i		j	k	l	m	n	b

To retrieve general or class-specific results from an entry in the images dataset (DatasetContextClassId), the ResultType parameter can be set to one of the following values. In this case, you can set the LabelOrIndex parameter to M_GENERAL or a specific class. Unless otherwise specified, you can retrieve results for any task (TaskType) or dataset (images or features).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

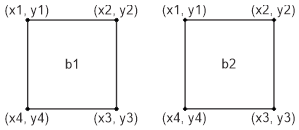
For retrieving general or class-specific entry results																
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned																
M_NUMBER_OF_INSTANCES +	Retrieves the number of instances. M_NUMBER_OF_INSTANCES is only available for M_OBJECT_DETECTION tasks.															
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Retrieves the number of instances.	c		e	f	g	a		i		j	k	l	m	n	b

M_NUMBER_OF_PREDICTIONS +	Retrieves the total number of class scores. For general results, the total number of class scores in an images dataset corresponds to M_PREDICTION_SIZE_X * M_PREDICTION_SIZE_Y * M_NUMBER_OF_CLASSES. The total number of scores in a features dataset is equal to the number of classes. For class-specific results, the total number of scores in an images dataset corresponds to M_PREDICTION_SIZE_X * M_PREDICTION_SIZE_Y. The total number of scores for a features dataset is one. The class score data that gets returned is organized planar-wise. Specifically, the first batch of M_PREDICTION_SIZE_X * M_PREDICTION_SIZE_Y scores are for the first class, and the subsequent batches are for the remaining classes.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Retrieves the total number of class scores.	c		e	f	g	a		i		j	k	l	m	n	b
M_PREDICTION_SCORES +	Retrieves the score of every class. Scores are returned in an array. The number of returned scores is retrievable with M_NUMBER_OF_PREDICTIONS. For general results, the returned values are organized planar-wise in a 3D volume of size M_PREDICTION_SIZE_X * M_PREDICTION_SIZE_Y * M_NUMBER_OF_CLASSES. Note, each band contains the score of one given class. For example, the index in this returned array of the score of class *C* at pixel (X, Y) is: *Index* = (*C* * M_PREDICTION_SIZE_X *M_PREDICTION_SIZE_Y) + (*Y* * M_PREDICTION_SIZE_X) + *X* . For class-specific results, the returned values are organized in a vector that corresponds to a 2D volume of size M_PREDICTION_SIZE_X * M_PREDICTION_SIZE_Y. For example, the index in this returned array of the score at location (X, Y) is: *Index* = (*Y* * M_PREDICTION_SIZE_X + *X*).	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies all of the class scores.	c		e	f	g	a		i		j	k	l	m	n	b

To retrieve results from a specific class, a specific object detection instance, or all instances from an entry in an images dataset (DatasetContextClassId), the ResultType parameter can be set to one of the following values. In this case, you must set the LabelOrIndex parameter to M_CLASS_INDEX(), M_INSTANCE_INDEX(), or M_ALL_INSTANCES and you must set the TaskType parameter to M_OBJECT_DETECTION.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving results for a specific class, a specific object detection instance, or all instances (images dataset)

ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned																
M_BOX_4_CORNERS +	Retrieves an array containing the coordinates of the four corners of the specified bounding boxes. The 8 coordinates of the first box (b1) are grouped, followed by the 8 coordinates of the second box (b2), repeating this process for all specified boxes in the order b1x1, b1y1, b1x2, b1y2, b1x3, b1y3, b1x4, b1y4, b2x1, b2y1, b2x2, b2y2, b2x3, b2y3, b2x4, b2y4, etc.  The size of the array will be equal to the number of specified instances (M_NUMBER_OF_INSTANCES) multiplied by 8.															
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CENTER_X +	Retrieves the X-coordinates of the centers of the specified bounding boxes. When retrieving results for all instances, or a specific class, the X-coordinates are returned in an array. The size of the array can be retrieved with M_NUMBER_OF_INSTANCES. Only one coordinate will be returned if you are retrieving results for a specific instance.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* **|** *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_CENTER_Y +	Retrieves the Y-coordinates for the centers of the specified bounding boxes. When retrieving results for all instances, or a specific class, the Y-coordinates are returned in an array. The size of the array can be retrieved with M_NUMBER_OF_INSTANCES. Only one coordinate will be returned if you are retrieving results for a specific instance.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* **|** *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_HEIGHT +	Retrieves the heights of the specified bounding boxes. When retrieving results for all instances, or a specific class, the heights are returned in an array. The size of the array can be retrieved with M_NUMBER_OF_INSTANCES. Only one height will be returned if you are retrieving results for a specific instance.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* **|** *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_WIDTH +	Retrieves the widths of the specified bounding boxes. When retrieving results for all instances, or a specific class, the widths are returned in an array. The size of the array can be retrieved with M_NUMBER_OF_INSTANCES. Only one coordinate will be returned if you are retrieving results for a specific instance.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* **|** *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for the values listed in For retrieving results for a specific class, a specific object detection instance, or all instances (images dataset); and for the following values: M_BEST_CLASS_INDEX; M_BEST_CLASS_SCORE; M_PREDICTION_SCORES.

You can add the following value to the above-mentioned values to determine the required number of elements in the array (array size).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For determining the required number of elements in the array (array size)																	
ResultType combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned																
M_NB_ELEMENTS	Retrieves the required array size (number of elements) to store the returned values.															
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for
Tables:

- For retrieving general entry results (images or features dataset)
- For retrieving general entry results (images dataset)
- For retrieving general or class-specific entry results

the values listed in all the parameter association tables except **For retrieving results for a specific class, a specific object detection instance, or all instances (images dataset)**.

You can add the following value to the above-mentioned values to determine whether a result is available to be returned.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For determining whether results are available to be returned																	
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned																
M_AVAILABLE	Retrieves whether the requested result type is available for retrieval.															
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that the requested result type is not available.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that the requested result type is available.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for
Tables:

- For retrieving general entry results (images or features dataset)
- For retrieving general entry results (images dataset)
- For retrieving general or class-specific entry results

the values listed in all the parameter association tables except **For retrieving results for a specific class, a specific object detection instance, or all instances (images dataset)**.

You can add one of the following values to the above-mentioned values to cast the requested results to the required data type.

> Note: In C++, when using a standard vector (std::vector) instead of an array, the values listed in this combination table must not be used. If you need to use a specific data type, instantiate the vector with the necessary data type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the data type																	
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned																
M_TYPE_AIL_DOUBLE	Casts the requested results to an *AIL_DOUBLE*.															
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* | *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_ID	Casts the requested results to an *AIL_ID*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_ID [optionally, in C++: a reference to a std::vector<AIL_ID>]* | *address of an AIL_ID*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT	Casts the requested results to an *AIL_INT*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]* | *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT32	Casts the requested results to an *AIL_INT32*.	c		e	f	g	a		i		j	k	l	m	n	b

ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]* **|** *address of an AIL_INT32*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT64	Casts the requested results to an *AIL_INT64*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]* **|** *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailclass11.lib.
DLL	Requires ail11.dll; ailclass11.dll.

MclassGetResultStat

Synopsis

Get results from a statistics classification result buffer.

Syntax

```
void MclassGetResultStat(
    AIL_ID StatResultClassId,  //in
    AIL_INT64 LabelOrIndex1,   //in
    AIL_INT64 LabelOrIndex2,   //in
    AIL_DOUBLE ScoreThreshold, //in
    AIL_INT64 ResultType,      //in
    void *ResultArrayPtr       //out
)
```

Description

This function retrieves results of the specified type from a statistics classification result buffer. Results are available after calling MclassStatCalculate().

Parameters

StatResultClassId

Specifies the identifier of the statistics classification result buffer from which to retrieve results. The result buffer must have been previously allocated on the system using MclassAllocResult() with M_STAT_ANO_RESULT, M_STAT_CNN_RESULT, M_STAT_DET_RESULT, M_STAT_SEG_RESULT, or M_STAT_TREE_ENSEMBLE_RESULT.

LabelOrIndex1

Specifies the first index for which to retrieve results. Supported indices depend on the statistics classification result buffer (StatResultClassId) and the result type (ResultType). For details about the index to set, see the description of the result type in the parameter associations section below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the first index for which to retrieve results

Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_GENERAL.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLASS_INDEX(AIL_INT ClassIndex)	Specifies the index of the class for which to get results. This is supported for specific result types (ResultType) retrieved from an image classification, object detection, or segmentation statistics result buffer (StatResultClassId). **Parameters** ClassIndex Specifies the index of the class. This must be an integer value between 0 and the total number of classes, inclusively. To get the total number of classes, call **MclassGetResultStat**() with M_NUMBER_OF_CLASSES for one of the supported statistics result buffer types you are using.	c		e	f	g	a		i		j	k	l	m	n	b
M_INSTANCE_INDEX(AIL_INT InstanceIndex)	Specifies the index of the instance (in a specific entry) for which to get results. This is supported for specific result types (ResultType) retrieved from an object detection statistics result buffer (StatResultClassId). **Parameters** InstanceIndex Specifies the index of the instance. This must be an integer value between 0 and the total number of instances, inclusively. To get the total number of instances, call MclassGetResult() with M_NUMBER_OF_INSTANCES.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_INDEX(AIL_INT RegionIndex)	Specifies the index of the region (in a specific entry) for which to get results. This is supported for specific result types (ResultType) retrieved from an object detection statistics result buffer (StatResultClassId). **Parameters** RegionIndex Specifies the index of the region. This must be an integer value between 0 and the total number of regions, inclusively. To get the total number of regions, call MclassInquireEntry() with M_NUMBER_OF_REGIONS.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL_INSTANCES	Specifies to retrieve results for all instances (in a specific entry). This is supported for specific result types (ResultType) retrieved from an object detection statistics result buffer (StatResultClassId).	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL_REGIONS	Specifies to retrieve results for all regions (in a specific entry). This is supported for specific result types (ResultType) retrieved from an object detection statistics result buffer (StatResultClassId).	c		e	f	g	a		i		j	k	l	m	n	b
M_ANOMALOUS	Specifies to get results related to anomalous entries only. This is supported for specific result types (ResultType) retrieved from an anomaly detection statistics result buffer (StatResultClassId).	c		e	f	g	a		i		j	k	l	m	n	b
M_GENERAL	Specifies to get general results. This is supported for specific result types (ResultType) retrieved from any classification statistics result buffer (StatResultClassId).	c		e	f	g	a		i		j	k	l	m	n	b
M_NO_CLASS	Specifies to get results related to entries not assigned to a class. This is supported for specific result types (ResultType) retrieved from an object detection statistics result buffer (StatResultClassId).	c		e	f	g	a		i		j	k	l	m	n	b

M_NON_ANOMALOUS	Specifies to get results related to non-anomalous entries only. This is supported for specific result types (ResultType) retrieved from an anomaly detection statistics result buffer (StatResultClassId).	c		e	f	g	a		i		j	k	l	m	n	b

LabelOrIndex2

Specifies the second index for which to retrieve results. Supported indices depend on the statistics classification result buffer (StatResultClassId) and the result type (ResultType). For details about the index to set, see the description of the result type in the parameter associations section below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the second index for which to retrieve results

Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_GENERAL.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLASS_INDEX(AIL_INT ClassIndex)	Specifies the index of the class for which to get results. This is supported for specific result types (ResultType) retrieved from an image classification, object detection, or segmentation statistics result buffer (StatResultClassId).	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	ClassIndex Specifies the index of the class. This must be an integer value between 0 and the total number of classes, inclusively. To get the total number of classes, call **MclassGetResultStat**() with M_NUMBER_OF_CLASSES for one of the supported statistics result buffer types you are using.															
M_ANOMALOUS	Specifies to get results related to anomalous entries only. This is supported for specific result types (ResultType) retrieved from an anomaly detection statistics result buffer (StatResultClassId).	c		e	f	g	a		i		j	k	l	m	n	b
M_GENERAL	Specifies to get general results. This is supported for specific result types (ResultType) retrieved from any classification statistics result buffer (StatResultClassId).	c		e	f	g	a		i		j	k	l	m	n	b
M_NO_CLASS	Specifies to get results related entries not assigned to a class. This is supported for specific result types (ResultType) retrieved from an object detection statistics result buffer (StatResultClassId).	c		e	f	g	a		i		j	k	l	m	n	b
M_NON_ANOMALOUS	Specifies to get results related to non-anomalous entries only. This is supported for specific result types (ResultType) retrieved from an anomaly detection statistics result buffer (StatResultClassId).	c		e	f	g	a		i		j	k	l	m	n	b

ScoreThreshold

Specifies the score threshold, when retrieving result types that use this information. When retrieving result types that do not use information, set this parameter to M_DEFAULT. When applicable, only results with scores greater than or equal to the specified threshold are returned. For details about when and how the score threshold applies, see the description of the result type in the parameter associations section below.

When specifying the score threshold to retrieve object detection statistics, matches with enough IOU (intersection over union) will still be considered misses (and not retrieved) if the score threshold is not met (IOU indicates the overlap between the ground-truth and predicted region; if the 2 regions overlap completely, the IOU is 1).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying score threshold, when retrieving result types that use this information

Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies that the score threshold does not apply.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL	Specifies to sequentially retrieve results from all score thresholds in an array, when applicable. Score thresholds can be adjusted using MclassControl() with M_SCORE_THRESHOLD_STEP. To retrieve the score threshold values, call **MclassGetResultStat**() with M_SCORE_THRESHOLDS.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the score threshold.	c		e	f	g	a		i		j	k	l	m	n	b

ResultType

Specifies the type of result to retrieve.

See the Parameter associations section for possible values that can be specified.

ResultArrayPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]
- array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]
- array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]
- array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]
- array of type AIL_UUID [optionally, in C++: a reference to a std::vector<AIL_UUID>]
- AIL_DOUBLE
- AIL_INT
- AIL_INT32
- AIL_INT64

Specifies the address of the array in which to write results.

See the Parameter associations section for possible values that can be returned.

Parameter associations

The tables below list possible values for the **ResultType** parameter and possible values returned to the **ResultArrayPtr** parameter.

- For any statistics result buffer
- For anomaly detection (pixel-level) or segmentation statistics result buffers
- For object detection (single-entry) statistics result buffers

To retrieve results for any statistics result buffer, set the ResultType parameter to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For any statistics result buffer

ResultType / ResultArrayPtr - Possible values returned	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_ACCURACY +	Retrieves the accuracy of a single class versus other classes according to the specified score threshold. To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId: Feature classification (M_STAT_TREE_ENSEMBLE_RESULT), image classification (M_STAT_CNN_RESULT), segmentation (M_STAT_SEG_RESULT), or object detection (M_STAT_DET_RESULT) statistics result ID; LabelOrIndex1: M_CLASS_INDEX(); LabelOrIndex2: M_GENERAL; ScoreThreshold: 0.0 <= Value <= 100.0, M_ALL, or M_DEFAULT StatResultClassId: Anomaly detection statistics result ID (M_STAT_ANO_RESULT); LabelOrIndex1: M_ANOMALOUS or M_NON_ANOMALOUS; LabelOrIndex2: M_GENERAL; ScoreThreshold: 0.0 <= Value <= 100.0, M_ALL, or M_DEFAULT															
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the accuracy.	c		e	f	g	a		i		j	k	l	m	n	b
M_ACCURACY_MACRO +	Retrieves the macro accuracy of the entire confusion matrix, giving equal importance to all classes. To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId: Any classification statistics result ID; LabelOrIndex1: M_GENERAL; LabelOrIndex2: M_GENERAL; ScoreThreshold: M_DEFAULT StatResultClassId: Object detection statistics result ID (M_STAT_DET_RESULT); LabelOrIndex1: M_GENERAL; LabelOrIndex2: M_GENERAL; ScoreThreshold: 0.0 <= Value <= 100.0 or M_ALL	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the macro accuracy.	c		e	f	g	a		i		j	k	l	m	n	b
M_ACCURACY_OVERALL +	Retrieves the overall accuracy of the entire confusion matrix, giving equal importance to all samples. To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId: Any classification statistics result ID; LabelOrIndex1: M_GENERAL; LabelOrIndex2: M_GENERAL; ScoreThreshold: M_DEFAULT StatResultClassId: Object detection statistics result ID (M_STAT_DET_RESULT); LabelOrIndex1: M_GENERAL; LabelOrIndex2: M_GENERAL; ScoreThreshold: 0.0 <= Value <= 100.0 or M_ALL	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the overall accuracy.	c		e	f	g	a		i		j	k	l	m	n	b

M_ACCURACY_WEIGHTED +	Retrieves the weighted accuracy of the entire confusion matrix, computed by taking the accuracy for each class and then averaging them, weighted by the number of true instances for each class. To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId | LabelOrIndex1 | LabelOrIndex2 | ScoreThreshold Any classification statistics result ID | M_GENERAL | M_GENERAL | M_DEFAULT Object detection statistics result ID (M_STAT_DET_RESULT) | M_GENERAL | M_GENERAL | 0.0 <= Value <= 100.0 or M_ALL	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the weighted accuracy.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUC_ROC +	Retrieves the area under the curve of the receiver operating characteristics (recall as a function of false positive rate) for a single class vs other classes. This result is not supported for object detection (M_STAT_DET_RESULT). To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId | LabelOrIndex1 | LabelOrIndex2 | ScoreThreshold Feature classification (M_STAT_TREE_ENSEMBLE_RESULT), image classification (M_STAT_CNN_RESULT), or segmentation (M_STAT_SEG_RESULT) statistics result ID | M_CLASS_INDEX() | M_GENERAL | M_DEFAULT Anomaly detection statistics result ID (M_STAT_ANO_RESULT) | M_ANOMALOUS or M_NON_ANOMALOUS | M_GENERAL | M_DEFAULT	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID	Specifies that you cannot retrieve area under the curve value (can occur if either all of the ground instances are of the given class, or none of them are).	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the area under the curve.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUC_ROC_MACRO +	Retrieves the average area under the curve of the receiver operating characteristics of all classes, giving the same importance to all classes. This result is not supported for object detection (M_STAT_DET_RESULT). To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId | LabelOrIndex1 | LabelOrIndex2 | ScoreThreshold Anomaly detection (M_STAT_ANO_RESULT), feature classification (M_STAT_TREE_ENSEMBLE_RESULT), image classification (M_STAT_CNN_RESULT), or segmentation (M_STAT_SEG_RESULT) statistics result ID | M_GENERAL | M_GENERAL | M_DEFAULT	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID	Specifies that you cannot retrieve area under the curve This can occur if one of the class-index specific M_AUC_ROC result is M_INVALID.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the area under the curve.	c		e	f	g	a		i		j	k	l	m	n	b

M_AUC_ROC_WEIGHTED +	Retrieves the weighted area under the curve of the receiver operating characteristics of all classes, computed by taking the area under the curve for each class and then averaging them, weighted by the number of true instances for each class. This result is not supported for object detection (M_STAT_DET_RESULT). To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId: Anomaly detection (M_STAT_ANO_RESULT), feature classification (M_STAT_TREE_ENSEMBLE_RESULT), image classification (M_STAT_CNN_RESULT), or segmentation (M_STAT_SEG_RESULT) statistics result ID; LabelOrIndex1: M_GENERAL; LabelOrIndex2: M_GENERAL; ScoreThreshold: M_DEFAULT	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID	Specifies that you cannot retrieve area under the curve This can occur if one of the class-index specific M_AUC_ROC result is M_INVALID.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the area under the curve.	c		e	f	g	a		i		j	k	l	m	n	b
M_AVERAGE_PRECISION +	Retrieves an approximation of the area under the precision-recall curve for a single class versus other classes. It consists of the averaged interpolated precision across all selected score thresholds for the selected IOU threshold. To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId: Feature classification (M_STAT_TREE_ENSEMBLE_RESULT), image classification (M_STAT_CNN_RESULT), object detection (M_STAT_DET_RESULT), or segmentation (M_STAT_SEG_RESULT) statistics result ID; LabelOrIndex1: M_CLASS_INDEX(); LabelOrIndex2: M_GENERAL; ScoreThreshold: M_DEFAULT StatResultClassId: Anomaly detection statistics result ID (M_STAT_ANO_RESULT); LabelOrIndex1: M_ANOMALOUS or M_NON_ANOMALOUS; LabelOrIndex2: M_GENERAL; ScoreThreshold: M_DEFAULT	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the average precision.	c		e	f	g	a		i		j	k	l	m	n	b
M_AVERAGE_PRECISION_MACRO +	Retrieves the mean average precision of all classes, giving the same importance to all classes. To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId: Any classification statistics result ID; LabelOrIndex1: M_GENERAL; LabelOrIndex2: M_GENERAL; ScoreThreshold: M_DEFAULT	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the mean average precision.	c		e	f	g	a		i		j	k	l	m	n	b
M_AVERAGE_PRECISION_WEIGHTED +	Retrieves the weighted mean average precision of all classes, computed by taking the mean average precision for each class and then averaging them, weighted by the number of true instances for each class. To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId: Any classification statistics result ID; LabelOrIndex1: M_GENERAL; LabelOrIndex2: M_GENERAL; ScoreThreshold: M_DEFAULT	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the weighted mean average precision.	c		e	f	g	a		i		j	k	l	m	n	b

M_CONFUSION_MATRIX +	Retrieves, in row major order, the full NxN confusion matrix across all classes, using best class index to populate the matrix. Rows of the confusion matrix are associated to ground truths classes and columns to predicted classes. To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId | LabelOrIndex1 | LabelOrIndex2 | ScoreThreshold Any classification statistics result ID | M_GENERAL | M_GENERAL | M_DEFAULT Object detection statistics result ID (M_STAT_DET_RESULT) | M_GENERAL | M_GENERAL | 0.0 <= Value <= 100.0 For object detection, an extra row/column is added at the end for the no class type. The ScoreThreshold parameter setting is also supported as a criterion to populate the matrix. For anomaly detection, the confusion matrix is 2x2, for M_ANOMALOUS in the first row/column and M_NON_ANOMALOUS in the second row/column. M_CONFUSION_MATRIX can also return, in row major order, the 2x2 1 vs other classes confusion matrix, depending on LabelOrIndex1 parameter setting. To do this, for feature classification (M_STAT_TREE_ENSEMBLE_RESULT), image classification (M_STAT_CNN_RESULT), object detection (M_STAT_DET_RESULT), or segmentation (M_STAT_SEG_RESULT) statistics result IDs, you must set the LabelOrIndex1 parameter to M_CLASS_INDEX(); for an anomaly detection statistics result ID (M_STAT_CNN_RESULT), you must set the LabelOrIndex1 parameter to M_ANOMALOUS or M_NON_ANOMALOUS. In these cases, you can use the ScoreThreshold parameter setting as a criterion on the selected class to populate the matrix. Note that for anomaly detection, M_ANOMALOUS and M_NON_ANOMALOUS are specified to get that class versus the other.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the confusion matrix values.	c		e	f	g	a		i		j	k	l	m	n	b
M_CONFUSION_MATRIX_COLUMN +	Retrieves the same value as the main confusion matrix result type, except for a single column of the confusion matrix. To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId | LabelOrIndex1 | LabelOrIndex2 | ScoreThreshold Feature classification (M_STAT_TREE_ENSEMBLE_RESULT), image classification (M_STAT_CNN_RESULT), or segmentation (M_STAT_SEG_RESULT) statistics result ID | M_CLASS_INDEX() | M_GENERAL | M_DEFAULT Object detection statistics result ID (M_STAT_DET_RESULT) | M_CLASS_INDEX() or M_NO_CLASS | M_GENERAL | 0.0 <= Value <= 100.0 Anomaly detection statistics result ID (M_STAT_ANO_RESULT) | M_ANOMALOUS or M_NON_ANOMALOUS | M_GENERAL | M_DEFAULT For more information about the confusion matrix information that is returned, and about the related parameters (StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold), see the description of M_CONFUSION_MATRIX.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the confusion matrix values.	c		e	f	g	a		i		j	k	l	m	n	b

M_CONFUSION_MATRIX_ELEMENT +	Retrieves the same value as the main confusion matrix result type, except for a single element of the confusion matrix. The LabelOrIndex1 and LabelOrIndex2 parameters specify row and column, respectively. To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId: Feature classification (M_STAT_TREE_ENSEMBLE_RESULT), image classification (M_STAT_CNN_RESULT), or segmentation (M_STAT_SEG_RESULT) statistics result ID — LabelOrIndex1: M_CLASS_INDEX() — LabelOrIndex2: M_CLASS_INDEX() — ScoreThreshold: M_DEFAULT StatResultClassId: Object detection statistics result ID (M_STAT_DET_RESULT) — LabelOrIndex1: M_CLASS_INDEX() or M_NO_CLASS — LabelOrIndex2: M_CLASS_INDEX() or M_NO_CLASS — ScoreThreshold: 0.0 <= Value <= 100.0 StatResultClassId: Anomaly detection statistics result ID (M_STAT_ANO_RESULT) — LabelOrIndex1: M_ANOMALOUS or M_NON_ANOMALOUS — LabelOrIndex2: M_ANOMALOUS or M_NON_ANOMALOUS — ScoreThreshold: M_DEFAULT For more information about the confusion matrix information that is returned, and about the related parameters (StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold), see the description of M_CONFUSION_MATRIX.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the confusion matrix element.	c		e	f	g	a		i		j	k	l	m	n	b
M_CONFUSION_MATRIX_PERCENTAGE +	Retrieves the same type of result as M_CONFUSION_MATRIX, with the values in each row normalized to sum up to 100%. Class indices with no ground truth instances will have the elements of their corresponding row set to M_INVALID. To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId: Any classification statistics result ID — LabelOrIndex1: M_GENERAL — LabelOrIndex2: M_GENERAL — ScoreThreshold: M_DEFAULT StatResultClassId: Object detection statistics result ID (M_STAT_DET_RESULT) — LabelOrIndex1: M_GENERAL — LabelOrIndex2: M_GENERAL — ScoreThreshold: 0.0 <= Value <= 100.0 For more information about the confusion matrix information that is returned, and about the related parameters (StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold), see the description of M_CONFUSION_MATRIX.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID	Specifies an invalid result.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the percentage.	c		e	f	g	a		i		j	k	l	m	n	b

		c		e	f	g	a		i		j	k	l	m	n	b
M_CONFUSION_MATRIX_ROW +	Retrieves the same value as the confusion matrix result type, except for a single row of the confusion matrix. To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId: Feature classification (M_STAT_TREE_ENSEMBLE_RESULT), image classification (M_STAT_CNN_RESULT), or segmentation (M_STAT_SEG_RESULT) statistics result ID — LabelOrIndex1: M_CLASS_INDEX() — LabelOrIndex2: M_GENERAL — ScoreThreshold: M_DEFAULT StatResultClassId: Object detection statistics result ID (M_STAT_DET_RESULT) — LabelOrIndex1: M_CLASS_INDEX() or M_NO_CLASS — LabelOrIndex2: M_GENERAL — ScoreThreshold: 0.0 <= Value <= 100.0 StatResultClassId: Anomaly detection statistics result ID (M_STAT_ANO_RESULT) — LabelOrIndex1: M_ANOMALOUS or M_NON_ANOMALOUS — LabelOrIndex2: M_GENERAL — ScoreThreshold: M_DEFAULT For more information about the confusion matrix information that is returned, and about the related parameters (StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold), see the description of M_CONFUSION_MATRIX.														
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the confusion matrix values.	c		e	f	g	a		i		j	k	l	m	n	b
M_CONFUSION_MATRIX_USED_ENTRIES +	Retrieves the dataset entry keys composing the confusion matrix. To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId: Any classification statistics result ID — LabelOrIndex1: M_GENERAL — LabelOrIndex2: M_GENERAL — ScoreThreshold: M_DEFAULT StatResultClassId: Object detection statistics result ID (M_STAT_DET_RESULT) — LabelOrIndex1: M_GENERAL — LabelOrIndex2: M_GENERAL — ScoreThreshold: 0.0 <= Value <= 100.0 or M_ALL For more information about the confusion matrix information that is returned, and about the related parameters (StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold), see the description of M_CONFUSION_MATRIX.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_UUID [optionally, in C++: a reference to a std::vector<AIL_UUID>]*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the UUID.	c		e	f	g	a		i		j	k	l	m	n	b
M_CONFUSION_MATRIX_USED_ENTRIES_COLUMN +	Retrieves the corresponding column of the confusion matrix of entries, representing the key of the entries in the dataset which constitute the corresponding column in the multi-class matrix. To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId: Feature classification (M_STAT_TREE_ENSEMBLE_RESULT), image classification (M_STAT_CNN_RESULT), or segmentation (M_STAT_SEG_RESULT) statistics result ID — LabelOrIndex1: M_CLASS_INDEX() — LabelOrIndex2: M_GENERAL — ScoreThreshold: M_DEFAULT StatResultClassId: Object detection statistics result ID (M_STAT_DET_RESULT) — LabelOrIndex1: M_CLASS_INDEX() or M_NO_CLASS — LabelOrIndex2: M_GENERAL — ScoreThreshold: 0.0 <= Value <= 100.0 StatResultClassId: Anomaly detection statistics result ID (M_STAT_ANO_RESULT) — LabelOrIndex1: M_ANOMALOUS or M_NON_ANOMALOUS — LabelOrIndex2: M_GENERAL — ScoreThreshold: M_DEFAULT For more information about the confusion matrix information that is returned, and about the related parameters (StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold), see the description of M_CONFUSION_MATRIX.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_UUID [optionally, in C++: a reference to a std::vector<AIL_UUID>]*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the UUID.	c		e	f	g	a		i		j	k	l	m	n	b

M_CONFUSION_MATRIX_USED_ENTRIES_ELEMENT +

Retrieves the corresponding element of the confusion matrix of entries, representing the key of the entries in the dataset which constitute the corresponding element in the multi-class matrix. The LabelOrIndex1 and LabelOrIndex2 parameters specify row and column, respectively.

To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following.

StatResultClassId	LabelOrIndex1	LabelOrIndex2	ScoreThreshold
Feature classification (M_STAT_TREE_ENSEMBLE_RESULT), image classification (M_STAT_CNN_RESULT), or segmentation (M_STAT_SEG_RESULT) statistics result ID	M_CLASS_INDEX()	M_CLASS_INDEX()	M_DEFAULT
Object detection statistics result ID (M_STAT_DET_RESULT)	M_CLASS_INDEX() or M_NO_CLASS	M_CLASS_INDEX() or M_NO_CLASS	0.0 <= Value <= 100.0
Anomaly detection statistics result ID (M_STAT_ANO_RESULT)	M_ANOMALOUS or M_NON_ANOMALOUS	M_ANOMALOUS or M_NON_ANOMALOUS	M_DEFAULT

For more information about the confusion matrix information that is returned, and about the related parameters (StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold), see the description of M_CONFUSION_MATRIX.

c e f g a i j k l m n b

ResultArrayPtr extra info

Data type info: Data type: *array of type AIL_UUID [optionally, in C++: a reference to a std::vector<AIL_UUID>]*

c e f g a i j k l m n b

Value

Specifies the UUID.

M_CONFUSION_MATRIX_USED_ENTRIES_ROW +

Retrieves the corresponding row of the confusion matrix of entries, representing the key of the entries in the dataset which constitute the corresponding row in the multi-class matrix.

To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following.

StatResultClassId	LabelOrIndex1	LabelOrIndex2	ScoreThreshold
Feature classification (M_STAT_TREE_ENSEMBLE_RESULT), image classification (M_STAT_CNN_RESULT), or segmentation (M_STAT_SEG_RESULT) statistics result ID	M_CLASS_INDEX()	M_GENERAL	M_DEFAULT
Object detection statistics result ID (M_STAT_DET_RESULT)	M_CLASS_INDEX() or M_NO_CLASS	M_GENERAL	0.0 <= Value <= 100.0
Anomaly detection statistics result ID (M_STAT_ANO_RESULT)	M_ANOMALOUS or M_NON_ANOMALOUS	M_GENERAL	M_DEFAULT

For more information about the confusion matrix information that is returned, and about the related parameters (StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold), see the description of M_CONFUSION_MATRIX.

c e f g a i j k l m n b

ResultArrayPtr extra info

Data type info: Data type: *array of type AIL_UUID [optionally, in C++: a reference to a std::vector<AIL_UUID>]*

c e f g a i j k l m n b

Value

Specifies the UUID.

c e f g a i j k l m n b

M_ERROR_ENTRIES +

Retrieves the dataset entry keys that were not processed because of an error. To retrieve the reported cause of the error, use M_ERROR_ENTRIES_STATUS.

To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following.

StatResultClassId	LabelOrIndex1	LabelOrIndex2	ScoreThreshold
Any classification statistics result ID	M_GENERAL	M_GENERAL	M_DEFAULT

c e f g a i j k l m n b

ResultArrayPtr extra info

Data type info: Data type: *array of type AIL_UUID [optionally, in C++: a reference to a std::vector<AIL_UUID>]*

c e f g a i j k l m n b

M_ERROR_ENTRIES_STATUS +	Retrieves the error status of each entry in error. To retrieve the key of the entries in error, use M_ERROR_ENTRIES. To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId: Any classification statistics result ID LabelOrIndex1: M_GENERAL LabelOrIndex2: M_GENERAL ScoreThreshold: M_DEFAULT	c	e	f	g	a	i	j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]*	c	e	f	g	a	i	j	k	l	m	n	b
M_FAILED_TO_RESTORE_GROUND_TRUTH_MASK	Specifies that the ground truth mask could not be restored.	c	e	f	g	a	i	j	k	l	m	n	b
M_FAILED_TO_RESTORE_PREDICTION_MASK	Specifies that the prediction mask could not be restored.	c	e	f	g	a	i	j	k	l	m	n	b
M_FAILED_TO_RESTORE_PREDICTION_SCORE_FILE	Specifies that the prediction score file could not be restored.	c	e	f	g	a	i	j	k	l	m	n	b
M_FCNET_SEGMENTATION_NOT_SUPPORTED	Specifies that segmentation with FCNET (legacy classifier) is not supported.	c	e	f	g	a	i	j	k	l	m	n	b
M_GROUND_TRUTH_MASK_DOES_NOT_EXIST	Specifies that the ground truth mask does not exist.	c	e	f	g	a	i	j	k	l	m	n	b
M_GROUND_TRUTH_MASK_INVALID_DIMENSIONS	Specifies that the dimensions of the ground truth mask are invalid.	c	e	f	g	a	i	j	k	l	m	n	b
M_IMAGE_FILE_INVALID	Specifies that the image file exists but cannot be restored.	c	e	f	g	a	i	j	k	l	m	n	b
M_INTERNAL_ERROR	Specifies that an unexpected error occurred during the operation.	c	e	f	g	a	i	j	k	l	m	n	b
M_MULTIPLE_GROUND_TRUTHS_NOT_SUPPORTED	Specifies that multiple ground truths are not supported.	c	e	f	g	a	i	j	k	l	m	n	b
M_NO_GROUND_TRUTH	Specifies that there is no ground truth.	c	e	f	g	a	i	j	k	l	m	n	b
M_NO_PREDICT_INFO	Specifies that there is no prediction information.	c	e	f	g	a	i	j	k	l	m	n	b
M_NUMBER_OF_CLASSES_MISMATCH	Specifies that there is a mismatch in the number of classes.	c	e	f	g	a	i	j	k	l	m	n	b
M_PREDICTION_MASK_DOES_NOT_EXIST	Specifies that the prediction mask does not exist.	c	e	f	g	a	i	j	k	l	m	n	b
M_PREDICTION_MASK_INVALID_DIMENSIONS	Specifies that the dimensions of the prediction mask are invalid.	c	e	f	g	a	i	j	k	l	m	n	b
M_SCORE_FILE_DOES_NOT_EXIST	Specifies that the score files does not exist.	c	e	f	g	a	i	j	k	l	m	n	b
M_SCORE_FILE_INVALID_DIMENSIONS	Specifies that the dimensions of the score file are invalid.	c	e	f	g	a	i	j	k	l	m	n	b
M_UNLABELED_GROUND_TRUTH_PIXELS	Specifies that ground truth pixels are unlabeled.	c	e	f	g	a	i	j	k	l	m	n	b
M_ERROR_RATE +	Retrieves the error rate of a single class versus other classes. To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId: Feature classification (M_STAT_TREE_ENSEMBLE_RESULT), image classification (M_STAT_CNN_RESULT), segmentation (M_STAT_SEG_RESULT), or object detection (M_STAT_DET_RESULT) statistics result ID LabelOrIndex1: M_CLASS_INDEX() LabelOrIndex2: M_GENERAL ScoreThreshold: 0.0 <= Value <= 100.0, M_ALL, or M_DEFAULT StatResultClassId: Anomaly detection statistics result ID (M_STAT_ANO_RESULT) LabelOrIndex1: M_ANOMALOUS or M_NON_ANOMALOUS LabelOrIndex2: M_GENERAL ScoreThreshold: 0.0 <= Value <= 100.0, M_ALL, or M_DEFAULT	c	e	f	g	a	i	j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c	e	f	g	a	i	j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the error rate.	c	e	f	g	a	i	j	k	l	m	n	b

M_ERROR_RATE_MACRO +	Retrieves the macro error rate of the entire confusion matrix, giving equal importance to all classes. To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId | LabelOrIndex1 | LabelOrIndex2 | ScoreThreshold Any classification statistics result ID | M_GENERAL | M_GENERAL | M_DEFAULT Object detection statistics result ID (M_STAT_DET_RESULT) | M_GENERAL | M_GENERAL | 0.0 <= Value <= 100.0 or M_ALL	c	e	f	g	a	i	j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c	e	f	g	a	i	j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the macro error rate.	c	e	f	g	a	i	j	k	l	m	n	b
M_ERROR_RATE_OVERALL +	Retrieves the overall error rate of the entire confusion matrix, giving equal importance to all samples. To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId | LabelOrIndex1 | LabelOrIndex2 | ScoreThreshold Any classification statistics result ID | M_GENERAL | M_GENERAL | M_DEFAULT Object detection statistics result ID (M_STAT_DET_RESULT) | M_GENERAL | M_GENERAL | 0.0 <= Value <= 100.0 or M_ALL	c	e	f	g	a	i	j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c	e	f	g	a	i	j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the overall error rate.	c	e	f	g	a	i	j	k	l	m	n	b
M_ERROR_RATE_WEIGHTED +	Retrieves the weighted error rate of the entire confusion matrix, computed by taking the error rate for each class and then averaging them, weighted by the number of true instances for each class. To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId | LabelOrIndex1 | LabelOrIndex2 | ScoreThreshold Any classification statistics result ID | M_GENERAL | M_GENERAL | M_DEFAULT Object detection statistics result ID (M_STAT_DET_RESULT) | M_GENERAL | M_GENERAL | 0.0 <= Value <= 100.0 or M_ALL	c	e	f	g	a	i	j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c	e	f	g	a	i	j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the weighted error rate.	c	e	f	g	a	i	j	k	l	m	n	b
M_F1SCORE +	Retrieves the F1-score of a single class versus other classes. To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId | LabelOrIndex1 | LabelOrIndex2 | ScoreThreshold Feature classification (M_STAT_TREE_ENSEMBLE_RESULT), image classification (M_STAT_CNN_RESULT), segmentation (M_STAT_SEG_RESULT), or object detection (M_STAT_DET_RESULT) statistics result ID | M_CLASS_INDEX() | M_GENERAL | 0.0 <= Value <= 100.0, M_ALL, or M_DEFAULT Anomaly detection statistics result ID (M_STAT_ANO_RESULT) | M_ANOMALOUS or M_NON_ANOMALOUS | M_GENERAL | 0.0 <= Value <= 100.0, M_ALL, or M_DEFAULT	c	e	f	g	a	i	j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c	e	f	g	a	i	j	k	l	m	n	b

M_INVALID	Specifies that you cannot retrieve the F1-score result (occurs when TP, FP and FN are 0 with respect to the given class index).	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the F1-score.	c		e	f	g	a		i		j	k	l	m	n	b
M_F1SCORE_MACRO +	Retrieves the macro F1-score of the entire confusion matrix, giving equal importance to all classes. To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId | LabelOrIndex1 | LabelOrIndex2 | ScoreThreshold Any classification statistics result ID | M_GENERAL | M_GENERAL | M_DEFAULT Object detection statistics result ID (M_STAT_DET_RESULT) | M_GENERAL | M_GENERAL | 0.0 <= Value <= 100.0 or M_ALL	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the macro F1-score.	c		e	f	g	a		i		j	k	l	m	n	b
M_F1SCORE_MICRO +	Retrieves the micro F1-score of the entire confusion matrix, giving equal importance to all samples. To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId | LabelOrIndex1 | LabelOrIndex2 | ScoreThreshold Any classification statistics result ID | M_GENERAL | M_GENERAL | M_DEFAULT Object detection statistics result ID (M_STAT_DET_RESULT) | M_GENERAL | M_GENERAL | 0.0 <= Value <= 100.0 or M_ALL	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the micro F1-score.	c		e	f	g	a		i		j	k	l	m	n	b
M_F1SCORE_WEIGHTED +	Retrieves the weighted F1-score of the entire confusion matrix, computed by taking the F1-score for each class and then averaging them, weighted by the number of true instances for each class. To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId | LabelOrIndex1 | LabelOrIndex2 | ScoreThreshold Any classification statistics result ID | M_GENERAL | M_GENERAL | M_DEFAULT Object detection statistics result ID (M_STAT_DET_RESULT) | M_GENERAL | M_GENERAL | 0.0 <= Value <= 100.0 or M_ALL	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the weighted F1-score.	c		e	f	g	a		i		j	k	l	m	n	b

M_FALSE_NEGATIVES +	Retrieves the number of false negatives out of a confusion matrix for a 1 versus other classes, using the ScoreThreshold parameter as the criterion on the selected class to populate the matrix. To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId: Feature classification (M_STAT_TREE_ENSEMBLE_RESULT), image classification (M_STAT_CNN_RESULT), segmentation (M_STAT_SEG_RESULT), or object detection (M_STAT_DET_RESULT) statistics result ID — LabelOrIndex1: M_CLASS_INDEX() — LabelOrIndex2: M_GENERAL — ScoreThreshold: 0.0 <= Value <= 100.0, M_ALL, or M_DEFAULT StatResultClassId: Anomaly detection statistics result ID (M_STAT_ANO_RESULT) — LabelOrIndex1: M_ANOMALOUS or M_NON_ANOMALOUS — LabelOrIndex2: M_GENERAL — ScoreThreshold: 0.0 <= Value <= 100.0, M_ALL, or M_DEFAULT	c	e	f	g	a	i	j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]*	c	e	f	g	a	i	j	k	l	m	n	b
Value >= 0	Specifies the false negatives.	c	e	f	g	a	i	j	k	l	m	n	b
M_FALSE_POSITIVES +	Retrieves the number of false positives out of a confusion matrix for a 1 versus other classes, using the ScoreThreshold parameter as the criterion on the selected class to populate the matrix. To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId: Feature classification (M_STAT_TREE_ENSEMBLE_RESULT), image classification (M_STAT_CNN_RESULT), segmentation (M_STAT_SEG_RESULT), or object detection (M_STAT_DET_RESULT) statistics result ID — LabelOrIndex1: M_CLASS_INDEX() — LabelOrIndex2: M_GENERAL — ScoreThreshold: 0.0 <= Value <= 100.0, M_ALL, or M_DEFAULT StatResultClassId: Anomaly detection statistics result ID (M_STAT_ANO_RESULT) — LabelOrIndex1: M_ANOMALOUS or M_NON_ANOMALOUS — LabelOrIndex2: M_GENERAL — ScoreThreshold: 0.0 <= Value <= 100.0, M_ALL, or M_DEFAULT	c	e	f	g	a	i	j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]*	c	e	f	g	a	i	j	k	l	m	n	b
Value >= 0	Specifies the false positives.	c	e	f	g	a	i	j	k	l	m	n	b
M_FDR +	Retrieves the False Discovery Rate (FDR) of a single class versus other classes. To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId: Feature classification (M_STAT_TREE_ENSEMBLE_RESULT), image classification (M_STAT_CNN_RESULT), segmentation (M_STAT_SEG_RESULT), or object detection (M_STAT_DET_RESULT) statistics result ID — LabelOrIndex1: M_CLASS_INDEX() — LabelOrIndex2: M_GENERAL — ScoreThreshold: 0.0 <= Value <= 100.0, M_ALL, or M_DEFAULT StatResultClassId: Anomaly detection statistics result ID (M_STAT_ANO_RESULT) — LabelOrIndex1: M_ANOMALOUS or M_NON_ANOMALOUS — LabelOrIndex2: M_GENERAL — ScoreThreshold: 0.0 <= Value <= 100.0, M_ALL, or M_DEFAULT	c	e	f	g	a	i	j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c	e	f	g	a	i	j	k	l	m	n	b
M_INVALID	Specifies that you cannot retrieve the FDR result (occurs when none of the predictions are of the given class; that is, there is no TP/FP).	c	e	f	g	a	i	j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the FDR.	c	e	f	g	a	i	j	k	l	m	n	b

M_FDR_MACRO +	Retrieves the macro false discovery rate of the entire confusion matrix, giving equal importance to all classes. To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId | LabelOrIndex1 | LabelOrIndex2 | ScoreThreshold Any classification statistics result ID | M_GENERAL | M_GENERAL | M_DEFAULT Object detection statistics result ID (M_STAT_DET_RESULT) | M_GENERAL | M_GENERAL | 0.0 <= Value <= 100.0 or M_ALL	c	e	f	g	a	i	j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c	e	f	g	a	i	j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the macro false discovery rate.	c	e	f	g	a	i	j	k	l	m	n	b
M_FDR_MICRO +	Retrieves the micro false discovery rate of the entire confusion matrix, giving equal importance to all samples. To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId | LabelOrIndex1 | LabelOrIndex2 | ScoreThreshold Any classification statistics result ID | M_GENERAL | M_GENERAL | M_DEFAULT Object detection statistics result ID (M_STAT_DET_RESULT) | M_GENERAL | M_GENERAL | 0.0 <= Value <= 100.0 or M_ALL	c	e	f	g	a	i	j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c	e	f	g	a	i	j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the micro false discovery rate.	c	e	f	g	a	i	j	k	l	m	n	b
M_FDR_WEIGHTED +	Retrieves the weighted false discovery rate of the entire confusion matrix, computed by taking the false discovery rate for each class and then averaging them, weighted by the number of true instances for each class. To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId | LabelOrIndex1 | LabelOrIndex2 | ScoreThreshold Any classification statistics result ID | M_GENERAL | M_GENERAL | M_DEFAULT Object detection statistics result ID (M_STAT_DET_RESULT) | M_GENERAL | M_GENERAL | 0.0 <= Value <= 100.0 or M_ALL	c	e	f	g	a	i	j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c	e	f	g	a	i	j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the weighted false discovery rate.	c	e	f	g	a	i	j	k	l	m	n	b
M_FNR +	Retrieves the False Negative Rate (FNR) of a single class versus other classes. To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId | LabelOrIndex1 | LabelOrIndex2 | ScoreThreshold Feature classification (M_STAT_TREE_ENSEMBLE_RESULT), image classification (M_STAT_CNN_RESULT), segmentation (M_STAT_SEG_RESULT), or object detection (M_STAT_DET_RESULT) statistics result ID | M_CLASS_INDEX() | M_GENERAL | 0.0 <= Value <= 100.0, M_ALL, or M_DEFAULT Anomaly detection statistics result ID (M_STAT_ANO_RESULT) | M_ANOMALOUS or M_NON_ANOMALOUS | M_GENERAL | 0.0 <= Value <= 100.0, M_ALL, or M_DEFAULT	c	e	f	g	a	i	j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c	e	f	g	a	i	j	k	l	m	n	b

	M_INVALID	Specifies that you cannot retrieve the FNR result (occurs when none of the ground truth instances are of the given class; that is, there is no TP/FN).	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 100.0	Specifies the FNR.	c		e	f	g	a		i		j	k	l	m	n	b
M_FNR_MACRO +		Retrieves the macro false negative rate of the entire confusion matrix, giving equal importance to all classes. To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId | LabelOrIndex1 | LabelOrIndex2 | ScoreThreshold Any classification statistics result ID | M_GENERAL | M_GENERAL | M_DEFAULT Object detection statistics result ID (M_STAT_DET_RESULT) | M_GENERAL | M_GENERAL | 0.0 <= Value <= 100.0 or M_ALL	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 100.0	Specifies the macro false negative rate.	c		e	f	g	a		i		j	k	l	m	n	b
M_FNR_MICRO +		Retrieves the micro false negative rate of the entire confusion matrix, giving equal importance to all samples. To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId | LabelOrIndex1 | LabelOrIndex2 | ScoreThreshold Any classification statistics result ID | M_GENERAL | M_GENERAL | M_DEFAULT Object detection statistics result ID (M_STAT_DET_RESULT) | M_GENERAL | M_GENERAL | 0.0 <= Value <= 100.0 or M_ALL	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 100.0	Specifies the micro false negative rate.	c		e	f	g	a		i		j	k	l	m	n	b
M_FNR_WEIGHTED +		Retrieves the weighted false negative rate of the entire confusion matrix, computed by taking the false negative rate for each class and then averaging them, weighted by the number of true instances for each class. To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId | LabelOrIndex1 | LabelOrIndex2 | ScoreThreshold Any classification statistics result ID | M_GENERAL | M_GENERAL | M_DEFAULT Object detection statistics result ID (M_STAT_DET_RESULT) | M_GENERAL | M_GENERAL | 0.0 <= Value <= 100.0 or M_ALL	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 100.0	Specifies the weighted false negative rate.	c		e	f	g	a		i		j	k	l	m	n	b

M_FPR +	Retrieves the False Positive Rate (FPR) of a single class versus other classes. To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId: Feature classification (M_STAT_TREE_ENSEMBLE_RESULT), image classification (M_STAT_CNN_RESULT), segmentation (M_STAT_SEG_RESULT), or object detection (M_STAT_DET_RESULT) statistics result ID — LabelOrIndex1: M_CLASS_INDEX() — LabelOrIndex2: M_GENERAL — ScoreThreshold: 0.0 <= Value <= 100.0, M_ALL, or M_DEFAULT StatResultClassId: Anomaly detection statistics result ID (M_STAT_ANO_RESULT) — LabelOrIndex1: M_ANOMALOUS or M_NON_ANOMALOUS — LabelOrIndex2: M_GENERAL — ScoreThreshold: 0.0 <= Value <= 100.0, M_ALL, or M_DEFAULT	c	e	f	g	a	i	j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c	e	f	g	a	i	j	k	l	m	n	b
M_INVALID	Specifies that you cannot retrieve the FPR result (occurs when all ground truth instances are of the given class; that is, there is no FP/TN).	c	e	f	g	a	i	j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the FPR.	c	e	f	g	a	i	j	k	l	m	n	b
M_FPR_MACRO +	Retrieves the macro false positive rate of the entire confusion matrix, giving equal importance to all classes. To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId: Any classification statistics result ID — LabelOrIndex1: M_GENERAL — LabelOrIndex2: M_GENERAL — ScoreThreshold: M_DEFAULT StatResultClassId: Object detection statistics result ID (M_STAT_DET_RESULT) — LabelOrIndex1: M_GENERAL — LabelOrIndex2: M_GENERAL — ScoreThreshold: 0.0 <= Value <= 100.0 or M_ALL	c	e	f	g	a	i	j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c	e	f	g	a	i	j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the macro false positive rate.	c	e	f	g	a	i	j	k	l	m	n	b
M_FPR_MICRO +	Retrieves the micro false positive rate of the entire confusion matrix, giving equal importance to all samples. To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId: Any classification statistics result ID — LabelOrIndex1: M_GENERAL — LabelOrIndex2: M_GENERAL — ScoreThreshold: M_DEFAULT StatResultClassId: Object detection statistics result ID (M_STAT_DET_RESULT) — LabelOrIndex1: M_GENERAL — LabelOrIndex2: M_GENERAL — ScoreThreshold: 0.0 <= Value <= 100.0 or M_ALL	c	e	f	g	a	i	j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c	e	f	g	a	i	j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the micro false positive rate.	c	e	f	g	a	i	j	k	l	m	n	b

M_FPR_WEIGHTED +	Retrieves the weighted false positive rate of the entire confusion matrix, computed by taking the false positive rate for each class and then averaging them, weighted by the number of true instances for each class. To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId: Any classification statistics result ID; LabelOrIndex1: M_GENERAL; LabelOrIndex2: M_GENERAL; ScoreThreshold: M_DEFAULT StatResultClassId: Object detection statistics result ID (M_STAT_DET_RESULT); LabelOrIndex1: M_GENERAL; LabelOrIndex2: M_GENERAL; ScoreThreshold: 0.0 <= Value <= 100.0 or M_ALL	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the weighted false positive rate.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_CLASSES +	Retrieves the number of class definitions processed from the dataset. This result is not supported for anomaly detection (M_STAT_ANO_RESULT). To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId: Any classification statistics result ID; LabelOrIndex1: M_GENERAL; LabelOrIndex2: M_GENERAL; ScoreThreshold: M_DEFAULT	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the number of classes.	c		e	f	g	a		i		j	k	l	m	n	b
M_PRECISION +	Retrieves the precision of a single class versus other classes. To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId: Feature classification (M_STAT_TREE_ENSEMBLE_RESULT), image classification (M_STAT_CNN_RESULT), segmentation (M_STAT_SEG_RESULT), or object detection (M_STAT_DET_RESULT) statistics result ID; LabelOrIndex1: M_CLASS_INDEX(); LabelOrIndex2: M_GENERAL; ScoreThreshold: 0.0 <= Value <= 100.0, M_ALL, or M_DEFAULT StatResultClassId: Anomaly detection statistics result ID (M_STAT_ANO_RESULT); LabelOrIndex1: M_ANOMALOUS or M_NON_ANOMALOUS; LabelOrIndex2: M_GENERAL; ScoreThreshold: 0.0 <= Value <= 100.0, M_ALL, or M_DEFAULT	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID	Specifies that you cannot retrieve the precision result (occurs when none of the predictions are of the given class; that is, there is no TP/FP).	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the precision.	c		e	f	g	a		i		j	k	l	m	n	b
M_PRECISION_MACRO +	Retrieves the macro precision of the entire confusion matrix, giving equal importance to all classes. To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId: Any classification statistics result ID; LabelOrIndex1: M_GENERAL; LabelOrIndex2: M_GENERAL; ScoreThreshold: M_DEFAULT StatResultClassId: Object detection statistics result ID (M_STAT_DET_RESULT); LabelOrIndex1: M_GENERAL; LabelOrIndex2: M_GENERAL; ScoreThreshold: 0.0 <= Value <= 100.0 or M_ALL	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

0.0 <= *Value* <= 100.0	Specifies the macro precision.	c		e	f	g	a		i		j	k	l	m	n	b
M_PRECISION_MICRO +	Retrieves the micro precision of the entire confusion matrix, giving equal importance to all samples. To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId | LabelOrIndex1 | LabelOrIndex2 | ScoreThreshold Any classification statistics result ID | M_GENERAL | M_GENERAL | M_DEFAULT Object detection statistics result ID (M_STAT_DET_RESULT) | M_GENERAL | M_GENERAL | 0.0 <= Value <= 100.0 or M_ALL	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the micro precision.	c		e	f	g	a		i		j	k	l	m	n	b
M_PRECISION_WEIGHTED +	Retrieves the weighted precision of the entire confusion matrix, computed by taking the precision for each class and then averaging them, weighted by the number of true instances for each class. To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId | LabelOrIndex1 | LabelOrIndex2 | ScoreThreshold Any classification statistics result ID | M_GENERAL | M_GENERAL | M_DEFAULT Object detection statistics result ID (M_STAT_DET_RESULT) | M_GENERAL | M_GENERAL | 0.0 <= Value <= 100.0 or M_ALL	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the weighted precision.	c		e	f	g	a		i		j	k	l	m	n	b
M_RECALL +	Retrieves the recall value of a single class versus other classes. To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId | LabelOrIndex1 | LabelOrIndex2 | ScoreThreshold Feature classification (M_STAT_TREE_ENSEMBLE_RESULT), image classification (M_STAT_CNN_RESULT), segmentation (M_STAT_SEG_RESULT), or object detection (M_STAT_DET_RESULT) statistics result ID | M_CLASS_INDEX() | M_GENERAL | 0.0 <= Value <= 100.0, M_ALL, or M_DEFAULT Anomaly detection statistics result ID (M_STAT_ANO_RESULT) | M_ANOMALOUS or M_NON_ANOMALOUS | M_GENERAL | 0.0 <= Value <= 100.0, M_ALL, or M_DEFAULT	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID	Specifies that you cannot retrieve the recall value (occurs when none of the ground truth instances are of the given class; that is, there is no TP/FN).	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the recall.	c		e	f	g	a		i		j	k	l	m	n	b

M_RECALL_MACRO +	Retrieves the macro recall of the entire confusion matrix, giving equal importance to all classes. To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId | LabelOrIndex1 | LabelOrIndex2 | ScoreThreshold Any classification statistics result ID | M_GENERAL | M_GENERAL | M_DEFAULT Object detection statistics result ID (M_STAT_DET_RESULT) | M_GENERAL | M_GENERAL | 0.0 <= Value <= 100.0 or M_ALL	c	e	f	g	a	i	j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c	e	f	g	a	i	j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the macro recall.	c	e	f	g	a	i	j	k	l	m	n	b
M_RECALL_MICRO +	Retrieves the micro recall of the entire confusion matrix, giving equal importance to all samples. To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId | LabelOrIndex1 | LabelOrIndex2 | ScoreThreshold Any classification statistics result ID | M_GENERAL | M_GENERAL | M_DEFAULT Object detection statistics result ID (M_STAT_DET_RESULT) | M_GENERAL | M_GENERAL | 0.0 <= Value <= 100.0 or M_ALL	c	e	f	g	a	i	j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c	e	f	g	a	i	j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the micro recall.	c	e	f	g	a	i	j	k	l	m	n	b
M_RECALL_WEIGHTED +	Retrieves the weighted recall of the entire confusion matrix, computed by taking the recall for each class and then averaging them, weighted by the number of true instances for each class. To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId | LabelOrIndex1 | LabelOrIndex2 | ScoreThreshold Any classification statistics result ID | M_GENERAL | M_GENERAL | M_DEFAULT Object detection statistics result ID (M_STAT_DET_RESULT) | M_GENERAL | M_GENERAL | 0.0 <= Value <= 100.0 or M_ALL	c	e	f	g	a	i	j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c	e	f	g	a	i	j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the weighted recall.	c	e	f	g	a	i	j	k	l	m	n	b
M_SCORE_HISTOGRAM +	Retrieves the number of occurrences of the given class associated with each score threshold. Number of elements is the same as for M_SCORE_THRESHOLDS. The ith bin is filled with score occurrences in the range [threshold_i, threshold_i+1).That is, greater or equal to threshold_i but lower than threshold_i+1. For object detection, M_GENERAL is supported and returns the number of occurrences of all classes combined. To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId | LabelOrIndex1 | LabelOrIndex2 | ScoreThreshold Feature classification (M_STAT_TREE_ENSEMBLE_RESULT), image classification (M_STAT_CNN_RESULT), or segmentation (M_STAT_SEG_RESULT) statistics result ID | M_CLASS_INDEX() | M_GENERAL | M_DEFAULT Object detection statistics result ID (M_STAT_DET_RESULT) | M_CLASS_INDEX() or M_GENERAL | M_GENERAL | M_DEFAULT Anomaly detection statistics result ID (M_STAT_ANO_RESULT) | M_ANOMALOUS or M_NON_ANOMALOUS | M_GENERAL | M_DEFAULT	c	e	f	g	a	i	j	k	l	m	n	b

ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the number of occurrences of the given class associated with each score threshold.	c		e	f	g	a		i		j	k	l	m	n	b
M_SCORE_HISTOGRAM_OTHER_CLASSES +	Retrieves the number of occurrences of all classes other than the given class associated with each score threshold. Number of elements is the same as for M_SCORE_THRESHOLDS. The ith bin is filled with score occurrences in the range [threshold_i, threshold_i+1).That is, greater or equal to threshold_i but lower than threshold_i+1. This does not apply to object detection. This result is not supported for object detection (M_STAT_DET_RESULT). To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId: Feature classification (M_STAT_TREE_ENSEMBLE_RESULT), image classification (M_STAT_CNN_RESULT), or segmentation (M_STAT_SEG_RESULT) statistics result ID; LabelOrIndex1: M_CLASS_INDEX(); LabelOrIndex2: M_GENERAL; ScoreThreshold: M_DEFAULT StatResultClassId: Anomaly detection statistics result ID (M_STAT_ANO_RESULT); LabelOrIndex1: M_ANOMALOUS or M_NON_ANOMALOUS; LabelOrIndex2: M_GENERAL; ScoreThreshold: M_DEFAULT	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the number of occurrences of all classes other than the given class associated with each score threshold.	c		e	f	g	a		i		j	k	l	m	n	b
M_SCORE_HISTOGRAM_USED_ENTRIES +	Retrieves the array of entries contributed to occurrences of the given class associated with the specified score threshold. Number of elements is the number of occurrences. It is filled with entry keys of the occurrences in the range [threshold_value, threshold_value+1). That is, greater or equal to threshold_value but lower than threshold_value+1. For object detection, M_GENERAL is supported and returns the number of occurrences of all classes combined. To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId: Feature classification (M_STAT_TREE_ENSEMBLE_RESULT), image classification (M_STAT_CNN_RESULT), or segmentation (M_STAT_SEG_RESULT) statistics result ID; LabelOrIndex1: M_CLASS_INDEX(); LabelOrIndex2: M_GENERAL; ScoreThreshold: 0.0 <= Value <= 100.0 StatResultClassId: Object detection statistics result ID (M_STAT_DET_RESULT); LabelOrIndex1: M_CLASS_INDEX() or M_GENERAL; LabelOrIndex2: M_GENERAL; ScoreThreshold: 0.0 <= Value <= 100.0 StatResultClassId: Anomaly detection statistics result ID (M_STAT_ANO_RESULT); LabelOrIndex1: M_ANOMALOUS or M_NON_ANOMALOUS; LabelOrIndex2: M_GENERAL; ScoreThreshold: 0.0 <= Value <= 100.0	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_UUID [optionally, in C++: a reference to a std::vector<AIL_UUID>]*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the UUID.	c		e	f	g	a		i		j	k	l	m	n	b
M_SCORE_HISTOGRAM_USED_ENTRIES_OTHER_CLASSES +	Retrieves the array of entry keys of the occurrences of all classes other than the given class associated with the specified score threshold. Number of elements is the number of corresponding entries. It is filled with entry keys of the occurrences in the range [threshold_value, threshold_value +1). That is, greater or equal to threshold_value but lower than threshold_value+1. This does not apply to object detection. This result is not supported for object detection (M_STAT_DET_RESULT). To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId: Feature classification (M_STAT_TREE_ENSEMBLE_RESULT), image classification (M_STAT_CNN_RESULT), or segmentation (M_STAT_SEG_RESULT) statistics result ID; LabelOrIndex1: M_CLASS_INDEX(); LabelOrIndex2: M_GENERAL; ScoreThreshold: M_DEFAULT StatResultClassId: Anomaly detection statistics result ID (M_STAT_ANO_RESULT); LabelOrIndex1: M_ANOMALOUS or M_NON_ANOMALOUS; LabelOrIndex2: M_GENERAL; ScoreThreshold: M_DEFAULT	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_UUID [optionally, in C++: a reference to a std::vector<AIL_UUID>]*	c		e	f	g	a		i		j	k	l	m	n	b

Value	Specifies the UUID.	c		e	f	g	a		i		j	k	l	m	n	b
M_SCORE_THRESHOLDS +	Retrieves the score thresholds used in the thresholding analysis. The same set of thresholds is applied to every class. Each score returned is a value between 0.0 and 100.0, inclusively. For tasks predicting probability distribution over all classes (feature classification, image classification, and segmentation), thresholding analysis is done using a "one class versus the rest" strategy. Thus, a threshold selected for a given class must be applied to scores associated to that class. Occurrences with scores greater or equal to the threshold are considered to be part of the class. One the other hand, the score predicted for object detection is a confidence score not associated with any specific class. In this case, only the detections with score greater or equal to the threshold are considered relevant. To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId: Any classification statistics result ID LabelOrIndex1: M_GENERAL LabelOrIndex2: M_GENERAL ScoreThreshold: M_DEFAULT	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the score thresholds.	c		e	f	g	a		i		j	k	l	m	n	b
M_STATUS +	Retrieves the status of the statistics calculation. To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId: Any classification statistics result ID LabelOrIndex1: M_GENERAL LabelOrIndex2: M_GENERAL ScoreThreshold: M_DEFAULT	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_CALCULATE_NOT_PERFORMED	Specifies that the calculate operation was not performed.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPLETE	Specifies that the statistics calculation completed successfully.	c		e	f	g	a		i		j	k	l	m	n	b
M_CURRENTLY_CALCULATING	Specifies that the calculate operation is currently ongoing. You can only get this status if you are retrieving it from another thread.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERNAL_ERROR	Specifies that an unexpected error occurred during the operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_STOPPED_BY_REQUEST	Specifies that the current execution of the operation was explicitly stopped by calling MclassControl() with M_STOP_CALCULATE.	c		e	f	g	a		i		j	k	l	m	n	b
M_TIMEOUT_REACHED	Specifies that the operation ended because the timeout limit was reached. The timeout is specified by calling MclassControl() with M_TIMEOUT.	c		e	f	g	a		i		j	k	l	m	n	b
M_ZERO_USED_ENTRIES	Specifies that no entries were used because they are all in error.	c		e	f	g	a		i		j	k	l	m	n	b
M_TNR +	Retrieves the true negative rate of a single class versus other classes. To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId: Feature classification (M_STAT_TREE_ENSEMBLE_RESULT), image classification (M_STAT_CNN_RESULT), segmentation (M_STAT_SEG_RESULT), or object detection (M_STAT_DET_RESULT) statistics result ID LabelOrIndex1: M_CLASS_INDEX() LabelOrIndex2: M_GENERAL ScoreThreshold: 0.0 <= Value <= 100.0, M_ALL, or M_DEFAULT StatResultClassId: Anomaly detection statistics result ID (M_STAT_ANO_RESULT) LabelOrIndex1: M_ANOMALOUS or M_NON_ANOMALOUS LabelOrIndex2: M_GENERAL ScoreThreshold: 0.0 <= Value <= 100.0, M_ALL, or M_DEFAULT	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_INVALID	Specifies that you cannot retrieve the true negative rate result (occurs when all ground truth instances are of the given class; that is, there is no FP/TN).	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the true negative rate.	c		e	f	g	a		i		j	k	l	m	n	b
M_TNR_MACRO +	Retrieves the macro true negative rate of the entire confusion matrix, giving equal importance to all classes. To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId | LabelOrIndex1 | LabelOrIndex2 | ScoreThreshold Any classification statistics result ID | M_GENERAL | M_GENERAL | M_DEFAULT Object detection statistics result ID (M_STAT_DET_RESULT) | M_GENERAL | M_GENERAL | 0.0 <= Value <= 100.0 or M_ALL	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the macro true negative rate.	c		e	f	g	a		i		j	k	l	m	n	b
M_TNR_MICRO +	Retrieves the micro true negative rate of the entire confusion matrix, giving equal importance to all samples. To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId | LabelOrIndex1 | LabelOrIndex2 | ScoreThreshold Any classification statistics result ID | M_GENERAL | M_GENERAL | M_DEFAULT Object detection statistics result ID (M_STAT_DET_RESULT) | M_GENERAL | M_GENERAL | 0.0 <= Value <= 100.0 or M_ALL	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies micro macro true negative rate.	c		e	f	g	a		i		j	k	l	m	n	b
M_TNR_WEIGHTED +	Retrieves the weighted true negative rate of the entire confusion matrix, computed by taking the true negative rate for each class and then averaging them, weighted by the number of true instances for each class. To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId | LabelOrIndex1 | LabelOrIndex2 | ScoreThreshold Any classification statistics result ID | M_GENERAL | M_GENERAL | M_DEFAULT Object detection statistics result ID (M_STAT_DET_RESULT) | M_GENERAL | M_GENERAL | 0.0 <= Value <= 100.0 or M_ALL	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the weighted true negative rate.	c		e	f	g	a		i		j	k	l	m	n	b

M_TRUE_NEGATIVES +	Retrieves the number of true negatives out of a confusion matrix for a 1 versus other classes, using the ScoreThreshold parameter as the criterion on the selected class to populate the matrix. To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId: Feature classification (M_STAT_TREE_ENSEMBLE_RESULT), image classification (M_STAT_CNN_RESULT), segmentation (M_STAT_SEG_RESULT), or object detection (M_STAT_DET_RESULT) statistics result ID; LabelOrIndex1: M_CLASS_INDEX(); LabelOrIndex2: M_GENERAL; ScoreThreshold: 0.0 <= Value <= 100.0, M_ALL, or M_DEFAULT StatResultClassId: Anomaly detection statistics result ID (M_STAT_ANO_RESULT); LabelOrIndex1: M_ANOMALOUS or M_NON_ANOMALOUS; LabelOrIndex2: M_GENERAL; ScoreThreshold: 0.0 <= Value <= 100.0, M_ALL, or M_DEFAULT	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the true negatives.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE_POSITIVES +	Retrieves the number of true positives out of a confusion matrix for a 1 versus other classes, using the ScoreThreshold parameter as the criterion on the selected class to populate the matrix. To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId: Feature classification (M_STAT_TREE_ENSEMBLE_RESULT), image classification (M_STAT_CNN_RESULT), segmentation (M_STAT_SEG_RESULT), or object detection (M_STAT_DET_RESULT) statistics result ID; LabelOrIndex1: M_CLASS_INDEX(); LabelOrIndex2: M_GENERAL; ScoreThreshold: 0.0 <= Value <= 100.0, M_ALL, or M_DEFAULT StatResultClassId: Anomaly detection statistics result ID (M_STAT_ANO_RESULT); LabelOrIndex1: M_ANOMALOUS or M_NON_ANOMALOUS; LabelOrIndex2: M_GENERAL; ScoreThreshold: 0.0 <= Value <= 100.0, M_ALL, or M_DEFAULT	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the true positives.	c		e	f	g	a		i		j	k	l	m	n	b

To retrieve results for anomaly detection (pixel-level) or segmentation statistics result buffers, set the ResultType parameter to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For anomaly detection (pixel-level) or segmentation statistics result buffers																	
ResultType ResultArrayPtr - Possible values returned	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_IOU +	Retrieves the Intersection-over-Union (IoU) metric of a single class vs other classes. To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId: Pixel-level anomaly detection statistics result ID (M_STAT_ANO_RESULT); LabelOrIndex1: M_ANOMALOUS or M_NON_ANOMALOUS; LabelOrIndex2: M_GENERAL; ScoreThreshold: 0.0 <= Value <= 100.0, M_ALL, or M_DEFAULT StatResultClassId: Segmentation (M_STAT_SEG_RESULT) statistics result ID; LabelOrIndex1: M_CLASS_INDEX(); LabelOrIndex2: M_GENERAL; ScoreThreshold: 0.0 <= Value <= 100.0, M_ALL, or M_DEFAULT															
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the IoU.	c		e	f	g	a		i		j	k	l	m	n	b

M_IOU_MACRO +	Retrieves the macro Intersection-over-Union (IoU) of the entire confusion matrix, giving equal importance to all classes. To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId: Pixel-level anomaly detection (M_STAT_ANO_RESULT) or segmentation (M_STAT_SEG_RESULT) statistics result ID LabelOrIndex1: M_GENERAL LabelOrIndex2: M_GENERAL ScoreThreshold: M_DEFAULT	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the macro IoU.	c		e	f	g	a		i		j	k	l	m	n	b
M_IOU_MICRO +	Retrieves the micro Intersection-over-Union (IoU) of the entire confusion matrix, giving equal importance to all samples. To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId: Pixel-level anomaly detection (M_STAT_ANO_RESULT) or segmentation (M_STAT_SEG_RESULT) statistics result ID LabelOrIndex1: M_GENERAL LabelOrIndex2: M_GENERAL ScoreThreshold: M_DEFAULT	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the micro IoU.	c		e	f	g	a		i		j	k	l	m	n	b
M_IOU_WEIGHTED +	Retrieves the weighted Intersection-over-Union (IoU) of the entire confusion matrix, computed by taking the IoU for each class and then averaging them, weighted by the number of true instances for each class. To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId: Pixel-level anomaly detection (M_STAT_ANO_RESULT) or segmentation (M_STAT_SEG_RESULT) statistics result ID LabelOrIndex1: M_GENERAL LabelOrIndex2: M_GENERAL ScoreThreshold: M_DEFAULT	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the weighted IoU.	c		e	f	g	a		i		j	k	l	m	n	b

To retrieve results for object detection (single-entry) statistics result buffers, set the ResultType parameter to one of the following values. To calculate single-entry statistic results for object detection, you must specify a specific dataset entry index or key (EntryIndex or EntryKey) when calling MclassStatCalculate().

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For object detection (single-entry) statistics result buffers																
ResultType ResultArrayPtr - Possible values returned	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_MATCHED_INSTANCE +	Retrieves the index of the detection instance matched to the ground truth region. This is only possible for bounding box regions (MclassEntryAddRegion() with M_DESCRIPTOR_TYPE_BOX). To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId: Single-entry object detection statistics result ID (M_STAT_DET_RESULT) LabelOrIndex1: M_REGION_INDEX() or M_ALL_REGIONS LabelOrIndex2: M_GENERAL ScoreThreshold: M_DEFAULT															
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID	Specifies that the region is not matched to any instance.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_IGNORED	Specifies that the region's M_REGION_USE control is set to M_IGNORE.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNSUPPORTED_DESCRIPTOR_TYPE	Specifies that the region is not a bounding box.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the index.	c		e	f	g	a		i		j	k	l	m	n	b

M_MATCHED_REGION +	Retrieves the index of the ground truth region matched to the detection instance. To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId: Single-entry object detection statistics result ID (M_STAT_DET_RESULT) LabelOrIndex1: M_INSTANCE_INDEX() or M_ALL_INSTANCES LabelOrIndex2: M_GENERAL ScoreThreshold: M_DEFAULT	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID	Specifies that the region is not matched.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 1	Specifies the index.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_INSTANCES +	Retrieves the number of detection instances in the entry. To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId: Single-entry object detection statistics result ID (M_STAT_DET_RESULT) LabelOrIndex1: M_GENERAL LabelOrIndex2: M_GENERAL ScoreThreshold: M_DEFAULT	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the number of detection instances.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_REGIONS +	Retrieves the number of regions in the entry. To retrieve this result, set the StatResultClassId, LabelOrIndex1, LabelOrIndex2 and ScoreThreshold parameters to the following. StatResultClassId: Single-entry object detection statistics result ID (M_STAT_DET_RESULT) LabelOrIndex1: M_GENERAL LabelOrIndex2: M_GENERAL ScoreThreshold: M_DEFAULT	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 1	Specifies number of regions.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for
Tables:

- For any statistics result buffer
- For anomaly detection (pixel-level) or segmentation statistics result buffers
- For object detection (single-entry) statistics result buffers

the values listed in all the parameter association tables.

You can add the following value to the above-mentioned values to determine whether a result is available.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For determining whether results are available																
ResultType ResultArrayPtr - Possible values returned	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_AVAILABLE	Retrieves whether the requested result type is available for retrieval.															
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]* ┃ *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that the requested result type is not available.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that the requested result type is available.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_ACCURACY; M_ACCURACY_MACRO; M_ACCURACY_OVERALL; M_ACCURACY_WEIGHTED; M_CONFUSION_MATRIX; M_CONFUSION_MATRIX_COLUMN; M_CONFUSION_MATRIX_PERCENTAGE; M_CONFUSION_MATRIX_ROW; M_CONFUSION_MATRIX_USED_ENTRIES; M_CONFUSION_MATRIX_USED_ENTRIES_COLUMN; M_CONFUSION_MATRIX_USED_ENTRIES_ELEMENT; M_CONFUSION_MATRIX_USED_ENTRIES_ROW; M_ERROR_ENTRIES; M_ERROR_RATE; M_ERROR_RATE_MACRO; M_ERROR_RATE_OVERALL; M_ERROR_RATE_WEIGHTED; M_F1SCORE; M_F1SCORE_MACRO; M_F1SCORE_MICRO; M_F1SCORE_WEIGHTED; M_FALSE_NEGATIVES; M_FALSE_POSITIVES; M_FDR; M_FDR_MACRO; M_FDR_MICRO; M_FDR_WEIGHTED; M_FNR; M_FNR_MACRO; M_FNR_MICRO; M_FNR_WEIGHTED; M_FPR; M_FPR_MACRO; M_FPR_MICRO; M_FPR_WEIGHTED; M_PRECISION; M_PRECISION_MACRO; M_PRECISION_MICRO; M_PRECISION_WEIGHTED; M_RECALL; M_RECALL_MACRO; M_RECALL_MICRO; M_RECALL_WEIGHTED; M_SCORE_HISTOGRAM; M_SCORE_HISTOGRAM_OTHER_CLASSES; M_SCORE_HISTOGRAM_USED_ENTRIES;

M_SCORE_HISTOGRAM_USED_ENTRIES_OTHER_CLASSES; M_SCORE_THRESHOLDS; M_TNR; M_TNR_MACRO; M_TNR_MICRO; M_TNR_WEIGHTED; M_TRUE_NEGATIVES; M_TRUE_POSITIVES; M_IOU; M_MATCHED_INSTANCE; M_MATCHED_REGION.

You can add the following value to the above-mentioned values to determine the required array size (number of elements) to store the returned values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For determining the required array size (number of elements) to store the returned values																	
ResultType combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned																
M_NB_ELEMENTS	Retrieves the required array size (number of elements) to store the returned values.															
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for
Tables:

- For any statistics result buffer
- For anomaly detection (pixel-level) or segmentation statistics result buffers
- For object detection (single-entry) statistics result buffers

the values listed in all the parameter association tables.

You can add one of the following values to the above-mentioned values to cast the requested results to the required data type.

> Note: In C++, when using a standard vector (std::vector) instead of an array, the values listed in this combination table must not be used. If you need to use a specific data type, instantiate the vector with the necessary data type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the data type																	
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned																
M_TYPE_AIL_DOUBLE	Casts the requested results to an *AIL_DOUBLE*.															
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* **|** *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT	Casts the requested results to an *AIL_INT*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]* **|** *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT32	Casts the requested results to an *AIL_INT32*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]* **|** *address of an AIL_INT32*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT64	Casts the requested results to an *AIL_INT64*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]* **|** *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailclass11.lib.
DLL	Requires ail11.dll; ailclass11.dll.

MclassHookFunction

Synopsis

Hook a function to an event that occurs using classification objects.

Syntax

```
void MclassHookFunction(
    AIL_ID ClassId,                          //in
    AIL_INT64 HookType,                      //in
    AIL_CLASS_HOOK_FUNCTION_PTR HookHandlerPtr, //in
    void *UserDataPtr                        //in-out
)
```

Description

This function allows you to attach or detach a user-defined function to a classification object event (for example, during the execution of MclassTrain() or MclassPredict()). Once a hook-handler function is defined and hooked to an event, it is automatically called when the event occurs.

You can hook more than one function to an event by making separate calls to **MclassHookFunction**() for each function that you want to hook. Aurora Imaging Library automatically chains and keeps an internal list of all these hooked functions. When a function is hooked, this new function is added to the end of the list. When the event happens, all user-defined functions in the list will be executed in the same order that they were hooked to the event. You can also remove any function from the list; in this case, Aurora Imaging Library preserves the order of the remaining functions in the list.

You can obtain information concerning the event, using MclassGetHookInfo(), and take appropriate action before returning control to the application.

Parameters

ClassId

Specifies the identifier of the classification object onto which to attach a hook-handler. Set this parameter to one of the following values.

For specifying the identifier of the classification object	
Value	Description
ContextClassifierId	Specifies the identifier of a predefined CNN, predefined object detection, predefined segmentation, predefined anomaly detection, or tree ensemble classifier context. These contexts must have been previously allocated on the required system using MclassAlloc() with M_CLASSIFIER_CNN_PREDEFINED, M_CLASSIFIER_DET_PREDEFINED, M_CLASSIFIER_SEG_PREDEFINED, M_CLASSIFIER_ANO_PREDEFINED or M_CLASSIFIER_TREE_ENSEMBLE.
ContextDataPreparationId	Specifies the identifier of a data preparation context. This context must have been previously allocated on the required system using MclassAlloc() with M_PREPARE_IMAGES_CNN, M_PREPARE_IMAGES_DET or M_PREPARE_IMAGES_SEG.
ContextDatasetId	Specifies the identifier of an images or features dataset context. These contexts must have been previously allocated on the required system using MclassAlloc() with M_DATASET_IMAGES or M_DATASET_FEATURES.
ContextTrainId	Specifies the identifier of a CNN, object detection, segmentation, anomaly detection, or tree ensemble training context. These contexts must have been previously allocated on the required system using MclassAlloc() with M_TRAIN_CNN, M_TRAIN_DET, M_TRAIN_SEG, M_TRAIN_ANO or M_TRAIN_TREE_ENSEMBLE.

HookType

Specifies the event type. Set this parameter to one of the following values.

For specifying the event type	
Value	Description
M_DATASETS_PREPARED +	Calls the hook-handler function when the training and development datasets are prepared and available in the training results. You can use M_DATASETS_PREPARED with a training context when training a CNN, object detection, segmentation, or anomaly detection classifier (ClassId).
M_EPOCH_TRAINED +	Calls the hook-handler function each time the training process is completed for an epoch. This is typically used to plot a graph of the training process. You can use M_EPOCH_TRAINED with a training context when training a CNN, object detection, or segmentation classifier (ClassId).
M_MINI_BATCH_TRAINED +	Calls the hook-handler function each time the training process is completed for a given mini-batch within the current epoch. You can use M_MINI_BATCH_TRAINED with a training context when training a CNN, object detection, segmentation, or anomaly detection classifier (ClassId).
M_PREDICT_ENTRY +	Calls the hook-handler function each time the prediction is completed for a dataset entry. You can use M_PREDICT_ENTRY with a classifier context when performing a prediction with a trained CNN, object detection, segmentation, anomaly detection, or tree ensemble classifier and an images or features dataset (ClassId).
M_PREPARE_ENTRY_POST +	Calls the hook-handler function after each entry is prepared. This lets you track the progress of the data preparation, and how close it is to completing. The data preparation operation is done when the number of prepared entries is equal to the number of entries in the source dataset. It is possible that some source images (entries) are not used; to know which, access the status of the previous preparation. You can use M_PREPARE_ENTRY_POST when preparing data with a data preparation context (ClassId).
M_PREPARE_ENTRY_PRE +	Calls the hook-handler function before each entry is prepared. Set this hook type when you want to use a custom algorithm to determine the center around which to crop your images. That is, you are given the identifier of the image buffer, which you can then use in a customized way to select the center of the image, and then set that center with MclassSetHookInfo(). You can use M_PREPARE_ENTRY_PRE when preparing data with a data preparation context (ClassId).

M_SAMPLE_ADDED +	Calls the hook-handler function each time the training process is completed for a given sample. You can use M_SAMPLE_ADDED with a training context when training an anomaly detection classifier (ClassId).
M_TREE_TRAINED +	Calls the hook-handler function each time the training process is completed for a tree. You can use M_TREE_TRAINED with a training context when training a tree ensemble classifier (ClassId).
M_WRITE_NEW_FILE +	Calls the hook-handler function each time a file is written to disk when certain Aurora Imaging Library functions are called (for example, MclassEntryAddRegion() and MclassPredict() with a segmentation classifier context). You can use M_WRITE_NEW_FILE with an images dataset context (ClassId).

Combination value for any of the possible values of the HookType parameter.

You can add the following value to the above-mentioned values to set whether to detach the hook-handler function.

For the HookType parameter	
Combination value	Description
M_UNHOOK	Unhooks the specified function if hooked to the specified event. When you use M_UNHOOK, you must provide the same values for all the parameters as when you originally hooked the function.

HookHandlerPtr

Specifies the address of the function that should be called when an event occurs.

The hook-handler function, pointed to by HookHandlerPtr, must be declared as follows:

```
AIL_INT MFTYPE HookHandler(
    AIL_INT HookType,
    AIL_ID EventId,
    void *UserDataPtr
)
```

Parameters

HookType

Specifies the type of event that generated the call.

EventId

Event identifier to pass to MclassGetHookInfo() when inquiring about the hooked event.

UserDataPtr

User data pointer that was passed to the UserDataPtr parameter of **MclassHookFunction**().

Upon successful completion, the hook-handler function should return **M_NULL**. Note, *MFTYPE* and *AIL_CLASS_HOOK_FUNCTION_PTR* are reserved Aurora Imaging Library predefined types for functions and data pointers, respectively.

UserDataPtr

Specifies the address of the user data that you want to make available to the hook-handler function. This address is passed to the hook-handler function, through its UserDataPtr parameter, when the specified event occurs. Set this parameter to **M_NULL** if user data is not required.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailclass11.lib.
DLL	Requires ail11.dll; ailclass11.dll.

MclassImport

Synopsis

Import data to a dataset or classifier context.

Syntax

```
void MclassImport(
    AIL_CONST_TEXT_PTR FileNameOrFolderPath, //in
    AIL_INT64 FileOrPathFormat,              //in
    AIL_ID DatasetOrClassifierContextId,     //in
    AIL_INT64 Index,                         //in
    AIL_INT64 ImportType,                    //in
    AIL_INT64 ControlFlag                    //in
)
```

Description

This function imports data from a CSV file or folder to a dataset context (images or features), or imports data from an ONNX file to an ONNX classifier context. Aurora Imaging Library appends imported data from a CSV file or folder to any existing data in the dataset. To export data from a dataset, use MclassExport().

Parameters

FileNameOrFolderPath

Specifies the name and path of a file, or specifies a folder, from which to import. When specifying the name and path of a file, the function handles (internally) the opening and closing of that file. Also, for easier use with other Aurora Imaging software products, the specified file name must use the CSV or ONNX file extension. A CSV file represents a common format that holds comma separated values. An ONNX file is a platform-neutral and language-neutral open source file format for describing machine learning models.

Set this parameter to one of the values below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the file name and path																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which to interactively specify the file's drive, directory, and name.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*FileNameOrFolderPath*") [1]	Specifies the required information for the file or folder from which to import. For a file, you must specify its drive, directory, and name (for example, "*E:\Manual\MySweetSweetDataset.csv*" or "*E:\Manual\MySweetSweetClassifierContext.onnx*"). For a folder, you must specify its drive and directory (for example, "*E:\Manual\MySweetDatasetFolder*"). To specify a dataset file on a remote computer (under Distributed Aurora Imaging Library), prefix the file name string with "*remote:///*" (for example, "*remote:///E:\Manual\\MyEvenSweeterDataset.csv*").	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

FileOrPathFormat

Specifies the format of the file or path from which to import. Set this parameter to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the file format																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FORMAT_CSV	Specifies a CSV file format. In this case, set the DatasetOrClassifierContextId parameter to the identifier of an images or features dataset. When importing from a CSV file, note the following: • Field values with commas must delimit them within double quotation marks. • If an entry refers to a non-existing class label, the entry is added to the dataset with default initial values. • When no BOM (Byte-Order-Marker) is encoded in the file, the encoding character UTF-8 is used. The CSV file must adhere to formatting requirements and contain the required data; this can depend on what you are importing, as specified by the ImportType parameter.	c		e	f	g	a		i		j	k	l	m	n	b
M_IMAGE_DATASET_FOLDER	Specifies a path to an existing folder. In this case, set the DatasetOrClassifierContextId parameter to the identifier of an images dataset. When importing from a folder, note the following: • You should follow the Aurora Imaging Library folder structure, as described in MclassExport(). • If the dataset you are importing was exported with M_IMAGE_DATASET_FOLDER_ALLOW_RENAME, Aurora Imaging Library cannot guarantee that the import will successfully restore the exported dataset. • Aurora Imaging Library might modify the root path (M_ROOT_PATH) in the dataset, if one is specified, to maintain the dataset's coherence. For example, dataset paths should all be either relative or absolute.	c		e	f	g	a		i		j	k	l	m	n	b

M_ONNX_FILE	Specifies an ONNX file format. In this case, set the DatasetOrClassifierContextId parameter to the identifier of an ONNX classifier context. When importing an ONNX file, note the following: • The supported ONNX file format version (ir_version) is from 3 to 8, inclusive. The supported ONNX operator set version is from 7 to 15, inclusive. • The ONNX model must have a single operator set domain. This domain must be either "ai.onnx" or an empty string (""). • The network must have one input. ○ The input type must be a dense tensor, that is, an array where the elements are all sequential starting at index 0. ▪ The tensor element type must be either float32, uint8, uint16, int8, int16 or int32. ▪ The tensor dimensionality (rank) must range from 1 to 4. Aurora Imaging Library follows the NCHW convention, that is "Number in batch" x "Channels" x "Height" x "Width" or, in Aurora Imaging Library terminology, "size batch" x "size band" x "size Y" x "size X". Aurora Imaging Library will automatically assign values of 1 to the missing higher (leftmost) dimensions for inputs of lower dimensionality. For instance, given an input tensor with 2 dimensions, Aurora Imaging Library will assign values of 1 to N and C and define HW based on the tensor. ▪ Each dimension must be expressed either with an integer value greater than 0 or using a dimension variable string. ▪ Aurora Imaging Library only supports 1 as the integer value of dimension N. ▪ Dimension variables are allowed at import time but must be defined by the user with MclassControl() with M_TARGET_IMAGE_SIZE_X, M_TARGET_IMAGE_SIZE_Y, M_TARGET_IMAGE_SIZE_BAND calls prior to calling MclassPreprocess(). ▪ Dimension variables for the size batch (N) are internally set to 1. • The network must have at least one output. ○ All output type must be dense tensor. ▪ The tensor element types must be either float16, float32, float64, uint8, uint16, int8, int16, int32 or int64. ▪ The tensors must have at least one dimension. ▪ Each dimension must be expressed either with an integral value greater than 0 or using a dimension variable string. ○ Outputs are filled at MclassPredict() and are retrieved using MclassGetResult(). • The graph of the model must be free of cycles. • The ONNX model must pass the ONNX function `onnx::checker::check_model()` without error.	c		e	f	g	a		i		j	k	l	m	n	b

DatasetOrClassifierContextId

Specifies the identifier of the dataset or classifier context in which to import. Set this parameter to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the dataset or classifier context in which to import																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ContextClassifierId	Specifies the identifier of an ONNX classifier context. This context is allocated using MclassAlloc() with M_CLASSIFIER_ONNX.	c		e	f	g	a		i		j	k	l	m	n	b
ContextDatasetId	Specifies the identifier of a features or images dataset context. This context is allocated using MclassAlloc() with M_DATASET_FEATURES or M_DATASET_IMAGES.	c		e	f	g	a		i		j	k	l	m	n	b

Index

Reserved for future expansion and must be set to **M_DEFAULT**.

ImportType

Specifies the type of import operation to perform.

Typically, you will import an entire dataset with M_COMPLETE.

When importing specific files, the order in which you import data is important. You should import authors and class definitions first, and then import entries, as entries require authors and class definitions.

If you only import entries, Aurora Imaging Library automatically creates class definitions and authors according to the information in the entries. If you import class definitions or authors afterward, Aurora Imaging Library appends them to the already existing ones. If Aurora Imaging Library encounters non-existing class definitions or authors when importing entries, they are automatically added to the dataset using default values.

Set this parameter to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of the file that you can import	
Value	Description
M_DEFAULT	Specifies to import an ONNX file.
M_AUTHORS	Specifies to import authors from a CSV file. In this case, use the headers below (and provide the corresponding lines of information after the header) in the CSV file. `Key,AuthorName` Both of these fields are required.

M_CLASS_DEFINITIONS	Specifies to import class definitions from a CSV file. In this case, use the headers below (and provide the corresponding lines of information after the header) in the CSV file. `Key,Name,Color_R,Color_G,Color_B,Weight,Icon,Anomalous` The 'Key' and 'Name' fields are required; the others are optional.

M_COMPLETE	Specifies to import the entire folder into the dataset. This includes authors (CSV), class definitions (CSV), controls (CSV), entries (images and CSV), descriptors (images and CSV), segmentation fi (images).
	To use it, the FileOrPathFormat parameter must be set to M_IMAGE_DATASET_FOLDER. In this case, use the headers below (and provide the corresponding lines of information after the headers
	• Headers for *controls.csv* files.
	`ControlType,ControlValue`
	Note:
	◦ Currently supported control types (and associated control values) are M_NO_CLASS_DRAW_COLOR, M_NO_REGION_PIXEL_CLASS, and M_DONT_CARE_CLASS_DRAW_COLOR. If you control types (as specified in your folder) are imported into the dataset; otherwise, the control values (as specified in your folder) must be the same as those in the dataset you are
	• Headers for *descriptor_boxes.csv* files.
	`Key,FilePath,RegionIndex,ClassIdxGroundTruth,AuthorName,BoxCoords`
	Note:
	◦ When importing a dataset for object detection, this file is used to import information about the bounding boxes.
	◦ FilePath and AuthorName are optional.
	◦ BoxCoords is in the format "xmin, ymin, xmax, ymax" (each box is a line in the CSV).
	• Headers for *descriptor_masks.csv* files.
	`Key,FilePath,RegionIndex,ClassIdxGroundTruth,AuthorName,MaskPath`
	Note:
	◦ FilePath and AuthorName are optional.
	◦ MaskPath is the path to the mask image file, relative to the folder being imported.
	• Headers for *descriptor_polygons.csv* files.
	`Key,FilePath,RegionIndex,ClassIdxGroundTruth,AuthorName,PolygonCoords`
	Note:
	◦ FilePath and AuthorName are optional.
	◦ PolygonCoords is in the format "x1, y1, x2, y2, ..., xN, yN" (each polygon is a line in the CSV).
	• Headers for *results_detection.csv* files.
	`Key,FilePath,ClassScorePredicted,ClassIdxPredicted,BoxCoords`
	Note:
	◦ FilePath is optional.
	◦ BoxCoords is in the format "xmin, ymin, xmax, ymax" (each box is a line in the CSV).
	• Headers for *results_anomaly_detection.csv* files.
	`Key,FilePath,AnomalyScoresPath,ImageAnomalyScore,Threshold,SizeX,SizeY,ClassifierPredefinedType`
	Note:
	◦ AnomalyScoresPath is the reletive path to the anomaly scores.
	◦ SizeX and SizeY are the dimensions of the anomaly scores map.
	• Headers for *results_segmentation.csv* files.
	`Key,FilePath,SegmentationPath,SizeX,SizeY,NumClasses,PredefinedNetworkType,ReceptiveFieldSizeX,ReceptiveFieldSizeY,ReceptiveFieldOffse`
	`ReceptiveFieldOffsetY,ReceptiveFieldAsymetricOffsetX,ReceptiveFieldAsymetricOffsetY,ReceptiveFieldStrideX,ReceptiveFieldStrideY`
	Note:
	◦ Headers must be on one line; illustrated headers are on multiple lines for display purposes.
	◦ FilePath is optional.
	◦ SegmentationPath is the relative path to the segmentation scores.
	◦ SizeX, SizeY and NumClasses are the dimensions of the segmentation scores map (saved under SegmentationPath).
	◦ PredefinedNetworkType is the result returned by calling MclassGetResultEntry() with M_SEGMENTATION and M_CLASSIFIER_PREDEFINED_TYPE.
	◦ ReceptiveFieldSizeX/Y is the result returned by calling MclassGetResultEntry() with M_SEGMENTATION and M_RECEPTIVE_FIELD_SIZE_X / M_RECEPTIVE_FIELD_SIZE_Y.
	◦ The combination of ReceptiveFieldOffsetX/Y and ReceptiveFieldAsymetricOffsetX/Y represents the result returned by calling MclassGetResultEntry() with M_SEGMENTATION and M_
	◦ ReceptiveFieldStrideX/Y is the result returned by calling MclassGetResultEntry() with M_SEGMENTATION and M_RECEPTIVE_FIELD_STRIDE_X / M_RECEPTIVE_FIELD_STRIDE_Y.
	> You cannot specify M_COMPLETE with Distributed Aurora Imaging Library.

M_ENTRIES	Specifies to import the entries from a CSV file. In this case, use the headers below (and provide the corresponding lines of information after the headers) in the CSV file.
	• Headers for an entries file of an images dataset (M_DATASET_IMAGES); for example, *DatasetEntries_images.csv*.
	`Key,FilePath,AuthorName,AugmentationSource,RegionType,ClassIdxGroundTruth_...,ClassIdxPredicted_...,ClassScorePredicted_...,UserString`
	`ReceptiveFieldSizeX,ReceptiveFieldSizeY,ReceptiveFieldOffsetX,ReceptiveFieldOffsetY,ReceptiveFieldAsymetricOffsetX,ReceptiveFieldAsyme`
	Note:
	○ Headers must be on one line; illustrated headers are on multiple lines for display purposes.
	○ PredefinedNetworkType is the result returned by calling MclassGetResultEntry() with M_SEGMENTATION and M_CLASSIFIER_PREDEFINED_TYPE.
	○ ReceptiveFieldSizeX/Y is the result returned by calling MclassGetResultEntry() with M_SEGMENTATION and M_RECEPTIVE_FIELD_SIZE_X / M_RECEPTIVE_FIELD_SIZE_Y.
	○ The combination of ReceptiveFieldOffsetX/Y and ReceptiveFieldAsymetricOffsetX/Y represents the result returned by calling MclassGetResultEntry() with M_SEGMENTATION and M_
	○ ReceptiveFieldStrideX/Y is the result returned by calling MclassGetResultEntry() with M_SEGMENTATION and M_RECEPTIVE_FIELD_STRIDE_X / M_RECEPTIVE_FIELD_STRIDE_Y.
	• Headers for an entries file of a features dataset (M_DATASET_FEATURES); for example, *DatasetEntries_features.csv*.
	`Key,FilePath,AuthorName,AugmentationSource,Data_...,ClassIdxGroundTruth_...,ClassIdxPredicted_...,ClassScorePredicted_...,UserString,E`
	For entries from either and images or features dataset, the 'Key', 'FilePath', and 'ClassIdxGroundTruth_...' fields are required; the others are optional.
	The fields with an underscore and an ellipsis, such as 'Data_...', represent an array of values. You must replace the ellipsis with an integer, starting at 0 for the first array value, and increasing by
	`Key,FilePath,AuthorName,AugmentationSource,Data_0,Data_1,Data_2,ClassIdxGroundTruth`
	`0,E:\Images\Class1\0001.mim,Zebra,NOT_AUGMENTED,99,66,77,1`

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailclass11.lib.
DLL	Requires ail11.dll; ailclass11.dll.

MclassInquire

Synopsis

Inquire about a classifier context, dataset context, class definition, data preparation context, training context, training result buffer, prediction result buffer, or the source layer of a predefined classifier or ONNX classifier.

Syntax

```
AIL_INT MclassInquire(
    AIL_ID ContextOrResultClassId, //in
    AIL_INT64 LabelOrIndex,        //in
    AIL_INT64 InquireType,         //in
    void *UserVarPtr               //out
)
```

Description

This function inquires about a setting of a classifier context, a dataset context, a data preparation context, a training context, class definitions, training result buffer, prediction result buffer, or the source layer of a predefined classifier (for example, a CNN). Note, class definitions are held in dataset and classifier contexts.

If the inquired setting is set to M_DEFAULT (for example, using MclassControl()), **MclassInquire**() will return M_DEFAULT. To inquire the actual default value, add M_DEFAULT to the InquireType parameter.

Parameters

ContextOrResultClassId

Specifies the identifier of the classification context or result buffer about which to inquire.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the context or result buffer

Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ContextClassifierId	Specifies the identifier of a predefined CNN, object detection, segmentation, anomaly detection, tree ensemble, or ONNX classifier context. These contexts must have been previously allocated on the required system using MclassAlloc() with M_CLASSIFIER_CNN_PREDEFINED, M_CLASSIFIER_DET_PREDEFINED, M_CLASSIFIER_SEG_PREDEFINED, M_CLASSIFIER_ANO_PREDEFINED, M_CLASSIFIER_TREE_ENSEMBLE, or M_CLASSIFIER_ONNX.	c		e	f	g	a		i		j	k	l	m	n	b
ContextDataPreparationId	Specifies the identifier of a data preparation context (for image data). This context must have been previously allocated on the required system using MclassAlloc() with M_PREPARE_IMAGES_CNN, M_PREPARE_IMAGES_DET or M_PREPARE_IMAGES_SEG.	c		e	f	g	a		i		j	k	l	m	n	b
ContextDatasetId	Specifies the identifier of an images or features dataset context. These contexts must have been previously allocated on the required system using MclassAlloc() with M_DATASET_IMAGES or M_DATASET_FEATURES.	c		e	f	g	a		i		j	k	l	m	n	b
ContextTrainId	Specifies the identifier of a CNN, object detection, segmentation, anomaly detection, or tree ensemble training context. These contexts must have been previously allocated on the required system using MclassAlloc() with M_TRAIN_CNN, M_TRAIN_DET, M_TRAIN_SEG, M_TRAIN_ANO, or M_TRAIN_TREE_ENSEMBLE.	c		e	f	g	a		i		j	k	l	m	n	b
ResultPredictId	Specifies the identifier of a CNN, object detection, segmentation, anomaly detection, or tree ensemble prediction result buffer. These buffers must have been previously allocated on the required system using MclassAllocResult() with M_PREDICT_CNN_RESULT, M_PREDICT_DET_RESULT, M_PREDICT_SEG_RESULT, M_PREDICT_ANO_RESULT, M_PREDICT_TREE_ENSEMBLE_RESULT, or M_PREDICT_ONNX_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b
ResultTrainId	Specifies the identifier of a CNN, object detection, segmentation, anomaly detection, or tree ensemble training result buffer. These buffers must have been previously allocated on the required system using MclassAllocResult() with M_TRAIN_CNN_RESULT, M_TRAIN_DET_RESULT, M_TRAIN_SEG_RESULT, M_TRAIN_ANO_RESULT, or M_TRAIN_TREE_ENSEMBLE_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b

LabelOrIndex

Specifies what to inquire. This parameter can be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying what to inquire

Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTHOR_INDEX(AIL_INT AuthorIndex)	Specifies the author in a dataset context about which to inquire.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	AuthorIndex Set this parameter to the following:															
	0 <= *Value* < M_NUMBER_OF_AUTHORS Specifies the index of the author. You can inquire the total number of authors, using M_NUMBER_OF_AUTHORS.	c		e	f	g	a		i		j	k	l	m	n	b

M_CLASS_INDEX(AIL_INT ClassIndex)	Specifies the class definition in a classifier or dataset context about which to inquire.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	ClassIndex Set this parameter to the following:															
	0 <= *Value* < M_NUMBER_OF_CLASSES Specifies the index. If you inquire about a class definition that you did not explicitly control, you will not get back any information, but there will be no error.	c		e	f	g	a		i		j	k	l	m	n	b
M_PREDICT_ENGINE_INDEX(AIL_INT PredictEngineIndex)	Specifies the predict engine about which to inquire.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	PredictEngineIndex Set this parameter to the following:															
	0 <= *Value* < M_NUMBER_OF_PREDICT_ENGINES Specifies the index of the predict engine. You can inquire the total number of predict engines, using M_NUMBER_OF_PREDICT_ENGINES.	c		e	f	g	a		i		j	k	l	m	n	b
M_CONTEXT	Specifies to inquire about a global setting of a classification context.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT_SOURCE_LAYER	Specifies to inquire about the input (initial) layer of the classifier in a classification context (for example, the source layer of the predefined CNN). In this case, you must set the ContextOrResultClassId parameter to the identifier of a predefined CNN, object detection, segmentation, anomaly detection, or ONNX classifier.	c		e	f	g	a		i		j	k	l	m	n	b
M_GENERAL	Specifies to inquire about a general setting of a classification result buffer.	c		e	f	g	a		i		j	k	l	m	n	b

InquireType

Specifies the type of setting about which to inquire.

See the Parameter associations section for possible values that can be specified.

UserVarPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]
- array of type AIL_ID [optionally, in C++: a reference to a std::vector<AIL_ID>]
- array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]
- array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]
- array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]
- array of type AIL_TEXT_CHAR [optionally, in C++: a reference to an AIL_STRING]
- array of type AIL_UUID [optionally, in C++: a reference to a std::vector<AIL_UUID>]
- AIL_DOUBLE
- AIL_ID
- AIL_INT
- AIL_INT32
- AIL_INT64
- AIL_UUID

Specifies the address in which to write the requested information. Since the **MclassInquire**() function also returns the requested information, you can set this parameter to **M_NULL**.

See the Parameter associations section for possible values that can be returned.

Parameter associations

The tables below list possible values for the **InquireType** parameter and possible values returned to the **UserVarPtr** parameter.

- For a classifier context (predefined CNN, object detection, segmentation, anomaly detection, or ONNX)
- For a classifier context (segmentation or ONNX)
- For a classifier context (object detection)
- For a classifier context (object detection or anomaly detection)
- For a classifier context (tree ensemble)
- For a classifier context (predefined CNN, predefined object detection, predefined segmentation, predefined anomaly detection, tree ensemble, or ONNX)
- For a classifier context's source layer (predefined CNN, predefined segmentation, predefined object detection, or ONNX)
- For a data preparation context (CNN, object detection or segmentation)
- For a classifier context's source layer (predefined CNN, object detection, predefined segmentation, predefined anomaly detection, or ONNX) or a data preparation context
- For a dataset context (images or features)
- For a classifier or dataset context (images or features)
- For a class definition in a classifier or dataset context (for CNN, object detection, segmentation, anomaly detection, tree ensemble, or ONNX)
- For a training context (segmentation or tree ensemble)
- For a training context (tree ensemble)
- For a data preparation context (images) or a tree ensemble training context
- For a training context (CNN, object detection, segmentation, or anomaly detection)
- For a training context (CNN, object detection, or segmentation)

- For a training context (segmentation)
- For a training context (anomaly detection)
- For a training context (CNN, segmentation, or feature classification)
- For a statistics classification, classifier, or training context (CNN, object detection, segmentation, anomaly detection, or tree ensemble)
- For a prediction result buffer (object detection)
- For any supported context
- For any supported context or result buffer
- For statistics classification

To inquire about a global setting of a predefined CNN, object detection, segmentation, anomaly detection, or ONNX classifier context, set the InquireType parameter to one of the values below. In this case, set the ContextOrResultClassId parameter to the identifier of a predefined CNN, object detection, segmentation, or anomaly detection classifier context, or the identifier of an ONNX classifier context, and set the LabelOrIndex parameter to M_CONTEXT, unless otherwise specified.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a classifier context (predefined CNN, object detection, segmentation, anomaly detection, or ONNX)

InquireType UserVarPtr - Possible values returned	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_CLASSIFIER_PREDEFINED_TYPE +	Inquires the type of predefined classifier context that you are using. These contexts are built by Aurora Imaging and specified when you call MclassAlloc() with M_CLASSIFIER_CNN_PREDEFINED, M_CLASSIFIER_DET_PREDEFINED, M_CLASSIFIER_SEG_PREDEFINED, M_CLASSIFIER_ANO_PREDEFINED, or M_CLASSIFIER_ONNX.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_ADNET	Specifies an ADNet classifier context. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_CSNET_COLOR_XL	Specifies an extra large CSNet classifier context that is for color images. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_CSNET_M	Specifies a medium CSNet classifier context. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_CSNET_MONO_XL	Specifies an extra large CSNet classifier context that is for monochrome images. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_CSNET_S	Specifies a small CSNet classifier context. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_CSNET_XL	Specifies an extra large CSNet classifier context. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_CUSTOM	Specifies a custom classifier context. M_CLASSIFIER_PREDEFINED_TYPE can only return M_CUSTOM if you contacted Aurora Imaging (Zebra) and received a classifier context that was customized for you.	c		e	f	g	a		i		j	k	l	m	n	b
M_FCNET_COLOR_XL	Specifies an extra large legacy FCNet classifier context that is for color images.	c		e	f	g	a		i		j	k	l	m	n	b
M_FCNET_M	Specifies a medium legacy FCNet classifier context.	c		e	f	g	a		i		j	k	l	m	n	b
M_FCNET_MONO_XL	Specifies an extra large legacy FCNet classifier context that is for monochrome images.	c		e	f	g	a		i		j	k	l	m	n	b
M_FCNET_S	Specifies a small legacy FCNet classifier context.	c		e	f	g	a		i		j	k	l	m	n	b
M_FCNET_XL	Specifies an extra large legacy FCNet classifier context.	c		e	f	g	a		i		j	k	l	m	n	b
M_ICNET_COLOR_XL	Specifies an extra large ICNet classifier context that is for color images. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ICNET_M	Specifies a medium ICNet classifier context. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ICNET_MONO_XL	Specifies an extra large ICNet classifier context that is for monochrome images. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ICNET_S	Specifies a small ICNet classifier context. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ICNET_XL	Specifies an extra large ICNet classifier context. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ODNET	Specifies an ODNet classifier context. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_UNDEFINED	Specifies an undefined classifier context. M_UNDEFINED is returned if you allocate a predefined classifier context (M_CLASSIFIER_CNN_PREDEFINED, M_CLASSIFIER_DET_PREDEFINED, M_CLASSIFIER_SEG_PREDEFINED, or M_CLASSIFIER_ANO_PREDEFINED) with the ControlFlag parameter set to M_DEFAULT. If your classifier context is undefined, and you copy the result of a trained classifier context into it, the type of that undefined context is modified according to the type of the context copied into it. For example, if you copy an M_ICNET_M context into an undefined classifier context, that context's classifier type is now also M_ICNET_M.	c		e	f	g	a		i		j	k	l	m	n	b

	M_USER_ONNX	Specifies a user-defined ONNX classifier context. This requires importing an ONNX file (MclassImport()).	c		e	f	g	a		i		j	k	l	m	n	b
M_EXPIRY_DATE_STRING +		Inquires the expiration date of the context in string format. The date returned is formatted as follow: "2999-12-31 23:59:59".	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*	c		e	f	g	a		i		j	k	l	m	n	b
	AIL_TEXT("*2999-12-31 23:59:59*") 1	Specifies the expiration date, if there is one. Returns the date as a regular string, consisting of all characters including the terminating null character. The date in the string is organized as follows: *YYYY-MM-DD HH:MM:SS* , where *YYYY* refers to the year, *MM* refers to the month, *DD* refers to the day, *HH* refers to the hour, *MM* refers to the minute, and *SS* refers to the second. An example of an expiration date that Aurora Imaging Library might one day return is: "2999-12-31 23:59:59\0".	c		e	f	g	a		i		j	k	l	m	n	b
	AIL_TEXT("*None*") 1	Specifies that there is no expiration date.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_PREDICT_ENGINES +		Inquires the number of predict engines detected. You can inquire the specifics of each predict engine using M_PREDICT_ENGINE_PROVIDER, M_PREDICT_ENGINE_DESCRIPTION and M_PREDICT_ENGINE_PRECISION.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0	Specifies the number of predict engines.	c		e	f	g	a		i		j	k	l	m	n	b
M_PREDICT_ENGINE_DESCRIPTION +		Inquires the description of the predict engine at the index specified by setting LabelOrIndex to M_PREDICT_ENGINE_INDEX().	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*	c		e	f	g	a		i		j	k	l	m	n	b
	AIL_TEXT("") 1	The default description in the classifier context is an empty string. Note, this is only available by setting LabelOrIndex to M_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b
	AIL_TEXT("*Description*") 1	Specifies the text description of the engine type seen by the execution provider.	c		e	f	g	a		i		j	k	l	m	n	b
M_PREDICT_ENGINE_PRECISION +		Inquires the precision of the predict engine at the index specified by setting LabelOrIndex to M_PREDICT_ENGINE_INDEX().	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT_PREDICT_ENGINE_PRECISION	Specifies the default precision in classifier context. Note, this is only available by setting LabelOrIndex to M_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b
	M_FP16	Specifies that the precision used by the predict engine is floating-point 16-bit. > Note, selecting a floating-point 16-bit predict engine for a model trained on floating-point 32-bit might cause a loss of precision or in extreme circumstances, floating-point overflow.	c		e	f	g	a		i		j	k	l	m	n	b
	M_FP32	Specifies that the precision used by the predict engine is floating-point 32-bit.	c		e	f	g	a		i		j	k	l	m	n	b
M_PREDICT_ENGINE_PROVIDER +		Inquires the provider of the predict engine at the index specified by setting LabelOrIndex to M_PREDICT_ENGINE_INDEX(). You must have installed the appropriate Aurora Imaging Library add-ons and updates to perform GPU accelerated prediction using CUDA or OpenVINO.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_CUDA	Specifies that the predict engine provider is CUDA.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT_CPU	Specifies the default predict engine provider.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT_PREDICT_ENGINE_PROVIDER	Specifies the default predict engine provider in the classifier context. Note, this is only available by setting LabelOrIndex to M_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b
	M_OPENVINO	Specifies that the predict engine provider is OpenVINO.	c		e	f	g	a		i		j	k	l	m	n	b
M_PREDICT_ENGINE_USED +		Inquires which predict engine is used by the specified context. This value should match the value set by using M_PREDICT_ENGINE unless the order of the predict engines is modified. This can happen if the environment running Aurora Imaging Library is modified.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies that the predict engine being used is the predict engine that was selected in the **Aurora Imaging Configurator** utility.	c		e	f	g	a		i		j	k	l	m	n	b

M_INVALID	Specifies that the predict engine is not found.	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* < M_NUMBER_OF_PREDICT_ENGINES	Specifies the index of the predict engine used.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAINABLE_COMPLETE +	Inquires whether you can perform a complete type of training on the context. Note that this inquire type always returns M_FALSE when M_CLASSIFIER_PREDEFINED_TYPE is set to M_USER_ONNX. M_CLASSIFIER_ONNX is not trainable and will always return M_FALSE.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that you cannot perform a complete training. That is, you cannot set M_RESET_TRAINING_VALUES to M_COMPLETE.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that you can perform a complete training. That is, you can set M_RESET_TRAINING_VALUES to M_COMPLETE.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAINABLE_FINE_TUNING +	Inquires whether you can perform a fine tuning type of training on the context. Note that this inquire type returns M_FALSE when M_CLASSIFIER_PREDEFINED_TYPE is set to M_USER_ONNX. M_CLASSIFIER_ONNX is not trainable and will always return M_FALSE.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that you cannot perform a fine tuning type of training. That is, you cannot set M_RESET_TRAINING_VALUES to M_FINE_TUNING.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that you can perform a fine tuning type of training. That is, you can set M_RESET_TRAINING_VALUES to M_FINE_TUNING.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAINABLE_TRANSFER_LEARNING +	Inquires whether you can perform a transfer learning type of training on the context. Note that this inquire type returns M_FALSE when M_CLASSIFIER_PREDEFINED_TYPE is set to M_USER_ONNX. M_CLASSIFIER_ONNX is not trainable and will always return M_FALSE.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that you cannot perform a transfer learning type of training. That is, you cannot set M_RESET_TRAINING_VALUES to M_TRANSFER_LEARNING.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that you can perform a transfer learning type of training. That is, you can set M_RESET_TRAINING_VALUES to M_TRANSFER_LEARNING.	c		e	f	g	a		i		j	k	l	m	n	b

[1] If the value was passed in a variable, it won't be returned enclosed in AIL_TEXT().

To inquire about a global setting of predefined segmentation or ONNX classifier context, set the InquireType parameter to one of the values below. In this case, set the ContextOrResultClassId parameter to the identifier of a predefined segmentation or ONNX classifier context and set the LabelOrIndex parameter to M_CONTEXT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a classifier context (segmentation or ONNX)																
InquireType UserVarPtr - Possible values returned	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_TARGET_IMAGE_SIZE_BAND +	Inquires the number of bands of the image input to MclassPredict(). If the size is equal to 0 (default value), the preprocessing assumes the size of the target image is the same as the default source layer, M_SIZE_BAND. M_TARGET_IMAGE_SIZE_BAND must be set to a value greater than 0 when the source layer equals M_DEFINED_BY_TARGET_IMAGE_SIZE_BAND. Note, this is only available for ONNX classifiers. SET															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0. INFO	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the number of bands.	c		e	f	g	a		i		j	k	l	m	n	b
M_TARGET_IMAGE_SIZE_X +	Inquires the X-size of the image input to MclassPredict(). If the size is equal to 0 (default value), the preprocessing assumes the size of the target image is the same as the default source layer, M_SIZE_X. For ONNX classifiers, M_TARGET_IMAGE_SIZE_X must be set to a value greater than 0 if the X-size of the network is defined by the target image X-size; that is, if inquiring M_SIZE_X returns M_DEFINED_BY_TARGET_IMAGE_SIZE_X. Otherwise, M_TARGET_IMAGE_SIZE_X must be set to 0 (M_DEFAULT). SET	c		e	f	g	a		i		j	k	l	m	n	b

UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0. INFO	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the X-size.	c		e	f	g	a		i		j	k	l	m	n	b
M_TARGET_IMAGE_SIZE_Y +	Inquires the Y-size of the image input to MclassPredict(). If the size is equal to 0 (default value), the preprocessing assumes the size of the target image is the same as the default source layer, M_SIZE_Y. For ONNX classifiers, M_TARGET_IMAGE_SIZE_Y must be set to a value greater than 0 if the Y-size of the network is defined by the target image Y-size; that is, if inquiring M_SIZE_Y returns M_DEFINED_BY_TARGET_IMAGE_SIZE_Y. Otherwise, M_TARGET_IMAGE_SIZE_Y must be set to 0 (M_DEFAULT). SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0. INFO	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the Y-size.	c		e	f	g	a		i		j	k	l	m	n	b

To inquire about a global setting of predefined object detection classifier context, set the InquireType parameter to one of the values below. In this case, set the ContextOrResultClassId parameter to the identifier of a predefined object detection classifier context and set the LabelOrIndex parameter to M_CONTEXT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a classifier context (object detection)																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_NMS_IOU_THRESHOLD +	Inquires the IOU threshold value used by the NMS algorithm to determine if a given box proposal is already covered by an existing box proposal. SET															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 25.0.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the IOU threshold.	c		e	f	g	a		i		j	k	l	m	n	b
M_NMS_POST_K +	Inquires the maximum number of detections to keep after applying the NMS algorithm to the detection results specified by M_NMS_TOP_K. The rest of the detections will be discarded. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 100.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the number of detections to keep.	c		e	f	g	a		i		j	k	l	m	n	b
M_NMS_TOP_K +	Inquires the top K number of detection results (in terms of score) that will have the non-maximum suppression (NMS) algorithm applied. A lower value reduces the computation complexity of the algorithm. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1000.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the number of top detection results.	c		e	f	g	a		i		j	k	l	m	n	b

To inquire about a global setting of a predefined object detection or anomaly detection classifier context, set the InquireType parameter to one of the values below. In this case, set the ContextOrResultClassId parameter to the identifier of a predefined object detection or anomaly detection classifier context and set the LabelOrIndex parameter to M_CONTEXT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a classifier context (object detection or anomaly detection)																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_SCORE_THRESHOLD	Inquires the minimum score threshold for a successful detection result. For object detection, scores above this threshold are kept. For anomaly detection, only images or pixels above this score are considered anomalous. SET															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 50.0.	c		e	f	g	a		i		j	k	l	m	n	b

	0.0 <= *Value* <= 100.0	Specifies the score threshold.	c		e	f	g	a		i		j	k	l	m	n	b

To inquire about a global setting of a tree ensemble classifier context, set the InquireType parameter to one of the values below. In this case, set the ContextOrResultClassId parameter to the identifier of a tree ensemble classifier context and set the LabelOrIndex parameter to M_CONTEXT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a classifier context (tree ensemble)																	
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																	
M_NUMBER_OF_FEATURES +	Inquires the number of features in entries that are used to train the classifier. Note that all entries used for a predict must have exactly the number of features returned by this inquire. You can only retrieve this value after calling MclassPreprocess().																
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the number of features used to train the classifier.	c		e	f	g	a		i		j	k	l	m	n	b

To inquire about a global setting of a predefined CNN, predefined object detection, predefined segmentation, predefined anomaly detection, tree ensemble, or ONNX classifier context, set the InquireType parameter to one of the values below. In this case, set the ContextOrResultClassId parameter to the identifier of a predefined CNN, predefined object detection, predefined segmentation, predefined anomaly detection, tree ensemble, or ONNX classifier context unless otherwise specified, and set the LabelOrIndex parameter to M_CONTEXT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a classifier context (predefined CNN, predefined object detection, predefined segmentation, predefined anomaly detection, tree ensemble, or ONNX)																	
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																	
M_CLASSIFIER_STATUS +	Inquires the state of the classifier.																
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default state of the classifier. For a predefined CNN, predefined object detection, predefined segmentation, predefined anomaly detection, predefined or ONNX classifier context, M_DEFAULT is the same as M_UNDEFINED. For a tree ensemble classifier context, M_DEFAULT is the same as M_UNTRAINED.	c		e	f	g	a		i		j	k	l	m	n	b
M_IMPORTED_ONNX	Specifies a classifier that is imported by the user. This requires importing an ONNX file using MclassImport() with M_ONNX_FILE. A classifier in this state can be used for MclassPredict() but not for MclassTrain().	c		e	f	g	a		i		j	k	l	m	n	b
M_PRETRAINED	Specifies a classifier that was trained by Aurora Imaging. A classifier in this state can be used with MclassPredict() and MclassTrain(). Note, calling MclassTrain() with a pretrained classifier is typically for fine-tuning or to perform transfer learning.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNDEFINED	Specifies an undefined classifier. A predefined CNN, predefined object detection, predefined segmentation, anomaly detection, or ONNX context is considered undefined if it was allocated with the ControlFlag parameter set to M_DEFAULT. You cannot train (or predict) with such a classifier. Note that this state only occurs for predefined CNN, predefined object detection, predefined segmentation, predefined anomaly detection, or ONNX classifier contexts.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNTRAINED	Specifies a classifier that has not been trained yet. A classifier in this state can be used for MclassTrain().	c		e	f	g	a		i		j	k	l	m	n	b
M_USER_TRAINED	Specifies a classifier that is trained by the user. A classifier in this state can be used for MclassPredict() and MclassTrain().	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAINED_PARAMS_DATE +	Inquires the date at which the trained parameters changed in the context either by training or importing.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*2999-12-31 23:59:59*") 1	Specifies the date at which the trained parameters changed, if there is one. Returns the date as a regular string, consisting of all characters including the terminating null character. The date in the string is organized as follows: *YYYY-MM-DD HH:MM:SS* , where *YYYY* refers to the year, *MM* refers to the month, *DD* refers to the day, *HH* refers to the hour, *MM* refers to the minute, and *SS* refers to the second. An example of a date that Aurora Imaging Library might one day return is: "2999-12-31 23:59:59\0".	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*None*") 1	Specifies that there is no date.	c		e	f	g	a		i		j	k	l	m	n	b

M_UUID +	Inquires the unique identifier of a trained or imported context.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DEFAULT_UUID.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT_UUID	Specifies the default unique identifier.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the unique identifier of the classifier context.	c		e	f	g	a		i		j	k	l	m	n	b

[1] If the value was passed in a variable, it won't be returned enclosed in AIL_TEXT().

To inquire about the source layer of the classifier in a predefined CNN, predefined object detection, predefined segmentation, predefined anomaly detection, or ONNX classifier context, the InquireType parameter can be set to one of the following values. In this case, set the LabelOrIndex parameter to M_DEFAULT_SOURCE_LAYER.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a classifier context's source layer (predefined CNN, predefined segmentation, predefined object detection, or ONNX)																	
InquireType UserVarPtr - Possible values returned	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_BAND_ORDER +	Inquires the order of the input bands of the network. This corresponds to the order in which the network interpreted the bands of the images that were used in the training phase. If the bands in the target image (MclassPredict()) do not follow this order, Aurora Imaging Library internally compensates by temporarily copying the bands to be ordered as the network expects. Note, this is not supported when M_CLASSIFIER_PREDEFINED_TYPE is set to M_USER_ONNX but is supported for M_CLASSIFIER_ONNX contexts.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_RGB.	c		e	f	g	a		i		j	k	l	m	n	b
M_BGR	Specifies that the bands are ordered according to the BGR (Blue, Green, Red) convention.	c		e	f	g	a		i		j	k	l	m	n	b
M_RGB	Specifies that the bands are ordered according to the RGB (Red, Green, Blue) convention.	c		e	f	g	a		i		j	k	l	m	n	b
M_SIZE_BAND +	Inquires the number of input bands of the network. This corresponds to the number of bands of the images that were used in the training phase.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFINED_BY_TARGET_IMAGE_SIZE_BAND	Specifies that the input tensor of the imported ONNX model has a variable width dimension. The width dimension must be set using M_TARGET_IMAGE_SIZE_BAND prior to preprocessing. M_DEFINED_BY_TARGET_IMAGE_SIZE_BAND can only be returned if you are inquiring about an ONNX classifier (M_CLASSIFIER_ONNX).	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 1	Specifies the number of bands.	c		e	f	g	a		i		j	k	l	m	n	b
M_STEP_X +	Inquires the increment, in the X-direction, of the target image size for prediction. Note the following calculation, to establish the target image's X-size: M_TARGET_IMAGE_SIZE_X = M_SIZE_X + *m* * M_STEP_X , where *m* is an integer that is greater than or equal to 0. M_STEP_X is only available for legacy FCNet classifiers (outdated technology for performing image classification or segmentation).	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 1	Specifies the appropriate increment, in the X-direction.	c		e	f	g	a		i		j	k	l	m	n	b
M_STEP_Y +	Inquires the increment, in the Y-direction, of the target image size for prediction. Note the following calculation, to establish the target image's Y-size: M_TARGET_IMAGE_SIZE_Y = M_SIZE_Y + *m* * M_STEP_Y , where *m* is an integer that is greater than or equal to 0. M_STEP_Y is only available for legacy FCNet classifiers (outdated technology for performing image classification or segmentation).	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 1	Specifies the appropriate increment, in the X-direction.	c		e	f	g	a		i		j	k	l	m	n	b

M_TRAIN_IMAGE_MIN_SIZE_X +	Inquires the minimum image size, along the X-axis, to train this context. Aurora Imaging Library recommends using ICNet, CSNet, ODNet, or ADNet classifiers; for them, this is a suggested minimum. If you are using a legacy FCNet classifier, this is a required minimum. Note, this is not supported when M_CLASSIFIER_PREDEFINED_TYPE returns M_USER_ONNX and is not supported by M_CLASSIFIER_ONNX contexts. For legacy FCNets, valid image sizes are given by the following formulas (for X and Y), where *k* and *j* are integers greater than or equal to zero: M_TRAIN_IMAGE_MIN_SIZE_X + *k* * M_TRAIN_IMAGE_STEP_X and M_TRAIN_IMAGE_MIN_SIZE_Y + *j* * M_TRAIN_IMAGE_STEP_Y	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Inquires the minimum image size, in the X-axis, required to train this context.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAIN_IMAGE_MIN_SIZE_Y +	Inquires the minimum image size, along the Y-axis, to train this context. Aurora Imaging Library recommends using ICNet, ODNet, CSNet, or ADNet classifiers; for them, this is a suggested minimum. If you are using a legacy FCNet classifier, this is a required minimum. Note, this is not supported when M_CLASSIFIER_PREDEFINED_TYPE returns M_USER_ONNX and is not supported by M_CLASSIFIER_ONNX contexts. For legacy FCNets, valid image sizes are given by the following formulas (for X and Y), where *k* and *j* are integers greater than or equal to zero: M_TRAIN_IMAGE_MIN_SIZE_X + *k* * M_TRAIN_IMAGE_STEP_X and M_TRAIN_IMAGE_MIN_SIZE_Y + *j* * M_TRAIN_IMAGE_STEP_Y	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Inquires the minimum image size, in the Y-axis, required to train this context.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAIN_IMAGE_STEP_X +	Inquires the increment in image size, along the X-axis, to train an FCNet context. M_TRAIN_IMAGE_STEP_X is only available for legacy FCNet classifiers (outdated technology for performing image classification or segmentation). For legacy FCNets, valid image sizes are given by the following formulas (for X and Y), where *k* and *j* are integers greater than or equal to zero: M_TRAIN_IMAGE_MIN_SIZE_X + *k* * M_TRAIN_IMAGE_STEP_X and M_TRAIN_IMAGE_MIN_SIZE_Y + *j* * M_TRAIN_IMAGE_STEP_Y	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Inquires the minimum image size, in the X-axis, required to train this context.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAIN_IMAGE_STEP_Y +	Inquires the increment in image size, along the Y-axis, to train this context. M_TRAIN_IMAGE_STEP_X is only available for legacy FCNet classifiers (outdated technology for performing image classification or segmentation). For legacy FCNets, valid image sizes are given by the following formulas (for X and Y), where *k* and *j* are integers greater than or equal to zero: M_TRAIN_IMAGE_MIN_SIZE_X + *k* * M_TRAIN_IMAGE_STEP_X and M_TRAIN_IMAGE_MIN_SIZE_Y + *j* * M_TRAIN_IMAGE_STEP_Y	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Inquires the minimum image size, in the Y-axis, required to train this context.	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE +	Inquires the input type of the network. This corresponds to the data type and depth of the images that were used in the training phase (for example, M_UNSIGNED + 8). Note, this is not supported when M_CLASSIFIER_PREDEFINED_TYPE is set to M_USER_ONNX but is supported for M_CLASSIFIER_ONNX contexts.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_FLOAT + *Depth value*	Specifies the floating-point data type, along with the depth, in bits.	c		e	f	g	a		i		j	k	l	m	n	b
M_SIGNED + *Depth value*	Specifies the signed data type, along with the depth, in bits.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNSIGNED + *Depth value*	Specifies the unsigned data type, along with the depth, in bits.	c		e	f	g	a		i		j	k	l	m	n	b

To inquire about a data preparation context for image data, set the InquireType parameter to one of the values below. In this case, set the ContextOrResultClassId parameter to the identifier of a data preparation context, and set the LabelOrIndex parameter to M_CONTEXT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a data preparation context (CNN, object detection or segmentation)																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																

M_AUGMENT_BALANCING +	Inquires the proportion of augmented entries that will go towards class balancing. SET														
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 80.0.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the proportion of augmented entries that will go towards class balancing. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_AUGMENT_CONTEXT_ID +	Inquires the identifier of the internal image processing augmentation context. This identifier is then what gives the user access to set the required controls for the image processing augment context.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Augmentation context identifier	Specifies the identifier of the augmentation context.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUGMENT_NUMBER_ABSOLUTE +	Inquires the number of augmented entries to add to the destination dataset. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the number of augmented entries that should be added to the destination dataset. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_AUGMENT_NUMBER_FACTOR +	Inquires the ratio with which to establish the number of augmentations to add to the destination dataset. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 3.0. INFO	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the ratio. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_AUGMENT_NUMBER_MODE +	Inquires the mode with which to establish the number of augmentations to add to the destination dataset. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_AUTO.	c		e	f	g	a		i		j	k	l	m	n	b
M_ABSOLUTE	Specifies that the number of augmentations is established with M_AUGMENT_NUMBER_ABSOLUTE.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO	Specifies that Aurora Imaging Library automatically establishes the number of augmentations. When augmenting a dataset, M_AUTO is the same as M_FACTOR. When augmenting an image buffer, M_AUTO is the same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies that no augmentations are performed. This does not affect other data preparations (these are still performed).	c		e	f	g	a		i		j	k	l	m	n	b
M_FACTOR	Specifies that the number of augmentations is established with M_AUGMENT_NUMBER_FACTOR.	c		e	f	g	a		i		j	k	l	m	n	b
M_CENTER_MODE +	Inquires the mode with which to establish the center of the destination image. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_AUTO.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO	Specifies that the center of the destination image is taken to be the center of the source image. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_USER_DEFINED	Specifies the center for cropping is taken to be the user-defined M_CENTER_X and M_CENTER_Y. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_CENTER_X +	Inquires the X-coordinate of the center point used by M_CENTER_MODE. M_CENTER_X only has an effect if M_CENTER_MODE is set to M_USER_DEFINED. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_INVALID.	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID	Specifies an invalid pixel x center. Set the center to a valid value, using MclassControl() with M_CENTER_X.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the X-coordinate.	c		e	f	g	a		i		j	k	l	m	n	b

M_CENTER_Y +	Inquires the Y-coordinate of the center point used by M_CENTER_MODE. M_CENTER_Y only has an effect if M_CENTER_MODE is set to M_USER_DEFINED. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT		c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID	Specifies an invalid pixel y center. Set the center to a valid value, using MclassControl() with M_CENTER_Y.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the Y-coordinate.	c		e	f	g	a		i		j	k	l	m	n	b
M_DESTINATION_FOLDER_MODE +	Inquires the mode Aurora Imaging Library uses to manage the folder in which it saves the prepared images. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_CREATE.	c		e	f	g	a		i		j	k	l	m	n	b
M_CREATE	Specifies that the data preparation process creates a unique *prepare* destination folder. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_OVERWRITE	Specifies that the data preparation process uses the same *prepare* destination folder. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_PREPARE_ORIGINAL_IMAGES +	Inquires the mode with which to add source entries to destination dataset, when cropping/resizing images with MclassPrepareData(). SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_AUTO.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO	Specifies that the source dataset entries, for cropped/resized images, are added to the destination dataset, if they were not previously added. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies that the source dataset entries, for cropped/resized images, are not added to the destination dataset.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies that the source entries, for cropped/resized images, are added to the destination dataset, regardless of whether they were previously added. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_PREPARED_DATA_FOLDER +	Inquires the path of the destination folder at which the prepared dataset is saved. An empty string implies the default path, which you can inquire with M_PREPARED_DATA_FOLDER + M_DEFAULT_PATH. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*Path*") 1	Specifies the path of the prepared dataset folder.	c		e	f	g	a		i		j	k	l	m	n	b
M_PRESET_ASPECT_RATIO +	Inquires the aspect ratio augmentation. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable the aspect ratio augmentation operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable the aspect ratio augmentation operation with preset values.	c		e	f	g	a		i		j	k	l	m	n	b
M_PRESET_CROP +	Inquires whether the preset crop augmentation is applied. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable the crop augmentation operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable the crop augmentation operation with preset values.	c		e	f	g	a		i		j	k	l	m	n	b
M_PRESET_FLIP +	Inquires whether the preset flip augmentation is applied. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable the flip augmentation operation.	c		e	f	g	a		i		j	k	l	m	n	b

M_ENABLE	Specifies to enable the flip augmentation operation with preset values.	c		e	f	g	a		i		j	k	l	m	n	b
M_PRESET_GAMMA +	Inquires whether the gamma augmentation is applied. This operation is not supported for binary (1-bit) images. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable the gamma augmentation operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable the gamma augmentation operation with preset values.	c		e	f	g	a		i		j	k	l	m	n	b
M_PRESET_HUE_OFFSET +	Inquires whether the preset hue offset augmentation is applied. This operation is not supported for binary (1-bit) and grayscale (1-band) images. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable the hue offset augmentation operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable the hue offset augmentation operation with preset values.	c		e	f	g	a		i		j	k	l	m	n	b
M_PRESET_INTENSITY_ADD +	Inquires whether the preset intensity add augmentation is applied. This operation is not supported for binary (1-bit) images. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable the intensity add augmentation operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable the intensity add augmentation operation with preset values.	c		e	f	g	a		i		j	k	l	m	n	b
M_PRESET_INTENSITY_MULTIPLY +	Inquires whether the preset intensity multiply augmentation is applied. This operation is not supported for binary (1-bit) images. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable the intensity multiply augmentation operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable the intensity multiply augmentation operation with preset values.	c		e	f	g	a		i		j	k	l	m	n	b
M_PRESET_LIGHTING_DIRECTIONAL +	Inquires whether the preset directional lighting augmentation is applied. This operation is not supported for binary (1-bit) images. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable the directional lighting augmentation operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable the directional lighting augmentation operation with preset values.	c		e	f	g	a		i		j	k	l	m	n	b
M_PRESET_NOISE_GAUSSIAN_ADDITIVE +	Inquires whether the Gaussian additive noise augmentation is applied. This operation is not supported for binary (1-bit) images. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable the Gaussian additive noise augmentation operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable the Gaussian additive noise augmentation operation with preset values.	c		e	f	g	a		i		j	k	l	m	n	b

M_PRESET_NOISE_SALT_PEPPER +	Inquires whether the preset salt and pepper augmentation is applied. This operation is not supported for binary (1-bit) images. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable the salt and pepper noise augmentation operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable the salt and pepper noise augmentation operation with preset values.	c		e	f	g	a		i		j	k	l	m	n	b
M_PRESET_REDUCE +	Inquires whether the preset reduce augmentation is applied. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable the reduce augmentation operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable the reduce augmentation operation with preset values.	c		e	f	g	a		i		j	k	l	m	n	b
M_PRESET_ROTATION +	Inquires whether the preset rotation augmentation is applied. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable the rotation augmentation operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable the rotation augmentation operation with preset values.	c		e	f	g	a		i		j	k	l	m	n	b
M_PRESET_SATURATION_GAIN +	Inquires whether the preset saturation gain augmentation is applied. This operation is not supported for binary (1-bit) and grayscale (1-band) images. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable the saturation gain augmentation operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable the saturation gain augmentation operation with preset values.	c		e	f	g	a		i		j	k	l	m	n	b
M_PRESET_SCALE +	Inquires whether the preset scale augmentation is applied. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable the scale augmentation operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable the scale augmentation operation with preset values.	c		e	f	g	a		i		j	k	l	m	n	b
M_PRESET_SMOOTH_GAUSSIAN +	Inquires whether the preset smooth Gaussian augmentation is applied. This operation is not supported for binary (1-bit) images. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable the smooth Gaussian augmentation operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable the smooth Gaussian augmentation operation with preset values.	c		e	f	g	a		i		j	k	l	m	n	b
M_PRESET_TRANSLATION +	Inquires whether the preset translation is applied. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b

M_DISABLE	Specifies to disable the translation augmentation operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable the translation augmentation operation with preset values.	c		e	f	g	a		i		j	k	l	m	n	b
M_RESIZE_SCALE_FACTOR +	Inquires whether the preset resize scale factor augmentation is applied. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_FILL_DESTINATION_BILINEAR	Specifies to make all entries scale to fill the destination image size x/y determined by M_SIZE_MODE, M_SIZE_X, M_SIZE_Y and classifier passed in the MclassPrepareData() call using bilinear interpolation. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_FILL_DESTINATION_NEAREST_NEIGHBOR	Specifies to make all entries scale to fill the destination image size x/y determined by M_SIZE_MODE, M_SIZE_X, M_SIZE_Y and classifier passed in the MclassPrepareData() call using nearest neighbor interpolation. INFO	c		e	f	g	a		i		j	k	l	m	n	b
0.0 < *Value* <= 16.0	Specifies the factor by which to resize. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_SIZE_MODE +	Inquires the mode with which to establish the size of the destination image. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_AUTO.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO	Specifies that the size of the destination image will be the same as the source image if no classifier was given. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_USER_DEFINED	Specifies the size of the destination image will be the user-defined M_SIZE_X and M_SIZE_Y. INFO	c		e	f	g	a		i		j	k	l	m	n	b

[1] If the value was passed in a variable, it won't be returned enclosed in AIL_TEXT().

To inquire about the source layer of the classifier in a predefined CNN, predefined object detection, predefined segmentation, predefined anomaly detection, or ONNX classifier context, or a data preparation context for image data, the InquireType parameter can be set to one of the following values. In this case, set the ContextOrResultClassId parameter to the identifier of a predefined CNN, predefined object detection, predefined segmentation, predefined anomaly detection, or ONNX classifier context, and set the LabelOrIndex parameter to M_DEFAULT_SOURCE_LAYER, or set the ContextOrResultClassId parameter to the identifier of a data preparation context and set the LabelOrIndex parameter to M_CONTEXT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a classifier context's source layer (predefined CNN, object detection, predefined segmentation, predefined anomaly detection, or ONNX) or a data preparation context																	
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_SIZE_X +	Inquires the width of the network's source layer (the width of the images that were used in the training phase) when specifying a classifier context, or the width of the destination image when specifying a data preparation context (in pixels). SET															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFINED_BY_TARGET_IMAGE_SIZE_X	Specifies that the input tensor of the imported ONNX model has a variable width dimension. The width dimension must be set using M_TARGET_IMAGE_SIZE_X prior to preprocessing. M_DEFINED_BY_TARGET_IMAGE_SIZE_X is only supported by M_CLASSIFIER_ONNX contexts.	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID	Specifies an invalid width of the destination image for a data preparation context. Set the size to a valid value, using MclassControl() with M_SIZE_X.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	For a data preparation context, this value specifies the width of the destination image. For a classifier context, this value specifies the width of the network's source layer.	c		e	f	g	a		i		j	k	l	m	n	b
M_SIZE_Y +	Inquires the height of the network's source layer (the height of the images that were used in the training phase) when specifying a classifier context, or the height of the destination image when specifying a data preparation context (in pixels). SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFINED_BY_TARGET_IMAGE_SIZE_Y	Specifies that the input tensor of the imported ONNX model has a variable width dimension. The width dimension must be set using M_TARGET_IMAGE_SIZE_Y prior to preprocessing. M_DEFINED_BY_TARGET_IMAGE_SIZE_Y is only supported by M_CLASSIFIER_ONNX contexts.	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID	Specifies an invalid height of the destination image for a data preparation context. Set the size to a valid value, using MclassControl() with M_SIZE_Y.	c		e	f	g	a		i		j	k	l	m	n	b

Value > 0.0	For a data preparation context, this value specifies the height of the destination image. For a classifier context, this value specifies the height of the network's source layer.	c		e	f	g	a		i		j	k	l	m	n	b

To inquire about a dataset context, set the InquireType parameter to one of the values below. In this case, set the ContextOrResultClassId parameter to the identifier of a dataset context, and set the LabelOrIndex parameter to M_CONTEXT, unless otherwise specified.

Note, these values apply to an images or a features dataset context, unless otherwise specified.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a dataset context (images or features)																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_ACTIVE_AUTHOR_INDEX +	Inquires the author that is automatically added for labeling operations performed on a dataset. SET															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_CURRENT_USER.	c		e	f	g	a		i		j	k	l	m	n	b
M_CURRENT_USER	Specifies the system's current username as the author that is automatically defined for each dataset entry. To get the index of the current user, use M_CURRENT_USER_AUTHOR_INDEX.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_INT	Specifies the author index.	c		e	f	g	a		i		j	k	l	m	n	b
M_ACTIVE_AUTHOR_UPDATE +	Inquires the behavior of the entry's author after a modification was performed on the entry. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ENABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to keep the existing author's information.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies that when an entry's information (an entry's region information) is modified, its author is automatically modified according to M_ACTIVE_AUTHOR_INDEX.	c		e	f	g	a		i		j	k	l	m	n	b
M_ANOMALY_DETECTION_FOLDER +	Inquires the path at which to save the anomaly detection scores. This value only applies to an images dataset context. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*Path*") 1	Specifies the path.	c		e	f	g	a		i		j	k	l	m	n	b
M_ANOMALY_DETECTION_FOLDER_ABS +	Inquires the absolute path at which to save the anomaly detection scores. This path corresponds to the concatenation of M_ROOT_PATH + M_ANOMALY_DETECTION_FOLDER. This value only applies to an images dataset context.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*Path*") 1	Specifies the path.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTHOR_KEY +	Inquires the author's key (UUID). You must set the LabelOrIndex parameter to M_AUTHOR_INDEX().	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_UUID*	c		e	f	g	a		i		j	k	l	m	n	b
AIL_UUID Value	Specifies the key of the author. The key is defined as an Aurora Imaging Library universal unique identifier (UUID).	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTHOR_NAME +	Inquires the name of an author in the dataset. You must set the LabelOrIndex parameter to M_AUTHOR_INDEX(). SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*AuthorName*") 1	Specifies the name of an author.	c		e	f	g	a		i		j	k	l	m	n	b
M_CURRENT_USER_AUTHOR_INDEX +	Inquires the author index of the current user.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID	Specifies the current user does not exist as an author.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_INT	Specifies the author index of the current user.	c		e	f	g	a		i		j	k	l	m	n	b

M_DONT_CARE_CLASS_DRAW_COLOR +	Inquires the color of regions where M_CLASS_INDEX_GROUND_TRUTH has been set to M_DONT_CARE_CLASS. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_COLOR_DONT_CARE_CLASS.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_BLACK	Specifies the color black.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_BLUE	Specifies the color blue.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_BRIGHT_GRAY	Specifies the color bright gray.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_CYAN	Specifies the color cyan.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_BLUE	Specifies the color dark blue.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_CYAN	Specifies the color dark cyan.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_GREEN	Specifies the color dark green.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_MAGENTA	Specifies the color dark magenta.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_RED	Specifies the color dark red.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_YELLOW	Specifies the color dark yellow.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DONT_CARE_CLASS	Specifies the color (255, 255, 255).	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_GRAY	Specifies the color gray.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_GREEN	Specifies the color green.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_LIGHT_BLUE	Specifies the color light blue.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_LIGHT_GRAY	Specifies the color light gray.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_LIGHT_GREEN	Specifies the color light green.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_LIGHT_WHITE	Specifies the color light white.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_MAGENTA	Specifies the color magenta.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_RED	Specifies the color red.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_WHITE	Specifies the color white.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_YELLOW	Specifies the color yellow.	c		e	f	g	a		i		j	k	l	m	n	b
Byte-encoded RGB value	Specifies an encoded RGB value. To verify if the value is a byte-encoded RGB value, use the **M_IS_RGB888** macro. To retrieve the R, G, and B bands, use the **M_RGB888_R**, **M_RGB888_G**, and **M_RGB888_B** macros.	c		e	f	g	a		i		j	k	l	m	n	b
M_NO_CLASS_DRAW_COLOR +	Inquires the color used for unlabeled pixels when M_NO_REGION_PIXEL_CLASS is set to M_NO_CLASS. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_COLOR_NO_CLASS.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_BLACK	Specifies the color black. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_BLUE	Specifies the color blue. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_BRIGHT_GRAY	Specifies the color bright gray. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_CYAN	Specifies the color cyan. INFO	c		e	f	g	a		i		j	k	l	m	n	b

	M_COLOR_DARK_BLUE	Specifies the color dark blue. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_DARK_CYAN	Specifies the color dark cyan. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_DARK_GREEN	Specifies the color dark green. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_DARK_MAGENTA	Specifies the color dark magenta. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_DARK_RED	Specifies the color dark red. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_DARK_YELLOW	Specifies the color dark yellow. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_GRAY	Specifies the color gray. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_GREEN	Specifies the color green. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_LIGHT_BLUE	Specifies the color light blue. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_LIGHT_GRAY	Specifies the color light gray. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_LIGHT_GREEN	Specifies the color light green. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_LIGHT_WHITE	Specifies the color light white. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_MAGENTA	Specifies the color magenta. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_NO_CLASS	Specifies the color (16, 0, 0).	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_RED	Specifies the color red. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_WHITE	Specifies the color white. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_YELLOW	Specifies the color yellow. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	Byte-encoded RGB value	Specifies an encoded RGB value. To verify if the value is a byte-encoded RGB value, use the **M_IS_RGB888** macro. To retrieve the R, G, and B bands, use the **M_RGB888_R**, **M_RGB888_G**, and **M_RGB888_B** macros.	c		e	f	g	a		i		j	k	l	m	n	b
M_NO_REGION_PIXEL_CLASS +		Inquires the class of any pixels that are not associated with any regions and therefore do not have an assigned value for M_CLASS_INDEX_GROUND_TRUTH. As M_NO_REGION_PIXEL_CLASS is associated with the UUID of the specified class, the returned index will vary with the index of the class. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_NO_CLASS.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DONT_CARE_CLASS	Specifies that all pixels not assigned to a region will not contribute to training loss.	c		e	f	g	a		i		j	k	l	m	n	b
	M_NO_CLASS	Specifies that no class index is used for pixels not assigned to a region.	c		e	f	g	a		i		j	k	l	m	n	b
	0 <= *Value* < M_NUMBER_OF_CLASSES	Specifies the index of the class that is used for pixels not assigned to a region.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_AUGMENTED_ENTRIES +		Inquires the number of entries that are augmented.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0	Specifies the number of augmented entries.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_AUTHORS +		Inquires the number of authors currently defined in the dataset.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0	Specifies the number of authors.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_ENTRIES +		Inquires the number of entries defined in the dataset.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

Value >= 0	Specifies the number of entries in the dataset.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_ENTRIES_GROUND_TRUTH +	Inquires the number of entries that have their ground truth specified (M_CLASS_INDEX_GROUND_TRUTH). To inquire the number of entries for every class in the context, set the LabelOrIndex parameter to M_CONTEXT. To inquire the number of entries for a specific class in the context, set the LabelOrIndex parameter to that class using M_CLASS_INDEX().	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the number of entries that have a ground truth.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_ENTRIES_PREDICTED +	Inquires the number of entries that have their predicted class specified, for classification predictions (not for segmentation). Specifically, M_NUMBER_OF_ENTRIES_PREDICTED returns the number of entries that return M_TRUE when calling MclassGetResultEntry() with M_CLASSIFICATION and M_PREDICT_INFO. To inquire the number of entries for every class in the context, set the LabelOrIndex parameter to M_CONTEXT. To inquire the number of entries for a specific class in the context, set the LabelOrIndex parameter to that class using M_CLASS_INDEX().	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the number of entries that have a predicted class.	c		e	f	g	a		i		j	k	l	m	n	b
M_PREDICTED_SCORE_AVERAGE +	Inquires the average predicted best score among the entries counted in M_NUMBER_OF_ENTRIES_PREDICTED.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the average predicted score among the entries counted in M_NUMBER_OF_ENTRIES_PREDICTED.	c		e	f	g	a		i		j	k	l	m	n	b
M_PREDICTED_SCORE_MAX +	Inquires the maximum predicted best score among the entries counted in M_NUMBER_OF_ENTRIES_PREDICTED.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the maximum predicted score among the entries counted in M_NUMBER_OF_ENTRIES_PREDICTED.	c		e	f	g	a		i		j	k	l	m	n	b
M_PREDICTED_SCORE_MIN +	Inquires the minimum predicted best score among the entries counted in M_NUMBER_OF_ENTRIES_PREDICTED.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the minimum predicted score among the entries counted in M_NUMBER_OF_ENTRIES_PREDICTED.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_MASKS_FOLDER +	Inquires the path at which region masks are saved as they are added by MclassEntryAddRegion(). This value only applies to an images dataset context. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*Path*") 1	Specifies the path of the region masks folder.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_MASKS_FOLDER_ABS +	Inquires the absolute path at which to save region masks. This path corresponds to the concatenation of M_ROOT_PATH + M_REGION_MASKS_FOLDER. This value only applies to an images dataset context.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*Path*") 1	Specifies the path.	c		e	f	g	a		i		j	k	l	m	n	b
M_ROOT_PATH +	Inquires the root file path, such that M_ROOT_PATH + M_ENTRY_IMAGE_PATH, M_REGION_MASK_PATH, M_SEGMENTATION_PATH, or M_ANOMALY_DETECTION_PATH gives a valid path. If M_ROOT_PATH is empty, M_ENTRY_IMAGE_PATH should be an absolute path, otherwise it should be a relative path to M_ROOT_PATH. This value only applies to an images dataset context. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("") 1	Specifies an empty path. INFO	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*///DATASET///*") 1	Specifies the folder path from where the dataset context is restored, using MclassRestore(). INFO	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*RootFilePath*") 1	Specifies the root file path. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_SEGMENTATION_FOLDER +	Inquires the path at which to save the segmentation scores. This value only applies to an images dataset context. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*	c		e	f	g	a		i		j	k	l	m	n	b

AIL_TEXT("*Path*") 1	Specifies the path.	c		e	f	g	a		i		j	k	l	m	n	b
M_SEGMENTATION_FOLDER_ABS +	Inquires the absolute path at which to save the segmentation scores. This path corresponds to the concatenation of M_ROOT_PATH + M_SEGMENTATION_FOLDER. This value only applies to an images dataset context.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*Path*") 1	Specifies the path.	c		e	f	g	a		i		j	k	l	m	n	b

[1] If the value was passed in a variable, it won't be returned enclosed in AIL_TEXT().

To inquire about a classifier or dataset context, set the InquireType parameter to one of the values below. In this case, set the ContextOrResultClassId parameter to the identifier of a classifier or dataset context, and set the LabelOrIndex parameter to M_CONTEXT, unless otherwise specified.

Note, these values apply to a classifier context or an images or a features dataset context, unless otherwise specified.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a classifier or dataset context (images or features)																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_NUMBER_OF_CLASSES +	Inquires the number of class definitions (classes).															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the number of classes.	c		e	f	g	a		i		j	k	l	m	n	b

To inquire about a class definition in a classifier or dataset context, set the InquireType parameter to one of the values below. Unless otherwise specified, set the ContextOrResultClassId parameter to the identifier of any type of dataset or classifier context, and set the LabelOrIndex parameter to a specific class definition using M_CLASS_INDEX().

Note, these values apply to class definitions in a predefined CNN, predefined object detection, predefined segmentation, predefined anomaly detection, tree ensemble, or ONNX classifier context, or in an images or features dataset context, unless otherwise specified.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a class definition in a classifier or dataset context (for CNN, object detection, segmentation, anomaly detection, tree ensemble, or ONNX)																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_ANOMALOUS +	Inquires whether the specified class is considered anomalous. Note, the ContextOrResultClassId parameter must be the identifier of an anomaly detection classifier context or images dataset used for anomaly detection. SET															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that images containing this class, either at the image or pixel level, will not be considered anomalous unless it also contains a class with M_ANOMALOUS set to M_TRUE.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that images containing this class, either at the image or pixel level, will be considered anomalous.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLASS_DRAW_COLOR +	Inquires the color assigned to a class definition. This color is used as a visual label. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_BLACK	Specifies the color black. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_BLUE	Specifies the color blue. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_BRIGHT_GRAY	Specifies the color bright gray. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_CYAN	Specifies the color cyan. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_BLUE	Specifies the color dark blue. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_CYAN	Specifies the color dark cyan. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_GREEN	Specifies the color dark green. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_MAGENTA	Specifies the color dark magenta. INFO	c		e	f	g	a		i		j	k	l	m	n	b

M_COLOR_DARK_RED	Specifies the color dark red. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_YELLOW	Specifies the color dark yellow. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_GRAY	Specifies the color gray. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_GREEN	Specifies the color green. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_LIGHT_BLUE	Specifies the color light blue. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_LIGHT_GRAY	Specifies the color light gray. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_LIGHT_GREEN	Specifies the color light green. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_LIGHT_WHITE	Specifies the color light white. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_MAGENTA	Specifies the color magenta. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_RED	Specifies the color red. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_WHITE	Specifies the color white. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_YELLOW	Specifies the color yellow. INFO	c		e	f	g	a		i		j	k	l	m	n	b
Byte-encoded RGB value	Specifies an encoded RGB value. To verify if the value is a byte-encoded RGB value, use the **M_IS_RGB888** macro. To retrieve the R, G, and B bands, use the **M_RGB888_R**, **M_RGB888_G**, and **M_RGB888_B** macros.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLASS_ICON_ID +	Inquires the identifier of the image associated to the specified class definition. The identifier returned is not the same as the one supplied using McLassControl() with M_CLASS_ICON_ID. Aurora Imaging Library copies that image to the context and assigns it a new identifier, which is returned here. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies that the image associated with the class definition is freed. Note that if the specified class definition does not have an associated image, this setting has no effect.	c		e	f	g	a		i		j	k	l	m	n	b
Image identifier	Specifies the identifier of the image buffer that Aurora Imaging Library associates to the class definition.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLASS_KEY +	Inquires the UUID (key), which is to be used to link between datasets, class labels, and a context class definition.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_UUID*	c		e	f	g	a		i		j	k	l	m	n	b
AIL_UUID Value	Specifies the unique identifier of the class definition.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLASS_NAME +	Inquires the name of a class definition. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*ClassDefinitionName*") 1	Specifies the name of a class definition.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLASS_WEIGHT +	Inquires the class definition's weight. M_CLASS_WEIGHT can only have an effect if M_CLASS_WEIGHT_MODE is set to M_USER_DEFINED). Note that this inquire is only supported for a features dataset context, an images dataset context being used for segmentation, or a tree ensemble classifier context. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the weight.	c		e	f	g	a		i		j	k	l	m	n	b

[1] If the value was passed in a variable, it won't be returned enclosed in AIL_TEXT().

To inquire about a global setting of a segmentation or tree ensemble training context, set the InquireType parameter to one of the values below. In this case, set the ContextOrResultClassId parameter to the identifier of a segmentation or tree ensemble training context, unless otherwise specified, and set the LabelOrIndex parameter to M_CONTEXT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a training context (segmentation or tree ensemble)															
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

UserVarPtr - Possible values returned															
M_CLASS_WEIGHT_MODE +	Inquires how to establish the weight of a class definition. SET														
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_AUTO.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO	Specifies that Aurora Imaging Library automatically sets how the weight is established. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_BALANCE	Specifies a balanced weight. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_INVERSE_CLASS_FREQUENCY	Specifies to use weight factors equal to the inverse of each class frequency. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_NONE	Specifies no weight.	c		e	f	g	a		i		j	k	l	m	n	b
M_SQUARED_INVERSE_CLASS_FREQUENCY	Specifies to use weight factors equal to the squared inverse of each class frequency. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_USER_DEFINED	Specifies to use the weight defined with M_CLASS_WEIGHT.	c		e	f	g	a		i		j	k	l	m	n	b

To inquire about a global setting of a tree ensemble training context, set the InquireType parameter to one of the values below. In this case, set the ContextOrResultClassId parameter to the identifier of a tree ensemble training context, and set the LabelOrIndex parameter to M_CONTEXT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a training context (tree ensemble)																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_BOOTSTRAP +	Inquires whether to use the bootstrap samples when building trees. SET															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_WITH_REPLACEMENT.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to use bootstrap aggregation.	c		e	f	g	a		i		j	k	l	m	n	b
M_WITH_REPLACEMENT	Specifies to use bootstrap aggregation, and allow replacement (chosen entries in a training set can be chosen again).	c		e	f	g	a		i		j	k	l	m	n	b
M_WITHOUT_REPLACEMENT	Specifies to use bootstrap aggregation, and not allow replacement (chosen entries in a training set cannot be chosen again).	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPUTE_OUT_OF_BAG_RESULTS +	Inquires whether to use out-of-bag dataset entries to estimate the generalization accuracy. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ENABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to use out-of-bag entries.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to use out-of-bag entries.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPUTE_PROXIMITY_MATRIX +	Inquires whether to calculate the proximity measure matrix when building trees. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to calculate the proximity measure matrix when building trees.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to calculate the proximity measure matrix when building trees.	c		e	f	g	a		i		j	k	l	m	n	b
M_FEATURE_IMPORTANCE_MODE +	Inquires how to establish the importance of the features. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_MEAN_DECREASE_IMPURITY.	c		e	f	g	a		i		j	k	l	m	n	b

	M_DISABLE	Specifies to disable the feature importance mode. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_DROP_COLUMN	Specifies that the feature importance is based on the drop column of features. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MEAN_DECREASE_IMPURITY	Specifies that the feature importance is based on a decreasing impurity process. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_PERMUTATION	Specifies that the feature importance is based on feature permutation. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_FEATURE_IMPORTANCE_SET +		Inquires how to establish the group of features with which to determine feature importance. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_AUTO.	c		e	f	g	a		i		j	k	l	m	n	b
	M_AUTO	Specifies to automatically establish the feature set to use. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEV_DATASET	Specifies to use the development dataset.	c		e	f	g	a		i		j	k	l	m	n	b
	M_OUT_OF_BAG	Specifies to use the out-of-bag set. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_IMPURITY_DECREASE +		Inquires a decrease of impurity requirement for splitting a node. That is, Aurora Imaging Library will split a node if it induces a decrease of impurity greater than or equal to the specified value. The weighted impurity decrease equation is as follows: *N_t* / *N* * (*impurity* - *N_t_R* / *N_t* * *right_impurity* - *N_t_L* / *N_t* * *left_impurity*), where *N* is the total number of dataset entries, *N_t* is the number of dataset entries related to the current node, *N_t_L* is the number of dataset entries related to the left child, and *N_t_R* is the number of dataset entries related to the right child. *N*, *N_t*, *N_t_R*, and *N_t_L* all refer to the weighted sum, if weights (entry weights and class weights) are used. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0.0	Specifies the impurity value.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_NUMBER_OF_ENTRIES_LEAF +		Inquires the minimum number of related dataset entries, or a percentage of the minimum number of related dataset entries, that Aurora Imaging Library uses to identify a leaf node. Aurora Imaging Library uses this value to establish M_MIN_NUMBER_OF_ENTRIES_LEAF_MODE. Note that if Round((Value/100) * *NumberEntriesLeaf*) == 0, then *NumberEntriesLeaf* = 1. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 1.	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 < *MinNumPercent* <= 100.0	Specifies the minimum number of entries, as a percentage. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 1	Specifies the minimum number of entries. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_NUMBER_OF_ENTRIES_LEAF_MODE +		Inquires how to establish the minimum number of related dataset entries required to be at a leaf node. A split point at any depth will only be considered if it leaves at least the minimum number of entries (leaf training entries) in each of the left and right branches. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_USER_DEFINED_VALUE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_USER_DEFINED_PERCENTAGE	Specifies to use a percentage, determined from the minimum number of entries defined with M_MIN_NUMBER_OF_ENTRIES_LEAF.	c		e	f	g	a		i		j	k	l	m	n	b
	M_USER_DEFINED_VALUE	Specifies to use the minimum number of entries defined with M_MIN_NUMBER_OF_ENTRIES_LEAF.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_NUMBER_OF_ENTRIES_SPLIT +		Inquires the minimum number of related dataset entries, or a percentage of the minimum number of related dataset entries, that Aurora Imaging Library uses to determine how to best split the internal nodes of the trees. Aurora Imaging Library uses this value to establish M_MIN_NUMBER_OF_ENTRIES_SPLIT_MODE. Note that if Round((Value/100) * *NumberEntriesSplit*) == 0, then *NumberEntriesSplit* = 2. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 2.	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 < *MinNumPercent* <= 100.0	Specifies the minimum number of entries, as a percentage. INFO	c		e	f	g	a		i		j	k	l	m	n	b

	Value >= 1	Specifies the minimum number of entries. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_NUMBER_OF_ENTRIES_SPLIT_MODE +		Inquires how to establish the minimum number of related dataset entries required to split an internal node. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_USER_DEFINED_VALUE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_USER_DEFINED_PERCENTAGE	Specifies to use a percentage, determined from the minimum number of entries defined with M_MIN_NUMBER_OF_ENTRIES_SPLIT.	c		e	f	g	a		i		j	k	l	m	n	b
	M_USER_DEFINED_VALUE	Specifies to use the minimum number of entries defined with M_MIN_NUMBER_OF_ENTRIES_SPLIT.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_WEIGHT_FRACTION_LEAF +		Inquires the minimum weighted fraction of the sum total of weights (of all the dataset entries) required to be at a leaf node. Entries gave equal weight when M_CLASS_WEIGHT_MODE is set to M_NONE. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 0.5	Specifies the minimum weighted fraction.	c		e	f	g	a		i		j	k	l	m	n	b
M_NODE_MAX_NUMBER_OF_FEATURES +		Inquires the maximum number of features, or a percentage of the maximum number of features that Aurora Imaging Library can use to determine how to best split the nodes of the trees. Note that if Round((Value/100)**NumberOfFeatures*) == 0, then *NumberOfFeatures* = 1 (see M_NODE_MAX_NUMBER_OF_FEATURES_MODE). SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_ALL.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ALL	Specifies no maximum.	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *MaxNumPercent* <= 100.0	Specifies the percentage with which to determine the maximum number of features. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 1	Specifies the value with which to determine the maximum number of features. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_NODE_MAX_NUMBER_OF_FEATURES_MODE +		Inquires how to establish the maximum number of features that Aurora Imaging Library uses to determine how to best split the nodes of the trees. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_SQUARE_ROOT.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ALL	Specifies that all available features are used.	c		e	f	g	a		i		j	k	l	m	n	b
	M_LOG2	Specifies to use a base 2 logarithm, determined from the total number of features. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_SQUARE_ROOT	Specifies to use a square root, determined from the total number of features. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_USER_DEFINED_PERCENTAGE	Specifies to use a percentage, determined from the total number of features, where the percentage is set by M_NODE_MAX_NUMBER_OF_FEATURES. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_USER_DEFINED_VALUE	Specifies to use the maximum number of features defined with M_NODE_MAX_NUMBER_OF_FEATURES.	c		e	f	g	a		i		j	k	l	m	n	b
M_SPLIT_QUALITY_TYPE +		Inquires the function with which to measure the quality of a node split. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_GINI.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENTROPY	Specifies that Aurora Imaging Library uses an Entropy based function to measure the quality of a node split.	c		e	f	g	a		i		j	k	l	m	n	b
	M_GINI	Specifies that Aurora Imaging Library uses a Gini based function to measure the quality of a node split.	c		e	f	g	a		i		j	k	l	m	n	b
M_TREE_MAX_DEPTH +		Inquires the maximum depth of the trees. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

	M_DEFAULT	Same as M_NONE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_NONE	Specifies no maximum depth.	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0	Specifies the maximum depth.	c		e	f	g	a		i		j	k	l	m	n	b
M_TREE_MAX_NUMBER_OF_LEAF_NODES +		Inquires the maximum number of terminal nodes in the tree. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_NONE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_NONE	Specifies no maximum.	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 1	Specifies the maximum number of terminal nodes in the tree.	c		e	f	g	a		i		j	k	l	m	n	b

To inquire about a global setting of a tree ensemble training context, set the InquireType parameter to one of the values below. In this case, set the ContextOrResultClassId parameter to the identifier of a tree ensemble training context, and set the LabelOrIndex parameter to M_CONTEXT.

To inquire about a data preparation context for image data, set the InquireType parameter to one of the values below. In this case, set the ContextOrResultClassId parameter to the identifier of a data preparation context, and set the LabelOrIndex parameter to M_CONTEXT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a data preparation context (images) or a tree ensemble training context																	
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_SEED_MODE +	Inquires the mode with which to initialize the seed for tree ensemble training or image data augmentation. SET															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default mode with which to initialize the seed. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO	Specifies that the seed is chosen at random, and that Aurora Imaging Library generates and uses a new seed each time you call MclassTrain(). INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_USER_DEFINED	Specifies the user-defined seed set with M_SEED_VALUE.	c		e	f	g	a		i		j	k	l	m	n	b
M_SEED_VALUE +	Inquires the seed value that Aurora Imaging Library uses to generate random numbers for tree ensemble training or image data augmentation. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 1	Specifies the seed value.	c		e	f	g	a		i		j	k	l	m	n	b

To inquire about a global setting of a CNN, object detection, segmentation, anomaly detection, training context, set the InquireType parameter to one of the values below. In this case, set the ContextOrResultClassId parameter to the identifier of a CNN, object detection, segmentation, anomaly detection training context, and set the LabelOrIndex parameter to M_CONTEXT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a training context (CNN, object detection, segmentation, or anomaly detection)																	
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_CPU_TRAIN_ENGINE_LOAD_STATUS +	Inquires the status of the CPU train engine. This information can help you detect issues related to training with the CPU. M_CPU_TRAIN_ENGINE_LOAD_STATUS is only available if M_TRAIN_ENGINE is set to M_CPU, or if it is set to M_AUTO or M_DEFAULT and Aurora Imaging Library selected the CPU.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_FAILURE	Specifies that an unknown error occurred while trying to load the CPU train engine.	c		e	f	g	a		i		j	k	l	m	n	b
M_SUCCESS	Specifies that the CPU train engine loaded successfully (no issues were detected) and is ready for training.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNABLE_TO_FIND_CPU_TRAIN_ENGINE	Specifies that the CPU train engine was not found.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNABLE_TO_LOAD_CPU_TRAIN_ENGINE	Specifies that the CPU train engine was not loaded.	c		e	f	g	a		i		j	k	l	m	n	b

M_GPU_TRAIN_ENGINE_LOAD_STATUS +	Inquires the status of the GPU train engine. This information can help you detect issues related to training with the GPU. M_GPU_TRAIN_ENGINE_LOAD_STATUS is only available if M_TRAIN_ENGINE is set to M_GPU, or if it is set to M_AUTO or M_DEFAULT and Aurora Imaging Library selected the GPU.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_CUDA_FAIL	Specifies that an unknown error occurred with CUDA.	c		e	f	g	a		i		j	k	l	m	n	b
M_FAILURE	Specifies that an unknown error occurred while trying to load the GPU train engine.	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID_GPU_DRIVER_VERSION	Specifies that the GPU driver version is not up to date. Verify that the latest NVIDIA drivers and display drivers are installed. Refer to Chapter 43 of the Aurora Imaging Library User Guide for more information.	c		e	f	g	a		i		j	k	l	m	n	b
M_JIT_COMPILATION_REQUIRED	Specifies that the GPU has a compute capability that is higher than the train engine. There could be a delay before the training starts.	c		e	f	g	a		i		j	k	l	m	n	b
M_JIT_COMPILER_NOT_FOUND	Specifies that a JIT compiler was not found. A JIT compilation is required; verify that the proper NVIDIA drivers are installed.	c		e	f	g	a		i		j	k	l	m	n	b
M_SUCCESS	Specifies that the GPU train engine loaded successfully (no issues were detected) and is ready for training.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNABLE_TO_FIND_GPU_TRAIN_ENGINE	Specifies that the GPU train engine was not found.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNABLE_TO_FIND_VALID_GPU	Specifies that an NVIDIA GPU was not found, or that the GPU's compute capability is lower than 3.7.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNABLE_TO_LOAD_GPU_TRAIN_ENGINE	Specifies that the GPU train engine was not loaded.	c		e	f	g	a		i		j	k	l	m	n	b
M_MINI_BATCH_SIZE +	Inquires the size of each mini-batch. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value. For image classification (M_TRAIN_CNN), the default is 32. For segmentation (M_TRAIN_SEG) and object detection (M_TRAIN_DET), the default is 4. For anomaly detection (M_TRAIN_ANO), the default is 1.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the size.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAIN_ENGINE +	Inquires the train engine specified. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_AUTO.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO	Specifies the engine automatically. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_CPU	Specifies the CPU.	c		e	f	g	a		i		j	k	l	m	n	b
M_GPU	Specifies the GPU. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAIN_ENGINE_IS_INSTALLED +	Inquires whether a train engine is installed.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that a train engine is not installed.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that a train engine is installed.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAIN_ENGINE_USED +	Inquires which train engine is used. Note, the context must be preprocessed.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_CPU	Specifies that a CPU is used as the train engine.	c		e	f	g	a		i		j	k	l	m	n	b
M_GPU	Specifies that a GPU is used as the train engine.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAIN_ENGINE_USED_DESCRIPTION +	Inquires the description of the engine (device) on which training is done (M_TRAIN_ENGINE_USED). To inquire this information, the context must be preprocessed.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*	c		e	f	g	a		i		j	k	l	m	n	b

	AIL_TEXT("*TrainingDeviceName*") [1]	Specifies the description of the engine. Examples of descriptions you can retrieve are "GeForce RTX (tm) 2060 6GB" and "Intel(R) Core(TM) i7-8700 CPU 3.20GHZ".	c		e	f	g	a		i		j	k	l	m	n	b

[1] If the value was passed in a variable, it won't be returned enclosed in AIL_TEXT().

To inquire about a global setting of a CNN, object detection, or segmentation, training context, set the InquireType parameter to one of the values below. In this case, set the ContextOrResultClassId parameter to the identifier of a CNN, object detection, or segmentation training context, and set the LabelOrIndex parameter to M_CONTEXT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a training context (CNN, object detection, or segmentation)																	
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_FOCAL_LOSS_GAMMA +	Inquires the focusing value for the focal loss when training. SET															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 2.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the focusing value. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_INITIAL_LEARNING_RATE +	Inquires the initial learning rate of training. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value. For image classification (M_TRAIN_CNN), the default is 0.005. For segmentation (M_TRAIN_SEG) and object detection (M_TRAIN_DET), the default is 0.001.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the learning rate.	c		e	f	g	a		i		j	k	l	m	n	b
M_LATEST_USED_RESET_TRAINING_VALUES +	Inquires the latest value passed to M_RESET_TRAINING_VALUES. Note that this inquire does not indicate if any of the values set by M_RESET_TRAINING_VALUES have been changed since the last call with M_RESET_TRAINING_VALUES. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_COMPLETE.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPLETE	Specifies a complete training mode. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_FINE_TUNING	Specifies a fine tuning training mode. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_TRANSFER_LEARNING	Specifies a transfer learning mode. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_LEARNING_RATE_DECAY +	Inquires the learning rate decay of training. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value. For image classification (M_TRAIN_CNN), the default is 0.1. For segmentation (M_TRAIN_SEG) and object detection (M_TRAIN_DET), the default is 0.05.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the learning rate decay.	c		e	f	g	a		i		j	k	l	m	n	b
M_LOSS_FUNCTION_TYPE +	Inquires the type of loss function to use for training. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_AUTO.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO	Specifies that Aurora Imaging Library automatically establishes the type of loss function to use.	c		e	f	g	a		i		j	k	l	m	n	b
M_CROSS_ENTROPY	Specifies a cross entropy type of loss.	c		e	f	g	a		i		j	k	l	m	n	b
M_DET_LOSS	Specifies a detection type of loss.	c		e	f	g	a		i		j	k	l	m	n	b
M_DICE_LOSS	Specifies a dice type of loss.	c		e	f	g	a		i		j	k	l	m	n	b
M_FOCAL_LOSS	Specifies a focal type of loss.	c		e	f	g	a		i		j	k	l	m	n	b

M_MAX_EPOCH +	Inquires the maximum number of epochs. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 60.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the maximum number.	c		e	f	g	a		i		j	k	l	m	n	b
M_PREPARE_DATA_CONTEXT_ID +	Inquires the identifier of the internal prepare data context, allowing access to the prepare data API. MclassStream() with M_SAVE or M_LOAD, MclassSave(), and MclassRestore() allows you to save/load a copy of the internal prepare data context.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Data preparation context identifier	Specifies the identifier of the data preparation context.	c		e	f	g	a		i		j	k	l	m	n	b
M_SCHEDULER_TYPE +	Inquires the scheduler type. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_CYCLICAL_DECAY	Specifies to decay the learning rate at an internally established cyclical schedule.	c		e	f	g	a		i		j	k	l	m	n	b
M_DECAY	Specifies to decay the learning rate, as weights are updated.	c		e	f	g	a		i		j	k	l	m	n	b
M_FACTOR_DECAY	Specifies to decay the learning rate by a factor at every specified epoch.	c		e	f	g	a		i		j	k	l	m	n	b
M_SPLIT_PERCENTAGE +	Inquires the split percentage parameter of MclassSplitDataset() used in single dataset training (percentage of the source dataset that has been assigned to the training dataset). SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 80.0.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the percentage.	c		e	f	g	a		i		j	k	l	m	n	b
M_SPLIT_SEED_MODE +	Inquires the split seed mode used in single dataset training. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_RANDOM.	c		e	f	g	a		i		j	k	l	m	n	b
M_FIXED	Specifies to use the M_SPLIT_CONTEXT_FIXED_SEED predefined split classification context, for image classification (CNN) or feature classification (tree ensemble), or the M_SPLIT_SEG_CONTEXT_FIXED_SEED predefined split classification context, for segmentation.	c		e	f	g	a		i		j	k	l	m	n	b
M_RANDOM	Specifies to use the M_SPLIT_CONTEXT_DEFAULT predefined split classification context, for image classification (CNN) or feature classification (tree ensemble), or the M_SPLIT_SEG_CONTEXT_DEFAULT predefined split classification context, for segmentation.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAIN_DESTINATION_FOLDER +	Inquires the path of the destination folder in which to save training results. An empty string implies the default path, which you can inquire with M_TRAIN_DESTINATION_FOLDER + M_DEFAULT_PATH. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*Path*") 1	Specifies the path of the train destination folder.	c		e	f	g	a		i		j	k	l	m	n	b

[1] If the value was passed in a variable, it won't be returned enclosed in AIL_TEXT().

To inquire about a global setting of a segmentation training context, set the InquireType parameter to one of the values below. In this case, set the ContextOrResultClassId parameter to the identifier of a segmentation training context, and set the LabelOrIndex parameter to M_CONTEXT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a training context (segmentation)																
InquireType UserVarPtr - Possible values returned	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_CLASS_WEIGHT_STRENGTH +	Inquires the class weight strength when training with an inverse class frequency weight mode (M_INVERSE_CLASS_FREQUENCY). SET															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

M_DEFAULT	Specifies the default value; the default value is 50.0.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the proportion of weighting to assign to M_INVERSE_CLASS_FREQUENCY. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_FEATURE_SIZE_X +	Inquires the width (X-size) of the smallest feature to segment. This acts as a lower bound to the X-size of the class regions that the classifier expects during training. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_MIN.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN	Specifies the smallest size possible.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies that the classifier will adjust to encompass features of the size you set, provided they are larger than the minimum possible feature size.	c		e	f	g	a		i		j	k	l	m	n	b
M_FEATURE_SIZE_Y +	Inquires the height (Y-size) of the smallest feature to segment. This acts as a lower bound to the Y-size of the class regions that the classifier expects during training. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_SAME.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN	Specifies the smallest size possible.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAME	Specifies to use the same value as M_FEATURE_SIZE_X.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies that the classifier will adjust to encompass features of the size you set, provided they are larger than the minimum possible feature size.	c		e	f	g	a		i		j	k	l	m	n	b
M_USE_MASK_DESCRIPTORS +	Inquires whether to use mask descriptors for training. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ENABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to use mask descriptors for training.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to use mask descriptors for training.	c		e	f	g	a		i		j	k	l	m	n	b
M_USE_POLYGON_DESCRIPTORS +	Inquires whether to use polygon descriptors for training. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ENABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to use polygon descriptors for training.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to use polygon descriptors for training.	c		e	f	g	a		i		j	k	l	m	n	b

To inquire about a global setting of an anomaly detection training context, set the InquireType parameter to one of the values below. In this case, set the ContextOrResultClassId parameter to the identifier of an anomaly detection training context, and set the LabelOrIndex parameter to M_CONTEXT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a training context (anomaly detection)																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_MINI_BATCHES_PER_SAMPLING +	Inquires how many mini-batches are loaded into memory before sampling when M_TRAIN_MODE is set to M_ITERATIVE_SAMPLING.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 1	Specifies the number of mini-batches.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAMPLES_FIXED_SIZE +	Inquires the number of samples learned by the classifier.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the number of samples.	c		e	f	g	a		i		j	k	l	m	n	b

M_TRAIN_MODE +	Inquires the mode in which the samples are selected.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO	Specifies to automatically select the mode in which the samples are selected. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_GLOBAL_SAMPLING	Specifies a global selection mode. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ITERATIVE_SAMPLING	Specifies an iterative selection mode. INFO	c		e	f	g	a		i		j	k	l	m	n	b

To inquire about a global setting of a CNN, segmentation, or tree ensemble training context, set the InquireType parameter to the value below. In this case, set the ContextOrResultClassId parameter to the identifier of a CNN, segmentation, or tree ensemble training context, and set the LabelOrIndex parameter to M_CONTEXT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a training context (CNN, segmentation, or feature classification)																	
InquireType UserVarPtr - Possible values returned	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_SUPPORT_MISSING_GROUND_TRUTH +	Inquires whether entries without a ground truth on training and development datasets are supported. SET															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies that entries without a ground truth are not allowed. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies that entries without a ground truth are allowed. INFO	c		e	f	g	a		i		j	k	l	m	n	b

To inquire about a global setting of a statistics classification context, a classifier context, or a training context, the InquireType parameter can be set to one of the following values, unless otherwise specified. In this case, set the ContextOrResultClassId parameter to the identifier of a statistics classification context, or a classifier or training context, and set the LabelOrIndex parameter to M_CONTEXT.

Note, these values apply to a statistics classification context, a CNN, object detection, segmentation, anomaly detection, or tree ensemble classifier or training context, or an ONNX classifier context, unless otherwise specified.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a statistics classification, classifier, or training context (CNN, object detection, segmentation, anomaly detection, or tree ensemble)																	
InquireType UserVarPtr - Possible values returned	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NUMBER_OF_TREES +	Inquires the number of trees in the classifier or training context. You can only retrieve this value after calling MclassPreprocess(). Note, this is only available for tree ensemble training or classifier contexts.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value. For tree ensemble training context, the default value is 10. > Note that you cannot specify this value for tree ensemble classifier contexts.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 1	Specifies the number of trees in the classifier or training context.	c		e	f	g	a		i		j	k	l	m	n	b
M_PREPROCESSED +	Inquires whether the classification or training context is preprocessed.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that the context has not been preprocessed using MclassPreprocess().	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that the context has been preprocessed using MclassPreprocess().	c		e	f	g	a		i		j	k	l	m	n	b
M_TIMEOUT +	Inquires the maximum calculation time for MclassPredict(), in msec. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies an infinite amount of time.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the maximum time, in msec.	c		e	f	g	a		i		j	k	l	m	n	b

To inquire about a general setting of a prediction result buffer, the InquireType can be set to the following value. Unless otherwise specified, set the ContextOrResultClassId parameter to the identifier of an object detection prediction result buffer and set the LabelOrIndex parameter to M_GENERAL.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a prediction result buffer (object detection)																	
InquireType UserVarPtr - Possible values returned	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_RESULT_OUTPUT_UNITS +	Inquires which units and coordinate system to use for prediction results. This setting only affects positional results within the Aurora Imaging Library Classification module. Note, this setting can be changed at any time to get results in the required units. SET																
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_ACCORDING_TO_CALIBRATION	Specifies to use world units if the target image is associated with a camera calibration context; otherwise, specifies to use pixel units.	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL	Specifies to use pixel units with respect to the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_WORLD	Specifies to use world units with respect to the relative coordinate system. INFO	c		e	f	g	a		i		j	k	l	m	n	b

To inquire about any classification context supported by this function, set the InquireType the value below. In this case, set the ContextOrResultClassId parameter to the identifier of a classifier context, a dataset context, or a training context, and set the LabelOrIndex parameter to M_CONTEXT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For any supported context																	
InquireType UserVarPtr - Possible values returned	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_MODIFICATION_COUNT +	Inquires the current value of the modification counter. The modification counter is increased by one each time settings for the context are modified. Although you cannot identify the modification counter's contents, you can compare them throughout your application to know if the context has been altered. If the modification counter has changed you can, for example, prompt the user to save before closing the application.																
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the current value of the modification counter.	c		e	f	g	a		i		j	k	l	m	n	b

To inquire about any classification context or result buffer supported by this function, set the InquireType to the value below. In this case, set the ContextOrResultClassId parameter to the identifier of a classifier context, a dataset context, a training context, a prediction result, or a training result and set the LabelOrIndex parameter to M_CONTEXT or M_GENERAL.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For any supported context or result buffer																	
InquireType UserVarPtr - Possible values returned	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_OWNER_SYSTEM +	Inquires the identifier of the system on which the context or result was allocated. SET																
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

To inquire about statistics classification, set the InquireType parameter to one of the values below. In addition, you can also inquire M_TIMEOUT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For statistics classification																	
InquireType UserVarPtr - Possible values returned	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_MATCH_IOU_THRESHOLD +	Inquires the IOU threshold for an object detection stat context, based on which predicted boxes are compared to their matched ground truth boxes. To use this inquire, you must set the ContextOrResultClassId parameter to the identifier of a statistics classification context for object detection (M_STAT_DET) and set the LabelOrIndex parameter to M_CONTEXT. SET															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

	M_DEFAULT	Specifies the default value; the default value is 50.0.	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 100.0	Specifies the IOU threshold.	c		e	f	g	a		i		j	k	l	m	n	b
M_MODE +		Inquires whether to calculate statistics for anomaly detection on an image level (default) or a pixel level. To use this inquire, you must set the ContextOrResultClassId parameter to the identifier of a statistics classification context for anomaly detection (M_STAT_ANO) and set the LabelOrIndex parameter to M_CONTEXT. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_IMAGE_LEVEL.	c		e	f	g	a		i		j	k	l	m	n	b
	M_IMAGE_LEVEL	Specifies to calculate statistics for anomaly detection on an image level. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_PIXEL_LEVEL	Specifies to calculate statistics for anomaly detection on a pixel level. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_SCORE_THRESHOLD_STEP +		Inquires the step increment of threshold values from 0.0 to 100.0. To use this inquire, you must set the ContextOrResultClassId parameter to the identifier of any statistics classification context (M_STAT_ANO, M_STAT_CNN, M_STAT_DET, M_STAT_SEG, or M_STAT_TREE_ENSEMBLE) and set the LabelOrIndex parameter to M_CONTEXT. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.001.	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 100.0	Specifies the step increment for threshold.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_PREPARED_DATA_FOLDER; M_TRAIN_DESTINATION_FOLDER.

You can add the following value to the above-mentioned values to determine the default path of the folder.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring the default path of the folder																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_DEFAULT_PATH +	Inquires the default path of the specified folder related inquire type. The default path depends on whether the dataset paths are relative or is absolute. For relative type datasets, the default is under M_ROOT_PATH \ *FolderName*. For absolute type datasets, the default path is under C:\Users\Public\Documents\Classification\ *FolderName* on Windows and /var/lib/aurora_imaging/Classification/ *FolderName* on Linux. You can modify the default absolute path with the **Default destination folder** option under the **DL Classifiers** item in the **Aurora Imaging Configurator** utility.															
UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*DefaultPath*") 1	Specifies the path.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_EXPIRY_DATE_STRING; M_PREDICT_ENGINE_DESCRIPTION; M_ANOMALY_DETECTION_FOLDER; M_ANOMALY_DETECTION_FOLDER_ABS; M_AUTHOR_NAME; M_REGION_MASKS_FOLDER_ABS; M_ROOT_PATH; M_SEGMENTATION_FOLDER; M_SEGMENTATION_FOLDER_ABS; M_CLASS_NAME; M_TRAIN_ENGINE_USED_DESCRIPTION; M_DEFAULT_PATH.

You can add the following value to the above-mentioned values to get the size of a string.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring the size of a string																
InquireType combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_STRING_SIZE +	Inquires the number of characters in the string. This number accounts for every character, including the terminating null character.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for the values listed in For a classifier context's source layer (predefined CNN, predefined segmentation, predefined object detection, or ONNX), For a data preparation context (CNN, object detection or segmentation), For a classifier context's source layer (predefined CNN, object detection, predefined segmentation, predefined anomaly detection, or ONNX) or a data preparation context, For a training context (segmentation or tree ensemble), For a training context (tree ensemble), For a data preparation context (images) or a tree ensemble training context, For statistics classification; and for the following values: M_TARGET_IMAGE_SIZE_BAND; M_TARGET_IMAGE_SIZE_X; M_TARGET_IMAGE_SIZE_Y; M_ACTIVE_AUTHOR_INDEX; M_ACTIVE_AUTHOR_UPDATE; M_CLASS_DRAW_COLOR; M_CLASS_ICON_ID; M_CLASS_WEIGHT; M_MINI_BATCH_SIZE; M_TRAIN_ENGINE; M_FOCAL_LOSS_GAMMA; M_INITIAL_LEARNING_RATE; M_LATEST_USED_RESET_TRAINING_VALUES; M_LEARNING_RATE_DECAY; M_LOSS_FUNCTION_TYPE; M_MAX_EPOCH; M_PREPARE_DATA_CONTEXT_ID; M_SCHEDULER_TYPE; M_SPLIT_PERCENTAGE; M_SPLIT_SEED_MODE; M_TIMEOUT; M_OWNER_SYSTEM.

You can add the following value to the above-mentioned values to determine the default value of an inquire type, regardless of the current value of the inquire type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring the default value of an inquire type																		
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																	
M_DEFAULT		Inquires the default value of the specified inquire type.																
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for
Tables:

- For a classifier context (predefined CNN, object detection, segmentation, anomaly detection, or ONNX)
- For a classifier context (tree ensemble)
- For a classifier context (predefined CNN, predefined object detection, predefined segmentation, predefined anomaly detection, tree ensemble, or ONNX)
- For a classifier context's source layer (predefined CNN, predefined segmentation, predefined object detection, or ONNX)
- For a data preparation context (CNN, object detection or segmentation)
- For a classifier context's source layer (predefined CNN, object detection, predefined segmentation, predefined anomaly detection, or ONNX) or a data preparation context
- For a dataset context (images or features)
- For a class definition in a classifier or dataset context (for CNN, object detection, segmentation, anomaly detection, tree ensemble, or ONNX)
- For a training context (segmentation or tree ensemble)
- For a training context (tree ensemble)
- For a data preparation context (images) or a tree ensemble training context
- For a training context (CNN, object detection, segmentation, or anomaly detection)
- For a training context (CNN, object detection, or segmentation)
- For a training context (segmentation)
- For a training context (anomaly detection)
- For a training context (CNN, segmentation, or feature classification)
- For a statistics classification, classifier, or training context (CNN, object detection, segmentation, anomaly detection, or tree ensemble)
- For any supported context
- For any supported context or result buffer
- For statistics classification

the values listed in all the parameter association tables except **For a classifier context (segmentation or ONNX)**, **For a classifier context (object detection)**, **For a classifier context (object detection or anomaly detection)**, **For a classifier or dataset context (images or features)**, **For a prediction result buffer (object detection)**.

You can add one of the following values to the above-mentioned values to determine whether an inquire type is supported.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring whether an inquire type is supported																		
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																	
M_HAS_DEFAULT		Inquires whether the specified inquire type has a default value.																
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_FALSE	Specifies that the inquire type does not have a default value.	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRUE	Specifies that the inquire type has a default value.	c		e	f	g	a		i		j	k	l	m	n	b
M_SUPPORTED		Inquires whether the specified inquire type is supported for the classification context.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_FALSE	Specifies that the inquire type is not supported.	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRUE	Specifies that the inquire type is supported.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for the values listed in For a classifier context (object detection), For a classifier context (tree ensemble), For a classifier context (predefined CNN, predefined object detection, predefined segmentation, predefined anomaly detection, tree ensemble, or ONNX), For a classifier context's source layer (predefined CNN, predefined segmentation, predefined object detection, or ONNX), For a data preparation context (CNN, object detection or segmentation), For a classifier context's source layer (predefined CNN, object detection, predefined segmentation, predefined anomaly detection, or ONNX) or a data preparation context, For a class definition in a classifier or dataset context (for CNN, object detection, segmentation, anomaly detection, tree ensemble, or ONNX), For a training context (segmentation or tree ensemble), For a training context (tree ensemble), For a data preparation context (images) or a tree ensemble training context, For a training context (CNN, object detection, segmentation, or anomaly detection), For a training context (CNN, object detection, or segmentation), For a training context (segmentation), For a training context (anomaly detection), For a training context (CNN, segmentation, or feature classification), For a statistics classification, classifier, or training context (CNN, object detection, segmentation, anomaly detection, or tree ensemble), For a prediction result buffer (object detection), For any supported context, For any supported context or result buffer, For statistics classification; (exceptions might exist) and for the following values: M_CLASSIFIER_PREDEFINED_TYPE; M_NUMBER_OF_PREDICT_ENGINES; M_PREDICT_ENGINE_PRECISION; M_PREDICT_ENGINE_PROVIDER; M_PREDICT_ENGINE_USED; M_TRAINABLE_COMPLETE; M_TRAINABLE_FINE_TUNING; M_TRAINABLE_TRANSFER_LEARNING; M_TARGET_IMAGE_SIZE_BAND; M_TARGET_IMAGE_SIZE_X; M_TARGET_IMAGE_SIZE_Y; M_TRAIN_IMAGE_MIN_SIZE_X; M_TRAIN_IMAGE_MIN_SIZE_Y; M_TRAIN_IMAGE_STEP_X; M_TRAIN_IMAGE_STEP_Y; M_ACTIVE_AUTHOR_INDEX; M_ACTIVE_AUTHOR_UPDATE; M_CURRENT_USER_AUTHOR_INDEX; M_DONT_CARE_CLASS_DRAW_COLOR; M_NO_CLASS_DRAW_COLOR; M_NO_REGION_PIXEL_CLASS; M_NUMBER_OF_AUGMENTED_ENTRIES; M_NUMBER_OF_AUTHORS; M_NUMBER_OF_ENTRIES; M_NUMBER_OF_ENTRIES_GROUND_TRUTH; M_NUMBER_OF_ENTRIES_PREDICTED; M_PREDICTED_SCORE_AVERAGE; M_PREDICTED_SCORE_MAX; M_PREDICTED_SCORE_MIN; M_NUMBER_OF_CLASSES; M_PREPROCESSED; M_TIMEOUT; M_OWNER_SYSTEM; M_STRING_SIZE.

You can add one of the following values to the above-mentioned values to cast the requested information to a required data type.

> Note: In C++, when using a standard vector (std::vector) instead of an array, the values listed in this combination table must not be used. If you need to use a specific data type, instantiate the vector with the necessary data type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the data type																		
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_TYPE_AIL_DOUBLE	Casts the requested information to an *AIL_DOUBLE*.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_ID	Casts the requested information to an *AIL_ID*. Note that M_TYPE_AIL_ID should only be used with M_OWNER_SYSTEM.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_ID* **|** *array of type AIL_ID [optionally, in C++: a reference to a std::vector<AIL_ID>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT	Casts the requested information to an *AIL_INT*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT* **|** *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT32	Casts the requested information to an *AIL_INT32*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT32* **|** *array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT64	Casts the requested information to an *AIL_INT64*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64* **|** *array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_UUID	Casts the requested information to an *AIL_UUID*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_UUID* **|** *array of type AIL_UUID [optionally, in C++: a reference to a std::vector<AIL_UUID>]*	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the requested information, cast to an *AIL_INT*. If the requested information does not fit into an *AIL_INT*, this function will return **M_NULL** or truncate the information.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailclass11.lib.
DLL	Requires ail11.dll; ailclass11.dll.

MclassInquireEntry

Synopsis

Inquire about an entry in a dataset context.

Syntax

```
AIL_INT MclassInquireEntry(
    AIL_ID DatasetContextClassId, //in
    AIL_INT64 EntryIndex,         //in
    AIL_UUID EntryKey,            //in
    AIL_INT64 RegionIndex,        //in
    AIL_INT64 InquireType,        //in
    void *UserVarPtr              //out
)
```

Description

This function allows you to inquire about an entry in a dataset context using its index or key. You can typically control these settings, using MclassControlEntry().

Parameters

DatasetContextClassId

Specifies the identifier of the dataset about which to inquire settings. These contexts are allocated using MclassAlloc() with M_DATASET_IMAGES or M_DATASET_FEATURES.

EntryIndex

Specifies the entry to inquire about, using the entry's index. Set this parameter to one of the following values.

When you specify an index with EntryIndex, you must set the EntryKey parameter to M_DEFAULT_KEY.

> Note that EntryIndex and EntryKey cannot simultaneously be set to M_DEFAULT and M_DEFAULT_KEY respectively.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the index of an entry to inquire about

Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies that the index of an entry is not required.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the index of an entry.	c		e	f	g	a		i		j	k	l	m	n	b

EntryKey

Specifies the entry to inquire about, using the entry's unique key (AIL_UUID). Set this parameter to one of the following values.

When you specify an entry key with EntryKey, you must set the EntryIndex parameter to M_DEFAULT.

> Note that EntryIndex and EntryKey cannot simultaneously be set to M_DEFAULT and M_DEFAULT_KEY respectively.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the key (UUID) of an entry to inquire about

Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_KEY	Specifies that the unique key of an entry is not required.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_UUID Value	Specifies the unique key of an entry.	c		e	f	g	a		i		j	k	l	m	n	b

RegionIndex

Specifies which region in an entry to inquire. For an images dataset, there is always at least one region in the dataset, at index 0. For a features dataset, use M_DEFAULT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying which region to inquire

Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies that the function call refers to the entry, and not to a specific region within the entry.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_INDEX(AIL_INT RegionIndex)	Specifies the region to inquire, using its index. Refers to a specific region within the entry. Note that there is always at least one region in an entry. This value only applies to an images dataset context.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	RegionIndex Set this parameter to the following:															

			0 <= *Value* < M_NUMBER_OF_REGIONS	Specifies the index.	c		e	f	g	a		i		j	k	l	m	n	b

InquireType

Specifies the type of inquire.

See the Parameter associations section for possible values that can be specified.

UserVarPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]
- array of type AIL_ID [optionally, in C++: a reference to a std::vector<AIL_ID>]
- array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]
- array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]
- array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]
- array of type AIL_TEXT_CHAR [optionally, in C++: a reference to an AIL_STRING]
- AIL_DOUBLE
- AIL_ID
- AIL_INT
- AIL_INT32
- AIL_INT64
- AIL_UUID

Specifies the address in which to write the requested information.

See the Parameter associations section for possible values that can be returned.

Parameter associations

The tables below list possible values for the **InquireType** parameter and possible values returned to the **UserVarPtr** parameter.

- For an entry in a dataset context (images or features)
- For an entry or a region of an entry in a dataset context (images or features)

To inquire about an entry in a dataset, the InquireType parameter can be set to the following values. In this case, set the DatasetContextClassId parameter to the identifier of either an images dataset context or a features dataset context, unless otherwise specified, and set the RegionIndex parameter to M_DEFAULT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For an entry in a dataset context (images or features)																		
InquireType UserVarPtr - Possible values returned	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_ANOMALOUS +	Inquires whether an entry is considered anomalous. This is based on the entry's regions and their ground truths. If an entry's whole image region (M_WHOLE_IMAGE) has a ground truth, only that class is used to determine whether the entry is considered anomalous (if other regions exist, their ground truth classes are not checked). If the whole image region does not have a ground truth, all of the entry's regions are checked to determine whether the entry is considered anomalous. If any entry region checked is associated to an anomalous ground truth class, the entry is considered anomalous. If M_NO_REGION_PIXEL_CLASS is set to the index of an anomalous class, unlabeled pixels are also checked, which can lead to an entry being considered anomalous. This value only applies to an images dataset context.																
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that the entry is not considered anomalous.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that the entry is considered anomalous.	c		e	f	g	a		i		j	k	l	m	n	b
M_ANOMALY_DETECTION_PATH +	Inquires the file path to the anomaly detection scores files, such that M_ROOT_PATH + M_ANOMALY_DETECTION_PATH gives a valid path. If M_ROOT_PATH is empty, M_ANOMALY_DETECTION_PATH should be an absolute path, otherwise it should be a relative path to M_ROOT_PATH. This value only applies to an images dataset context.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*Path*") 1	Specifies the path.	c		e	f	g	a		i		j	k	l	m	n	b
M_ANOMALY_DETECTION_PATH_ABS +	Inquires the absolute path to the anomaly detection scores file which corresponds to the concatenation of M_ROOT_PATH + M_ANOMALY_DETECTION_PATH. This value only applies to an images dataset context.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*Path*") 1	Specifies the absolute path.	c		e	f	g	a		i		j	k	l	m	n	b

M_AUGMENTATION_SOURCE +	Inquires whether an entry is an augmented entry, according to the index. For entries added to a dataset by MclassPrepareData(), this index refers to the prepared original image should it be present in the dataset. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_NOT_AUGMENTED	Specifies that the entry is not an augmented entry.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNKNOWN	Specifies that the entry is augmented, however, the original data is not known.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies that the entry is augmented, and that it originates from (was performed with) the specified entry index. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_AUGMENTATION_SOURCE_KEY +	Inquires whether an entry is an augmented entry, according to the defined key (UUID). For entries added to a dataset by MclassPrepareData(), this key refers to the prepared original image should it be present in the dataset. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_UUID*	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL_KEY	Specifies that the entry is not an augmented entry.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNKNOWN_KEY	Specifies that the entry is augmented, and that the entry key (UUID) from which it was augmented is not known.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_UUID Value	Specifies that the entry is augmented, and that it originates from (was performed with) the specified key. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ENTRY_IMAGE_PATH +	Inquires the file path, such that M_ROOT_PATH + M_ENTRY_IMAGE_PATH gives a valid path. If M_ROOT_PATH is empty, M_ENTRY_IMAGE_PATH should be an absolute path, otherwise it should be a relative path to M_ROOT_PATH. This value only applies to an images dataset context. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*///DATASET///FilePath*") 1	Specifies the file path under the folder path from where the dataset context is restored, using MclassRestore(). INFO	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*FilePath*") 1	Specifies the file path.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENTRY_IMAGE_PATH_ABS +	Inquires the absolute path of the entry image, which is the concatenation of M_ROOT_PATH + M_ENTRY_IMAGE_PATH. This value only applies to an images dataset context.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*FilePath*") 1	Specifies the file path.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENTRY_KEY +	Inquires the unique identifier for an entry.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_UUID*	c		e	f	g	a		i		j	k	l	m	n	b
M_ENTRY_USER_STRING +	Inquires the user string that holds meta information. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*String*") 1	Specifies a string containing meta information. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ENTRY_WEIGHT +	Inquires the weight that indicates the importance of the feature entry. This value only applies to a features dataset context. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the weight, as a *double*.	c		e	f	g	a		i		j	k	l	m	n	b
M_EXTERNAL_SOURCE_KEY +	Inquires whether an entry has an external augmentation source, specified by its key (UUID). This value is set during a call to MclassPrepareData() and refers to the UUID of the source image in the source dataset. This value only applies to an images dataset context.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_UUID*	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL_KEY	Specifies that the entry has no external augmentation source. This is the default value for entries created outside of MclassPrepareData().	c		e	f	g	a		i		j	k	l	m	n	b
AIL_UUID Value	Specifies that the entry has an external augmentation source with this key (UUID).	c		e	f	g	a		i		j	k	l	m	n	b

M_NUMBER_OF_REGIONS +	Inquires the number of regions in this entry. The number of regions is equal to 1 plus the number of regions added, using MclassEntryAddRegion(). The number of regions starts at 1 because region 0 refers to the region representing the whole image (M_WHOLE_IMAGE region), which is always present. This value only applies to an images dataset context.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 1	Specifies the number of regions, as an *integer*.	c		e	f	g	a		i		j	k	l	m	n	b
M_RAW_DATA +	Inquires an array of feature values (set of features). This value only applies to a features dataset context. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies no values.	c		e	f	g	a		i		j	k	l	m	n	b
M_SEGMENTATION_PATH +	Inquires the file path to the segmentation scores files, such that M_ROOT_PATH + M_SEGMENTATION_PATH gives a valid path. If M_ROOT_PATH is empty, M_SEGMENTATION_PATH should be an absolute path, otherwise it should be a relative path to M_ROOT_PATH. If there is more than one file associated with this entry (because the number of classes is > 1024), the pipe delimiter is used between the files to differentiate them (for example, *GUID_0.mbufi|GUID_1.mbufi|GUID_2.mbufi*). This value only applies to an images dataset context.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*Path*") 1	Specifies the path.	c		e	f	g	a		i		j	k	l	m	n	b
M_SEGMENTATION_PATH_ABS +	Inquires the absolute path to the segmentation scores file which corresponds to the concatenation of M_ROOT_PATH + M_SEGMENTATION_PATH. If there is more than one file associated with this entry (because the number of classes is > 1024), the pipe delimiter is used between the files to differentiate them (for example, *Abs\Path\To\Segmentations\GUID_0.mbufi|Abs\Path\To\Segmentations\GUID_1.mbufi|Abs\Path\To\Segmentations\GUID_2.mbufi*). This value only applies to an images dataset context.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*Path*") 1	Specifies the absolute region mask path.	c		e	f	g	a		i		j	k	l	m	n	b
M_USER_CONFIDENCE +	Inquires how confident you are in the entry's ground truth (M_CLASS_INDEX_GROUND_TRUTH). This value only applies to an images dataset context. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the user confidence.	c		e	f	g	a		i		j	k	l	m	n	b

[1] If the value was passed in a variable, it won't be returned enclosed in AIL_TEXT().

To inquire about a region in an entry or to inquire about an entry, the InquireType parameter can be set to the following values. In this case, set the DatasetContextClassId parameter to the identifier of any dataset context, unless otherwise specified. For an images dataset context, set the RegionIndex parameter to M_REGION_INDEX(), and for a features dataset context, set the RegionIndex parameter to M_DEFAULT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For an entry or a region of an entry in a dataset context (images or features)																	
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_AUTHOR_NAME +	Inquires the author's name. SET															
UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*Name*") 1	Specifies the author's name.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLASS_INDEX_GROUND_TRUTH +	Inquires the index of the class definition that represents the entry's ground truth. If there is no ground truth index, M_CLASS_INDEX_GROUND_TRUTH returns nothing to UserVarPtr. To inquire if there is any ground truth index set, use M_CLASS_INDEX_GROUND_TRUTH + M_NB_ELEMENTS. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_DONT_CARE_CLASS	Specifies that pixels assigned to the region do not contribute to the training loss, when using descriptor based regions (that is, any region index other than 0). INFO	c		e	f	g	a		i		j	k	l	m	n	b

0 <= *Value* < M_NUMBER_OF_CLASSES	Specifies the class definition index (ground truth), as an *integer*.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_DESCRIPTOR_TYPE_BOX +	Inquires the number of bounding box-type descriptors. This value only applies to an images dataset context.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the number of bounding box-type descriptors.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_DESCRIPTOR_TYPE_MASK +	Inquires the number of mask-type descriptors. This value only applies to an images dataset context.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the number of mask-type descriptors.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_DESCRIPTOR_TYPE_POLYGON +	Inquires the number of polygon-type descriptors. This value only applies to an images dataset context.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the number of polygon-type descriptors.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_MASK_PATH +	Inquires the path of a region descriptor context, relative to M_ROOT_PATH. This value only applies to an images dataset context. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*Path*") 1	Specifies the path of the region descriptor context.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_MASK_PATH_ABS +	Inquires the absolute region mask path. This value only applies to an images dataset context.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*Path*") 1	Specifies the absolute region mask path.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_TYPE +	Inquires the type of the region. This value only applies to an images dataset context.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_WHOLE_IMAGE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DESCRIPTOR_BASED	Specifies that the region is descriptor based. Regions at any index other than 0 will be of this type.	c		e	f	g	a		i		j	k	l	m	n	b
M_WHOLE_IMAGE	Specifies that the region type of the entry is set to the whole image. Regions at index 0 are always of this type.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_USE +	Inquires whether a region is to be used. This value only applies to an images dataset context.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_IGNORE	Specifies that the region is to be ignored.	c		e	f	g	a		i		j	k	l	m	n	b
M_USE	Specifies that the region can be used.	c		e	f	g	a		i		j	k	l	m	n	b

[1] If the value was passed in a variable, it won't be returned enclosed in AIL_TEXT().

Combination value for M_RAW_DATA; M_CLASS_INDEX_GROUND_TRUTH.

You can add the following value to the above-mentioned values to determine the required array size (number of elements) to store the returned values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring the required array size (number of elements) to store the returned values															
InquireType combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned															
M_NB_ELEMENTS	Retrieves the required array size (number of elements) to store the returned values.														

	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_ANOMALY_DETECTION_PATH_ABS; M_AUGMENTATION_SOURCE; M_AUGMENTATION_SOURCE_KEY; M_SEGMENTATION_PATH_ABS; M_USER_CONFIDENCE; M_REGION_MASK_PATH; M_REGION_MASK_PATH_ABS; M_REGION_TYPE; M_REGION_USE.

You can add the following value to the above-mentioned values to determine the default value of an inquire type, regardless of the current value of the inquire type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring the default value of an inquire type																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_DEFAULT		Inquires the default value of the specified inquire type.															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_ANOMALY_DETECTION_PATH; M_ENTRY_IMAGE_PATH; M_ENTRY_IMAGE_PATH_ABS; M_ENTRY_USER_STRING; M_SEGMENTATION_PATH; M_AUTHOR_NAME.

You can add the following value to the above-mentioned values to get the size of a string.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring the size of a string																	
InquireType combination value		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_STRING_SIZE +		Inquires the number of characters in the string. This number accounts for every character, including the terminating null character.															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for the values listed in For an entry in a dataset context (images or features), For an entry or a region of an entry in a dataset context (images or features).

You can add one of the following values to the above-mentioned values to determine whether an inquire type is supported.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring whether an inquire type is supported																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_HAS_DEFAULT		Inquires whether the specified inquire type has a default value.															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_FALSE	Specifies that the inquire type does not have a default value.	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRUE	Specifies that the inquire type has a default value.	c		e	f	g	a		i		j	k	l	m	n	b
M_SUPPORTED		Inquires whether the specified inquire type is supported for the dataset entry.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_FALSE	Specifies that the inquire type is not supported.	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRUE	Specifies that the inquire type is supported.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for M_ANOMALOUS; M_ANOMALY_DETECTION_PATH_ABS; M_AUGMENTATION_SOURCE; M_ENTRY_KEY; M_NUMBER_OF_REGIONS; M_RAW_DATA; M_SEGMENTATION_PATH_ABS; M_USER_CONFIDENCE; M_CLASS_INDEX_GROUND_TRUTH; M_NUMBER_OF_DESCRIPTOR_TYPE_BOX; M_NUMBER_OF_DESCRIPTOR_TYPE_MASK; M_NUMBER_OF_DESCRIPTOR_TYPE_POLYGON; M_REGION_MASK_PATH; M_REGION_MASK_PATH_ABS; M_REGION_TYPE; M_REGION_USE; M_STRING_SIZE.

You can add one of the following values to the above-mentioned values to cast the requested results to the required data type.

> Note: In C++, when using a standard vector (std::vector) instead of an array, the values listed in this combination table must not be used. If you need to use a specific data type, instantiate the vector with the necessary data type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For casting the requested information to the required data type																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																

M_TYPE_AIL_DOUBLE	Casts the requested results to an *AIL_DOUBLE*.														
UserVarPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* **|** *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_ID	Casts the requested results to an *AIL_ID*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_ID [optionally, in C++: a reference to a std::vector<AIL_ID>]* **|** *address of an AIL_ID*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT	Casts the requested results to an *AIL_INT*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]* **|** *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT32	Casts the requested results to an *AIL_INT32*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]* **|** *address of an AIL_INT32*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT64	Casts the requested results to an *AIL_INT64*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]* **|** *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the requested information, cast to an *AIL_INT*. If the requested information does not fit into an *AIL_INT*, this function will return **M_NULL** or truncate the information.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailclass11.lib.
DLL	Requires ail11.dll; ailclass11.dll.

MclassPredict

Synopsis

Predict the class to which the target belongs or the position of objects to which each entry of a dataset belongs.

Syntax

```
void MclassPredict(
    AIL_ID ContextClassId,                //in
    AIL_ID TargetAilObjectId,             //in
    AIL_ID DatasetContextOrResultClassId, //out
    AIL_INT64 ControlFlag                 //in
)
```

Description

This function uses a trained classifier context to predict the class to which the target belongs or the position of objects in the target. For a trained CNN, segmentation, object detection, or anomaly detection classifier context, you can specify an image or an images dataset context as the target. Image classification and segmentation support buffers with up to 1024 bands. For a trained tree ensemble classifier context, you can specify an array of features or a features dataset context as the target. For a trained ONNX classifier, you can only specify an image.

This function is typically called with a target image (for image classification, segmentation, object detection, or anomaly detection) or with a target array of features (for feature classification). In these cases, results are stored in a result buffer (DatasetContextOrResultClassId) and retrievable using MclassGetResult().

You can also call this function with a target dataset. In this case, the predicted data is stored in a dataset context (DatasetContextOrResultClassId). Calling **MclassPredict**() with a dataset can be done, for example, to predict the class that represents the dataset's entries, which you can retrieve using MclassGetResultEntry() with M_BEST_CLASS_INDEX. You can then use these predictions to help manually define the ground truth of the entries during training.

Before calling **MclassPredict**(), the classifier context must be in a preprocessed state (MclassPreprocess()) and it must be trained (MclassTrain()). To inquire this information, use M_PREPROCESSED and M_CLASSIFIER_STATUS.

If your target is a dataset context, you can hook functions to prediction events, using MclassHookFunction() (for example, with M_PREDICT_ENTRY). To get information about the events that caused the hook-handler function to execute, call MclassGetHookInfo().

Classification contexts and result buffers must have been previously allocated on the required system using MclassAlloc() and MclassAllocResult(), or restored from disk using MclassRestore().

Note, you can also call **MclassPredict**() with an images dataset context to perform assisted labeling (that is, label images in an existing dataset).

By default, **MclassPredict**() uses the default predict engine established by the **Aurora Imaging Configurator** utility. You can modify this by calling MclassControl() with M_PREDICT_ENGINE or in **Aurora Imaging Configurator**.

> To take advantage of all available hardware resources, such as accelerating prediction using OpenVINO and CUDA, you should install the required Aurora Imaging Library add-on. For more information, see the Aurora Imaging Library add-ons and updates subsection of the Requirements, recommendations, and troubleshooting section of Chapter 47: Machine learning with the Aurora Imaging Library Classification module.

Parameters

ContextClassId

Specifies the identifier of the trained classifier context to use for the prediction operation. You can specify a trained CNN classifier context (M_CLASSIFIER_CNN_PREDEFINED), a trained segmentation classifier context (M_CLASSIFIER_SEG_PREDEFINED), a trained object detection classifier context (M_CLASSIFIER_DET_PREDEFINED), a trained anomaly detection classifier (M_CLASSIFIER_ANO_PREDEFINED), a trained tree ensemble classifier context (M_CLASSIFIER_TREE_ENSEMBLE), or a trained ONNX classifier context (M_CLASSIFIER_ONNX).

TargetAilObjectId

Specifies the identifier of the target to classify. Set this parameter to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the target to classify																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ArrayBufId	Specifies the identifier of a one-dimensional 32-bit float array that holds a list of features. In this case, the prediction operation will predict the best class to which the array of features belongs. The number of features in the array must be the same as the number of trained features in the dataset used to train the classifier context (DatasetContextOrResultClassId). To specify an array, you must set the ContextClassId parameter to the identifier of a trained tree ensemble classifier context.	c		e	f	g	a		i		j	k	l	m	n	b
ContextDatasetId	Specifies the identifier of a features or images dataset context. For CNN and tree ensemble classifiers, the prediction operation will predict the best class to which each dataset entry (feature list or image) belongs. For segmentation classifiers, the prediction operation will predict the best class to which each pixel belongs in a image entry. For object detection classifiers, the prediction operation will detect the positions of each object in an image entry. For anomaly detection classifiers, the prediction operation will predict the anomaly score of the target image as well as the anomaly score for each pixel within the image. Image classification and segmentation support buffers with up to 1024 bands. For image datasets, you must set the ContextClassId parameter to the identifier of a trained CNN, segmentation, object detection, or anomaly detection classifier context. For feature datasets, you must set the ContextClassId parameter to the identifier of a trained tree ensemble.	c		e	f	g	a		i		j	k	l	m	n	b

ImageBufId	Specifies the identifier of a 1- or 3-band target image. Image classification and segmentation support buffers with up to 1024 bands. When ContextClassId is a trained CNN classifier context, the prediction operation will predict the best class to which the target image belongs. When ContextClassId is a trained segmentation classifier context, the prediction operation will predict the class of each pixel within the target image. When ContextClassId is a trained object detection classifier context, the prediction operation will detect the position of each object within the image. When ContextClassId is a trained anomaly detection classifier context, the prediction operation will predict the anomaly score of the target image as well as the anomaly score for each pixel within the image. You must set the ContextClassId parameter to the identifier of a trained CNN, segmentation, object detection, anomaly detection or ONNX classifier context. When predicting for image classification (using M_ICNET_...), the target image size must be the same as the training images. When predicting for segmentation (using M_CSNET_...), or if you are using a predefined legacy classifier (M_FCNET_...), the target image size can be bigger than the training images (this requires calling MclassControl() with M_TARGET_IMAGE_SIZE_X and M_TARGET_IMAGE_SIZE_Y). By default, Aurora Imaging Library considers the size of the target image to be the same as the source layer size.	c		e	f	g	a		i		j	k	l	m	n	b

DatasetContextOrResultClassId

Specifies the identifier of the result buffer or dataset in which to write the results of the prediction operation. Set this parameter to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the result buffer or dataset in which to write the results of the prediction																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ContextDatasetId	Specifies the identifier of a features dataset context (M_DATASET_FEATURES) or images dataset context (M_DATASET_IMAGES). You must call **MclassPredict**() with a features or images dataset context as your target (TargetAilObjectId). The dataset context specified by the DatasetContextOrResultClassId parameter must either be empty, or be the same dataset as your target. If the destination is an images dataset context, Aurora Imaging Library might modify its root path (M_ROOT_PATH), if one is specified, to maintain the dataset's coherence. For example, dataset paths should all be either relative or absolute.	c		e	f	g	a		i		j	k	l	m	n	b
ResultBufPredictId	Specifies the identifier of a CNN prediction result buffer (M_PREDICT_CNN_RESULT), a segmentation prediction result buffer (M_PREDICT_SEG_RESULT), an object detection prediction result buffer (M_PREDICT_DET_RESULT), an anomaly detection prediction result buffer (M_PREDICT_ANO_RESULT) a tree ensemble prediction result buffer (M_PREDICT_TREE_ENSEMBLE_RESULT), or an ONNX prediction result buffer (M_PREDICT_ONNX_RESULT). In this case, you must call **MclassPredict**() with an image or an array of features as your target (TargetAilObjectId).	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailclass11.lib.
DLL	Requires ail11.dll; ailclass11.dll.

MclassPrepareData

Synopsis

Prepare images in a dataset, or individual images, for training or prediction.

Syntax

```
void MclassPrepareData(
    AIL_ID PrepareDataContextClassId,     //in
    AIL_ID SrcDatasetContextOrImageBufId, //in
    AIL_ID DstDatasetContextOrImageBufId, //out
    AIL_ID ClassifierContextClassId,      //in
    AIL_INT64 ControlFlag                 //in
)
```

Description

This function creates prepared (modified) versions of images in a dataset for training (MclassTrain()), or an individual image for prediction (MclassPredict()). You can, for example, use this function to resize or crop images, or add augmented images to a dataset. Some preparations are only available for training.

MclassPrepareData() requires a data preparation context (MclassAlloc() with M_PREPARE_IMAGES_CNN, M_PREPARE_IMAGES_DET or M_PREPARE_IMAGES_SEG) that is preprocessed (MclassPreprocess()). To inquire the context's preprocessing state, call MclassInquire() with M_PREPROCESSED. Note that **MclassPrepareData**() does not support preparing signed images. Image classification and segmentation support buffers with up to 1024 bands.

The data preparation context (PrepareDataContextClassId) holds the settings with which to modify the specified source (SrcDatasetContextOrImageBufId). To change the data preparation context settings, call MclassControl(). For example, you can use the M_SIZE_... controls to set the size of the images that **MclassPrepareData**() produces, or you can use the M_AUGMENT_NUMBER_ABSOLUTE control to set the number of augmented entries to add to the destination dataset.

> You can consider data preparation as taking the specified source data (SrcDatasetContextOrImageBufId), modifying it (according to your data preparation settings), and copying the modified version into the specified destination (DstDatasetContextOrImageBufId). One image in your source can result in multiple modified images in your destination, depending on your data preparation settings. Source data is not modified. Class definitions and authors are copied from the source dataset to the destination, if they do not exist in the destination. If using a newly allocated destination dataset for segmentation, control values for M_NO_REGION_PIXEL_CLASS, M_NO_CLASS_DRAW_COLOR, and M_DONT_CARE_CLASS_DRAW_COLOR are copied to the destination; otherwise, these values must be the same for the source and destination datasets.

To prepare your data with specific, preset augmentations, you can call MclassControl() and enable the M_PRESET... augmentation operation setting. Some preset augmentation settings are not available for binary (1-bit) and grayscale (1-band) images. Alternatively, you can access additional types of augmentation operations using the data preparation context's internal augmentation context. This context is automatically managed by Aurora Imaging Library (you need not allocate it or free it) and is an internal version of the image processing context for augmentation (that is, MimAlloc() with M_AUGMENTATION_CONTEXT).

To specify the additional augmentations, you must first inquire the identifier of this internal augmentation context, by calling MclassInquire() with M_AUGMENT_CONTEXT_ID. You can then use that augmentation context identifier with MimControl() and enable the augmentation operation (M_AUG_..._OP).

Aurora Imaging Library only performs augmentations on a source dataset (SrcDatasetContextOrImageBufId); if you specify a source image buffer, augmentation settings are ignored (cropping/resizing settings are applied). **MclassPrepareData**() only adds augmented images to the destination dataset if at least one augmentation operation is enabled; by default; no augmentations are enabled.

The modified (cropped/resized or augmented) images that this function produces are placed in the specified dataset context or image buffer (DstDatasetContextOrImageBufId), depending on the source you want to prepare (SrcDatasetContextOrImageBufId). The images are also placed in the destination folder, specified by the M_PREPARED_DATA_FOLDER control. By default, each time you call this function, a new *PrepareX* folder is created, where *X* is the next available value to create a unique folder name. You can change this behavior with the M_DESTINATION_FOLDER_MODE control.

All the images that this function produces (and copies into the prepared data folder) will have a MIM file extension and be named as follows: *OriginalName_Prp_AugmentationNumber*.mim. For example, if **MclassPrepareData**() produces three augmentations of *Abyssinian.mim*, they would be named *Abyssinian_Prp_1.mim*, *Abyssinian_Prp_2.mim*, and *Abyssinian_Prp_3.mim*. The suffix '0' is reserved for images that are prepared but not augmented (for example, *Abyssinian_Prp_0.mim* would be a cropped/resized copy of the original image).

To hook functions to data preparation events, call MclassHookFunction().

Parameters

PrepareDataContextClassId

Specifies the identifier of the data preparation context that this function uses to modify the source image data. This context must have been previously allocated on the required system using MclassAlloc() with M_PREPARE_IMAGES_CNN, M_PREPARE_IMAGES_DET, or M_PREPARE_IMAGES_SEG and it must be in a preprocessed state before calling **MclassPrepareData**().

SrcDatasetContextOrImageBufId

Specifies the identifier of the source images dataset context or image buffer. You must have previously allocated the specified source on the required system using either MclassAlloc() with M_DATASET_IMAGES or MbufAlloc...() with M_IMAGE.

If you specify an images dataset context, all specified augmentations are applied. If you specify an image buffer, augmentations are not applied; however, any specified cropping or resizing operations are performed, such as those set with the M_SIZE_... controls. This is typically done to make images conform to those used during the training process.

DstDatasetContextOrImageBufId

Specifies the identifier of the destination images dataset context or image buffer. The type of destination must be same as the source (SrcDatasetContextOrImageBufId); that is, the destination must be an image dataset context if the source is an image dataset context, and an image buffer if the source is an image buffer. You cannot specify the same dataset or image for both the SrcDatasetContextOrImageBufId and DstDatasetContextOrImageBufId parameters.

If you specify a destination image buffer, it must have the same number of bands, data type, and depth as the source image buffer. The size of the destination buffer is established by the predefined CNN, predefined object detection, or predefined segmentation classifier context that you are using, as well as the mode (M_SIZE_MODE) and size controls (M_SIZE_X and M_SIZE_Y) of the data preparation context. Aurora Imaging Library chooses a default size that is typically suitable, however the requirements of your application might require you to change it.

If you specify an images dataset context, all specified augmentations are applied. If you specify an image buffer, augmentations are not applied; however, any specified cropping or resizing operations are performed, such as those set with the M_SIZE_... controls. This is typically done to make images conform to those used in the training dataset.

If you specify an images dataset context, Aurora Imaging Library might modify its root path (M_ROOT_PATH), if one is specified, to maintain the dataset's coherence. For example, dataset paths should all be either relative or absolute.

ClassifierContextClassId

Specifies the identifier of the predefined CNN, predefined object detection, or predefined segmentation classifier context that this function uses to prepare your image data. This context is allocated using MclassAlloc() with M_CLASSIFIER_CNN_PREDEFINED, M_CLASSIFIER_DET_PREDEFINED, or M_CLASSIFIER_SEG_PREDEFINED. To not use a classifier context, or if the size mode specified by MclassControl() is M_USER_DEFINED, you must set this parameter to **M_NULL**.

If you specify an untrained CNN, object detection, or segmentation classifier context, it must hold an ICNet, ODNet, or CSNet classifier (if necessary, it can also hold a legacy FCNet classifier). Aurora Imaging Library uses the sizing requirements of the classifier to establish the image size. If you specify a trained CNN, object detection, or segmentation classifier context, Aurora Imaging Library uses the size of the source layer to establish the image size. If you do not specify a classifier context, Aurora Imaging Library establishes the size of the destination images using the size mode control (M_SIZE_MODE) and, depending on its specified value, the resizing controls (M_RESIZE_SCALE_FACTOR, M_SIZE_X, and M_SIZE_Y), or the source image size. If a trained classifier is passed, you must set M_SIZE_MODE to M_AUTO.

Typically, the ClassifierContextClassId parameter is set to an untrained classifier when preparing datasets (SrcDatasetContextOrImageBufId), and to a trained classifier when preparing images (SrcDatasetContextOrImageBufId). This function is usually called with a trained classifier when you want to modify images to conform to the already established training data.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailclass11.lib.
DLL	Requires ail11.dll; ailclass11.dll.

MclassPreprocess

Synopsis

Preprocess a classification context.

Syntax

```
void MclassPreprocess(
    AIL_ID ContextClassId, //in
    AIL_INT64 ControlFlag  //in
)
```

Description

This function preprocesses a classification context for data preparation (MclassPrepareData()), training (MclassTrain()), statistics calculation (MclassStatCalculate()), or prediction (MclassPredict()). You must call **MclassPreprocess**() before the first call to any of these functions.

During preprocessing, Aurora Imaging Library analyzes the content of the context and makes internal refinements to execute an optimized and robust operation. Changes to a context or any of its content often require preprocessing the context again. To inquire whether you should preprocess the context, call MclassInquire() with M_PREPROCESSED. The context must be in a preprocessed state before calling the data preparation, training, or prediction operation.

Saving a context does not save preprocessing changes. Upon restoration, you must preprocess the context again before calling MclassPrepareData(), MclassTrain(), MclassStatCalculate(), or MclassPredict().

Parameters

ContextClassId

Specifies the identifier of the classification context to preprocess. The classification context must have been previously allocated on the required system using MclassAlloc() with M_PREPARE_IMAGES_CNN, M_PREPARE_IMAGES_DET, M_PREPARE_IMAGES_SEG, M_TRAIN_CNN, M_TRAIN_DET, M_TRAIN_SEG, M_TRAIN_ANO, M_TRAIN_TREE_ENSEMBLE, M_CLASSIFIER_CNN_PREDEFINED, M_CLASSIFIER_DET_PREDEFINED, M_CLASSIFIER_SEG_PREDEFINED, M_CLASSIFIER_ANO_PREDEFINED, or M_CLASSIFIER_TREE_ENSEMBLE.

ControlFlag

Specifies whether to preprocess the classification context. Set this parameter to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying whether to preprocess the context																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Preprocesses the context.	c		e	f	g	a		i		j	k	l	m	n	b
M_RESET	Un-preprocesses the context. Un-preprocessing the context can be useful if you want to conserve system memory within an application and preserve context settings.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailclass11.lib.
DLL	Requires ail11.dll; ailclass11.dll.

MclassRestore

Synopsis

Restore an Aurora Imaging Library Classification context (classifier, dataset, data preparation, or training) from disk.

Syntax

```
AIL_ID MclassRestore(
    AIL_CONST_TEXT_PTR FileName, //in
    AIL_ID SysId,                //in
    AIL_INT64 ControlFlag,       //in
    AIL_ID *ContextClassIdPtr    //out
)
```

Description

This function restores an Aurora Imaging Library Classification context (classifier, dataset, data preparation, or training) that was previously saved to a file, using MclassSave() or MclassStream().

This function restores all of the context's settings that were in effect when it was saved. A restored context is not preprocessed; if the context requires preprocessing, you must call MclassPreprocess() before calling MclassTrain() or MclassPredict().

Note, it is possible to restore a pretrained CNN, object detection, segmentation, or anomaly detection classifier context that Aurora Imaging has built and trained to address your specific requirements (as opposed to a predefined CNN, object detection, segmentation, anomaly detection classifier context that you have trained). For more information about having Zebra provide this service to you, contact customer support.

When the restored classifier, dataset, data preparation, or training context is no longer required, release it using MclassFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the context's lifetime and you must not manually free it.

Parameters

FileName

Specifies the name and path of the file from which to restore the classification context (classifier, dataset, data preparation, or training). The function handles (internally) the opening and closing of the file. For easier use with other Aurora Imaging software products, classifier context, data preparation context, and training context files should have an MCLASS extension. Dataset context files should have an MCLASSD extension.

Set this parameter to one of the values below:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the file name and path of the context to restore																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which you can interactively specify the drive, directory, and name of the file.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*FileName*") [1]	Specifies the drive, directory, and name of the classification context file (for example, "*C:\mydirectory\WePutTheArtInArtificialIntelligence.mclass*"). Classifier context, data preparation context, and training context files should have an MCLASS extension. Dataset context files should have an MCLASSD extension. To specify a context file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\WePutTheArtInArtificialIntelligence.mclass*").	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

SysId

Specifies the identifier of the system on which to restore the classification context (classifier, dataset, data preparation, or training) to disk.

Set this parameter to one of the values below:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the system identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

ContextClassIdPtr

Specifies the address of the variable in which to write the classifier, dataset, data preparation, or training context identifier or specifies the data type that the function should use to return the context identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated classifier, dataset, data preparation, or training data context ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated classifier, dataset, data preparation, or training data context ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_CLASS_ID* is returned instead of a standard Aurora Imaging Library identifier. > This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the classifier, dataset, data preparation, or training data context (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the classifier context identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated classifier context. If restoration fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the data preparation context identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated data preparation context. If restoration fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the dataset context identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated dataset context. If restoration fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the training context identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated training context. If restoration fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the classifier context identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_CLASS_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailclass11.lib.
DLL	Requires ail11.dll; ailclass11.dll.

MclassSave

Synopsis

Save an Aurora Imaging Library Classification context (classifier, dataset, data preparation, or training) to a file.

Syntax

```
void MclassSave(
    AIL_CONST_TEXT_PTR FileName, //in
    AIL_ID ContextClassId,       //in
    AIL_INT64 ControlFlag        //in
)
```

Description

This function saves all the information about the previously allocated Aurora Imaging Library Classification context (classifier, dataset, data preparation, or training) to disk. This information can be reloaded, using MclassRestore() or MclassStream().

When you save the context, the preprocessing changes are not saved. Upon restoration, you must preprocess the context again.

Parameters

FileName

Specifies the name and path of the file in which to save the classification context (classifier, dataset, data preparation, or training). The function internally handles the opening and closing of this file. For easier use with other Aurora Imaging software products, classifier contexts, data preparation contexts, and training context files should have an MCLASS extension. Dataset context files should have an MCLASSD extension.

Set this parameter to one of the values below:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the file name and path																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which you can interactively specify the drive, directory, and name of the file.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("FileName") [1]	Specifies the drive, directory, and name of the classification context file (for example, "*C:\mydirectory\WePutTheArtInArtificialIntelligence.mclass*"). Classifier context, data preparation context, and training context files should have an MCLASS extension. Dataset context files should have an MCLASSD extension. To specify a context file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\WePutTheArtInArtificialIntelligence.mclass*").	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

ContextClassId

Specifies the identifier of the classification context to save (that is, a classifier context, a dataset context, a data preparation context, or a training context).

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailclass11.lib.
DLL	Requires ail11.dll; ailclass11.dll.

MclassSetHookInfo

Synopsis

Set information about a classification object event.

Syntax

```
void MclassSetHookInfo(
    AIL_ID EventId,      //in
    AIL_INT64 InfoType,  //in
    AIL_DOUBLE InfoValue //in
)
```

Description

This function allows you to set information about the event that caused the hook-handler function to be called. **MclassSetHookInfo**() must only be called within the scope of a classification object hook-handler function (see MclassHookFunction()).

Parameters

EventId

Specifies the classification object event identifier received by the hook-handler function. See MclassHookFunction() for more information. The event will be of the type passed to the HookType parameter of the hook-handler function.

InfoType

Specifies the type of information to set.

See the Parameter associations section for possible values that can be specified.

InfoValue

Specifies the required value for the type of information to set.

See the Parameter associations section for possible values that can be specified.

Parameter associations

The table below lists possible values for the **InfoType** and **InfoValue** parameters.

- For a data preparation context using PREPARE_ENTRY_PRE

To set information about the event that caused the hook-handler function to be called, the InfoType and corresponding InfoValue parameter settings can be set to one of the following values. These settings are only available if the hook-handler function was called due to an M_PREPARE_ENTRY_PRE event type on a data preparation context object.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a data preparation context using PREPARE_ENTRY_PRE																		
InfoType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	InfoValue																	
M_CENTER_X		Sets the X-coordinate of the center position in the given image.																
	Value >= 0	Specifies the center's X-coordinate.	c		e	f	g	a		i		j	k	l	m	n	b
M_CENTER_Y		Sets the Y-coordinate of the center position in the given image.	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0	Specifies the center's Y-coordinate.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailclass11.lib.
DLL	Requires ail11.dll; ailclass11.dll.

MclassSplitDataset

Synopsis

Split the source dataset context into two destination dataset contexts.

Syntax

```
void MclassSplitDataset(
    AIL_ID SplitContextClassId,            //in
    AIL_ID SrcDatasetContextClassId,       //in
    AIL_ID DstFirstDatasetContextClassId,  //out
    AIL_ID DstSecondDatasetContextClassId, //out
    AIL_DOUBLE Percentage,                 //in
    AIL_ID SplitResultClassId,             //in
    AIL_INT64 ControlFlag                  //in
)
```

Description

This function splits the source dataset context into two destination dataset contexts, according to a percentage. This is typically done to split all of your data (SrcDatasetContextClassId) into the training dataset context (DstFirstDatasetContextClassId) and the development dataset context (DstSecondDatasetContextClassId), which you will use with MclassTrain() to train your classifier context. This function ensures that the proper portion of entries for each class in the source dataset context are in both the first and second destination dataset contexts.

You can consider the split to be a type of copy of dataset entries from the source dataset to either the first or second destination dataset. All source dataset entries are copied (including class definitions) and unaltered. Entries already in the destination are overwritten.

All datasets (source and destinations) must be for the same classification task; that is, you must have allocated all of them with either M_DATASET_IMAGES (for image classification, object detection, segmentation, or anomaly detection) or M_DATASET_FEATURES (for feature classification).

To establish the first and second destination dataset contexts, Aurora Imaging Library randomly selects source entries for the split. Typically, a different random selection of entries is chosen each time you call this function. To select the same random entries, use M_SPLIT_..._FIXED_SEED.

If the source dataset context contains augmented entries, they can end up in both the first and second destination dataset contexts. In this case, you must not use one of these destinations as your development dataset context or testing dataset context, since only the training dataset context can contain augmented entries. To inquire whether the source dataset contexts have augmented entries, use M_NUMBER_OF_AUGMENTED_ENTRIES.

An augmented entry is never split from the source entry with which it was augmented. This can cause the percentage of entries in the first and second destination dataset contexts to differ from the specified percentage. If you do not have augmented data, source entries are only copied to one destination.

Parameters

SplitContextClassId

Specifies the predefined split classification context. This predefined context establishes whether to use a different random selection of entries to split the source dataset context, or to use the random selection of entries associated to a fixed seed.

Set this parameter to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the predefined split classification context																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_SPLIT_ANO_CONTEXT_DEFAULT	Specifies to use a different random selection of entries from the source dataset context to establish the first and second destination dataset contexts, for anomaly detection.	c		e	f	g	a		i		j	k	l	m	n	b
M_SPLIT_ANO_CONTEXT_FIXED_SEED	Specifies to use the random selection of entries that is associated to a fixed seed, for anomaly detection. In this case, the first and second destination dataset contexts are established using the same random selection of entries from the source dataset context. This allows you to repeat the same split operation, provided you make an identical call to **MclassSplitDataset**() (that is, you specify the same dataset contexts and split percentage).	c		e	f	g	a		i		j	k	l	m	n	b
M_SPLIT_CONTEXT_DEFAULT	Specifies to use a different random selection of entries from the source dataset context to establish the first and second destination dataset contexts, for image classification (CNN) or feature classification (tree ensemble).	c		e	f	g	a		i		j	k	l	m	n	b
M_SPLIT_CONTEXT_FIXED_SEED	Specifies to use the random selection of entries that are associated to a fixed seed, for image classification (CNN) or feature classification (tree ensemble). In this case, the first and second destination dataset contexts are established using the same random selection of entries from the source dataset context. This allows you to repeat the same split operation, provided you make an identical call to **MclassSplitDataset**() (that is, you specify the same dataset contexts and split percentage).	c		e	f	g	a		i		j	k	l	m	n	b
M_SPLIT_DET_CONTEXT_DEFAULT	Specifies to use a different random selection of entries from the source dataset context to establish the first and second destination dataset contexts, for object detection.	c		e	f	g	a		i		j	k	l	m	n	b
M_SPLIT_DET_CONTEXT_FIXED_SEED	Specifies to use the random selection of entries that is associated to a fixed seed, for object detection. In this case, the first and second destination dataset contexts are established using the same random selection of entries from the source dataset context. This allows you to repeat the same split operation, provided you make an identical call to **MclassSplitDataset**() (that is, you specify the same dataset contexts and split percentage).	c		e	f	g	a		i		j	k	l	m	n	b
M_SPLIT_SEG_CONTEXT_DEFAULT	Specifies to use a different random selection of entries from the source dataset context to establish the first and second destination dataset contexts, for segmentation.	c		e	f	g	a		i		j	k	l	m	n	b

M_SPLIT_SEG_CONTEXT_FIXED_SEED	Specifies to use the random selection of entries that is associated to a fixed seed, for segmentation. In this case, the first and second destination dataset contexts are established using the same random selection of entries from the source dataset context. This allows you to repeat the same split operation, provided you make an identical call to **MclassSplitDataset**() (that is, you specify the same dataset contexts and split percentage).	c		e	f	g	a		i		j	k	l	m	n	b

SrcDatasetContextClassId

Specifies the identifier of the source dataset context.

When using **MclassSplitDataset**() to split a dataset with anomalous entries (that is, the source dataset has classes with M_ANOMALOUS set to M_TRUE), all anomalous entries will be in the second destination dataset (DstSecondDatasetContextClassId). In this case, the Percentage parameter controls what percentage of good (non-anomalous) entries go into the first and second destination datasets. Note that datasets with anomalous entries cannot be used for training.

DstFirstDatasetContextClassId

Specifies the identifier of the first destination dataset context.

DstSecondDatasetContextClassId

Specifies the identifier of the second destination dataset context.

Percentage

Specifies the percentage of entries in the source dataset context that should be in the first destination dataset context. The remaining entries in the source dataset context will be in the second destination dataset context. The percentage value can range between 0.0 to 100.0, inclusive.

SplitResultClassId

Reserved for future expansion and must be set to **M_NULL**.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailclass11.lib.
DLL	Requires ail11.dll; ailclass11.dll.

MclassStatCalculate

Synopsis

Calculates statistics on the datasets.

Syntax

```
void MclassStatCalculate(
    AIL_ID StatContextClassId,    //in
    AIL_ID DatasetContextClassId, //in
    AIL_INT64 EntryIndex,         //in
    AIL_UUID EntryKey,            //in
    AIL_ID StatResultClassId,     //out
    AIL_INT64 ControlFlag         //in
)
```

Description

This function calculates statistics on the classification datasets. This requires specifying a statistics classification context, which you must have allocated using MclassAlloc() with M_STAT_ANO, M_STAT_CNN, M_STAT_DET, M_STAT_SEG, or M_STAT_TREE_ENSEMBLE. To control a statistics classification context, use MclassControl().

You can calculate statistics for specific dataset entries, using the EntryIndex or EntryKey parameters, if you are specifying M_STAT_DET, M_STAT_SEG, or pixel-level M_STAT_ANO.

After calling **MclassStatCalculate**(), you can obtain the statistics from the result buffer, using MclassGetResultStat().

For datasets with more than 256 class definitions, using default colors will result in repeated colors.

Before calling **MclassStatCalculate**(), the statistics classification context must be in a preprocessed state (MclassPreprocess()). To inquire this, call MclassInquire() with M_PREPROCESSED.

Parameters

StatContextClassId

Specifies the identifier of a statistics classification context. This context must have been previously allocated on the required system using MclassAlloc() with M_STAT_ANO, M_STAT_CNN, M_STAT_DET, M_STAT_SEG, or M_STAT_TREE_ENSEMBLE.

DatasetContextClassId

Specifies the identifier of the images or features dataset on which to calculate statistics. These contexts are allocated using MclassAlloc() with M_DATASET_IMAGES or M_DATASET_FEATURES. Note that M_STAT_ANO, M_STAT_CNN, M_STAT_DET, M_STAT_SEG require an images dataset, and M_STAT_TREE_ENSEMBLE requires a feature dataset.

EntryIndex

Specifies the index of the dataset entry for which to calculate statistic results. This is only possible when using M_STAT_DET, M_STAT_SEG, and when calculating pixel level statistics for M_STAT_ANO.

When you specify an index with EntryIndex, you must set the EntryKey parameter to M_DEFAULT_KEY.

> Note that both EntryIndex and EntryKey cannot be set to M_DEFAULT and M_DEFAULT_KEY simultaneously.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the index of an entry																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies that the index of an entry is not required.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the index of an entry.	c		e	f	g	a		i		j	k	l	m	n	b

EntryKey

Specifies the key (*AIL_UUID*) of the entry for which to calculate statistic results. This is only possible when using M_STAT_DET, M_STAT_SEG, and when calculating pixel level statistics for M_STAT_ANO.

When you specify an entry key with EntryKey, you must set the EntryIndex parameter to M_DEFAULT.

> Note that both EntryIndex and EntryKey cannot be set to M_DEFAULT and M_DEFAULT_KEY simultaneously.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the unique key of an entry																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_KEY	Specifies that the unique key of an entry is not required.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the unique key of an entry.	c		e	f	g	a		i		j	k	l	m	n	b

StatResultClassId

Specifies the identifier of the statistics classification result buffer in which to write the results of the statistics calculation. The statistics classification result buffer must have been previously allocated on the required system using MclassAllocResult() with M_STAT_ANO_RESULT, M_STAT_CNN_RESULT, M_STAT_DET_RESULT, M_STAT_SEG_RESULT, or M_STAT_TREE_ENSEMBLE_RESULT.

The task related to the statistics result buffer (StatResultClassId) and the statistics classification context (StatContextClassId) must be the same. For example, you can only specify

the identifier of a statistics classification result buffer for anomaly detection (M_STAT_ANO_RESULT) if you specify the identifier of a statistics classification context for anomaly detection (M_STAT_ANO).

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailclass11.lib.
DLL	Requires ail11.dll; ailclass11.dll.

MclassStream

Synopsis

Load, restore, or save an Aurora Imaging Library Classification context (classifier, dataset, data preparation, or training) from/to a file or a memory stream.

Syntax

```
void MclassStream(
    AIL_TEXT_PTR MemPtrOrFileName, //in-out
    AIL_ID SysId,                  //in
    AIL_INT64 Operation,           //in
    AIL_INT64 StreamType,          //in
    AIL_DOUBLE Version,            //in
    AIL_INT64 ControlFlag,         //in
    AIL_ID *ContextClassIdPtr,     //in-out
    AIL_INT *SizeByteVarPtr        //out
)
```

Description

This function can load or restore an Aurora Imaging Library Classification context (classifier, dataset, data preparation, or training) from a file or memory stream. All of the context settings that were in effect when the context was saved will be restored. A loaded or restored context is not preprocessed.

This function can also save the context to a file or memory stream. All information about the previously allocated context is saved. Preprocessing changes are not saved.

To inquire the number of bytes necessary to save the context to memory stream, first call **MclassStream**() with M_INQUIRE_SIZE_BYTE.

The content saved to memory stream is equivalent to the content saved to file. In addition, any file saved using this function is equivalent to a file saved with MclassSave().

You can use this and other Aurora Imaging Library stream functions, for example, to save all required Aurora Imaging Library objects, as well as any other custom data, for your application to a memory stream. Once in a memory stream, you can write the stream to a single file or transfer it over a network. You are responsible for concatenating the streams and for saving the stream to file.

Using **MclassStream**(), you can choose to save a backwards-compatible version of the classification context, which will work using a version of Aurora Imaging Library that is up to one major release older than the current version (depending on which version is specified). However, all settings and features unique to the higher version will be ignored when restored using the lower version. Besides saving backwards-compatible versions, you can also load or restore classification contexts saved using Aurora Imaging Library version 10.0 or above. Settings that do not exist in the lower version will be filled with default values when the classification context is loaded or restored.

Parameters

MemPtrOrFileName
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- AIL_TEXT_PTR [optionally, in C++: a reference to an AIL_STRING]
- Optionally, in C++: a reference to a std::vector of type AIL_UINT8

Specifies the file or memory stream.

See the Parameter associations section for possible values that can be returned/specified.

SysId

Specifies the identifier of the system on which to restore the classification context.

See the Parameter associations section for possible values that can be specified.

Operation

Specifies the operation to perform on the classification context.

See the Parameter associations section for possible values that can be specified.

StreamType

Specifies the type of stream in which to store/from which to restore the classification context. Set this parameter to one of the values below:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of stream																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FILE	Specifies a file stream.	c		e	f	g	a		i		j	k	l	m	n	b
M_MEMORY	Specifies a memory stream. You are responsible for allocating a block of memory for the stream.	c		e	f	g	a		i		j	k	l	m	n	b

Version

Specifies the Aurora Imaging Library version of the classification context.

See the Parameter associations section for possible values that can be specified.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

ContextClassIdPtr

Specifies the address of the variable in which to write or from which to read the identifier of the classification context (that is, a classifier, a dataset, a data preparation, or a training context).

See the Parameter associations section for possible values that can be returned/specified.

SizeByteVarPtr

Specifies the address of the variable in which to write the size of the classification context, in bytes. If size is not required, set this parameter to **M_NULL**.

> The size of the context varies depending on the Aurora Imaging Library version specified.

Parameter associations

The table below lists possible values for the **MemPtrOrFileName**, **SysId**, **Operation**, **Version**, and **ContextClassIdPtr** parameters.

- For performing the stream operation

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For performing the stream operation																	
Operation	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
MemPtrOrFileName - Possible values passed or returned																
SysId																
Version																
ContextClassIdPtr - Possible values passed or returned																
M_INQUIRE_SIZE_BYTE	Inquires the number of bytes required to save the context to memory stream. This operation is not supported when the StreamType parameter is set to M_FILE.															
MemPtrOrFileName	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
SysId	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
Version	Specifies the Aurora Imaging Library version with which the classification context must be compatible when saved. The size of a context might differ from one Aurora Imaging Library version to another.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the current version of Aurora Imaging Library.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_100_SP7	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 7.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_110	Specifies the version as being Aurora Imaging Library 11.0.	c		e	f	g	a		i		j	k	l	m	n	b
ContextClassIdPtr	Specifies the address of the variable containing the identifier of the classification context.	c		e	f	g	a		i		j	k	l	m	n	b
M_LOAD	Loads the content of a specified file or memory stream into a previously allocated classification context.	c		e	f	g	a		i		j	k	l	m	n	b
MemPtrOrFileName	Specifies the file or memory stream from which to load the context.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which you can interactively specify the drive, directory, and name of the context file, when the StreamType parameter is set to M_FILE.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the context file (for example, "*C:\mydirectory\WePutTheArtInArtificialIntelligence.mclass*"), when the StreamType parameter is set to M_FILE. The function handles (internally) the opening and closing of the file. Classifier context, training context, and data preparation context files should have an MCLASS extension. Dataset context files should have an MCLASSD extension. To specify a context file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\WePutTheArtInArtificialIntelligence.mclass*").	c		e	f	g	a		i		j	k	l	m	n	b
MemPtr	Specifies the address of the block of memory, when the StreamType parameter is set to M_MEMORY. The block of memory should be of type *AIL_UINT8*. The specified address must correspond to the first memory address in which the object is located, and the block of memory must contain the entire object. Note that when using a C compiler (not a C++ or other compiler), you must cast the *AIL_UINT8* pointer to an *AIL_TEXT_PTR*.	c		e	f	g	a		i		j	k	l	m	n	b
SysId	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
Version	This parameter must be set to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b
ContextClassIdPtr	Specifies the address of the variable containing the identifier of the classification context. All previous settings are overwritten with those of the loaded context, or are set to the default value if the settings didn't exist in the version of Aurora Imaging Library from which the context is being loaded.	c		e	f	g	a		i		j	k	l	m	n	b
M_RESTORE	Restores the context from a file or memory stream and assigns it an Aurora Imaging Library identifier.	c		e	f	g	a		i		j	k	l	m	n	b

MemPtrOrFileName	Specifies the file or memory stream from which to restore the context.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which you can interactively specify the drive, directory, and name of the context file, when the StreamType parameter is set to M_FILE.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the context file (for example, "*C:\mydirectory\WePutTheArtInArtificialIntelligence.mclass*"), when the StreamType parameter is set to M_FILE. The function handles (internally) the opening and closing of the file. Classifier context, training context, and data preparation files should have an MCLASS extension. Dataset context files should have an MCLASSD extension. To specify a context file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\WePutTheArtInArtificialIntelligence.mclass*").	c		e	f	g	a		i		j	k	l	m	n	b
MemPtr	Specifies the address of the block of memory, when the StreamType parameter is set to M_MEMORY. The block of memory should be of type *AIL_UINT8*. The specified address must correspond to the first memory address in which the object is located, and the block of memory must contain the entire object. Note that when using a C compiler (not a C++ or other compiler), you must cast the *AIL_UINT8* pointer to an *AIL_TEXT_PTR*.	c		e	f	g	a		i		j	k	l	m	n	b
SysId	Specifies the system on which to allocate the classification context.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b
Version	This parameter must be set to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b
ContextClassIdPtr	Specifies the address of the variable in which to write the identifier of the classification context. If the operation is not successful, **M_NULL** is returned.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAVE	Saves the context to a specified file or memory stream.	c		e	f	g	a		i		j	k	l	m	n	b
MemPtrOrFileName	Specifies the file or memory stream.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which you can interactively specify the drive, directory, and name of the context file, when the StreamType parameter is set to M_FILE.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the context file (for example, "*C:\mydirectory\WePutTheArtInArtificialIntelligence.mclass*"), when the StreamType parameter is set to M_FILE. The function handles (internally) the opening and closing of the file. If the file already exists, it will be overwritten. Classifier context, training context, and data preparation files should have an MCLASS extension. Dataset context files should have an MCLASSD extension. To specify a context file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\WePutTheArtInArtificialIntelligence.mclass*").	c		e	f	g	a		i		j	k	l	m	n	b
MemPtr	Specifies the address of the block of memory, when the StreamType parameter is set to M_MEMORY. The block of memory should be of type *AIL_UINT8*. The specified address must correspond to the first memory address in which to write, and the block of memory must be large enough to stream the entire object. To determine the required size, call this function with M_INQUIRE_SIZE_BYTE. Note that when using a C compiler (not a C++ or other compiler), you must cast the *AIL_UINT8* pointer to an *AIL_TEXT_PTR*.	c		e	f	g	a		i		j	k	l	m	n	b
SysId	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
Version	Specifies the Aurora Imaging Library version with which the classification context must be compatible when saved.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the current version of Aurora Imaging Library.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_100_SP7	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 7.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_110	Specifies the version as being Aurora Imaging Library 11.0.	c		e	f	g	a		i		j	k	l	m	n	b
ContextClassIdPtr	Specifies the address of the variable containing the identifier of the classification context to save.	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailclass11.lib.
DLL	Requires ail11.dll; ailclass11.dll.

MclassTrain

Synopsis

Train a classifier context.

Syntax

```
void MclassTrain(
    AIL_ID TrainContextClassId,        //in
    AIL_ID ClassifierContextClassId,   //in
    AIL_ID TrainDatasetContextClassId, //in
    AIL_ID DevDatasetContextClassId,   //in
    AIL_ID TrainResultClassId,         //out
    AIL_INT64 ControlFlag              //in
)
```

Description

This function trains a classifier context. This context gets trained according to a training context and dataset contexts. All training results are stored in a classification result buffer. To get these results, use MclassGetResult(). You can also copy dataset-specific training results from the result buffer to a dataset, using MclassCopyResult(). If you are using an images dataset, dataset-specific results are retrievable directly from that dataset, using MclassGetResultEntry().

To modify training related settings, call MclassControl(). To hook functions to training events, call MclassHookFunction().

A training context must be preprocessed before it is trained. To inquire a context's preprocessing state, call MclassInquire() with M_PREPROCESSED.

You can train a predefined CNN, object detection, segmentation, anomaly detection classifier context, or a tree ensemble classifier context (ClassifierContextClassId). All other training contexts (TrainContextClassId, TrainDatasetContextClassId, and DevDatasetContextClassId) must be for the same type of classifier. Unless otherwise specified, contexts must have been previously allocated on the required system using MclassAlloc(). Image classification and segmentation support buffers with up to 1024 bands.

A classifier context is not trained when you allocate it. You must call **MclassTrain**() to train it, using the ClassifierContextClassId parameter. If you set this parameter to **M_NULL**, **MclassTrain**() automatically selects a new classifier context to train.

You can also call this function to retrain a trained classifier context. To do so, you must copy the trained classifier context result that **MclassTrain**() produces (TrainResultClassId) into a classifier context, using MclassCopyResult(). You can then train that classifier context with **MclassTrain**(). Before training a trained classifier context, you should modify your training settings. You can repeat this process (train, copy result, modify settings, train) as many times as required. Note, this is not possible for object detection.

When your classifier context is properly trained, you can copy the result that **MclassTrain**() produces into a classifier context (MclassCopyResult()) and use that trained classifier context with MclassPredict().

The training process for image classification, object detection, segmentation, and anomaly detection requires two images datasets (M_DATASET_IMAGES): a training dataset and a development dataset. You can, with the exception of anomaly detection, specify just one dataset, using the TrainDatasetContextClassId parameter, and set the development dataset parameter (DevDatasetContextClassId) to **M_NULL**. When doing so, Aurora Imaging Library automatically splits your specified dataset into the training and development datasets, according to the related split control settings (for example, M_SPLIT_PERCENTAGE and M_SPLIT_SEED_MODE). At this time, if data preparation controls were set, Aurora Imaging Library prepares the datasets using an internal data preparation context. To inquire the identifier of this internal data preparation context, or the location at which prepared images are stored, use M_PREPARE_DATA_CONTEXT_ID and M_TRAIN_DESTINATION_FOLDER. Note that signed images and anomaly detection tasks are not supported for single dataset training since, MclassPrepareData() does not support preparing signed images.

When training for feature classification, the development dataset is optional; typically, only the training dataset is used. In this case, if you specify one dataset, using the TrainDatasetContextClassId parameter, and you set the DevDatasetContextClassId parameter to **M_NULL**, Aurora Imaging Library does not train with a development dataset (your data is not split), and the development dataset results will not be available (such as M_DEV_DATASET_ACCURACY). For feature classification, you must explicitly specify the identifier of the development dataset to use it (DevDatasetContextClassId).

> Training time can vary considerably, depending on the complexity of the application, the available hardware, and the accuracy required. To produce a properly trained classifier, **MclassTrain**() might have to run for an extended period, and you might have to call it several times after modifying your training setup (for example, your training settings or your datasets). To take advantage of all available resources, such as GPU training, you should install the required Aurora Imaging Library add-on. For more information, see the Aurora Imaging Library add-ons and updates subsection of the Requirements, recommendations, and troubleshooting section of Chapter 47: Machine learning with the Aurora Imaging Library Classification module.

Parameters

TrainContextClassId

Specifies the identifier of the training context with which to train the classifier context. Training settings (MclassControl()) are held in the training context.

ClassifierContextClassId

Specifies the identifier of the classifier context to train. Depending on this parameter's setting, you can start a new training process, or continue a training process.

To start a new training process, you can set this parameter to **M_NULL**. In this case, Aurora Imaging Library automatically selects the classifier to train (ClassifierContextClassId) according to the specified training context (TrainContextClassId).

- For an image classification training context (M_TRAIN_CNN), Aurora Imaging Library selects a predefined medium ICNet classifier (M_ICNET_M).
- For an object detection training context (M_TRAIN_DET), Aurora Imaging Library selects an object detection classifier (M_ODNET).
- For a segmentation training context (M_TRAIN_SEG), Aurora Imaging Library selects a predefined medium CSNet classifier (M_CSNET_M).
- For an anomaly detection training context (M_TRAIN_ANO), Aurora Imaging Library selects a predefined ADNet classifier (M_ADNET).
- For a feature classification training context (M_TRAIN_TREE_ENSEMBLE), Aurora Imaging Library selects a tree ensemble classifier (M_CLASSIFIER_TREE_ENSEMBLE).

To start a new training process for image classification (M_TRAIN_CNN), object detection (M_TRAIN_DET), segmentation (M_TRAIN_SEG), or anomaly detection (M_TRAIN_ANO), and override the training context that Aurora Imaging Library automatically selects when you specify **M_NULL**, set this parameter to the identifier of an original (untrained) classifier context that was predefined by Aurora Imaging.

To continue the training process, set this parameter to the identifier of a trained classifier context. Note, this is not possible for object detection.

> When you call **MclassTrain**(), the class definitions in the classifier context are discarded and replaced by the class definitions in the training dataset context.

By default, **MclassTrain**() uses the default train engine established by the **Aurora Imaging Configurator** utility. You can modify this by calling MclassControl() with M_TRAIN_ENGINE or in **Aurora Imaging Configurator**.

TrainDatasetContextClassId

Specifies the identifier of the training dataset context with which to train the classifier context. If training with an image classification (M_TRAIN_CNN), object detection (M_TRAIN_DET), or segmentation (M_TRAIN_SEG) training context, and you set the DevDatasetContextClassId parameter to **M_NULL**, the specified dataset (TrainDatasetContextClassId) is split into the training dataset and the development dataset.

> For anomaly detection, the training dataset must only contain good (non-anomalous) images. Image classification and segmentation support buffers with up to 1024 bands.

You can set both the TrainDatasetContextClassId and the DevDatasetContextClassId parameters to **M_NULL** if there are valid training and development datasets in your training results (TrainResultClassId); in this case, those datasets are used. This is meant for quickly testing different training settings with those same datasets (this is faster since those datasets are already split and prepared).

DevDatasetContextClassId

Specifies the identifier of the development dataset context with which to evaluate the performance of the classifier context's training. If not needed, set this parameter to **M_NULL**. If you explicitly provide the development dataset, ensure that it does not contain data (image or feature entries) that is in the training dataset, and that it does not contain augmented data.

> For anomaly detection, the development dataset must only contain good (non-anomalous) images. Image classification and segmentation support buffers with up to 1024 bands.

You can set both the DevDatasetContextClassId and the TrainDatasetContextClassId parameters to **M_NULL** if there are valid training and development datasets in your training results (TrainResultClassId); in this case, those datasets are used. This is meant for quickly testing different training settings with those same datasets (this is faster since those datasets are already split and prepared).

TrainResultClassId

Specifies the identifier of the classification result buffer in which to write the results of the training operation. This result buffer must have been previously allocated on the required system using MclassAllocResult() with M_TRAIN_CNN_RESULT, M_TRAIN_DET_RESULT, M_TRAIN_SEG_RESULT, M_TRAIN_ANO_RESULT, or M_TRAIN_TREE_ENSEMBLE_RESULT.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailclass11.lib.
DLL	Requires ail11.dll; ailclass11.dll.